

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIEROS DE TELECOMUNICACIÓN



MÁSTER UNIVERSITARIO EN
INGENIERÍA DE SISTEMAS
ELECTRÓNICOS TRABAJO FIN DE
MÁSTER

IMPLEMENTACIÓN DE UN SISTEMA
EMBEBIDO CON CONECTIVIDAD
INALÁMBRICA PARA MONITORIZACIÓN
HEMODINÁMICA

BILAL BOULIBYA

MÁSTER UNIVERSITARIO EN INGENIERÍA DE SISTEMAS ELECTRÓNICOS

TRABAJO FIN DE MÁSTER

Título: Implementación de un sistema embebido con conectividad inalámbrica para monitorización hemodinámica

Autor: D. Bilal Boulibya

Tutor: D. Pedro José Malagón Marzo

Departamento: Ingeniería Electrónica

MIEMBROS DEL TRIBUNAL

Presidente: D.

Vocal: D.

Secretario: D.

Suplente: D.

Los miembros del tribunal arriba nombrados acuerdan otorgar la calificación de:
.....

Madrid, a de de 2026

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIEROS DE TELECOMUNICACIÓN



MÁSTER UNIVERSITARIO EN
INGENIERÍA DE SISTEMAS
ELECTRÓNICOS TRABAJO FIN DE
MÁSTER

**Implementación de un sistema
embebido con conectividad
inalámbrica para monitorización
hemodinámica**

Bilal Boulibya

2026

RESUMEN

En una donación de sangre quedan registrados el código ISBT 128, el hemograma de laboratorio y las constantes que toma el personal sanitario. Lo que ocurre *durante* la extracción no se registra: la respuesta hemodinámica del donante se pierde. La reacción vasovagal —la complicación más frecuente en donación— viene precedida de una caída de la perfusión periférica que una señal continua podría anticipar, extremo que este Trabajo no evalúa y deja al modelo inferencial futuro; y, aun registrada, esa señal viviría en un fichero desligado del registro clínico.

Este Trabajo construye **BloodTwin**, un gemelo digital de la donación: un dispositivo embebido de pulsioximetría y la cadena completa que lleva su señal, con trazabilidad por diseño, hasta el registro clínico. Sus etapas se verificaron por separado; la integración con el almacén de objetos quedó pendiente de validación de extremo a extremo.

El núcleo es un *firmware* en C sobre Zephyr RTOS para el ARM Cortex-M4 del SoC nRF52840, con el sensor MAX30102. Se organiza en capas: una capa de abstracción del sistema operativo (OSAL), que desacopla la lógica de Zephyr y permite probarla en el anfitrión, y sobre ella un motor de control implementado como **red de Petri interpretada** de seis lugares de estado y veinticinco transiciones, donde el orden de declaración fija la prioridad entre eventos concurrentes. La adquisición no la rige un temporizador propio sino los punteros de la FIFO del sensor. Las tramas de veinte muestras se serializan con Protocol Buffers (nanopb) y viajan por un perfil GATT propio sobre Bluetooth Low Energy cifrado y autenticado.

Una aplicación Android y otra web —un puente local en Python que dialoga por **bleak**— implementan la misma cadena de procesamiento: eliminación de línea base, componente alterna por RMS, estimación de la ratio R y detección de picos. Ambas consumen la interfaz REST de una plataforma en Flask que recorre las cinco etapas del protocolo y reparte cada dato: las variables clínicas a REDCap y las señales en bruto a Cloud Storage, vinculadas por el identificador de registro. Una herramienta de diagnóstico por USB extrae los informes de fallo conservados en la caja negra.

La validación se articula en **tres niveles**. En el formal, el modelo se traduce a Promela y SPIN verifica diecisiete propiedades de lógica temporal lineal. En el unitario, sesenta y ocho pruebas con Ceedling, Unity y CMock ejercitan el motor real y alcanzan el 96,6 % de cobertura de líneas y el 100 % de funciones. Sobre hardware real se provocan los mecanismos que no admiten simulación, y el trazado CTF del núcleo contrasta cadencia y ocupación de CPU frente a las constantes del código. Lo sostiene un subsistema que clasifica treinta y un códigos por severidad; dieciocho son candidatos a persistencia, y la caja negra conserva hasta cuatro informes.

Los resultados acreditan un *firmware* determinista y verificable —su consumo real, pendiente de caracterización experimental— y una base para un estudio clínico y la futura certificación como dispositivo médico.

SUMMARY

A blood donation leaves behind the ISBT 128 code, the laboratory blood count and the vital signs taken by clinical staff. What happens *during* the extraction is recorded nowhere: the donor's haemodynamic response is lost. The vasovagal reaction —the most frequent complication in donation— is preceded by a drop in peripheral perfusion that a continuous signal could anticipate, a claim this Thesis does not evaluate and leaves to a future inferential model; and even if recorded, it would live in a file detached from the clinical record.

This Thesis builds **BloodTwin**, a digital twin of the donation: an embedded pulse oximetry device and the complete chain carrying its signal, with traceability by design, into the clinical record. Its stages were verified separately; integration with the object store remains pending end-to-end validation.

At its core is a firmware in C on Zephyr RTOS for the ARM Cortex-M4 of the nRF52840 SoC, with the MAX30102 sensor. It is organised in layers: an Operating System Abstraction Layer (OSAL) decoupling the logic from Zephyr and making it testable on the host, and above it a control engine implemented as an **interpreted Petri net** of six state places and twenty-five transitions, where declaration order sets priority between concurrent events. Acquisition is driven by the sensor's FIFO pointers, not by a timer. Frames of twenty samples are serialised with Protocol Buffers (nanopb) and travel over a custom GATT profile on an encrypted, authenticated Bluetooth Low Energy link.

An Android and a web application —a local Python bridge using **bleak**— implement the same processing chain: baseline removal, AC component by RMS, ratio R estimation and peak detection. Both consume the REST interface of a Flask platform that walks the five stages of the protocol, routing clinical variables to REDCap and raw signals to Cloud Storage, linked by the record identifier. A USB tool extracts the fault reports retained by the black box.

Validation is structured in **three levels**: the model is translated into Promela and SPIN verifies seventeen linear temporal logic properties; sixty-eight tests with Ceedling, Unity and CMock exercise the actual engine, reaching 96.6 % line and 100 % function coverage; and on real hardware the mechanisms admitting no simulation are triggered physically, while the kernel CTF trace contrasts real cadence and CPU occupancy against the declared constants. Underpinning it is a diagnostics subsystem classifying thirty-one error codes by severity; eighteen are persistence candidates, and the black box retains up to four reports.

The results demonstrate a deterministic, verifiable firmware —its actual consumption still pending experimental characterisation— and a foundation for a clinical study and future certification as a medical device.

PALABRAS CLAVE

Donación de Sangre, Gemelo Digital, Trazabilidad Clínica, ISBT 128, Sistemas Embebidos, Zephyr RTOS, Red de Petri, Verificación Formal, SPIN, Pulsioximetría, Bluetooth Low Energy, Protocol Buffers, Caja Negra.

KEYWORDS

Blood Donation, Digital Twin, Clinical Traceability, ISBT 128, Embedded Systems, Zephyr RTOS, Petri Net, Formal Verification, SPIN, Pulse Oximetry, Bluetooth Low Energy, Protocol Buffers, Black Box.

Agradecimientos

Cuando uno llega al final de un trabajo como este, lo primero que siente no es orgullo sino alivio, y lo segundo es gratitud. Gratitud hacia todas las personas que, de un modo u otro, han puesto un poco de su tiempo, su energía o su fe en algo que durante muchos meses no era más que código roto y notas en una libreta.

A mi tutor, **Pedro José Malagón Marzo**, le debo mucho más de lo que una sección de agradecimientos puede reflejar. Su manera de mirar un problema técnico —siempre buscando la solución más limpia, la más honesta con el hardware y con quien va a mantener ese código después— ha cambiado la forma en que yo pienso como ingeniero. Hay conversaciones tuyas que se me han quedado grabadas no porque las haya apuntado, sino porque tenían esa clase de claridad que no se olvida. Gracias por la paciencia en las revisiones, por las preguntas incómodas que mejoraron el proyecto, y por tomarse esto tan en serio como yo.

A mis padres, que nunca han entendido del todo qué es un RTOS ni por qué alguien pasa noches depurando interrupciones, pero que siempre han estado ahí. No necesitaron entenderlo para creer en mí, y eso, pensándolo bien, es lo más difícil. Este trabajo también es suyo.

A mis compañeros del máster, con quienes he compartido más horas de las que me gustaría contar, incluidas algunas en las que ninguno teníamos muy claro por qué el BLE dejaba de anunciar o por qué Zephyr se negaba a compilar sin un motivo aparente. Esas sesiones de depuración colectiva, donde alguien señalaba lo obvio que todos habíamos pasado por alto, me han enseñado que la ingeniería colaborativa no es solo más eficiente: es también menos solitaria.

Quiero mencionar también, aunque parezca raro agradecer a una comunidad de personas que nunca he conocido, a todos los que contribuyen al ecosistema de Zephyr RTOS, a las herramientas de Nordic Semiconductor y a nanopb. Encontrar documentación bien escrita, ejemplos que funcionan y foros donde alguien ya había tenido exactamente tu mismo problema es, en el día a día del desarrollo embebido, una diferencia enorme.

Por último, a la **Universidad Politécnica de Madrid** y al **Departamento de Ingeniería Electrónica (DIE – ETSIT-UPM)**, por los recursos, los laboratorios y el entorno que han hecho posible que este trabajo pasara del papel al hardware real.

A todos, gracias.

Madrid, septiembre de 2026

Índice

Resumen y Palabras Clave	II
Agradecimientos	v
Lista de acrónimos	XIII
1. Introducción y objetivos	1
1.1. Introducción	1
1.1.1. Contexto y Motivación	2
1.1.2. Alcance y Estructura del Documento	4
1.1.3. Herramientas de asistencia empleadas	5
1.2. Objetivos	5
1.2.1. Objetivo General	5
1.2.2. Objetivos Específicos	6
1.2.3. Relación entre Objetivos y Fases de Trabajo	8
2. Marco teórico y estado del arte	9
2.1. Fundamentos físicos de la fotoplethysmografía y oximetría de pulso	9
2.2. Estado del arte: pulsioximetría conectada y dispositivos vestibles	12
2.2.1. Del pulsioxímetro hospitalario al dispositivo vestible	12
2.2.2. El eslabón menos documentado: el software embebido	14
2.2.3. Trabajos previos en esta línea	15
2.3. Arquitecturas de procesamiento y sistemas operativos de tiempo real	15
2.3.1. El microcontrolador SoC Nordic Semiconductor nRF52840	15
2.3.2. El ecosistema de desarrollo Zephyr RTOS	17
2.4. Comunicaciones inalámbricas de bajo consumo (Bluetooth Low Energy)	18
2.4.1. Roles GATT: servidor y cliente	19
2.4.2. El servicio GATT del dispositivo	19
2.4.3. Serialización compacta mediante Protocol Buffers	20
2.4.4. Sistemas de captura de datos de investigación clínica	21
2.4.5. Seguridad del enlace y sus implicaciones	22
2.5. Modelado formal de concurrencia y robustez mediante redes de Petri	22
2.5.1. Definición matemática formal	23
2.5.2. Regla de disparo y dinámica de la red	23
2.5.3. Propiedades de comportamiento	24
2.5.4. Redes de Petri Interpretadas	25
2.6. Verificación formal mediante comprobación de modelos	25
2.6.1. Lógica temporal lineal: seguridad y vivacidad	26
2.6.2. El comprobador de modelos SPIN y el lenguaje Promela	26
2.6.3. Explosión de estados y equidad	27
2.7. Validación de firmware embebido en entorno de anfitrión	28
2.7.1. Inversión de dependencias y dobles de prueba	28

2.7.2. Observabilidad: trazado del núcleo	28
3. Diseño e implementación del firmware	29
3.1. Análisis de requisitos y restricciones del sistema	29
3.1.1. Requisitos funcionales (RF)	29
3.1.2. Requisitos de robustez y tolerancia a fallos (RR)	30
3.1.3. Restricciones de diseño (RS)	31
3.2. Diseño e implementación de la arquitectura del firmware	31
3.2.1. Modelado formal de estados mediante red de Petri reactiva	32
3.2.2. Sistema de detección y gestión de fallos (Fault Detection & Management)	39
3.2.3. Persistencia de fallos y Caja Negra en Flash NVS	55
3.2.4. Adquisición dirigida por la FIFO del sensor	59
3.2.5. Seguridad del enlace de radio	60
4. Verificación formal del modelo de control	63
4.1. Del firmware al modelo en Promela	63
4.1.1. Alcance de la abstracción	63
4.1.2. Codificación de la prioridad de las transiciones	64
4.1.3. El entorno y la cota de reinicios	65
4.2. Propiedades verificadas	65
4.2.1. Seguridad	66
4.2.2. Vivacidad	66
4.2.3. Alcanzabilidad	66
4.2.4. Lógica de negocio	67
4.3. Resultados	68
4.4. Lecciones del proceso de verificación	69
5. Herramienta de extracción diagnóstica	71
5.1. Decisiones de diseño	71
5.1.1. Conexión exclusivamente por USB	71
5.1.2. Una única tabla de códigos, compartida con el firmware	72
5.2. Arquitectura de la aplicación	72
5.3. Protocolo de diagnóstico	72
5.4. Formato de las tramas y decodificación	73
5.5. Interfaz de usuario	74
6. Cliente Android de donación	78
6.1. El flujo de trabajo determina la estructura	78
6.1.1. Navegación e indicador de progreso	79
6.2. Arquitectura del software	79
6.2.1. Enlace de radio y decodificación de tramas	80
6.3. Procesado digital de la señal	80
6.3.1. Filtro eliminador de continua	81
6.3.2. Detección de dedo y amplitud pulsátil	81
6.3.3. Saturación de oxígeno	81
6.3.4. Frecuencia cardíaca	82
6.4. Almacenamiento local y trabajo sin cobertura	82
6.5. Validación en el cliente	83

6.6. Empaquetado como aplicación Android	83
6.7. Simulación en escritorio	84
7. Plataforma de integración clínica	85
7.1. Arquitectura del ecosistema	85
7.2. El puente Bluetooth de la aplicación web	86
7.3. La interfaz de integración	87
7.3.1. El orden no es opcional	88
7.3.2. Autenticación y protección	89
7.4. La decisión de almacenamiento	90
7.4.1. El problema	90
7.4.2. La solución	90
7.4.3. Una limitación del despliegue de pruebas	92
7.4.4. Por qué REDCap y no una base de datos propia	93
7.5. Continuidad del servicio sin conexión	93
8. Validación experimental	95
8.1. Pruebas unitarias sobre el ordenador anfitrión	95
8.1.1. Entorno y estrategia	95
8.1.2. Cobertura alcanzada	95
8.2. Validación funcional del motor de control	96
8.3. Validación de los mecanismos de recuperación física	99
8.3.1. Ventana de diagnóstico y consola de emergencia	99
8.3.2. Despertar del sueño profundo	100
8.3.3. Tolerancia a fallos del sensor en marcha	100
8.4. Consumo: referencias de los fabricantes y caracterización pendiente	101
8.5. Análisis de temporización en tiempo real mediante trazas Perfetto	102
8.5.1. Configuración e instrumentación del sistema de trazado	102
8.5.2. Vista general del sistema	105
8.5.3. El ciclo de sondeo de la FIFO	107
8.5.4. El periodo de trama y su cuantificación	110
8.5.5. Periodicidad del motor de la Red de Petri	113
8.5.6. Cuatro relojes independientes	115
8.5.7. Efecto de la reforma de la adquisición	116
8.5.8. Tabla de conformidad con los requisitos de tiempo real	117
9. Conclusiones y líneas futuras	120
9.1. Conclusiones	120
9.1.1. Limitaciones del Sistema	122
9.2. Líneas futuras	123
Bibliografía	126
Anexo A: Aspectos éticos, económicos, sociales y ambientales	129
A.1. Introducción	129
A.2. Descripción de impactos relevantes relacionados con el proyecto	129
A.3. Análisis detallado de alguno de los principales impactos	130
A.4. Conclusiones	131
Anexo B: Presupuesto económico	133

B.1. Desglose de costes directos	133
B.1.1. Costes de mano de obra	133
B.1.2. Amortización de recursos materiales	133
B.2. Presupuesto total acumulado	134
Anexo C: Batería completa de pruebas unitarias	135
C.1. Observaciones sobre la batería	140
Anexo D: Especificación de la interfaz de integración	142
D.1. Convenios generales	142
D.2. Autenticación	142
D.3. Secuencia de invocación	143
D.4. Puntos de entrada	143
D.4.1. Registro de la donación	143
D.4.2. Datos demográficos y clínicos	144
D.4.3. Medidas del dispositivo	144
D.4.4. Resultados de laboratorio	145
D.5. Códigos de respuesta	146
D.6. Documentación en línea	146

Índice de figuras

2.1. Modos de adquisición y morfología de la señal PPG	10
2.2. Absortividad molar de HbO_2 y Hb	11
2.3. Pulsioxímetro clínico multiparamétrico portátil	13
2.4. Dispositivo vestibular con fotoplethysmografía de reflexión	13
2.5. Diagrama de bloques del SoC nRF52840	16
2.6. Arquitectura en capas de la pila BLE	19
2.7. Ocupación de la trama dentro de la MTU	21
2.8. Red de Petri básica y su marcado inicial	24
2.9. Flujo de la comprobación de modelos con SPIN	27
3.1. Arquitectura por capas del firmware	31
3.2. Ciclo de vida del dispositivo: vista simplificada	33
5.1. Interfaz tras la conexión USB	75
5.2. Anillo de informes de la Caja Negra	76
5.3. Borrado de la Caja Negra tras la extracción	77
5.4. Listado de vínculos Bluetooth	77
6.1. Pantallas del cliente móvil	79
7.1. Arquitectura del ecosistema completo	85
7.2. Página de adquisición con sesión en curso	87
7.3. Formulario del protocolo clínico	89
7.4. Registro clínico creado por la interfaz	91
7.5. Formulario de medidas del dispositivo	92
7.6. Registro de sesiones sincronizadas	94
8.1. Vista global de la traza Perfetto del sistema	105
8.2. Periodo de sondeo de la FIFO	108
8.3. Anatomía de un sondeo de la FIFO	109
8.4. Periodo de trama medido entre dos transmisiones	110
8.5. Métricas de un ciclo de trama completo	112
8.6. Periodicidad del motor de la Red de Petri	114
D.1. Autenticación, estado y registro de la donación	147
D.2. Punto de entrada de datos basales	148
D.3. Punto de entrada de constantes vitales	149
D.4. Punto de entrada de la señal del dispositivo	150
D.5. Punto de entrada del hemograma	151

Índice de tablas

1.1. Matriz de trazabilidad entre fases de trabajo y objetivos específicos.	9
2.1. Especificaciones arquitectónicas y subsistemas clave del SoC nRF52840. . .	16
2.2. Subsistemas de Zephyr RTOS empleados en el proyecto	17
3.1. Estados de los canales LED según el marcado de la Red de Petri.	36
3.2. Transiciones de la Red de Petri, prioridades 1 a 3	37
3.3. Transiciones de la Red de Petri, prioridades 4 y 5	38
3.4. Tabla maestra de códigos de error del firmware	39
3.5. Cadena de tolerancia a fallos del sensor	49
3.6. Órdenes de la consola de diagnóstico por USB	55
3.7. Estructura del registro de la Caja Negra	57
3.8. Registros de la FIFO del MAX30102	59
3.9. Reparto de la memoria no volátil	62
4.1. Alcance de la abstracción del modelo formal	64
4.2. Propiedades de seguridad	66
4.3. Propiedades de vivacidad	66
4.4. Propiedades de alcanzabilidad	67
4.5. Propiedades de lógica de negocio	67
4.6. Resultados de la verificación formal	68
5.1. Protocolo de la herramienta de extracción	73
5.2. Prefijos de las tramas de diagnóstico	74
6.1. Los cinco pasos del cliente móvil	78
6.2. Capas del cliente móvil	80
7.1. Puntos de entrada de la plataforma	88
8.1. Cobertura de código alcanzada	96
8.2. Distribución de las pruebas unitarias	97
8.3. Configuración del trazado en <code>prj.conf</code>	103
8.4. Leyenda de las pistas visibles en la traza Perfetto.	106
8.5. Desglose de un ciclo de sondeo	109
8.6. Intervalos consecutivos entre tramas	111
8.7. Métricas de un ciclo de trama	113
8.8. Estadística del periodo del motor de control	115
8.9. Periodos medidos frente a las constantes del código	116
8.10. Comparativa entre las dos estrategias de adquisición	116
8.11. Conformidad de los requisitos de tiempo real	117
8.12. Reparto de CPU durante el streaming	118
B.1. Costes de recursos humanos y desarrollo.	133
B.2. Amortización de recursos materiales.	134

B.3. Presupuesto final acumulado.	134
C.1. Batería completa de pruebas unitarias	135
D.1. Campos del registro de donación	143
D.2. Campos demográficos y clínicos	144
D.3. Campos de las medidas del dispositivo	145
D.4. Códigos de respuesta de la interfaz	146

Lista de acrónimos

- ADC** *Analog-to-Digital Converter* (Conversor Analógico a Digital).
- API** *Application Programming Interface* (Interfaz de Programación de Aplicaciones).
- ARM** *Advanced RISC Machines*.
- ATT** *Attribute Protocol* (Protocolo de Atributos en BLE).
- BLE** *Bluetooth Low Energy* (Bluetooth de Bajo Consumo).
- CCC / CCCD** *Client Characteristic Configuration Descriptor* (Descriptor de Configuración de Característica de Cliente).
- CDC-ACM** *Communications Device Class - Abstract Control Model* (Clase de Dispositivo de Comunicaciones USB).
- CPU** *Central Processing Unit* (Unidad Central de Procesamiento).
- CTF** *Common Trace Format* (Formato de Trazado Común en Zephyr).
- DC** *Direct Current* (Componente de Corriente Continua o línea base en bioseñales).
- DSP** *Digital Signal Processing* (Procesamiento Digital de Señales).
- ETSIT** Escuela Técnica Superior de Ingenieros de Telecomunicación.
- FSM** *Finite State Machine* (Máquina de Estados Finitos).
- GAP** *Generic Access Profile* (Perfil de Acceso Genérico en Bluetooth).
- GATT** *Generic Attribute Profile* (Perfil de Atributos Genérico en Bluetooth).
- GUI** *Graphical User Interface* (Interfaz Gráfica de Usuario).
- HAL** *Hardware Abstraction Layer* (Capa de Abstracción de Hardware).
- Hb** Hemoglobina reducida o desoxigenada.
- HbO₂** Oxihemoglobina o hemoglobina oxigenada.
- HR** *Heart Rate* (Frecuencia Cardíaca).
- HTTPS** *HyperText Transfer Protocol Secure* (Protocolo de Transferencia de Hipertexto Seguro).
- I²C** *Inter-Integrated Circuit* (Circuito Inter-Integrado).
- IoT** *Internet of Things* (Internet de las Cosas).
- ISBT** *International Society of Blood Transfusion*. El estándar ISBT 128 define la codificación universal de los productos sanguíneos y del donante.
- JSON** *JavaScript Object Notation* (Notación de Objetos de JavaScript).
- JWT** *JSON Web Token*; testigo firmado empleado para autenticar las peticiones a la

plataforma de integración.

LED *Light Emitting Diode* (Diodo Emisor de Luz).

LTL *Linear Temporal Logic* (Lógica Temporal Lineal), en la que se expresan las propiedades verificadas con SPIN.

MCU *Microcontroller Unit* (Unidad de Microcontrolador).

MPU *Memory Protection Unit* (Unidad de Protección de Memoria).

MTU *Maximum Transmission Unit* (Unidad Máxima de Transmisión).

NVS *Non-Volatile Storage* (Almacenamiento No Volátil).

OSAL *Operating System Abstraction Layer* (Capa de Abstracción del Sistema Operativo).

PPG Fotopletismografía (o fotopletismograma).

REDCap *Research Electronic Data Capture*; plataforma de captura de datos clínicos para investigación, empleada como almacén de las variables estructuradas de la donación.

RGPD Reglamento General de Protección de Datos de la Unión Europea.

RMS *Root Mean Square* (Media Cuadrática).

RTOS *Real-Time Operating System* (Sistema Operativo en Tiempo Real).

SMP *Security Manager Protocol*; capa de Bluetooth Low Energy encargada del emparejamiento y del cifrado del enlace.

SOC / SoC *System on Chip* (Sistema en un Chip).

SpO₂ Saturación periférica de oxígeno.

TFM Trabajo Fin de Máster.

UART *Universal Asynchronous Receiver-Transmitter* (Receptor-Transmisor Asíncrono Universal).

UPM Universidad Politécnica de Madrid.

USB *Universal Serial Bus* (Bus Serie Universal).

WDT *Watchdog Timer* (Temporizador Perro Guardián).

1. Introducción y objetivos

1.1. Introducción

La sangre es uno de los pocos recursos terapéuticos que la industria no sabe fabricar. No existe sustituto sintético para una transfusión, y cada unidad que un hospital administra procede necesariamente de otra persona que, poco antes, se sentó voluntariamente en un sillón y permitió que le extrajeran cerca de medio litro. La medicina transfusional descansa por completo sobre esa cadena de actos individuales, y por eso ha desarrollado alrededor de ellos un aparato de trazabilidad que pocas ramas de la sanidad igualan: cada bolsa, cada donante y cada componente derivado se identifican con códigos normalizados que permiten reconstruir el recorrido completo, desde la vena que dio la sangre hasta la vena que la recibe.

Una sesión de donación es, vista de cerca, un procedimiento breve pero denso en información. Antes de la extracción se entrevista al candidato y se comprueba que donar no le perjudicará: se toman sus constantes y se verifica que su concentración de hemoglobina supera un umbral mínimo. Durante la extracción se recoge la unidad, ya etiquetada. Y después, en el laboratorio, la sangre obtenida se somete a un análisis que produce un hemograma completo. En menos de una hora, y sin que ese sea su propósito, el proceso genera un retrato fisiológico del donante acompañado de su verificación analítica.

Ese retrato existe, pero vive disperso. El dato clínico queda en el sistema de captura del estudio, la analítica en el laboratorio, y la exploración física —la que decidió si podía donar— se realiza, se anota y rara vez vuelve a consultarse. Nada obliga a que las tres piezas se encuentren, porque cada una cumple su función por separado y el procedimiento termina correctamente sin necesidad de reunirlos. El resultado es que una información valiosa se produce a diario, en volumen, y se pierde como conjunto en el mismo momento en que se genera.

Este Trabajo de Fin de Máster aborda esa dispersión desde la ingeniería. Su resultado es **BloodTwin**, un sistema que construye, para cada donación, una contrapartida digital fiel: un registro único que encadena la identidad normalizada del donante y de la bolsa, sus constantes en el momento de donar, la señal óptica cruda capturada por un dispositivo desarrollado a tal efecto, y el hemograma que el laboratorio produce horas después. El nombre no es decorativo. Lo que el sistema persigue es el principio del *gemelo digital* [1]: mantener, junto al objeto físico, una réplica informacional lo bastante completa y bien identificada como para poder razonar sobre él cuando ya no se le tiene delante.

La diferencia entre ese planteamiento y un simple archivo de medidas está en el propósito. Un archivo guarda; un gemelo permite *preguntar*. Y la pregunta que motiva este sistema es concreta: si la señal óptica que un sensor recoge del dedo de un donante y el hemo-

grama que el laboratorio produce de su sangre quedan asociados de forma inequívoca, y esa asociación se repite miles de veces, entonces se dispone del material necesario para averiguar cuánto de lo que el laboratorio mide pinchando puede deducirse de lo que la luz mide sin pinchar. Ese es el horizonte del proyecto; los capítulos que siguen construyen la infraestructura que lo hace posible.

1.1.1. Contexto y Motivación

El problema concreto: el cribado previo a la donación

Antes de aceptar a un donante hay que comprobar que donar no le perjudicará. La verificación central es la concentración de hemoglobina: extraer casi medio litro de sangre a alguien cuyas reservas de hierro ya están al límite lo expone a una anemia que la donación misma provoca. Por eso todo centro aplica un umbral mínimo, y quien no lo alcanza queda diferido.

Esa comprobación se realiza hoy mediante una **punción capilar en el dedo**. El procedimiento es rápido y está bien establecido, pero acarrea inconvenientes que se multiplican por el volumen: es invasivo y molesto en un acto que es voluntario y no remunerado — y la molestia desalienta a repetir—, consume fungibles y genera residuos biológicos por cada candidato evaluado, exige tiempo de personal cualificado, y somete al donante a un pinchazo que en una parte de los casos termina en diferimiento, es decir, en dolor sin donación.

La alternativa evidente sería estimar la hemoglobina **sin pinchar**, por vía óptica. La fotopletismografía —la misma técnica en que se apoya cualquier pulsioxímetro de dedo [2]— ilumina el tejido y mide cuánta luz lo atraviesa o se refleja en él; y puesto que la absorción depende de la composición de la sangre, la señal contiene información sobre algo más que la saturación de oxígeno. Extraer de ella una estimación fiable de parámetros hematológicos es, sin embargo, un problema abierto [3], y lo es por una razón concreta que da sentido a todo este trabajo.

Por qué la donación es el escenario adecuado

Un modelo que traduzca señal óptica en parámetros hematológicos solo puede construirse y validarse si se dispone de **pares de datos**: la señal capturada de un individuo y, junto a ella, el valor real medido en su sangre por un método de referencia. Reunir esos pares es normalmente costoso, porque obliga a extraer sangre con el único fin de investigar.

En la donación ese coste desaparece. La extracción se produce de todos modos, y de ella sale sistemáticamente un hemograma completo —recuento eritrocitario, hemoglobina, hematocrito, índices, serie blanca y plaquetas— realizado con instrumentación de laboratorio. Cada donación ofrece por tanto un **patrón de referencia** para la medida no invasiva tomada minutos antes, sin coste añadido para el donante. A ello se suman tres circunstancias que rara vez concurren:

- **Identidad normalizada.** Donante y bolsa se etiquetan con códigos ISBT 128 [4], el estándar internacional de trazabilidad en medicina transfusional, *antes* de tomar ninguna medida. El vínculo entre la señal y su referencia de laboratorio no depende de una conciliación posterior: nace identificado.

- **Entorno controlado.** El donante permanece sentado o tumbado, en reposo y con temperatura estable, lo que reduce los artefactos de movimiento que degradan la señal fotopletimográfica en cualquier escenario ambulatorio.
- **Volumen y repetición.** Un centro de donación evalúa a numerosos candidatos al día, y una parte considerable dona de forma recurrente. Es precisamente el régimen que un modelo de estimación necesita para ajustarse y, más adelante, para verificarse sobre sujetos no vistos.

La donación no es entonces un caso de uso escogido por conveniencia, sino el escenario donde el problema y su método de validación coinciden en el mismo acto.

Qué exige eso del sistema

Reconocido el objetivo, las decisiones de ingeniería dejan de ser preferencias y pasan a ser consecuencias.

El dato debe ser fiel, no vistoso. Si la señal ha de servir para ajustar un modelo contra una referencia de laboratorio, no puede llegar filtrada, interpolada ni suavizada por el camino. El dispositivo transmite las muestras *en bruto*, y cada trama viaja numerada, de modo que el cliente sabe si el enlace perdió alguna en lugar de suponer que recibió todo. Una serie con huecos silenciosos es peor que una serie corta: la primera engaña, la segunda solo es incompleta.

La adquisición debe ser determinista. Una cadencia de muestreo que fluctúe con la carga del procesador desplaza el contenido frecuencial de la señal y sesga cualquier parámetro derivado de ella. De ahí la reimplementación completa del *firmware* sobre un sistema operativo en tiempo real, con el control modelado como una red de Petri interpretada [5] y sometido a verificación formal: no por rigor académico, sino porque un dispositivo que se bloquea o que transmite estando averiado contamina el conjunto de datos con muestras que nadie podrá distinguir después de las buenas.

El registro debe sobrevivir al proyecto. Un dato clínico se consulta meses o años más tarde, por personas que no participaron en su captura. Por eso las variables clínicas se depositan en REDCap —la herramienta que la institución ya emplea para sus estudios— y las señales en bruto en un almacén de objetos, quedando ambos mundos unidos por el identificador que la plataforma asigna. El comportamiento previsto es que quien consulte el registro clínico encuentre en él la referencia al fichero de señal; comprobarlo sobre un despliegue con credenciales del almacén de objetos quedó fuera del alcance de las pruebas realizadas, como se detalla en el Capítulo 7.

El procedimiento debe imponerse al operador, y no al revés. La plataforma exige que la donación se identifique antes de admitir ninguna medida, y rechaza cualquier dato que no venga precedido de ese registro. Es una rigidez deliberada: reproduce el orden del procedimiento clínico real e impide que queden en el sistema medidas huérfanas, imposibles de atribuir y por tanto inservibles como pares de referencia.

Alcance de lo construido

Conviene delimitar con precisión qué se ha desarrollado y qué no. Este Trabajo construye la **infraestructura completa de captura e integración**: el dispositivo embebido y su *firmware*, los dos clientes que dialogan con él, la plataforma que valida y reparte los

datos, y las herramientas de verificación y diagnóstico. El resultado es un canal *diseñado* para producir de forma trazable y repetible el par (*señal óptica, referencia de laboratorio*) sobre el que descansa todo lo demás; su recorrido completo hasta el almacén de objetos no llegó a validarse por falta de credenciales durante el periodo de pruebas.

Lo que no se aborda aquí es el **modelo inferencial** que, alimentado por esos pares, traduciría la señal en una estimación de hemoglobina. Esa pieza pertenece al dominio del tratamiento de señal y del aprendizaje estadístico, requiere una cohorte que solo puede reunirse una vez el sistema está desplegado, y constituye la continuación natural del trabajo. El gemelo digital que aquí se levanta es, en rigor, su cimiento: la réplica fiel e identificada de cada donación, sin la cual ningún modelo posterior podría entrenarse ni merecer crédito.

1.1.2. Alcance y Estructura del Documento

El alcance de este proyecto cubre la totalidad del ciclo de desarrollo del *firmware* embebido: desde la configuración del entorno de compilación cruzada (*cross-compilation*) basado en Zephyr RTOS, hasta la validación del sistema completo sobre hardware de referencia compatible con la arquitectura nRF52 de Nordic Semiconductor. Se incluye también el desarrollo de los dos clientes que consumen el dispositivo —la aplicación Android y la aplicación web, esta última apoyada en un puente local en Python para el enlace de radio—, junto con la plataforma de integración que recibe sus datos y los deposita en la infraestructura clínica. Completa el conjunto una herramienta de escritorio dedicada al diagnóstico del propio dispositivo, que extrae por USB el registro de fallos almacenado en su memoria no volátil.

El documento se organiza siguiendo una progresión lógica que va de los fundamentos teóricos a la validación experimental:

- El **Capítulo 1**, el presente, contextualiza el proyecto, expone su motivación y define los objetivos y el alcance del trabajo.
- El **Capítulo 2** establece los fundamentos —la física de la fotopletiśmografía, la plataforma de ejecución, el enlace inalámbrico y el formalismo de modelado— y revisa el estado del arte para delimitar el hueco que este Trabajo ocupa.
- El **Capítulo 3** detalla la implementación del *firmware*: su arquitectura por capas, el modelado del control mediante una red de Petri interpretada, el sistema de detección y gestión de fallos, y el subsistema de diagnóstico persistente.
- El **Capítulo 4** somete el modelo de control a verificación formal, comprobando diecisiete propiedades de lógica temporal mediante exploración exhaustiva del espacio de estados.
- El **Capítulo 5** describe la herramienta de escritorio que extrae e interpreta el registro de fallos almacenado en la memoria no volátil del dispositivo.
- El **Capítulo 6** expone el diseño de el **cliente Android**, segundo cliente del protocolo clínico: dialoga con el dispositivo por radio sin puente intermedio y recorre los cinco pasos de la donación contra la misma interfaz REST que la aplicación web.
- El **Capítulo 7** describe la plataforma de integración clínica que recibe lo medido, lo valida y lo deposita en la infraestructura de investigación: la interfaz REST que ambos clientes consumen, el registro de variables clínicas en REDCap y el depósito de la señal en bruto en Cloud Storage.

- El **Capítulo 8** recoge la validación experimental: las pruebas unitarias sobre el ordenador anfitrión y su cobertura, la validación de los mecanismos de recuperación física, la delimitación de lo que puede afirmarse sobre el consumo y el análisis del comportamiento temporal sobre el hardware.
- El **Capítulo 9** recoge las conclusiones, las limitaciones reconocidas del diseño y las líneas de trabajo futuro.

El documento se complementa con cuatro anexos: los aspectos éticos, económicos, sociales y ambientales del proyecto y su presupuesto económico (Anexos A y B), la batería completa de pruebas unitarias del *firmware* (Anexo C) y la especificación de la interfaz REST de la plataforma de integración (Anexo D).

Este Trabajo se inscribe en una línea previa del Departamento de Ingeniería Electrónica: los tres trabajos que lo preceden, y la relación de este Trabajo con ellos, se detallan en la Sección 2.2.3.

1.1.3. Herramientas de asistencia empleadas

En la redacción de esta memoria se han utilizado herramientas de inteligencia artificial como apoyo lingüístico. Dado que el español no es la lengua materna del autor y que este se encuentra en proceso de aprendizaje, estas herramientas se emplearon para corregir la gramática y la ortografía, mejorar la claridad de la expresión y reformular algunos pasajes. La selección del contenido y la responsabilidad sobre el texto final corresponden al autor.

1.2. Objetivos

Todo proyecto de ingeniería comienza, antes que con código o esquemáticos, con una pregunta clara: ¿qué se quiere conseguir y cómo se va a saber si se ha conseguido? Esta sección responde a esa pregunta. Los objetivos que se presentan a continuación no nacen de una lista de deseos, sino del análisis previo de las carencias reales del sistema preexistente y de las necesidades concretas del contexto clínico al que este dispositivo está destinado.

1.2.1. Objetivo General

El objetivo central de este trabajo es diseñar, implementar y validar la arquitectura de software completa de **BloodTwin**: un dispositivo embebido de pulsioximetría y la cadena que lleva su señal, con trazabilidad por diseño, hasta el registro clínico de una donación de sangre. Esa trazabilidad está implementada, y sus etapas se comprobaron por separado; la ruta actual completa —dispositivo, plataforma, almacén de objetos y registro clínico— no pudo validarse de extremo a extremo porque las credenciales y los permisos de acceso al *bucket* del proyecto no estuvieron disponibles durante el periodo de pruebas, según se detalla en el Capítulo 7. Eso abarca desde el *firmware* y la pila de comunicaciones inalámbricas hasta los clientes que el personal sanitario tiene delante y la plataforma que enlaza la señal con la historia de la donación. El criterio de éxito no es únicamente que el sistema funcione, sino que lo haga de forma fiable y **demostrable**, y que esté diseñado para un funcionamiento energéticamente eficiente —quedando su caracterización eléctrica pendiente de validación experimental—: que cada muestra sepa a qué donación pertenece y que el comportamiento del control pueda someterse a verificación automática.

1.2.2. Objetivos Específicos

A continuación se detallan los objetivos específicos que articulan el camino hacia esa meta. Se presentan en el orden natural en que se abordan durante el desarrollo, desde la base del sistema hacia las capas superiores.

OE1: Análisis de requisitos y configuración del entorno de desarrollo:

Antes de escribir una sola línea de código, es necesario saber sobre qué se construye y para qué. Este objetivo comprende la definición de los requisitos funcionales y no funcionales del sistema, la selección de la plataforma hardware —microcontrolador nRF52840 de Nordic Semiconductor, con núcleo ARM CORTEX-M4— y la configuración del entorno de compilación cruzada sobre Zephyr RTOS, incluyendo el sistema de *build* (CMake/Kconfig) y las herramientas de depuración y análisis (J-Link para la programación y el volcado de memoria por SWD, y el subsistema de trazado CTF de Zephyr con visualización en Peretto).

OE2: Desarrollo de la capa de abstracción del sistema operativo (OSAL):

Uno de los riesgos más habituales en proyectos embebidos es que el código quede tan ligado a un RTOS concreto que resulte imposible probarlo fuera del hardware real. Para evitarlo, se diseñará e implementará una *Operating System Abstraction Layer* que desacople el *firmware* de las primitivas específicas de Zephyr, ofreciendo interfaces uniformes para hilos, semáforos, temporizadores, colas de mensajes y operaciones atómicas. El beneficio inmediato es la posibilidad de ejecutar pruebas unitarias en un entorno *host* sin necesidad de tener el hardware encima de la mesa.

OE3: Implementación de los controladores del sensor MAX30102: El sensor MAX30102 (Maxim Integrated) es el origen de la señal. Este objetivo comprende la integración del controlador `maxim,max30101` que Zephyr proporciona para esta familia, su configuración mediante Kconfig y árbol de dispositivos —tasa de conversión, promediado, corriente de pulso de los emisores y rango del convertidor—, y el desarrollo de una capa de abstracción de hardware propia que complementa al controlador con los accesos directos por I²C que este no expone: la entrada en modo de bajo consumo del sensor y la lectura de la temperatura del encapsulado.

OE4: Diseño de la máquina de estados mediante redes de Petri: El comportamiento de un dispositivo embebido médico no puede depender de condiciones de carrera ni de transiciones ambiguas entre modos de operación. Para garantizar un ciclo de vida determinista, se modelará e implementará el comportamiento del sistema mediante una red de Petri interpretada, con los estados BOOT, ADVERTISING, CONNECTED_IDLE, STREAMING, ERROR y DEEP_SLEEP, definiendo explícitamente condiciones de guarda, acciones diferidas y la ausencia formal de *deadlocks* no deseados en los estados operativos —DEEP_SLEEP es un estado final por diseño—.

OE5: Desarrollo del algoritmo de filtrado y procesamiento de señal: Los datos brutos del sensor contienen artefactos que deben eliminarse antes de derivar cualquier parámetro clínico. El dispositivo transmite la señal sin procesar —el procesado exigiría en él memoria y ciclos que se prefieren dedicar a una adquisición determinista—, de modo que los algoritmos residen en los

clientes: eliminación de la línea base (*DC blocker*), cálculo de la componente AC mediante RMS, estimación de la ratio R para la derivación de la saturación de oxígeno, y detección de picos con ventana de refractariedad para la frecuencia cardíaca. La misma cadena se implementa en los dos clientes con idénticos parámetros, condición necesaria —aunque no suficiente por sí sola, según se detalla en el Capítulo 7— para que ambas estimaciones sean directamente comparables.

OE6: Implementación de la pila de comunicaciones Bluetooth Low Energy:

La transmisión de datos biométricos en tiempo real exige un enlace radio robusto y bien gestionado. Se desarrollará el módulo BLE siguiendo el perfil GATT, con un servicio personalizado que incluya características de control (lectura/escritura) y notificación de datos (*notify*). Se implementarán también mecanismos de control de flujo mediante semáforos, gestión de la MTU y manejo correcto de los eventos de conexión, desconexión y cambios en la suscripción a notificaciones (CCC).

OE7: Diseño del protocolo de serialización de datos: Transmitir datos en formato JSON sobre BLE no es una opción razonable cuando el ancho de banda es limitado y la batería importa. Se definirá un esquema de mensajes eficiente mediante *Protocol Buffers* —nanopb en el lado embebido, protobuf en los clientes—, codificando las tramas PPG en un mensaje `DataFrame` de 20 muestras, cada una con los campos `red` e `ir`, dimensionado para que una trama completa quepa en un único paquete de la MTU negociada sin fragmentación. El mensaje incorpora además dos campos escalares que no son accesorios: `seq`, número de trama que permite al cliente detectar las que el *firmware* descarta deliberadamente cuando la radio se congestiona, y `t_ms`, tiempo de funcionamiento de la primera muestra del bloque.

OE8: Desarrollo de la aplicación Android: El dispositivo embebido necesita una contraparte en el bolsillo del usuario. Se diseñará e implementará una aplicación móvil que permita el descubrimiento de dispositivos BLE, el emparejamiento seguro, la visualización en tiempo real de SpO_2 y frecuencia cardíaca, la representación gráfica de la onda PPG, y el envío de comandos de control al dispositivo (inicio y parada del *streaming*).

OE9: Desarrollo de la aplicación web y su puente de radio: Un navegador no puede abrir un enlace BLE de forma fiable, de modo que el segundo cliente se articula en dos piezas: un proceso local en Python que dialoga con el dispositivo mediante `bleak`, decodifica las tramas Protocol Buffers y aplica la misma cadena de procesamiento que la aplicación móvil, y una interfaz de navegador que recibe por Socket.IO los resultados ya interpretados y representa la onda y los valores derivados. Ambas piezas consumen la misma interfaz REST que el cliente Android, de forma que ninguno de los dos sea privilegiado.

OE10: Desarrollo de la plataforma de integración clínica: Guardar datos es un problema resuelto; mantener asociados un registro clínico y una señal de miles de muestras cuando cada uno vive en un sistema distinto no lo es. Se diseñará e implementará una plataforma en Flask que exponga los cinco puntos de entrada REST del protocolo de donación —registro de los códigos ISBT 128, datos basales, constantes clínicas, señal del dispositivo y hemograma de laboratorio—, con autenticación mediante JWT, validación de formato en el

servidor y limitación de la tasa de peticiones. La plataforma reparte cada dato al almacén que le corresponde: las variables clínicas a REDCap y las señales en bruto a Cloud Storage, manteniendo el vínculo entre ambos mediante el identificador de registro que ella misma asigna.

OE11: Desarrollo de la herramienta de diagnóstico por USB: Un dispositivo que falla en el arranque no puede explicarse a sí mismo por radio, porque al declarar el fallo corta el enlace. Se implementará una aplicación de escritorio en Python que, conectada por el puerto serie virtual USB CDC-ACM, extraiga e interprete el informe de fallo almacenado en la memoria no volátil, el anillo de incidencias en RAM y los contadores de salud acumulados, traduciendo cada código numérico a su nombre y descripción.

OE12: Validación y verificación del sistema: Un sistema no está terminado hasta que se ha probado de forma sistemática, y probar no es lo mismo que demostrar. El plan de validación contempla tres niveles complementarios: (a) **verificación formal** del modelo de control, expresando en lógica temporal lineal las propiedades exigidas y comprobándolas por exploración exhaustiva del espacio de estados con el comprobador de modelos SPIN; (b) **pruebas unitarias** del firmware ejecutadas en el ordenador de desarrollo con Ceedling, Unity y CMock, con medida de cobertura, ejercitando el motor de control real y no una simulación; y (c) **validación sobre el hardware real**, que cubre tanto los mecanismos que solo pueden provocarse físicamente —como el despertar del sueño profundo o la desconexión del sensor en marcha— como el análisis del comportamiento temporal mediante el trazado del núcleo, para contrastar la ocupación real de CPU y la cadencia efectiva frente a los requisitos.

OE13: Optimización energética y análisis de consumo: La autonomía de un dispositivo portátil es, en última instancia, lo que determina si resulta útil en la práctica. Se implementarán políticas de gestión de energía apoyadas en el subsistema de *System OFF* del nRF52840 y en la detención de los periféricos fuera de sus fases activas, y se comprobará funcionalmente la entrada y el despertar del sueño profundo sobre el hardware. La caracterización eléctrica propiamente dicha —consumo por modo de operación y autonomía efectiva con una celda concreta— requiere instrumentación de la que no se dispuso, y se plantea como trabajo experimental posterior.

1.2.3. Relación entre Objetivos y Fases de Trabajo

La Tabla 1.1 recoge la correspondencia entre los objetivos específicos y las fases del plan de trabajo. Más que un formalismo, esta matriz sirve para hacer visible qué objetivos se abordan en paralelo y cuáles dependen de la finalización de otros, facilitando el seguimiento del progreso a lo largo del proyecto.

Cada objetivo tiene, por tanto, un lugar en el tiempo y una razón de ser. La intención es que al leer cualquier capítulo posterior, sea posible rastrear por qué se tomó cada decisión de diseño y qué objetivo concreto estaba detrás de ella.

Tabla 1.1: Matriz de trazabilidad entre fases de trabajo y objetivos específicos.

Fase / Objetivo	OE1	OE2	OE3	OE4	OE5	OE6	OE7	OE8	OE9	OE10	OE11	OE12	OE13
Fase 1: Requisitos y entorno	✓	✓											
Fase 2: Firmware y drivers		✓	✓	✓							✓		
Fase 3: Comunicaciones y clientes				✓	✓	✓	✓	✓	✓		✓		
Fase 4: Integración clínica								✓	✓	✓			
Fase 5: Validación y consumo											✓	✓	✓

2. Marco teórico y estado del arte

Este capítulo establece las bases teóricas y el estado del arte sobre los que se sustenta el desarrollo del dispositivo de monitorización hemodinámica. Se abordan, en este orden, la física de la fotopletismografía y la estimación de la saturación de oxígeno; el panorama de dispositivos existentes y el hueco que este Trabajo ocupa; la plataforma de ejecución, entendida como el microcontrolador y el sistema operativo de tiempo real que lo gobierna; el enlace Bluetooth Low Energy junto con la codificación y la protección de los datos que transporta; y, finalmente, los tres pilares metodológicos sobre los que descansa la aportación: el modelado del control mediante redes de Petri, su verificación formal por comprobación de modelos, y la validación de la implementación en el ordenador de desarrollo.

2.1. Fundamentos físicos de la fotopletismografía y oximetría de pulso

La fotopletismografía (PPG) es una técnica óptica no invasiva utilizada para medir variaciones en el volumen sanguíneo en los lechos tisulares microvasculares. Su principio de funcionamiento se basa en la iluminación de una zona de la piel y la medición de la cantidad de luz transmitida o reflejada por los tejidos hacia un fotodetector [2].

La atenuación de la luz al atravesar el tejido biológico está descrita por la ley de Beer-Lambert, que establece que la intensidad de la luz transmitida I a través de un medio absorbente disminuye exponencialmente con la concentración de los absorbentes C y la longitud del camino óptico d :

$$I = I_0 \cdot e^{-\epsilon \cdot C \cdot d} \quad (2.1)$$

donde I_0 es la intensidad de la luz incidente y ϵ es el coeficiente de extinción molar del absorbente a una longitud de onda específica.

En el tejido periférico, varios componentes contribuyen a la atenuación de la luz: la piel, los huesos, el tejido muscular, la sangre venosa y la sangre arterial. La señal de PPG capturada por el fotodetector consta de dos componentes principales [6]:

- **Componente continua (DC):** Representa la atenuación constante debida a los tejidos estáticos (huesos, piel, músculo) y el flujo constante de sangre venosa y capilar.
- **Componente alterna (AC):** Es una señal pulsátil que varía de forma sincrónica con el ciclo cardíaco, reflejando los cambios volumétricos de la sangre arterial con cada sístole y diástole.

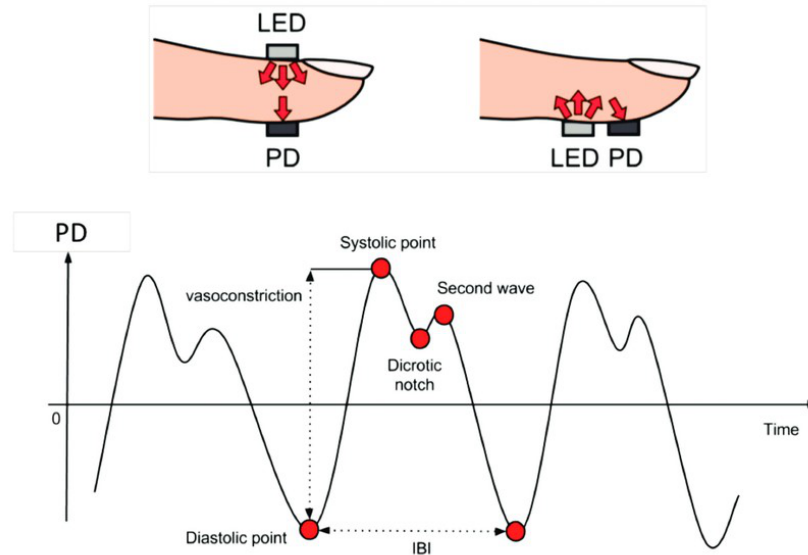


Figura 2.1: Modos de adquisición de la señal fotopletismográfica y morfología de la onda resultante. Arriba, las dos disposiciones posibles del emisor (LED) y el fotodetector (PD): por transmisión, con el tejido entre ambos, y por reflexión, con ambos en la misma cara. Abajo, la onda captada, en la que se distinguen el pico sistólico, la muesca dicrótica —producida por el cierre de la válvula aórtica— y el mínimo diastólico. El MAX30102 empleado en este Trabajo opera en modo de reflexión, que es el que permite integrar emisor y detector en un único encapsulado.

La pulsioximetría aprovecha esta señal pulsátil para estimar la saturación periférica de oxígeno (SpO_2), que representa la fracción de hemoglobina unida al oxígeno respecto a la hemoglobina total en sangre arterial:

$$SpO_2 = \frac{[HbO_2]}{[HbO_2] + [Hb]} \quad (2.2)$$

donde $[HbO_2]$ es la concentración de oxihemoglobina y $[Hb]$ es la concentración de hemoglobina desoxigenada (reducida).

Dado que HbO_2 y Hb tienen coeficientes de extinción molar significativamente diferentes a distintas longitudes de onda, se utilizan dos fuentes de luz en el sensor: una roja en torno a 660 nm y una infrarroja en la banda de 880 a 940 nm, cuyo valor exacto depende

del emisor concreto —880 nm en el MAX30102 empleado en este Trabajo—. Como se aprecia en la literatura clásica [7]:

- La hemoglobina desoxigenada (*Hb*) absorbe más luz en el espectro rojo (660 nm).
- La hemoglobina oxigenada (*HbO₂*) absorbe más luz en el espectro infrarrojo.

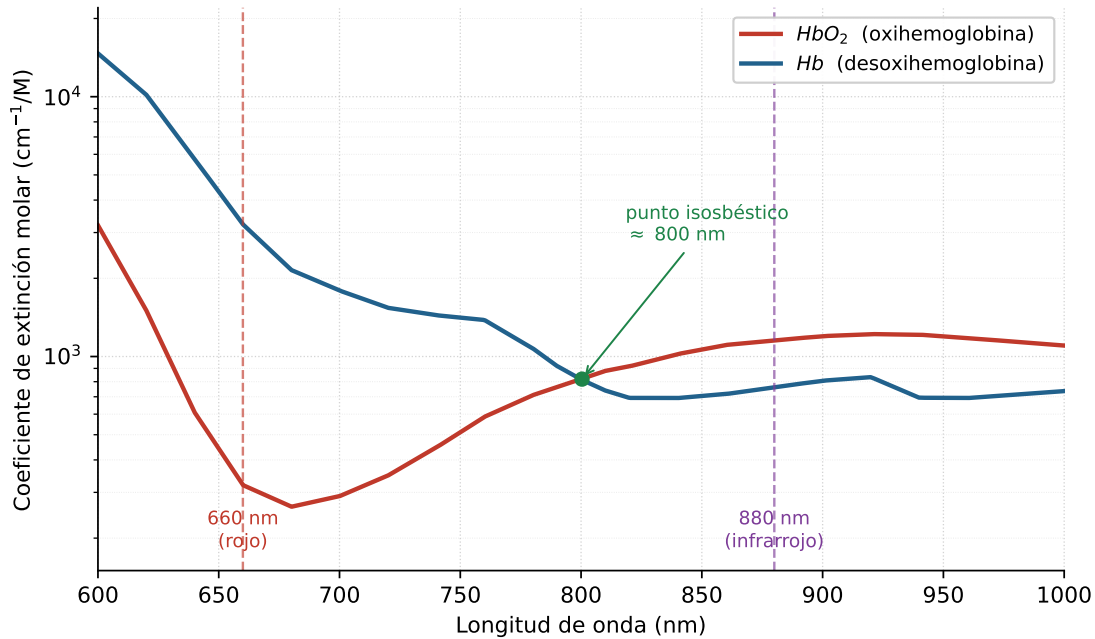


Figura 2.2: Coeficiente de extinción molar de la hemoglobina oxigenada y desoxigenada frente a la longitud de onda, en escala logarítmica. En 660 nm la desoxihemoglobina absorbe un orden de magnitud más que la oxihemoglobina, mientras que por encima del punto isobéptico —en torno a 800 nm, donde ambas curvas se cruzan y la absorción es independiente de la saturación— la relación se invierte. Esa inversión es la que hace posible el cociente de la Ecuación 2.3. Las curvas se han trazado a partir de los coeficientes tabulados recopilados por S. Prahl (Oregon Medical Laser Center), de dominio público.

Para normalizar las variaciones en la intensidad de los LEDs y la pigmentación de la piel, se calcula el denominado “ratio de ratios” (R) dividiendo las componentes AC y DC de cada canal óptico:

$$R = \frac{(AC_{red}/DC_{red})}{(AC_{ir}/DC_{ir})} \quad (2.3)$$

Una vez obtenido el valor de R , la saturación de oxígeno se calcula mediante una relación empírica lineal tomada de la documentación de referencia del fabricante para este sensor [8]. Conviene precisar su procedencia, porque condiciona lo que puede afirmarse del resultado: los coeficientes **no** proceden de una calibración experimental realizada en este Trabajo ni de un estudio de desaturación controlada, sino de esa referencia. La consecuencia se recoge entre las limitaciones del Capítulo 9:

$$SpO_2 = A - B \cdot R \quad (2.4)$$

donde A y B son constantes de calibración derivadas de estudios clínicos de desaturación, con valores típicos $A \approx 110$ y $B \approx 25$. Los valores concretos adoptados para el sensor empleado en este Trabajo, junto con la forma en que se estiman las componentes AC y DC a partir de la señal muestreada, se detallan en el Capítulo 6.

Conviene delimitar el alcance de esta medida antes de construir un sistema sobre ella. La estimación de SpO_2 por fotopletismografía es indirecta y descansa sobre hipótesis que la práctica no siempre respeta:

- **Perfusión suficiente:** la componente AC es dos órdenes de magnitud menor que la DC. Cuando la perfusión periférica cae —frío, vasoconstricción, hipotensión— la relación señal-ruido se degrada y el cociente R se vuelve inestable.
- **Ausencia de movimiento:** el desplazamiento relativo entre sensor y piel altera el camino óptico y genera artefactos cuyo contenido espectral se solapa con el de la propia señal cardíaca, lo que impide separarlos mediante un filtrado sencillo en frecuencia.
- **Pigmentación y grosor del tejido:** afectan a la componente DC de ambos canales. El cociente R está construido precisamente para cancelar ese efecto, pero la cancelación no es completa y la literatura documenta sesgos sistemáticos asociados a este factor.
- **Presencia de dishemoglobinas:** el modelo de dos longitudes de onda presupone que solo existen HbO_2 y Hb . La carboxihemoglobina absorbe en el rojo de forma similar a la oxihemoglobina, por lo que su presencia tiende a sobreestimar la saturación real.

Estas limitaciones no invalidan la técnica —es la base de la pulsioximetría clínica— pero delimitan su uso: un dispositivo de este tipo se concibe como herramienta de monitorización de tendencia y de detección de cambios, no como sustituto de una gasometría arterial.

2.2. Estado del arte: pulsioximetría conectada y dispositivos vestibles

Los fundamentos expuestos en la sección anterior son comunes a cualquier pulsioxímetro. Lo que distingue a unos sistemas de otros no es la física, sino las decisiones de ingeniería que se toman alrededor de ella: dónde se procesa la señal, cómo se transporta, qué garantías ofrece el firmware y qué ocurre cuando algo falla. Esta sección recorre esas decisiones en los sistemas existentes para delimitar con precisión qué aporta este Trabajo.

2.2.1. Del pulsioxímetro hospitalario al dispositivo vestible

La pulsioximetría clínica se consolidó en el entorno hospitalario a partir de los años ochenta como monitorización de cabecera: equipos certificados, calibrados contra cooximetría de laboratorio y conectados a la red eléctrica o a una estación de monitorización. Los modelos portátiles actuales, como el mostrado en la Figura 2.3, conservan esa precisión y añaden parámetros derivados —carboxihemoglobina y metahemoglobina— además de conectividad inalámbrica. Su limitación no es técnica sino de acceso: son dispositivos caros, de uso profesional, y su cadena de medida está cerrada al usuario.



Figura 2.3: Sistema multiparamétrico portátil de uso clínico con sensores de dedo y visualización de la onda PPG. Representa el extremo de precisión y coste del espectro de dispositivos.

En el extremo opuesto se sitúan los dispositivos vestibles de consumo. La miniaturización de los sensores ópticos integrados —emisor, fotodetector y convertidor en un único encapsulado de pocos milímetros— permitió incorporar fotopletimografía de reflexión a relojes y pulseras de actividad (Figura 2.4). Estos dispositivos aportan lo que falta al equipo clínico: continuidad, movilidad y coste bajo. A cambio, renuncian a la trazabilidad: el algoritmo que convierte la señal óptica en un número es propietario, la calibración no es auditable y el firmware no expone ninguna evidencia de su comportamiento interno.



Figura 2.4: Dispositivo vestible de consumo midiendo el ritmo cardíaco mediante fotopletimografía de reflexión en la muñeca.

Entre ambos extremos existe un espacio poco cubierto: dispositivos de bajo coste contruidos con componentes accesibles, pero cuyo firmware sea *auditable*, es decir, cuyo comportamiento pueda razonarse, demostrarse y examinarse tras un fallo. Es en ese espacio donde se sitúa este Trabajo.

2.2.2. El eslabón menos documentado: el software embebido

La literatura sobre pulsioximetría es abundante en el plano de la señal —modelos de absorción, algoritmos de estimación, corrección de artefactos de movimiento— y comparativamente escasa en el plano del *software* que ejecuta esa medida dentro del dispositivo. Los trabajos que describen prototipos de monitorización sobre microcontrolador suelen detallar con precisión el sensor, el algoritmo y la aplicación receptora, y tratar el firmware como un componente instrumental: se enuncia la máquina de estados que gobierna el ciclo de vida, pero rara vez se argumenta por qué es correcta, cómo se comprueba o qué ocurre cuando falla lejos del laboratorio.

La asimetría es comprensible, porque el interés científico se concentra en la medida. Pero deja sin cubrir tres cuestiones que resultan determinantes en cuanto el dispositivo abandona el banco de pruebas:

- **Corrección de la lógica de control.** Un dispositivo de monitorización es un sistema reactivo y concurrente: atiende al sensor, a la radio y a la batería a la vez, y los eventos llegan en cualquier orden. Afirmar que su control carece de bloqueos o de estados inalcanzables es una afirmación fuerte que la ejecución de pruebas no puede sostener por sí sola, porque las pruebas recorren los entrelazados que ocurren, no los que podrían ocurrir.
- **Diagnóstico posterior al fallo.** Cuando un dispositivo portátil se reinicia o se apaga en uso real, la información sobre la causa desaparece con la alimentación. Sin un registro persistente, el fallo solo puede reproducirse si vuelve a manifestarse, lo que rara vez sucede a demanda.
- **Capacidad de prueba sin hardware.** Un firmware acoplado directamente a los periféricos únicamente puede ejercitarse sobre la placa, lo que limita las pruebas a los escenarios que el hardware permite provocar. Los caminos de error —sensor ausente, bus caído, batería agotada— son precisamente los más difíciles de reproducir físicamente y los más importantes de verificar.

Este Trabajo se sitúa deliberadamente en ese eslabón. La aportación no consiste en mejorar la exactitud de la medida óptica, que depende del sensor y de su calibración, sino en elevar el nivel de garantía del *software* que la gobierna, y en hacerlo con métodos cuyo resultado sea comprobable por un tercero.

La decisión de diseño de la que se derivan las demás es el formalismo elegido para expresar el control. La lógica no se codifica como una estructura de decisión repartida por el firmware, sino como una **red de Petri interpretada** ejecutada por un motor dirigido por tablas sobre Zephyr RTOS. La elección tiene una consecuencia práctica que va más allá de la elegancia: el formalismo posee una semántica matemática —la que se desarrolla en la Sección 2.5— y el motor mantiene una correspondencia directa entre el modelo y el código que efectivamente se ejecuta. Esa correspondencia es la que permite someter el control a verificación automática (Sección 2.6) y ejercitarlo sin hardware (Sección 2.7). Las tres piezas no son independientes: se sostienen unas a otras, y ninguna tendría el mismo valor por separado.

Los objetivos concretos que se derivan de este planteamiento se detallan en la Sección 1.2.

2.2.3. Trabajos previos en esta línea

En el Departamento de Ingeniería Electrónica existe una línea previa sobre pulsioximetría en la que este Trabajo se inscribe, y respecto de la cual conviene situarlo con precisión. La preceden tres trabajos: un primer prototipo con componentes discretos y sin formalidad en la comprobación, de Andrés Moreno [9]; un segundo que diseña una PCB para el proyecto, de Esther Moreno [10]; y un tercero que prueba esa placa y desarrolla pruebas automáticas para ella, de Esther Ortega [11].

Se ha desarrollado de forma independiente: el *firmware*, la arquitectura de capas, el motor de control y la cadena de datos que aquí se describen se han escrito íntegramente para él, sobre hardware de referencia —el kit de desarrollo nRF52840 DK con el sensor MAX30102—. El diseño de una placa propia figura, de hecho, entre sus líneas futuras (Sección 9.2).

Su aportación diferencial se concentra en cuatro frentes: el **modelado formal del control** como red de Petri y su verificación con SPIN (Sección 2.6), el subsistema de **gestión de fallos** con persistencia en memoria no volátil, el **trazado temporal** sobre el hardware real y la **integración de la cadena de datos** hasta el registro clínico (Capítulo 7). El alcance y los límites de esa verificación formal se precisan en el Capítulo 4.

2.3. Arquitecturas de procesamiento y sistemas operativos de tiempo real

El diseño de dispositivos médicos portátiles impone severas restricciones de tamaño, peso y, especialmente, consumo de energía. La elección del microcontrolador y el software base es crítica para garantizar una larga autonomía de batería y un funcionamiento seguro en tiempo real [12].

2.3.1. El microcontrolador SoC Nordic Semiconductor nRF52840

El procesador seleccionado como plataforma de hardware para este trabajo es el chip **nRF52840** de Nordic Semiconductor [13]. Se trata de un *System on Chip* (SoC) multiprotocolo de consumo ultrabajo diseñado para aplicaciones médicas portátiles y dispositivos del Internet de las Cosas (IoT). La Tabla 2.1 resume sus especificaciones arquitectónicas y periféricos clave.

Parámetro / Subsistema	Descripción Técnica y Especificaciones
Núcleo de Procesamiento	ARM Cortex-M4F de 32 bits a 64 MHz con unidad de coma flotante (FPU) e instrucciones DSP integradas.
Subsistema de Memoria	1 MB de memoria Flash (NVMC) y 256 KB de memoria RAM con retención completa de contenido en bajo consumo.
Coprocesador Radio BLE	Transceptor de 2.4 GHz compatible con Bluetooth 5.0, Bluetooth Mesh, IEEE 802.15.4 (Thread/Zigbee) y sensibilidad de -96 dBm.
EasyDMA (Direct Memory Access)	Mecanismo de acceso directo a memoria sin intervención de la CPU para transmisión I ² C, SPI y BLE, reduciendo drásticamente el consumo del procesador.
Interconexión PPI	<i>Programmable Peripheral Interconnect</i> : permite conectar eventos de periféricos con tareas de otros periféricos por hardware sin despertar el núcleo Cortex-M4.
Convertidor Analógico (SAADC)	SAADC de 12 bits a 200 kSps con ganancia programable (1/6) y canal interno de supervisión de voltaje de batería.
Gestión de Energía	Convertidores DC/DC integrados y modos <i>System ON</i> y <i>System OFF</i> ; las cifras concretas de corriente dependen de la potencia de transmisión, de la tensión de alimentación y de la retención de RAM configurada, por lo que no se reproducen aquí.

Tabla 2.1: Especificaciones arquitectónicas y subsistemas clave del SoC nRF52840.

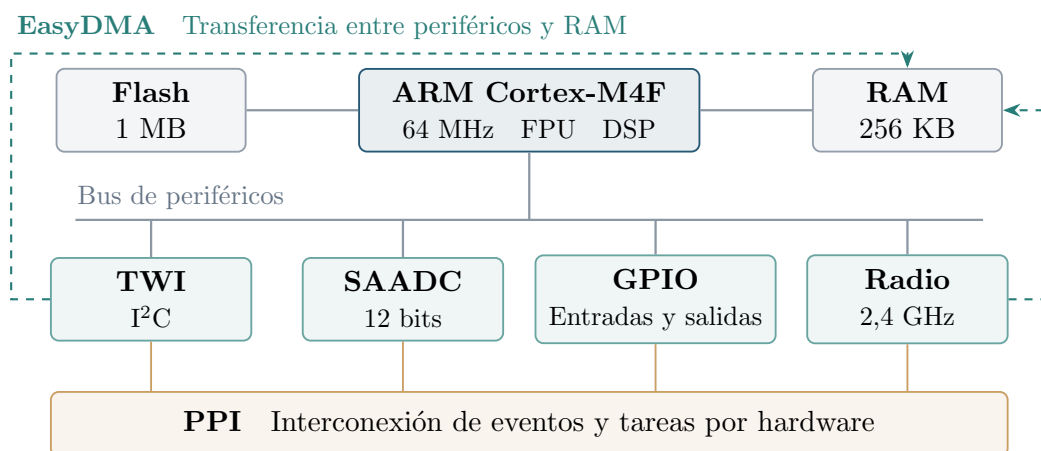


Figura 2.5: Bloques del SoC nRF52840 empleados en este Trabajo. Las rutas discontinuas representan EasyDMA, que transfiere datos entre periférico y RAM sin ocupar el núcleo; PPI enlaza eventos de un periférico con tareas de otro por hardware, sin despertarlo. Ambos mecanismos son los que permiten al SoC operar con la CPU detenida buena parte del tiempo, según muestran las trazas del Capítulo 8.

Entre las innovaciones clave del nRF52840 destaca el subsistema **EasyDMA**, que permite a los periféricos de comunicación (como el bus I²C del sensor MAX30102 y el controlador de radio BLE) transferir bloques de datos directamente a la memoria RAM sin sobrecargar la CPU. Asimismo, el sistema de interconexión periférica **PPI** (*Programmable Peripheral Interconnect*) permite desencadenar acciones en hardware (por ejemplo, iniciar la conversión del ADC) en respuesta a eventos de temporizadores o señales externas sin requerir

la ejecución de rutinas de interrupción (ISR), lo que minimiza la latencia del sistema y optimiza la eficiencia energética.

2.3.2. El ecosistema de desarrollo Zephyr RTOS

Para gestionar la concurrencia de la adquisición de muestras (100 Hz), el procesamiento numérico y la transmisión inalámbrica sin incurrir en retardos inaceptables, se hace necesario el uso de un Sistema Operativo de Tiempo Real (RTOS). Un RTOS permite construir una planificación temporal predecible mediante políticas de prioridad estricta [14]; que esa previsibilidad se traduzca en el cumplimiento efectivo de los plazos depende, además, de la asignación de prioridades, de los recursos compartidos entre tareas y de los tiempos de ejecución en el peor caso.

En este proyecto se utiliza **Zephyr RTOS**, un sistema operativo de tiempo real de código abierto apadrinado por la Fundación Linux, optimizado para microcontroladores restringidos en recursos. La Tabla 2.2 sintetiza las herramientas y subsistemas de Zephyr integrados en la solución desarrollada.

Subsistema / Herramienta	Función en la Arquitectura del Proyecto TFM
Planificador Multitarea	Planificación preemptiva basada en prioridades con soporte de hilos cooperativos, semáforos, colas de mensajes (<code>os_msgq</code>) y temporizadores de software.
DeviceTree (<code>app.overlay</code>)	Lenguaje de descripción de hardware que mapea jerárquicamente los pines I ² C, canales ADC (SAADC AIN0), LEDs GPIO y el nodo del sensor MAX30102.
Kconfig (<code>prj.conf</code>)	Sistema de configuración modular que permite habilitar la pila BLE, ajustar el tamaño del buffer MTU (247 bytes), corriente de pulso de cada LED ($0x2F \approx 9,4 \text{ mA}$) y optimizar el tamaño de las pilas.
Pila Bluetooth Low Energy	Pila de comunicaciones integrada de alta eficiencia con soporte para perfiles GATT/GAP personalizados, control de flujo por semáforos y notificaciones asíncronas.
Almacenamiento NVS	Subsistema <i>Non-Volatile Storage</i> [15] que implementa nivelación de desgaste (<i>wear leveling</i>) sobre la Flash interna para la persistencia del registro de fallos de la Caja Negra.
OSAL (<i>capa propia</i>)	Única fila de la tabla que no pertenece a Zephyr: capa desarrollada en este Trabajo que envuelve las primitivas anteriores tras una interfaz uniforme, de modo que el firmware pueda compilarse y ejercitarse también en el ordenador de desarrollo.

Tabla 2.2: Subsistemas de Zephyr RTOS empleados y, en la última fila, la capa propia construida sobre ellos.

La principal ventaja metodológica de Zephyr radica en la separación absoluta entre la descripción del hardware y la lógica del código:

- **DeviceTree:** Mediante el archivo de superposición `app.overlay`, se definen las

propiedades físicas de las interfaces (frecuencia del bus I²C, direcciones I²C de periféricos, canales ADC y asignación de pines GPIO).

- **Kconfig:** Mediante el archivo de configuración `prj.conf`, se activan o desactivan características del firmware en tiempo de compilación (como la habilitación del stack BLE, la memoria NVS, la compatibilidad con el driver `maxim,max30101` o el soporte de coma flotante en `printf`).

Esta arquitectura modular aísla la lógica de negocio del firmware de los registros específicos del silicio, garantizando la mantenibilidad, escalabilidad y portabilidad del código.

2.4. Comunicaciones inalámbricas de bajo consumo (Bluetooth Low Energy)

La conectividad inalámbrica del sensor se realiza a través de Bluetooth Low Energy (BLE), una variante del estándar Bluetooth diseñada específicamente para transmisiones periódicas de datos de bajo ancho de banda con un consumo mínimo de corriente [16].

La arquitectura de BLE está dividida en tres capas principales:

1. **Controlador (Controller):** Gestiona la capa física (modulación por desplazamiento de frecuencia GFSK en la banda ISM de 2.4 GHz) y la capa de enlace (control de acceso al medio y estados de publicidad y conexión).
2. **Host:** Define la pila de protocolos lógicos, incluyendo el protocolo de control y adaptación del enlace lógico (L2CAP) y el protocolo de atributos (ATT).
3. **Aplicación:** Utiliza los perfiles GAP (*Generic Access Profile*) para el descubrimiento y emparejamiento de dispositivos, y GATT (*Generic Attribute Profile*) para la estructuración y transmisión jerárquica de los datos.

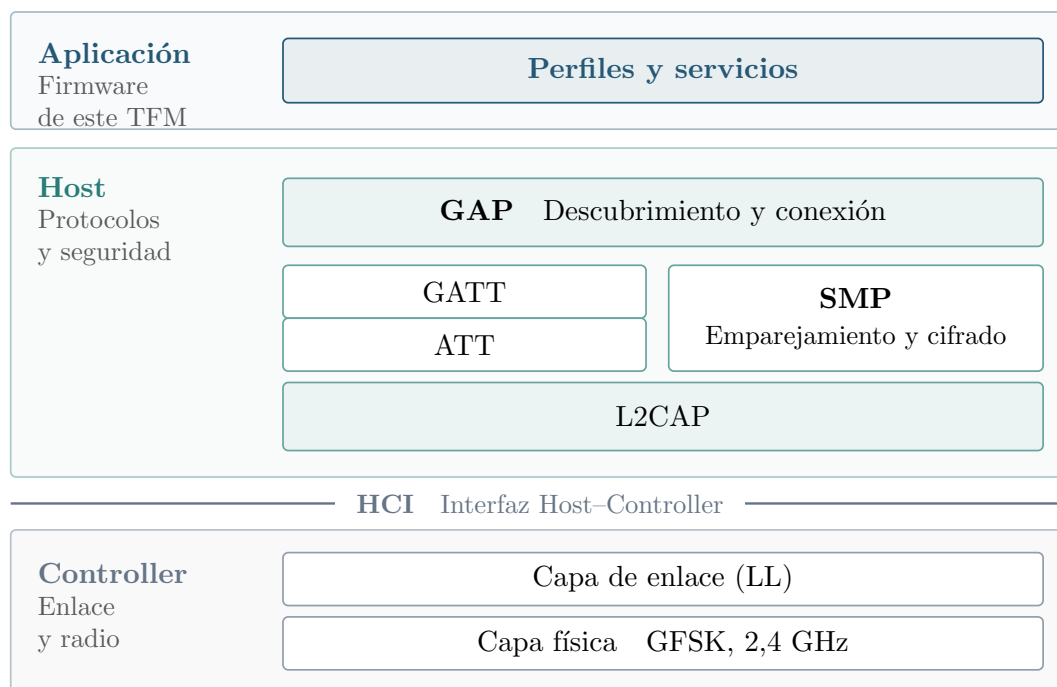


Figura 2.6: Arquitectura en capas de la pila Bluetooth Low Energy. GAP gobierna el descubrimiento y la conexión; GATT se apoya en ATT para estructurar los datos, y SMP opera en paralelo para el emparejamiento y el cifrado. La interfaz HCI separa el *Host* del *Controller* y es el punto por el que Zephyr permite alojarlos en procesadores distintos. El firmware desarrollado reside íntegramente en la capa de aplicación, sobre GATT.

2.4.1. Roles GATT: servidor y cliente

El protocolo GATT es asimétrico y define dos roles complementarios que conviene distinguir con precisión, porque este Trabajo implementa uno de ellos y sus aplicaciones terminales implementan el otro.

- El **servidor GATT** es el extremo que *posee* los datos y los organiza en una jerarquía de servicios, características y descriptores. No toma la iniciativa de la conexión ni decide cuándo se leen sus datos: se limita a exponerlos y a notificar cambios cuando el cliente lo ha solicitado. En este sistema, el servidor es el dispositivo embebido.
- El **cliente GATT** es el extremo que descubre la jerarquía del servidor, se suscribe a las características que le interesan y escribe en aquellas que aceptan comandos. En este sistema, los clientes son la aplicación móvil y la aplicación web.

Esta asimetría tiene una consecuencia de diseño que conviene subrayar: el firmware no debe presuponer nada sobre el cliente. Un cliente puede desconectarse sin avisar, suscribirse y no leer, o escribir comandos en un orden inesperado. El modelo de control descrito en la Sección 2.5 se hace cargo precisamente de esa incertidumbre, tratando cada acción del cliente como un evento externo cuya aceptación depende del estado actual del dispositivo.

2.4.2. El servicio GATT del dispositivo

La transferencia de las bioseñales PPG se realiza modelando el dispositivo embebido como un servidor GATT que expone un **Servicio Personalizado**. Este servicio expone cuatro características, identificadas por UUIDs de 128 bits derivados del UUID base del servicio:

- **Característica de Control (Lectura/Escritura):** Permite al cliente (aplicación Android o web) leer el estado actual del dispositivo y escribir los dos únicos comandos admitidos: iniciar (0x01) y detener (0x00) la transmisión de muestras. La superficie de escritura se mantiene deliberadamente mínima: cada comando adicional expuesto por radio es una vía de entrada que habría que validar y proteger. Por este motivo, la extracción de los informes de la caja negra no se realiza por radio sino por USB, decisión de diseño justificada en el Capítulo 5.
- **Característica de Datos (Notificación):** Utiliza el método de notificaciones (*notifications*) para enviar tramas de datos de forma asíncrona hacia el cliente sin necesidad de recibir una confirmación explícita en cada paquete, lo que minimiza el consumo energético de la radio. Para ello, el cliente debe habilitar previamente las notificaciones modificando el descriptor CCCD (*Client Characteristic Configuration Descriptor*) asociado.
- **Característica de Identificación (Lectura):** Expone un identificador único del dispositivo, necesario para que el cliente asocie inequívocamente cada flujo de muestras con la unidad física que las produce cuando varios dispositivos operan en la misma sala.
- **Característica de Temperatura (Lectura):** Devuelve la temperatura del encapsulado del sensor. No es un dato clínico: la respuesta espectral de los LEDs deriva con la temperatura, por lo que este valor permite al cliente descartar o corregir medidas tomadas fuera del rango de calibración.

2.4.3. Serialización compacta mediante Protocol Buffers

Exponer una característica no basta: hay que decidir cómo se codifican los datos que viajan por ella. La radio es, con diferencia, el subsistema que más energía consume, de modo que cada byte transmitido tiene un coste medible en autonomía. Una codificación textual como JSON resulta inadecuada en este contexto por su redundancia y por el coste de analizarla en un microcontrolador.

Protocol Buffers [17] resuelve este problema mediante un esquema declarado por separado del código: los campos se identifican por un número en lugar de por su nombre, los enteros se codifican con longitud variable de modo que los valores pequeños ocupan menos bytes, y los campos ausentes sencillamente no se transmiten. La implementación empleada, **nanopb** [18], está concebida para sistemas empotrados: genera código C sin asignación dinámica de memoria y con un consumo de pila acotado y conocido en tiempo de compilación.

La ventaja adicional, y no menor, es que el mismo esquema genera el código de decodificación para la aplicación móvil y para la aplicación web. El formato de las tramas deja así de ser un acuerdo implícito entre programas escritos en lenguajes distintos y pasa a ser un contrato único y versionado.

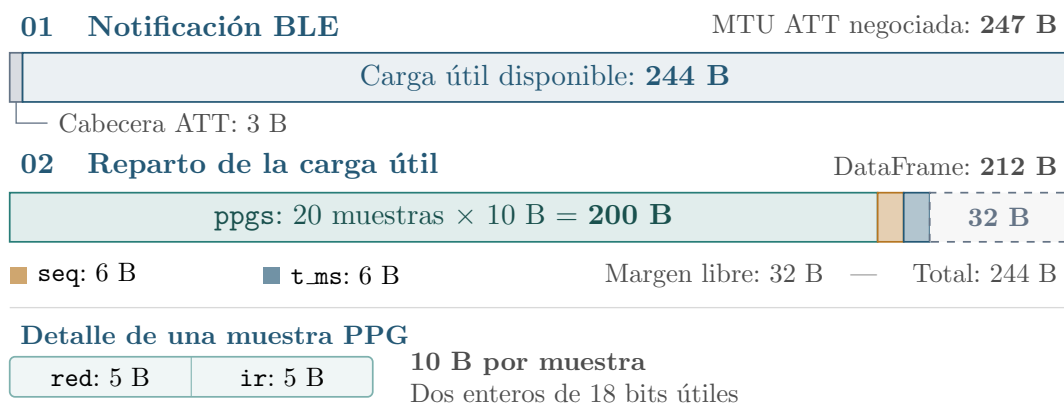


Figura 2.7: Reparto de la trama **DataFrame** dentro de la unidad máxima de transmisión. De los 247 bytes negociados, 3 corresponden a la cabecera del protocolo de atributos y 244 quedan disponibles. La trama ocupa 212, de modo que **cabe en una sola notificación** y no se fragmenta; los 32 bytes restantes son margen deliberado para admitir muestras que se codifiquen con más bytes de los típicos sin rebasar el límite. El tamaño del bloque es el que fija `max_count` en `simple.proto`, verificado en tiempo de compilación contra `SAMPLES_PER_FRAME`.

2.4.4. Sistemas de captura de datos de investigación clínica

Transmitir una medida es solo el primer tramo de su recorrido. Para que resulte útil en investigación, ese dato debe quedar asociado de forma inequívoca a un sujeto, a un instante y a un protocolo, y permanecer accesible y auditable años después. Ese cometido no lo resuelve una base de datos cualquiera, y ha dado lugar a una categoría propia de herramientas: los sistemas de captura electrónica de datos (*Electronic Data Capture*, EDC).

El más extendido en el ámbito académico es **REDCap** (*Research Electronic Data Capture*), desarrollado en la Universidad de Vanderbilt y utilizado por un consorcio internacional de instituciones de investigación. Su adopción no se explica por sus prestaciones técnicas —son modestas— sino por tres propiedades que un esquema relacional escrito a medida rara vez incorpora:

- **Trazabilidad de cambios.** Toda modificación queda registrada con su autor y su instante. En investigación clínica no basta con conocer el valor actual de una variable: hay que poder demostrar cuándo se introdujo y quién lo hizo.
- **Control de acceso por proyecto y por rol.** Los permisos se definen sobre el estudio, no sobre la base de datos, lo que permite que personal clínico, técnico e investigador vean subconjuntos distintos del mismo registro.
- **Interfaz programática.** Una API REST autenticada por token permite que los instrumentos escriban directamente en el estudio, sin transcripción manual, que es la fuente de error más frecuente en la captura de datos clínicos.

La consecuencia de diseño para un dispositivo de medida es que el sistema de captura ya existe y no debe reinventarse. El trabajo de integración consiste en traducir las magnitudes que produce el instrumento al vocabulario de variables que el estudio ya tiene definido, y en resolver la limitación práctica que estos sistemas comparten: están concebidos para variables escalares organizadas en formularios, no para series temporales de miles de

muestras. Cómo se resuelve esa tensión en este Trabajo se expone en el Capítulo 7.

2.4.5. Seguridad del enlace y sus implicaciones

Los datos que circulan por este enlace son señales fisiológicas asociadas a una persona, de modo que la seguridad del transporte no es un requisito accesorio. El estándar BLE ofrece los mecanismos necesarios a través del *Security Manager Protocol* (SMP): emparejamiento con intercambio de claves mediante Diffie-Hellman sobre curva elíptica, cifrado del enlace con AES-CCM, y almacenamiento de la relación de confianza (*bonding*) para que la reconexión no repita el proceso. Sobre ese cifrado, GATT permite además exigir un enlace autenticado como condición de acceso a cada característica individual.

Conviene distinguir dos cuestiones que se confunden con facilidad. El cifrado del enlace protege los datos frente a un tercero que escuche el canal radio, pero no identifica al interlocutor: sin autenticación, un emparejamiento del tipo *Just Works* es vulnerable a un intermediario activo. La autenticación requiere un canal fuera de banda —un código mostrado en pantalla, por ejemplo— del que un dispositivo sin interfaz de usuario no dispone. Es una restricción inherente al factor de forma, no una carencia de la pila.

El firmware desarrollado contempla esta política como un parámetro de compilación explícito (`APP_BLE_REQUIRE_ENCRYPTION`), que conmuta los permisos de todas las características entre acceso libre y acceso condicionado a un enlace cifrado. En la configuración final del dispositivo el mecanismo está **activado**: el emparejamiento se autentica mediante clave fija y las notificaciones no se emiten hasta que el enlace está cifrado, de modo que un cliente no emparejado no puede suscribirse a los datos. La alternativa —dejar el parámetro desactivado— se conservó únicamente durante las primeras campañas de trazado, para poder observar el enlace sin la negociación de seguridad; las trazas que se presentan en el Capítulo 8 corresponden ya a la configuración cifrada. El detalle de la implementación se desarrolla en la Sección 3.2.5 y sus implicaciones éticas se retoman en el Anexo A.

2.5. Modelado formal de concurrencia y robustez mediante redes de Petri

En los sistemas embebidos médicos de alta fiabilidad, la máquina de control que gobierna el ciclo de vida del dispositivo no puede presentar comportamientos indeterministas, bloqueos lógicos (*deadlocks*) o condiciones de carrera. Una máquina de estados finitos describe el sistema como un único punto de control: cada combinación de condiciones concurrentes —enlace de radio activo, sensor en fallo, batería baja— exige enumerar un estado distinto, de modo que el número de estados que el diseñador debe escribir crece como el producto de las condiciones independientes. Una red de Petri distribuye en cambio el control entre varios lugares que pueden estar marcados a la vez, y representa esa misma situación mediante un marcado en lugar de un estado enumerado explícitamente [19, 5].

Conviene precisar el alcance de esta ventaja, porque se malinterpreta con frecuencia. La red es más compacta como *descripción*, pero su grafo de alcanzabilidad puede crecer con la misma rapidez, y la exploración exhaustiva que se presenta en la Sección 2.6 se enfrenta exactamente al mismo problema combinatorio. Lo que el formalismo aporta no es la

desaparición de esa complejidad, sino una semántica matemática precisa y una separación limpia entre topología, guardas y acciones. Es esa semántica, y no la compacidad, la que hace posible el análisis automático del modelo.

2.5.1. Definición matemática formal

Una Red de Petri es una estructura matemática bipartita y dirigida que formaliza la concurrencia y sincronización de sistemas discretos. Matemáticamente, se define como una 5-tupla:

$$N = (P, T, F, W, M_0) \quad (2.5)$$

donde:

- $P = \{p_1, p_2, \dots, p_n\}$ es un conjunto finito y no vacío de nodos denominados **Lugares** (representados gráficamente por círculos).
- $T = \{t_1, t_2, \dots, t_m\}$ es un conjunto finito y no vacío de nodos denominados **Transiciones** (representados gráficamente por rectángulos), tal que $P \cap T = \emptyset$.
- $F \subseteq (P \times T) \cup (T \times P)$ es el conjunto de arcos dirigidos que conectan lugares con transiciones y viceversa. Los arcos son estrictamente bipartitos; no se permite la conexión directa entre dos lugares o dos transiciones.
- $W : F \rightarrow \mathbb{N}^+$ es la función de peso de los arcos, que asigna un valor entero positivo a cada arco en el grafo. Si el peso no se indica de forma explícita, se asume $W(f) = 1$.
- $M_0 : P \rightarrow \mathbb{N}$ es el **Marcado Inicial**, una función que asigna un número entero no negativo de marcas o fichas (*tokens*) a cada lugar del conjunto P (representadas por pequeños círculos negros). El marcado actual de la red en un instante dado se denota por el vector columna M .

Para cualquier transición $t \in T$, se define su conjunto de entrada (pre-set) y su conjunto de salida (post-set) como:

$$\bullet t = \{p \in P \mid (p, t) \in F\}, \quad t^\bullet = \{p \in P \mid (t, p) \in F\} \quad (2.6)$$

De forma análoga, para cualquier lugar $p \in P$, su conjunto de entrada y salida se denotan por $\bullet p$ y $p^\bullet$ respectivamente.

2.5.2. Regla de disparo y dinámica de la red

El comportamiento dinámico de una Red de Petri se rige por la habilitación y el disparo de sus transiciones:

1. **Habilitación (Enabling):** Una transición $t \in T$ se considera habilitada bajo un marcado actual M si y solo si cada lugar de entrada $p \in \bullet t$ contiene un número de marcas mayor o igual al peso del arco dirigido que los conecta:

$$M(p) \geq W(p, t) \quad \forall p \in \bullet t \quad (2.7)$$

2. **Disparo (Firing):** Si una transición t está habilitada, puede dispararse. El disparo de t es una operación atómica que altera el marcado de la red, transformando el

marcado M en un nuevo marcado M' de acuerdo con la siguiente regla de transición:

$$M'(p) = M(p) - W(p, t) + W(t, p) \quad \forall p \in P \quad (2.8)$$

Es decir, el disparo de una transición consume marcas de sus lugares de entrada y deposita marcas en sus lugares de salida.

La Figura 2.8 muestra de forma gráfica una Red de Petri básica que ilustra la aplicación práctica de estas reglas.

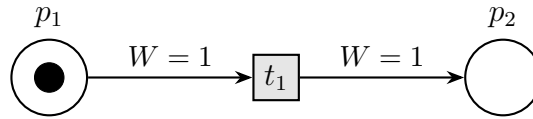


Figura 2.8: Grafo bipartito de una red de Petri básica con un marcado inicial $M_0 = [1, 0]^T$. La transición t_1 se encuentra habilitada debido a la presencia de la marca en p_1 .

2.5.3. Propiedades de comportamiento

El análisis matemático formal de las Redes de Petri permite certificar propiedades clave de seguridad antes de implementar el firmware:

- **Acotación (Boundedness):** Una red se considera acotada si el número de marcas en cualquier lugar de la red nunca supera un límite entero K para cualquier marcado alcanzable desde M_0 . Si $K = 1$, la red se denomina **segura** (*safe*). Esta propiedad es fundamental en sistemas embebidos, ya que garantiza que el número de marcas de cada lugar modelado se mantiene acotado; no implica, por sí sola, la ausencia de desbordamiento en estructuras que la red no representa —como un *buffer* de datos o el contenido de la memoria—, salvo que esas estructuras se modelen también como lugares con su propia capacidad.
- **Vivacidad (Liveness):** Una red es viva si, desde cualquier marcado alcanzable, siempre es posible disparar cualquier transición mediante una secuencia de disparos adecuada. Esto asegura la ausencia de bloqueos permanentes (*deadlocks*) en la parte operativa de la red. Conviene precisar que la red de este Trabajo no es viva en ese sentido estricto, y deliberadamente: el sueño profundo es un estado terminal del que sólo se sale por un despertar del hardware, de modo que lo que se verifica es la ausencia de bloqueos *no deseados* en los estados de servicio, no una actividad perpetua.
- **Alcanzabilidad (Reachability):** Determina si un marcado específico M_x puede ser obtenido a partir de M_0 mediante una secuencia de disparo. Permite verificar formalmente que un estado previsto —por ejemplo, el de error o el de sueño profundo— es alcanzable desde el arranque, es decir, que el diseño no contiene ramas muertas. Demostrar lo contrario —que un estado indeseado *nunca* se alcanza— no es una propiedad de alcanzabilidad sino de seguridad, y exige recorrer exhaustivamente el espacio de estados comprobando que el invariante se mantiene en todos ellos.

El análisis estructural clásico de estas propiedades —árbol de cobertura, invariantes de lugar y de transición— se aplica a la red desnuda, sin las guardas ni los eventos externos que la gobiernan en un sistema real. Como la red que aquí se implementa es interpretada, sus propiedades dependen tanto de la topología como de las condiciones lógicas asociadas

a cada transición, y el análisis estructural resulta insuficiente por sí solo. Por ese motivo la comprobación se aborda mediante exploración exhaustiva del espacio de estados, técnica que se introduce en la Sección 2.6.

2.5.4. Redes de Petri Interpretadas

Para el control directo de dispositivos físicos en tiempo real, se extiende el formalismo clásico introduciendo las **Redes de Petri Interpretadas (RPI)**. En una RPI, el disparo de las transiciones y las consecuencias del marcado están acopladas con las señales físicas del sistema embebido:

- **Eventos y Guardas lógicas:** Una transición en una RPI solo puede dispararse si está habilitada matemáticamente por el marcado, si ocurre un evento externo asíncrono (como la recepción de un comando BLE) y si se evalúa como verdadera una condición lógica de guarda (como un flag de estado atómico `can_stream == true`).
- **Acciones del Marcado:** Cada lugar p_i tiene asociada una acción de salida física (tipo máquina de Moore). La presencia de una marca en un lugar activa inmediatamente una función del firmware (por ejemplo, cambiar la coloración del LED físico del dispositivo, o invocar las rutinas de suspensión profunda de la CPU al marcar el lugar de reposo).

La ejecución de la red se delega en `rxnet` [20], un motor en C desarrollado por J. M. Moya en la Universidad Politécnica de Madrid, incorporado al firmware de este Trabajo bajo los términos de la licencia MIT que declaran sus ficheros fuente. El motor implementa la semántica de las RPI. En cada ciclo el motor recorre las transiciones en el orden fijo en que fueron declaradas, evalúa marcado y guarda, y dispara *todas* las que resultan habilitadas, aplicando el efecto de cada disparo antes de examinar la siguiente. Esa aplicación inmediata es la que convierte el orden de declaración en un orden de prioridad: entre transiciones que compiten por las mismas marcas, la declarada antes las consume y deja inhabilitadas a las posteriores; las que no compiten pueden dispararse en el mismo ciclo. El coste de un ciclo queda así acotado por el número de transiciones de la red, conocido en tiempo de compilación, y el motor no realiza ninguna asignación de memoria dentro del ciclo de control: las estructuras del marcado se reservan una sola vez en `rx_pn_net_init()` durante el arranque, y a partir de ahí la ejecución opera sobre memoria ya asignada. Ambas propiedades son las que permiten acotar el tiempo de respuesta del lazo de control, requisito ineludible en un dispositivo de uso médico.

2.6. Verificación formal mediante comprobación de modelos

La sección anterior ha enumerado las propiedades que interesa garantizar en la red de control —acotación, vivacidad, alcanzabilidad— pero no ha dicho cómo se demuestran. Esta sección cubre esa laguna, porque la diferencia entre afirmar que un sistema es correcto y demostrarlo es justamente la aportación metodológica de este Trabajo.

Probar un sistema concurrente ejecutándolo tiene un límite conocido: las pruebas recorren las trazas que el planificador decide producir, y los fallos de concurrencia suelen esconderse en los entrelazados que no se dieron. La *comprobación de modelos* (*model checking*) [21] invierte el planteamiento: en lugar de ejecutar el sistema, se construye un modelo

finito de él y se explora *exhaustivamente* el conjunto de sus estados alcanzables, comprobando en cada uno si la propiedad exigida se sostiene. Si existe una traza que la viola, el procedimiento la encuentra y la devuelve; si no la encuentra, es que no existe dentro del modelo.

2.6.1. Lógica temporal lineal: seguridad y vivacidad

Las propiedades no se expresan en lenguaje natural sino en **lógica temporal lineal** (LTL) [22], que extiende la lógica proposicional con operadores sobre la evolución del sistema en el tiempo. Los dos fundamentales son:

- $\Box\varphi$ (*always*): la propiedad φ se cumple en todo instante de toda ejecución.
- $\Diamond\varphi$ (*eventually*): la propiedad φ se cumple en algún instante de toda ejecución.

Esta distinción separa las dos familias clásicas de propiedades. Una propiedad de **seguridad** (*safety*) afirma que algo malo no ocurre nunca y tiene la forma $\Box\neg\varphi$; su contraejemplo es una traza finita que termina en el estado prohibido. Una propiedad de **vivacidad** (*liveness*) afirma que algo bueno acaba ocurriendo y tiene la forma $\Box(\varphi \rightarrow \Diamond\psi)$; su contraejemplo no es un estado sino un ciclo infinito en el que el sistema gira indefinidamente sin que ψ llegue a cumplirse.

La distinción es operativa, no académica. Que el dispositivo no transmita jamás estando en estado de error es una propiedad de seguridad. Que un error fatal conduzca siempre, antes o después, al estado de error es una propiedad de vivacidad. Ambas son necesarias y ninguna implica la otra.

2.6.2. El comprobador de modelos SPIN y el lenguaje Promela

La herramienta empleada es **SPIN** [23], comprobador de modelos orientado específicamente a la verificación de software concurrente. El modelo se describe en **Promela**, un lenguaje de procesos comunicantes cuya semántica de guardas y selección no determinista se corresponde de forma natural con las transiciones guardadas de una red de Petri interpretada.

El procedimiento de verificación consiste en traducir la negación de la propiedad LTL a un autómata de Büchi, componerlo con el autómata del modelo y comprobar si el producto acepta alguna traza. Una traza aceptada es un contraejemplo: la secuencia concreta de eventos que viola la propiedad. Esta capacidad de devolver el contraejemplo, y no un simple veredicto negativo, es lo que convierte al método en una herramienta de diseño y no solo de validación. Su aplicación concreta al modelo de control de este Trabajo se desarrolla en el Capítulo 4.

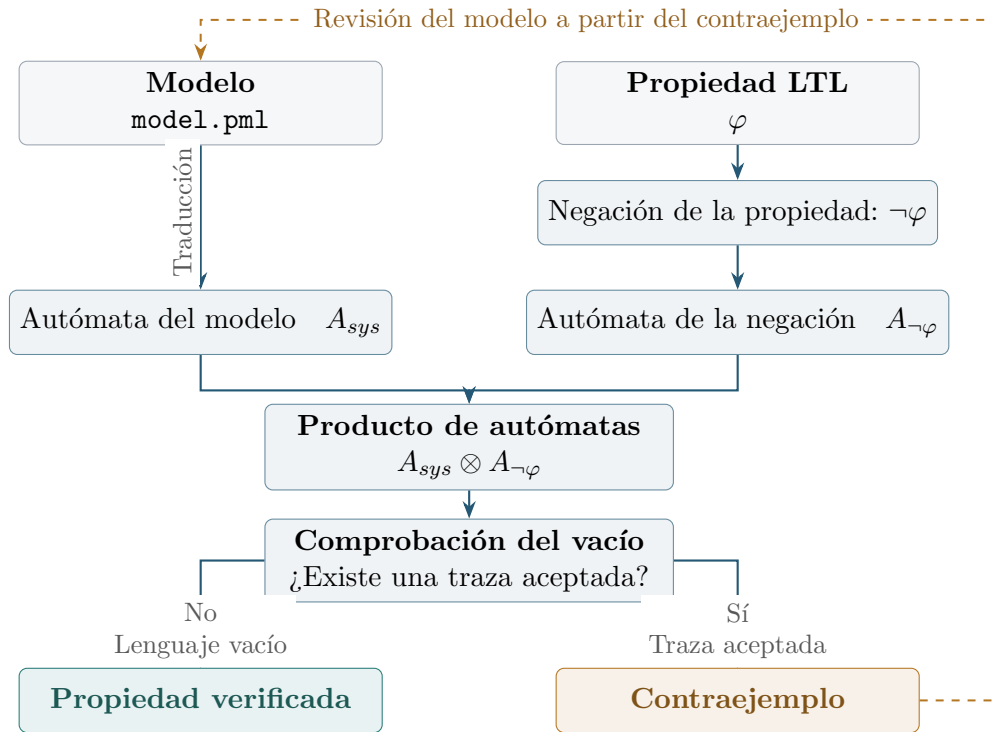


Figura 2.9: Flujo de trabajo de la comprobación de modelos. Ambas entradas se traducen a autómatas de Büchi: el modelo a A_{sys} y la *negación* de la propiedad a $A_{¬φ}$. Su producto acepta exactamente las ejecuciones del sistema que violan la propiedad, de modo que el veredicto se reduce a comprobar si ese lenguaje es vacío. Cuando no lo es, la traza aceptada constituye el contraejemplo, y la realimentación discontinua refleja el uso efectivo de la herramienta: ese contraejemplo corrige el modelo y se vuelve a verificar.

2.6.3. Explosión de estados y equidad

El método tiene un coste: el número de estados del producto crece de forma combinatoria con el número de procesos y de variables. Este fenómeno, conocido como **explosión de estados**, es el factor que determina si una verificación es viable en la práctica. Su mitigación no es únicamente una cuestión de recursos de cómputo, sino sobre todo de *cómo se escribe el modelo*: un modelo que refleja las restricciones reales del sistema — por ejemplo, que las transiciones se evalúan en un orden de prioridad determinado y no en cualquier orden— explora un espacio sensiblemente menor que uno que permite entrelazados que el sistema real nunca producirá.

Existe además una condición necesaria para verificar propiedades de vivacidad: la **equidad** (*fairness*). Sin ella, el comprobador considera válida una ejecución en la que un proceso listo para avanzar no avanza nunca, y prácticamente cualquier propiedad de vivacidad resulta refutada por un contraejemplo que no se corresponde con ningún comportamiento físico posible. SPIN implementa equidad débil a nivel de proceso, lo que obliga a tener presente qué garantiza exactamente esa hipótesis al interpretar los resultados.

2.7. Validación de firmware embebido en entorno de anfitrión

La verificación formal actúa sobre un modelo, no sobre el código. Garantiza que el modelo de la lógica de control satisface las propiedades enunciadas, pero no que su implementación en C lo haga, ni que las propiedades enunciadas agoten lo que significa «ser correcto». Ambos niveles son complementarios y ninguno sustituye al otro.

2.7.1. Inversión de dependencias y dobles de prueba

El obstáculo para probar firmware es su acoplamiento al hardware: una función que lee un registro del sensor no puede ejecutarse en un ordenador de desarrollo. La solución estándar es la **inversión de dependencias**: la lógica no invoca directamente las primitivas del sistema operativo ni los periféricos, sino una interfaz abstracta —la capa OSAL descrita en la Tabla 2.2— que en el dispositivo se resuelve contra Zephyr y en el ordenador de desarrollo se resuelve contra sustitutos.

Estos sustitutos, denominados genéricamente **dobles de prueba**, no son todos equivalentes. Un *stub* devuelve un valor fijo y permite recorrer una rama concreta. Un *fake* implementa una versión simplificada pero funcional del componente. Un *mock* va más allá: registra las llamadas que recibe y permite afirmar no solo qué resultado se obtuvo, sino *qué hizo* el código bajo prueba, lo que resulta imprescindible cuando el efecto observable es una acción sobre el hardware —apagar la radio, dormir el sensor— y no un valor de retorno. La ejecución de las pruebas y la generación automática de estos dobles a partir de las cabeceras [24, 25, 26] evita que la infraestructura de pruebas se degrade a medida que las interfaces evolucionan.

Sobre esta base, la **cobertura de código** proporciona una medida objetiva de qué porción de la lógica llegan a ejercitar las pruebas. Conviene interpretarla con cautela: una cobertura alta no demuestra la ausencia de defectos, pero una cobertura baja sí demuestra que existe código cuyo comportamiento nadie ha comprobado nunca.

2.7.2. Observabilidad: trazado del núcleo

Ni la verificación formal ni las pruebas unitarias responden a una pregunta que solo el hardware puede contestar: qué ocurre realmente en el tiempo. Cuándo despierta cada hilo, cuánto ocupa la CPU, si una tarea se retrasa cuando la radio transmite.

Para responderla se recurre al **trazado** del núcleo. Zephyr instrumenta los puntos de decisión del planificador —cambios de contexto, adquisición de semáforos, activación de temporizadores— y emite eventos en formato CTF (*Common Trace Format*), una codificación binaria compacta diseñada para minimizar la perturbación que la propia medida introduce en el sistema medido. Los eventos así capturados se convierten después a un formato de visualización que permite examinar la línea temporal de cada hilo y medir su ocupación real, cerrando el ciclo entre el modelo verificado, el código probado y el sistema en ejecución. Los resultados de estos dos últimos niveles se recogen en el Capítulo 8.

3. Diseño e implementación del firmware

Este capítulo detalla la ingeniería del dispositivo embebido. Comienza con el análisis de los requisitos funcionales, de robustez y de las restricciones de diseño; continúa con la arquitectura por capas del firmware y con el modelado de su ciclo de vida mediante una red de Petri interpretada, cuya tabla de transiciones se documenta de forma exhaustiva; y dedica la parte central al sistema de detección y gestión de fallos, organizado en cinco capas de defensa y rematado por el subsistema de diagnóstico persistente. Salvo indicación contraria, todos los identificadores, constantes y valores numéricos que aparecen en el capítulo se corresponden literalmente con el código fuente del firmware.

3.1. Análisis de requisitos y restricciones del sistema

El diseño del dispositivo de monitorización hemodinámica médica y sus aplicaciones asociadas responde a un conjunto estricto de especificaciones funcionales, de robustez y restricciones físicas.

3.1.1. Requisitos funcionales (RF)

RF-1 (Adquisición de señal PPG): El sistema debe adquirir de forma continua muestras de reflectancia luminosa roja e infrarroja del sensor MAX30102 a una frecuencia nominal de muestreo de 100 Hz ($\Delta t = 10$ ms). El criterio de aceptación es que el periodo medio de una trama de 20 muestras permanezca dentro de la tolerancia de $\pm 5\%$ respecto a los 200 ms nominales.

RF-2 (Transmisión inalámbrica): Los datos capturados deben transmitirse en tiempo real, con política *best-effort*, mediante notificaciones Bluetooth Low Energy (BLE) hacia un dispositivo móvil o PC. El sistema no garantiza la entrega de cada trama; las pérdidas deben ser detectables en el cliente mediante saltos en el número de secuencia `seq` y mediante los mecanismos diagnósticos disponibles.

RF-3 (Serialización eficiente): Las muestras de PPG deben empaquetarse y codificarse usando un protocolo de serialización binario ligero (Protocol Buffers) para reducir el ancho de banda y el consumo de la radio.

RF-4 (Interfaz de usuario local): El dispositivo debe proporcionar indicaciones visuales de su estado interno mediante un LED RGB, empleando cinco combinaciones cromáticas distinguibles (blanco, rojo, verde, violeta y azul) más el apagado.

RF-5 (Interfaz gráfica de usuario): El sistema debe incluir aplicaciones terminales capaces de recibir, decodificar, filtrar digitalmente y representar gráficamente las

ondas PPG en tiempo real, estimando el valor de SpO_2 y frecuencia cardíaca.

3.1.2. Requisitos de robustez y tolerancia a fallos (RR)

RR-1 (Tolerancia a fallos de hardware): El dispositivo debe ser capaz de detectar y recuperarse de errores físicos temporales del sensor en el bus I^2C . Si el error persiste durante tres ciclos consecutivos, debe declararse un estado de error fatal. La palabra *consecutivos* es la exigente: una lectura correcta rompe la racha y devuelve el contador a cero, de modo que tres fallos aislados repartidos a lo largo de una sesión —por ejemplo, tres movimientos del paciente— no deben provocar el apagado.

RR-2 (Detección de desconexión silenciosa): El firmware implementa una arquitectura de seguridad multinivel para detectar la pérdida imprevista del enlace BLE (por falta de cobertura o cierre abrupto de la aplicación cliente). En primer lugar, un temporizador de aplicación (*Software Barrier*) supervisa la **finalización local de las notificaciones**: cada vez que el controlador de radio da por terminado un envío se refresca una marca de tiempo, y si transcurren **2000 ms** sin que ningún envío se complete, se fuerza la transición segura al estado de publicidad (*advertising*). Conviene precisar el alcance de esta detección, porque delimita lo que el sistema puede y no puede garantizar: las notificaciones GATT **no llevan acuse de recibo** —solo lo llevan las indicaciones—, de modo que lo que se supervisa es que la radio local siga aceptando y despachando tramas, no que la aplicación cliente las esté recibiendo. Un cliente que cerrase la aplicación manteniendo abierto el enlace de transporte no sería detectado por esta vía; esa comprobación exigiría un *heartbeat* de nivel de aplicación, que se recoge como línea futura. Como segundo nivel de protección, el perro guardián por hardware (*Hardware Watchdog* – WDT) configurado a **5000 ms** actúa como salvaguarda crítica: si el sistema sufre un bloqueo grave, el WDT fuerza un reinicio completo del SoC para garantizar la auto-recuperación del dispositivo.

RR-3 (Registro de fallos post-mortem): El sistema debe incluir una *Caja Negra* persistente en memoria Flash no volátil (NVS) que registre, antes de un reinicio por fallo, no solo el código y el estado en que se produjo, sino el **contexto que condujo a él**: la secuencia de incidencias previas, los contadores acumulados de salud del equipo y el tiempo de funcionamiento. El registro debe ser verificable en integridad y la **Caja Negra no debe vaciarse nunca de forma automática**: conserva en todo momento los **cuatro informes más recientes**, y sólo al añadir un quinto cede su sitio el más antiguo. Esa rotación es el único desplazamiento automático admisible y es el precio de una memoria finita; el borrado *completo* exige siempre una orden explícita de mantenimiento, incluido el caso en que la partición resulte ilegible al arrancar, y esa reinicialización debe quedar a su vez registrada.

RR-4 (Protección de batería LiPo): El dispositivo debe monitorizar constantemente el voltaje de alimentación mediante un ADC. La evaluación sólo se realiza cuando el equipo funciona con batería: si se detecta alimentación externa por USB, la medida se registra pero no se juzga, porque sin celda conectada el canal del ADC queda flotando y su lectura no representa ninguna batería real. Con el equipo alimentado por batería, una lectura por debajo de 2000 mV es físicamente imposible para una celda de litio con circuito de protección, y se trata como ausencia de batería o fallo de la línea de medida, no como agotamiento: se registra y no se apaga. Entre 2000 y 3200 mV se considera batería baja real: el dispositivo debe apagar todos los periféricos y entrar en modo de suspensión profunda para evitar daños irreversibles

en la celda. Si el canal ADC no llega a configurarse, el dispositivo **no entra en servicio**: la justificación se desarrolla en la Sección 3.2.2.

3.1.3. Restricciones de diseño (RS)

- **Hardware:** Microcontrolador SoC nRF52840 (ARM Cortex-M4F) y sensor fotopletismográfico MAX30102.
- **Software base:** Zephyr RTOS como sistema operativo de tiempo real y la biblioteca rxnet [20] —desarrollada por J. M. Moya (Universidad Politécnica de Madrid) y utilizada bajo licencia MIT— como motor de ejecución de la red de Petri.
- **Eficiencia energética:** Autonomía mínima de 24 horas de transmisión continua con una batería de polímero de litio (LiPo) de 500 mAh.

3.2. Diseño e implementación de la arquitectura del firmware

La arquitectura del software embebido se ha diseñado con un enfoque modular y por capas para desacoplar el hardware físico de la lógica de negocio, lo que facilita enormemente la mantenibilidad y testabilidad del código fuente.

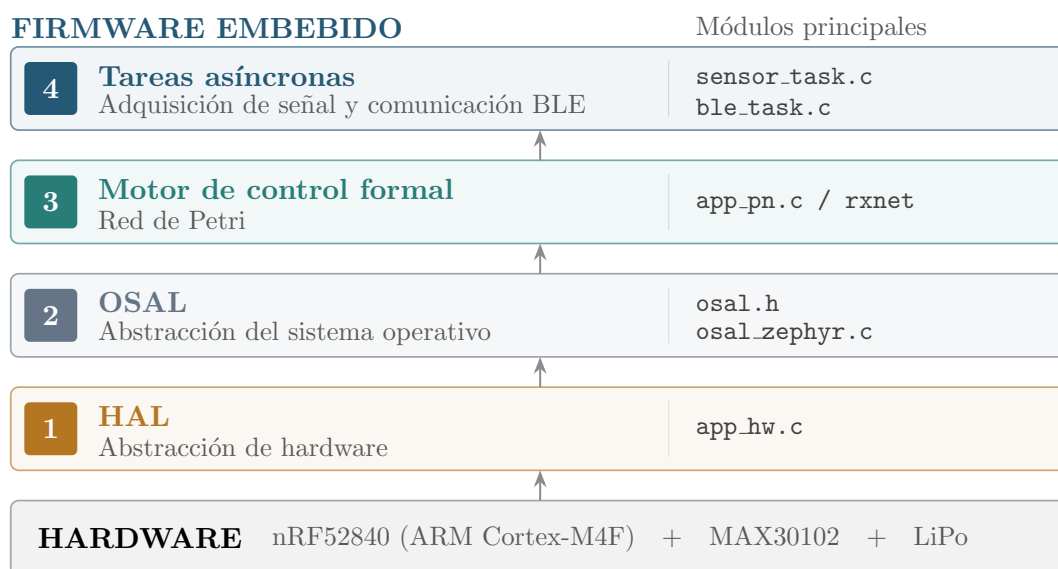


Figura 3.1: Arquitectura por capas del firmware embebido. Cada capa se apoya únicamente en la inmediatamente inferior, de modo que sustituir la capa OSAL por sustitutos permite ejecutar las capas 3 y 4 en el ordenador de desarrollo.

El firmware está compuesto por cuatro niveles principales:

1. **Capa de Hardware y Controladores (HAL):** Gestiona la interacción directa con el hardware. El archivo `app_hw.c` proporciona las rutinas para la configuración de los pines GPIO para LEDs, la lectura analógica de la batería mediante ADC, la inicialización y alimentación del perro guardián (*Watchdog Timer* WDT), la configuración del sensor MAX30102 a través de I^2C y las funciones de escritura/lectura en la memoria flash mediante el módulo NVS de Zephyr. Cabe destacar que Zephyr

RTOS no dispone de un controlador específico para el MAX30102; en su lugar se emplea el driver `maxim_max30101`, que es compatible con ambas variantes de la familia MAX3010x y se obtiene del *Device Tree* mediante `DT_COMPAT_GET_ANY_STATUS_OKAY(maxim_max30101)`, que selecciona el único nodo compatible declarado como operativo en `app.overlay`.

2. **Capa de Abstracción del Sistema Operativo (OSAL):** Definida en `osal.h` e implementada en `osal_zephyr.c`. Ofrece un envoltorio genérico para los servicios del kernel: hilos, semáforos, colas de mensajes, temporizadores y operaciones atómicas. Durante las pruebas unitarias en el ordenador host, esta capa redirige las llamadas a simuladores locales, evitando la necesidad de tener el microcontrolador físico.
3. **Motor de Control Formal (Red de Petri):** Implementado en `app_pn.c` utilizando la biblioteca `rxnet`. Controla la lógica de transición de estados de forma determinista y segura.
4. **Módulos de Tareas (Firmware Core):** Comprende las tareas asíncronas principales: la tarea de lectura del sensor (`sensor_task.c`) que muestrea el MAX30102, y la tarea de conectividad (`ble_task.c`) que gestiona la pila Bluetooth y el envío de tramas codificadas en Protocol Buffers a través del archivo `proto_codec.c`.

3.2.1. Modelado formal de estados mediante red de Petri reactiva

Para la gestión lógica del ciclo de vida del firmware, se ha descartado el uso de máquinas de estado tradicionales (FSM) basadas en estructuras condicionales anidadas, ya que son vulnerables a bloqueos y condiciones de carrera bajo múltiples eventos concurrentes. En su lugar, se implementa una Red de Petri reactiva que asocia el estado global de ejecución a la distribución de marcas (*tokens*) sobre el grafo, tal como se ilustra en el diagrama simplificado de la Figura 3.2.

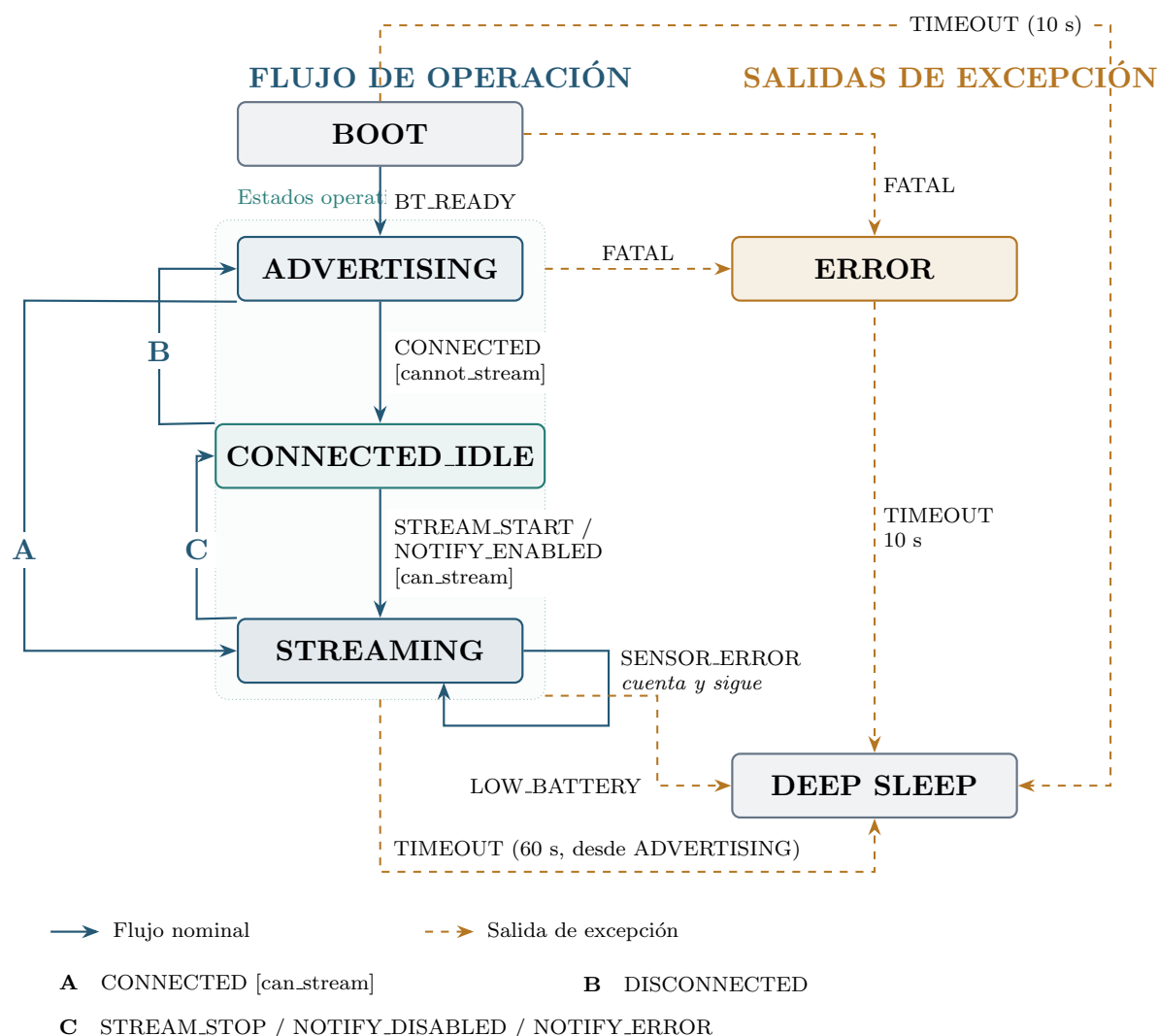


Figura 3.2: Vista simplificada del ciclo de vida del dispositivo. El eje vertical recoge el flujo nominal; a la derecha, las dos salidas de excepción. Las transiciones por fallo fatal y por batería baja existen desde *cada* estado operativo, por lo que se dibujan agrupadas desde el recuadro punteado en lugar de repetirlas una por una. La relación exhaustiva de las 25 transiciones, con sus guardas y acciones, figura en las Tablas 3.2 y 3.3. Obsérvese que **SENSOR_ERROR** es un autobucle sobre **P_STREAMING**: no interrumpe la transmisión.

La Red de Petri desarrollada se compone de 6 lugares de estado estables y 14 lugares de evento que operan como entradas asíncronas bloqueadas.

Lugares de estado (State Places)

Los lugares de estado modelan las condiciones estables de operación del sistema:

- **P_BOOT (Lugar 0):** Estado inicial de arranque.
- **P_ADVERTISING (Lugar 1):** Emisión de anuncios BLE. El dispositivo espera la conexión de un cliente.
- **P_CONNECTED_IDLE (Lugar 2):** Conexión activa establecida, pero el envío de datos está suspendido.

- **P_STREAMING (Lugar 3):** Transmisión asíncrona continua de muestras de reflectancia del sensor.
- **P_ERROR (Lugar 4):** Estado de error fatal. Se detiene el sistema para diagnóstico de hardware.
- **P_DEEP_SLEEP (Lugar 5):** Modo de apagado de bajo consumo (System OFF).

Lugares de evento (Event/Latch Input Places)

Los lugares de evento representan estímulos de entrada asíncronos gestionados mediante un mecanismo de pestillo (*latch*). A cada ciclo de ejecución, el motor reactivo pone a cero todas las marcas de evento y **vacía por completo** la cola de mensajes del sistema (`app_evt_msgq`) de forma no bloqueante, depositando una marca en el lugar correspondiente a cada evento recibido; varios lugares de evento pueden quedar marcados en el mismo ciclo, y un evento repetido no acumula marcas porque el lugar es de un solo uso. Un evento que llegue a un estado que no lo consume se descarta al limpiarse las marcas en el ciclo siguiente, que es el comportamiento buscado para no arrastrar órdenes obsoletas. Los eventos **críticos** —fallo fatal y batería baja— no viajan por la cola: se anotan en banderas atómicas propias que se leen *después* de vaciarla, de modo que ninguna presión de tráfico ordinario pueda desplazarlos. Los lugares de evento implementados son:

- **P_EVT_BT_READY (Lugar 6):** Inicialización del stack BLE completada con éxito.
- **P_EVT_CONNECTED (Lugar 7):** Conexión de radio BLE establecida.
- **P_EVT_DISCONNECTED (Lugar 8):** Pérdida de la conexión física de radio BLE.
- **P_EVT_NOTIFY_ENABLED (Lugar 9):** Suscripción del cliente a las notificaciones GATT.
- **P_EVT_NOTIFY_DISABLED (Lugar 10):** Cancelación de la suscripción a notificaciones.
- **P_EVT_STREAM_START (Lugar 11):** Comando remoto de inicio de transmisión de datos.
- **P_EVT_STREAM_STOP (Lugar 12):** Comando remoto de detención de transmisión de datos.
- **P_EVT_SENSOR_ERROR (Lugar 13):** Error de adquisición en el bus del sensor.
- **P_EVT_SENSOR_OK (Lugar 14):** Lectura correcta del sensor tras un fallo; rompe la racha de errores consecutivos.
- **P_EVT_NOTIFY_ERROR (Lugar 15):** Error en el subsistema de notificaciones BLE.
- **P_EVT_FATAL_ERROR (Lugar 16):** Error de hardware o software no recuperable.
- **P_EVT_TIMEOUT (Lugar 17):** Expiración de un temporizador del sistema.
- **P_EVT_LOW_BATTERY (Lugar 18):** Detección de nivel crítico de batería.
- **P_EVT_SILENT_DISCONNECT (Lugar 19):** Pérdida de comunicación silenciosa del host remoto.

Temporizadores de control

Para regular las transiciones temporizadas y comprobar el estado del hardware, se definen 3 temporizadores virtuales:

- **sleep_timer:** Controla el tiempo de inactividad. En el modo de publicidad, expira a los 60 segundos para forzar el apagado. En los estados de inicio o error, expira a los 10 segundos para permitir una ventana de diagnóstico USB antes de apagar el sistema.
- **silent_disc_timer:** Temporizador periódico de 1 segundo activo en el modo streaming. Compara el tiempo actual del sistema con la marca de tiempo del último envío que el controlador de radio dio por finalizado localmente —las notificaciones GATT no llevan acuse de recibo, de modo que lo que se supervisa es que la radio siga despachando tramas, no que el cliente las reciba—. Si la diferencia es superior a los 2000 ms (2 segundos), inyecta el evento de desconexión silenciosa.
- **battery_timer:** Temporizador periódico de 60 segundos encargado de invocar de forma regular la rutina de medición del voltaje analógico de la batería.

Guardas de transición y determinismo del sistema

Las guardas actúan como condiciones booleanas estrictas que evalúan de manera atómica el contexto global del sistema (`struct app_ctx`) antes de autorizar el paso de marcas por una transición habilitada en la Red de Petri. Esta arquitectura garantiza el determinismo del sistema, impidiendo cambios de estado no deseados o la activación redundante de periféricos. Se definen dos guardas mutuamente excluyentes:

- **guard_can_stream:** Evalúa la conjunción lógica de tres condiciones atómicas simultáneas:
 1. Existencia de una conexión BLE activa (`current_conn != NULL`).
 2. Suscripción del cliente a las notificaciones GATT habilitada (`notify_enabled == 1`).
 3. Comando de inicio de transmisión activo (`streaming_enabled == 1`).

Justificación técnica: Su propósito es garantizar la eficiencia energética e integridad de memoria, evitando encender el sensor óptico MAX30102 y saturar los búferes de transmisión si la pila BLE no está lista para despachar los paquetes de bioseñales.

- **guard_cannot_stream:** Representa la negación lógica estricta de la guarda anterior (`!guard_can_stream`).

Justificación técnica: Actúa como mecanismo de desenganche automático y seguro (*Failsafe*). Si durante la transmisión activa se interrumpe cualquiera de las tres condiciones (por ejemplo, si el usuario desactiva las notificaciones o solicita pausar el flujo), esta guarda fuerza de inmediato el retorno del sistema al estado de reposo conectado (`P_CONNECTED_IDLE`), desactivando el muestreo del sensor para maximizar la autonomía de la batería.

Acciones diferidas e indicadores LED

Cada cambio de marcado estable activa la ejecución de acciones diferidas (lógica Moore) que controlan el color de la indicación LED local y configuran los temporizadores:

- **Acción en P_BOOT:** Activa todos los canales RGB de salida, iluminando el LED en color Blanco.
- **Acción en P_ADVERTISING:** Ilumina el LED en color Rojo, apaga la supervisión de conexión y activa el temporizador de inactividad de 60 segundos.
- **Acción en P_CONNECTED_IDLE:** Ilumina el LED en color Verde, detiene los temporizadores de inactividad y transmisión, y desactiva la bandera de streaming activo.
- **Acción en P_STREAMING:** Ilumina el LED en color Violeta (canales Rojo y Azul activos), detiene el temporizador de inactividad de publicidad e inicia el temporizador periódico de supervisión de conexión.
- **Acción en P_ERROR:** Ilumina el LED en color Azul e invoca el algoritmo crítico de gestión de fallos.
- **Acción en P_DEEP_SLEEP:** Apaga todos los LEDs de estado y ordena el apagado físico del hardware.

La Tabla 3.1 resume la codificación de colores de las salidas del sistema.

Estado Estable	Canal Rojo	Canal Verde	Canal Azul	Color Resultante
P_BOOT	Encendido	Encendido	Encendido	Blanco
P_ADVERTISING	Encendido	Apagado	Apagado	Rojo
P_CONNECTED_IDLE	Apagado	Encendido	Apagado	Verde
P_STREAMING	Encendido	Apagado	Encendido	Violeta (Magenta)
P_ERROR	Apagado	Apagado	Encendido	Azul
P_DEEP_SLEEP	Apagado	Apagado	Apagado	Apagado

Tabla 3.1: Estados de los canales LED según el marcado de la Red de Petri.

Tabla exhaustiva de transiciones de la Red de Petri

El comportamiento dinámico completo del firmware queda determinado por las 25 transiciones declaradas en el motor reactivo. Un aspecto esencial de esta implementación es que **el orden de declaración de las transiciones establece su prioridad de resolución de conflictos**: el motor recorre la tabla de arriba abajo disparando todas las transiciones habilitadas, pero aplica el efecto de cada disparo antes de evaluar la siguiente. De ahí que, cuando varias compiten por las mismas marcas —señaladamente la del estado actual—, sólo se dispare la primera: las posteriores ya no encuentran la marca que necesitaban. Las que no compiten, como las dos que operan sobre el contador del sensor sin abandonar P_STREAMING, pueden dispararse en el mismo ciclo. Esta propiedad no es un detalle de implementación, sino la que confiere determinismo a la red y la que se traslada al modelo formal verificado con SPIN en el Capítulo 4.

Por ese motivo las Tablas 3.2 y 3.3 se presentan agrupadas por nivel de prioridad y no por estado de origen, reflejando el orden exacto del código fuente. La política de prioridades responde a un criterio de seguridad clínica: un fallo fatal debe atenderse antes que un aviso de batería, y este antes que una pérdida de enlace; los vencimientos de temporizador, al no representar ninguna urgencia, se resuelven en último lugar.

#	Pri.	Estado	Evento consumido	Estado destino	Acción asociada
<i>Prioridad 1 — Fallos fatales: se atienden antes que cualquier otro evento</i>					
1	1	BOOT	FATAL_ERROR	ERROR	enter_error
2	1	ADVERTISING	FATAL_ERROR	ERROR	enter_error
3	1	CONNECTED_IDLE	FATAL_ERROR	ERROR	enter_error
4	1	STREAMING	FATAL_ERROR	ERROR	enter_error
<i>Prioridad 2 — Batería baja: protección de la celda frente a descarga profunda</i>					
5	2	ADVERTISING	LOW_BATTERY	DEEP_SLEEP	enter_deep_sleep
6	2	CONNECTED_IDLE	LOW_BATTERY	DEEP_SLEEP	enter_deep_sleep
7	2	STREAMING	LOW_BATTERY	DEEP_SLEEP	enter_deep_sleep
<i>Prioridad 3 — Pérdida de enlace: el equipo vuelve a hacerse visible</i>					
8	3	CONNECTED_IDLE	DISCONNECTED	ADVERTISING	enter_advertising
9	3	STREAMING	DISCONNECTED	ADVERTISING	enter_advertising
10	3	STREAMING	SILENT_DISCONNECT	ADVERTISING	silent_disconnect

Tabla 3.2: Transiciones de la Red de Petri, prioridades 1 a 3 (Parte I). El orden de la tabla reproduce el orden de declaración en `app_pn.c`, que es el que resuelve los conflictos de disparo. Cuando una transición está sujeta a guarda, la condición se indica bajo el estado destino. Por brevedad se omiten los prefijos: los estados se declaran como `P_nombre`, los eventos como `P_EVT_nombre` y las acciones como `action_nombre`.

#	Pri.	Estado	Evento consumido	Estado destino	Acción asociada
<i>Prioridad 4 — Flujo nominal de operación</i>					
11	4	BOOT	BT_READY	ADVERTISING	enter_advertising
12	4	ADVERTISING	CONNECTED	STREAMING si can_stream	enter_streaming
13	4	ADVERTISING	CONNECTED	CONNECTED_IDLE si cannot_stream	enter_idle
14	4	CONNECTED_IDLE	STREAM_START	STREAMING si can_stream	enter_streaming
15	4	CONNECTED_IDLE	NOTIFY_ENABLED	STREAMING si can_stream	enter_streaming
16	4	CONNECTED_IDLE	STREAM_START	CONNECTED_IDLE si cannot_stream	—
17	4	CONNECTED_IDLE	NOTIFY_ENABLED	CONNECTED_IDLE si cannot_stream	—
18	4	STREAMING	STREAM_STOP	CONNECTED_IDLE	enter_idle
19	4	STREAMING	NOTIFY_DISABLED	CONNECTED_IDLE	enter_idle
20	4	STREAMING	NOTIFY_ERROR	CONNECTED_IDLE	enter_idle
21	4	STREAMING	SENSOR_OK	STREAMING	reset_sensor_errors
22	4	STREAMING	SENSOR_ERROR	STREAMING	handle_sensor_error
<i>Prioridad 5 — Vencimiento de temporizadores: la menor urgencia</i>					
23	5	BOOT	TIMEOUT	DEEP_SLEEP	enter_deep_sleep
24	5	ADVERTISING	TIMEOUT	DEEP_SLEEP	enter_deep_sleep
25	5	ERROR	TIMEOUT	DEEP_SLEEP	enter_deep_sleep

Tabla 3.3: Transiciones de la Red de Petri, prioridades 4 y 5 (Parte II). Se aplican las mismas convenciones de nomenclatura que en la Parte I.

Cuatro transiciones merecen un comentario específico por apartarse del patrón general: son las únicas cuyo lugar de salida coincide con el de entrada, de modo que se disparan sin mover la marca.

Las transiciones 21 y 22 son las dos únicas que, coincidiendo su lugar de salida con el de entrada, llevan además una acción asociada, y forman una pareja. La 22 recoge el error de lectura del sensor, que **no interrumpe la transmisión**: la marca permanece en P_STREAMING y la acción se limita a incrementar un contador. La 21 es su contrapartida, y va declarada *antes* a propósito: una lectura correcta posterior a un fallo devuelve ese contador a cero. Se trata de una decisión clínica deliberada: los artefactos de movimiento del paciente producen fallos de lectura aislados que no justifican cortar el servicio, y solo la reiteración del fallo —tres ocurrencias— se interpreta como avería real y desencadena el evento fatal.

Las transiciones 16 y 17 tampoco cambian el marcado y carecen de acción asociada. Su función es puramente formal: consumir el evento cuando la guarda impide avanzar a P_STREAMING. Sin ellas, el evento quedaría sin consumir y la red volvería a evaluarlo en el ciclo siguiente de forma indefinida. Su existencia es, por tanto, un requisito para la acotación de la red.

3.2.2. Sistema de detección y gestión de fallos (Fault Detection & Management)

El firmware incorpora un sistema de detección y gestión de fallos estructurado en un modelo de defensa en profundidad. Todos los códigos de error del sistema se declaran en una **única tabla maestra** (`APP_ERROR_TABLE`, en `src/core/app_errors.h`), implementada mediante la técnica de macro-X. De esa declaración única el preprocesador deriva simultáneamente el enumerado de códigos, la cadena con el nombre corto, la descripción legible y las dos clasificaciones que se explican a continuación. La herramienta de extracción diagnóstica descrita en el Capítulo 5 lee ese mismo fichero, de modo que resulta estructuralmente imposible que el firmware y la herramienta difieran en la interpretación de un código.

Cada código se clasifica según **dos ejes independientes**, distinción que constituye una de las decisiones de diseño centrales del subsistema de diagnóstico:

- **Severidad** —qué debe hacer el sistema—: FATAL obliga a abandonar la operación y pasar al estado de error; CRÍTICA degrada una capacidad del equipo pero permite seguir funcionando (por ejemplo, quedarse sin medida de batería); RECUPERABLE corresponde a incidencias transitorias que el propio firmware absorbe.
- **Persistencia** —dónde se guarda—: FLASH indica que el código es *candidato* a la Caja Negra. Conviene la precisión, porque el firmware no escribe en Flash al detectar la incidencia: la anota en el anillo volátil y la recuerda como «último código persistente». La escritura la realiza `action_enter_error()` cuando algún fallo genera un informe, y en ese momento el anillo entero —con los códigos RAM incluidos— viaja al informe como contexto. Un código FLASH sobrevive por tanto al reinicio *si* llega a producirse un informe; si el equipo funciona sin fallar, o si el fallo impide construir la propia red de Petri, no llega a escribirse. Los casos concretos en que eso ocurre se señalan más adelante; RAM indica que solo alimenta el anillo volátil de incidencias y los contadores de salud, sin desgastar la memoria no volátil.

Ambos ejes son ortogonales, y su independencia es intencionada. Un aviso de batería baja (código 50) es CRÍTICO porque obliga a apagar el equipo, pero se clasifica como RAM porque es un evento esperado en el uso normal y persistirlo desgastaría la Flash sin aportar información diagnóstica. La Tabla 3.4 recoge la tabla maestra completa.

Tabla 3.4: Tabla maestra de códigos de error del firmware (31 códigos), con su clasificación por severidad y por persistencia.

Cód.	Nombre	Severidad	Persist.	Descripción
<i>Grupo 1x — Hardware detectado durante el arranque</i>				
10	HW_SENSOR_ABSENT	Fatal	Flash	Sensor MAX3010x ausente o sin responder en el arranque
11	HW_SENSOR_TASK	Fatal	Flash	No se pudo crear el hilo de adquisición del sensor
12	HW_BLE_TASK	Fatal	Flash	No se pudo arrancar la tarea Bluetooth
13	HW_I2C_BUS	Fatal	Flash	El bus I ² C del sensor no está operativo
14	HW_ADC_INIT	Fatal	Flash	Fallo al configurar el canal ADC: sin protección de batería

continúa en la página siguiente

Cód.	Nombre	Severidad	Persist.	Descripción
15	HW_WDT_INIT	Fatal	Flash	Fallo al iniciar el watchdog: sin auto-recuperación ante bloqueo
<i>Grupo 2x — Sensor durante la adquisición</i>				
20	SENSOR_READ_LIMIT	Fatal	Flash	Alcanzado el límite de 3 fallos de lectura consecutivos
21	SENSOR_I2C_RUNTIME	Recuperable	RAM	Fallo puntual de lectura del sensor (aux = <i>errno</i>)
22	SENSOR_TEMP_READ	Recuperable	RAM	Fallo puntual de lectura de temperatura (aux = <i>errno</i>)
23	SENSOR_FIFO_OVERFLOW	Crítica	RAM	La FIFO del sensor desbordó: muestras perdidas (aux = cuántas)
24	SENSOR_FRAME_DROPPED	Crítica	RAM	Trama descartada por cola de muestras llena (aux = <i>seq</i> perdida)
<i>Grupo 3x — Sistema operativo y recursos</i>				
31	SYS_WATCHDOG_RESET	Fatal	Flash	Reinicio forzado por el watchdog (aux = máscara de tareas vivas)
32	SYS_NVS_UNAVAILABLE	Crítica	RAM	Partición NVS inutilizable: la Caja Negra no puede persistir (aux = <i>errno</i>)
33	SYS_QUEUE_INIT	Fatal	Flash	No se pudo crear una de las colas del sistema
34	SYS_THREAD_CREATE	Fatal	Flash	No se pudo crear el hilo de control
35	SYS_EVENT_QUEUE_FULL	Recuperable	RAM	Cola de eventos llena: evento descartado (aux = evento)
36	SYS_NVS_REINITIALIZED	Crítica	Flash	NVS reformateada por mantenimiento: el historial previo se perdió (aux = <i>errno</i> del montaje fallido, o 0 si el borrado fue una orden deliberada sobre una partición sana)
<i>Grupo 4x — Conectividad Bluetooth</i>				
40	BLE_STACK_INIT	Fatal	Flash	bt.enable falló: la pila Bluetooth no arrancó
41	BLE_ADV_START	Fatal	Flash	No se pudo iniciar el anuncio: equipo invisible
42	BLE_TX_STREAK	Recuperable	RAM	Racha de envíos fallidos alcanzada (aux = longitud)
43	BLE_SILENT_DISCONNECT	Recuperable	RAM	Enlace perdido sin aviso (aux = ms desde el último envío)
44	BLE_MTU_TOO_SMALL	Recuperable	RAM	La trama no cabe en el MTU negociado (aux = bytes)
45	BLE_ENCODE_FAIL	Recuperable	RAM	Fallo al codificar la trama Protocol Buffers
46	BLE_PAIRING_FAILED	Recuperable	RAM	Emparejamiento rechazado o fallido (aux = motivo de seguridad)
47	BLE_BOND_EVICTED	Recuperable	RAM	Lista de emparejamientos llena: se desalojó el vínculo más antiguo
<i>Grupo 5x — Energía</i>				
50	PWR_LOW_BATTERY	Crítica	RAM	Tensión de batería por debajo del umbral (aux = mV)
51	PWR_BATTERY_ABSENT	Crítica	Flash	Lectura imposible para una celda real: no hay batería o la línea de medida está rota (aux = mV)
<i>Grupo 6x — Genérico</i>				
60	UNKNOWN_FATAL	Fatal	Flash	Fallo fatal sin causa identificada
<i>Grupo 7x — Motor de control (red de Petri)</i>				
70	PN_NET_INIT	Fatal	Flash	Fallo al construir la estructura de la red de Petri

continúa en la página siguiente

Cód.	Nombre	Severidad	Persist.	Descripción
71	PN_RUNTIME_INIT	Fatal	Flash	Fallo al crear el entorno de ejecución de la red
72	PN_NET_REGISTER	Fatal	Flash	Fallo al registrar la red en el entorno de ejecución

A continuación, se describen con detalle los 19 puntos de detección de fallos (FDP) integrados en el firmware, agrupados en cinco capas de defensa según la fase de ejecución en que actúan. El capítulo se cierra con tres decisiones de diseño (DD-01 a DD-03) que no constituyen detección de fallos pero condicionan el comportamiento del sistema ante ellos.

Capa 1: Seguridad de arranque (Boot Safety)

FDP-01: Sensor Device NULL (Protección MPU Fault)

- **Fichero:** `app_pn.c` | **Función:** `app_pn_start()`
- **Detección:** Comprobación en el inicio de si el dispositivo sensor MAX30102 retornado por el HAL es un puntero nulo.
- **Severidad:** FATAL | **Código de Error:** 10 (`APP_ERR_HW_SENSOR_ABSENT`)
- **Descripción lógica del algoritmo:** La protección adoptada es deliberadamente simple: si `app_hw_sensor_get_device()` devuelve un puntero nulo, el arranque **no llega a crear** el hilo de adquisición, de modo que ninguna ruta del firmware puede desreferenciar el puntero inválido. No se recurre a punteros de relleno ni a construcciones equivalentes, que solo desplazarían el fallo a un punto más difícil de diagnosticar.
- **Discriminación de la causa raíz:** La ausencia de respuesta del sensor admite dos explicaciones muy distintas —el propio sensor averiado, o el bus I²C completo caído— que conducen a reparaciones diferentes. Por ello el código 10 solo se registra si la capa HAL no ha anotado previamente el código 13 (`APP_ERR_HW_I2C_BUS`): cuando el bus entero está inoperativo, prevalece esa causa. Atribuir al sensor un fallo del bus llevaría al técnico a sustituir la pieza equivocada.
- **La radio se arranca, pero se corta al declarar el fallo:** A diferencia del hilo del sensor, la tarea Bluetooth **sí llega a arrancarse** aunque el sensor esté ausente, porque `app_pn_start()` la lanza sin condición. Ahora bien, el fallo de arranque activa de inmediato la bandera atómica de `APP_EVT_FATAL_ERROR`, y en cuanto la red transita a `P_ERROR` la acción `action_enter_error()` invoca `ble_task_stop()`, que detiene el anuncio **y corta cualquier conexión establecida** mediante `bt_conn_disconnect()`. La radio queda por tanto activa durante un único ciclo del motor (`PN_TICK_PERIOD_MS`), no durante la ventana de diagnóstico.
- **Por qué se corta la radio:** Un equipo averiado que siguiera anunciándose podría recibir una conexión, y esa conexión cancelaba el plazo que debía llevarlo al sueño profundo, dejándolo indefinidamente en estado de error. Apagar la radio al declarar el fallo elimina la causa; la propiedad **p14** del Capítulo 4 convierte esa decisión en un invariante comprobado. En consecuencia, **el informe de fallo no se expone nunca por Bluetooth**: la caja negra se consulta exclusivamente por la consola USB descrita en el Capítulo 5.

- **Comportamiento diferenciado de la gestión de errores:** La función `action_enter_error()` bifurca según el momento del fallo. Si se detecta durante el arranque (`ctrl_state == APP_STATE_BOOT`), se abre una **ventana de diagnóstico** de `ERROR_WINDOW_MS` (10 segundos) mediante el temporizador de inactividad, transcurrida la cual la red transiciona a `P_DEEP_SLEEP`. Si el fallo ocurre en marcha, el firmware invalida el rastro del watchdog con `app_hw_wdt_breadcrumb_clear()` — paso imprescindible para que el arranque siguiente no atribuya al perro guardián un reinicio provocado por nosotros— y ejecuta `os_system_reset()`. En ambos casos, y antes de cualquier otra acción, el estado de control se fija a `APP_STATE_ERROR` y se detiene la radio mediante `ble_task_stop()`: el orden importa, porque la desconexión que esto provoca consulta `ctrl_state` para decidir si debe reactivar el plazo de sueño.

FDP-02: Fallo al crear la tarea del sensor

- **Fichero:** `app_pn.c` | **Función:** `app_pn_start()`
- **Detección:** Comprobación de si la rutina `sensor_task_start()` retorna un código de error distinto de cero.
- **Severidad:** FATAL | **Código de Error:** 11 (`APP_ERR_HW_SENSOR_TASK`)
- **Descripción lógica del algoritmo:** Si el planificador del sistema operativo no dispone de suficiente memoria estática para instanciar el hilo de adquisición del sensor, se captura la excepción, se guarda el código 11 en el contexto global y se fuerza el desvío lógico hacia el estado de error de arranque (`P_ERROR`), donde `action_enter_error()` detiene la radio como se ha descrito en FDP-01. Al tratarse de un fallo de inicialización (`ctrl_state == 0`), el sistema habilita una ventana de emergencia de 10 segundos para el diagnóstico por USB CDC-ACM; tras expirar este intervalo sin interacción, se conmuta automáticamente al estado `P_DEEP_SLEEP` para preservar la batería y evitar bucles de reinicio (*bootloops*).

FDP-03: Fallo al iniciar la tarea Bluetooth (BLE)

- **Fichero:** `app_pn.c` | **Función:** `app_pn_start()`
- **Detección:** Comprobación de si la rutina `ble_task_start()` retorna un valor distinto de cero en el arranque.
- **Severidad:** FATAL | **Código de Error:** 12 (`APP_ERR_HW_BLE_TASK`)
- **Descripción lógica del algoritmo:** Si el hardware de radio no responde o hay un desbordamiento al inicializar el pool de hilos de transmisión BLE, se intercepta la anomalía, se almacena el código de fallo 12 y se desencadena la transición a la rutina de gestión de errores (`action_enter_error()`). Dado que el error ocurre durante la fase de arranque (`ctrl_state == 0`), el *firmware* activa el temporizador de 10 segundos para permitir la extracción del informe NVS por puerto serie antes de transicionar de forma segura hacia el modo de reposo profundo (`P_DEEP_SLEEP`).

FDP-04: Fallo del Runtime del motor Petri

- **Fichero:** `app_pn.c` | **Función:** `app_pn_start()`
- **Detección:** Comprobación de si la rutina de inicialización de la biblioteca `rx_pn_runtime_init()` reporta un valor distinto de cero.

- **Severidad:** FATAL | **Código de Error:** 71 (APP_ERR_PN_RUNTIME_INIT)
- **Descripción lógica del algoritmo:** Si el motor reactivo no puede construir su entorno de ejecución, `app_pn_start()` registra el código 71 y retorna `-1` sin continuar. Sin motor de control no hay red que ejecutar, de modo que proseguir el arranque solo produciría un dispositivo que enciende sus LED sin gobernar nada.
- **Degradación a modo consola:** El retorno negativo no provoca un reinicio ni un apagado. La función `main()` registra la condición y cede el control a `app_console_run()`, que no retorna: el equipo queda **indefinidamente** disponible por USB para que el técnico extraiga el diagnóstico. Se prefiere esta degradación a un reinicio automático porque un fallo estructural del motor es determinista y reiniciar solo produciría un bucle.
- **Limitación conocida:** Los tres fallos del grupo 7x (códigos 70, 71 y 72) están clasificados como persistentes, pero en la práctica **no llegan a escribirse** en la Caja Negra. La escritura en Flash la realiza `action_enter_error()`, que forma parte de la red de Petri; si la red no ha podido construirse, esa acción nunca se ejecuta. El código queda registrado en el anillo volátil y es legible por consola, pero se pierde con la alimentación. Esta limitación es inherente al orden de arranque y se documenta explícitamente por su relevancia diagnóstica.

FDP-05: Fallo de compilación de la estructura de la Red de Petri

- **Fichero:** `app_pn.c` | **Función:** `app_pn_start()`
- **Detección:** Comprobación de si la compilación dinámica de la estructura de red `rx_pn_net_init()` retorna error.
- **Severidad:** FATAL | **Código de Error:** 70
- **Descripción lógica del algoritmo:** Si los vectores de arcos de entrada y salida o los punteros de guardas contienen inconsistencias geométricas en su grafo, la rutina aborta el inicio, almacena el código de fallo 70 en el contexto global y notifica un error de arranque crítico. Al igual que en FDP-04, la función retorna `-1` y el control pasa a la consola USB permanente; no se abre una ventana temporizada, porque la ventana de diagnóstico es una acción de la red de Petri y aquí la red no ha llegado a existir.

Capa 2: Monitorización de hardware

FDP-06: Voltaje de batería baja

- **Fichero:** `app_pn.c` | **Función:** `pn_handle_deferred_work()`
- **Detección:** Clasificación de la tensión medida mediante ADC en tres casos, evaluados en este orden: alimentación externa presente, lectura imposible por debajo de `BATTERY_ABSENT_THRESHOLD_MV` (2000 mV), y batería baja real entre ese valor y `BATTERY_LOW_THRESHOLD_MV` (3200 mV).
- **Severidad:** CRÍTICA | **Código de Error:** 50 (APP_ERR_PWR_LOW_BATTERY), o 51 (APP_ERR_PWR_BATTERY_ABSENT) en el caso de la lectura imposible | **Periodicidad:** `BATTERY_CHECK_PERIOD_MS` = 60 segundos.
- **Descripción lógica del algoritmo:** La medida no se realiza en el contexto del temporizador. La devolución de llamada `battery_timer_cb()` se limita a activar una bandera atómica (`battery_check_pending`); la conversión efectiva mediante

`app_hw.battery_read_mv()` y la decisión posterior las ejecuta el hilo de control en `pn_handle_deferred_work()`. Esta separación es obligatoria: la lectura del ADC y el registro de incidencias toman semáforos, operación prohibida en contexto de interrupción. Si la conversión falla, la rutina retorna sin actualizar la tensión almacenada, de modo que un error del ADC nunca se confunde con una batería llena. Con una lectura válida, la decisión atraviesa tres filtros. El primero comprueba si hay **alimentación externa**: con el equipo alimentado por USB el canal del ADC queda flotando y su lectura no representa ninguna celda, de modo que se registra el valor y se abandona la evaluación sin apagar nada; sin este filtro el equipo se auto-apagaba a los pocos segundos de *streaming* cada vez que se probaba por USB sin batería. El segundo descarta las lecturas **físicamente imposibles**: por debajo de 2000 mV ninguna celda de litio con circuito de protección puede entregar tensión, de modo que el suceso se atribuye a la ausencia de batería o a un fallo de la línea de medida, se anota con el código 51 en el anillo volátil y queda marcado como *último código persistente*, aunque no alcanza la memoria flash por sí solo: sólo llega a ella si más adelante se genera un informe de fallo, según se detalla en la Sección 3.2.2. Tampoco se apaga el equipo: apagar no protege a una celda que no existe. Sólo cuando la lectura supera esos dos filtros y cae por debajo de 3200 mV se interpreta como **batería baja real**: se registra el código 50 y se inyecta el evento `APP_EVT_LOW_BATTERY`, que la red atiende con prioridad 2 llevando el sistema a `P_DEEP_SLEEP`.

FDP-07: Perro Guardián físico (Watchdog Timer — WDT)

- **Fichero:** `app_hw.c` | **Función:** `app_hw_wdt_service()`
- **Detección:** Ausencia de señal de vida de alguna de las tareas vigiladas dentro de la ventana de `APP_WDT_TIMEOUT_MS` (5 segundos).
- **Severidad:** PREVENTIVA | **Código de Error:** 31 (`APP_ERR_SYS_WATCHDOG_RESET`, registrado en el arranque siguiente).
- **Perro guardián por tarea:** Un watchdog alimentado por un único hilo solo demuestra que *ese* hilo vive. Si el hilo de control sigue girando mientras la adquisición se ha bloqueado, el equipo transmitiría indefinidamente datos obsoletos sin que nada lo detecte. Para evitarlo, la vigilancia es colectiva: las tres tareas del sistema —control, sensor y Bluetooth— se registran con `app_hw_wdt_register()` y declaran su actividad con `app_hw_wdt_kick()` en cada iteración de su bucle. La rutina `app_hw_wdt_service()`, invocada por el hilo de control, **solo alimenta el periférico si todas las tareas registradas han dado señal** desde el ciclo anterior; acto seguido pone el registro de vida a cero para exigir una nueva confirmación. El bloqueo de cualquiera de las tres provoca, por tanto, el reinicio del SoC.
- **Rastro superviviente (*breadcrumb*):** Un reinicio por watchdog es, por definición, un fallo del que nadie pudo dejar constancia. Para no perder la información, inmediatamente antes de alimentar al perro se escribe en una zona de RAM declarada `_noinit` —que el arranque no inicializa— la máscara de las tareas que seguían vivas. Como esa región contiene basura tras un arranque en frío, el rastro se protege con una firma propia y un CRC: sin esa comprobación, un encendido normal se interpretaría como un reinicio por watchdog y el sistema fabricaría un fallo inexistente. En el arranque siguiente, `app_hw_watchdog_reset_check()` recupera la máscara y permite registrar el código 31 indicando en el campo `aux` exactamente **qué tarea dejó de responder**, que es la información que convierte un reinicio

inexplicable en un diagnóstico.

Qué hacer cuando la protección misma no arranca. Las dos capas anteriores comparten una propiedad incómoda: son subsistemas de *protección*, y su ausencia es invisible en marcha. Un equipo sin watchdog mide exactamente igual de bien que uno con él, y uno sin canal ADC también; la diferencia solo se manifiesta el día en que hace falta, que es precisamente el día en que ya no hay margen. Durante el desarrollo, el resultado de `app_hw_wdt_init()` y `app_hw_battery_init()` se descartaba con un `(void)`, de modo que un fallo en cualquiera de los dos dejaba el equipo funcionando con dos garantías declaradas —RR-2 y RR-4— que habían dejado de ser ciertas sin que nadie lo supiera.

La decisión adoptada es **no continuar en modo degradado**. Si cualquiera de los dos no arranca, el sistema fija la causa como error de arranque y entra en `P_ERROR`, que escribe la Caja Negra con el código concreto —14 o 15— y el *errno* del periférico en el campo auxiliar. El razonamiento tiene tres partes. La primera es que un equipo que se niega a entrar en servicio y deja un diagnóstico es preferible a uno que arranca callado y deja un misterio: la indisponibilidad es la misma, la información no. La segunda es que el resto del diseño *cuenta* con esos subsistemas, de modo que seguir sin ellos no es operar en modo reducido sino operar sobre supuestos falsos. La tercera, y la que zanja el caso de la batería, es que sin medida de tensión nada impide sobredescargar la celda de litio: eso ya no es una función perdida sino un riesgo físico.

Se admite una única excepción, y conviene explicarla porque distingue una avería de una decisión. El valor `-ENOTSUP` no significa que el periférico esté roto, sino que la función no existe en esta compilación —por ejemplo, una placa cuyo árbol de dispositivos no declara canal ADC—. Eso es una elección de construcción, ya señalada en tiempo de compilación mediante un `#warning`, y tratarla como fallo fatal en ejecución impediría construir el firmware para esa placa. Sólo los códigos de error restantes, que sí indican hardware que debía estar y no responde, retiran el equipo del servicio.

Cuatro pruebas unitarias cubren la decisión: el watchdog averiado, el ADC averiado, la ausencia por compilación —que debe arrancar con normalidad— y el caso de fallos múltiples, donde la causa registrada debe ser la primera de la secuencia de arranque, porque es la que explica a las demás y la que lleva al técnico a la pieza correcta.

Cuando la medida miente: distinguir una batería agotada de la ausencia de batería. La prueba sobre la placa reveló un caso que ninguna de las salvaguardas anteriores cubría. Alimentado sólo por el puerto del depurador y sin celda conectada, el equipo midió 400 mV en el canal ADC y se apagó por batería baja en mitad de una captura. La medida no era errónea: el convertidor leyó fielmente un pin al aire.

La primera defensa prevista —consultar si hay tensión USB antes de evaluar el umbral— no se activó, y por una razón que conviene explicar porque delimita su alcance. La detección lee el registro `USBREGSTATUS` del periférico de alimentación, que sólo refleja la tensión presente en el conector USB *del propio microcontrolador*. La placa de desarrollo tiene dos puertos, y el del depurador alimenta la placa sin llegar a ese pin. La comprobación es por tanto correcta para el producto —donde el único USB es el del SoC— pero inaplicable a ese montaje de laboratorio, y el propio registro de la sesión lo confirma: nunca aparece el mensaje de anfitrión USB conectado.

La segunda defensa, que sí cubre ambos casos, no consulta al hardware sino a la propia

medida. Una celda de litio con circuito de protección no entrega menos de unos 2,5 V: por debajo de esa tensión su propio circuito ya ha cortado. Un valor de 400 mV es, sencillamente, **imposible** para una celda conectada, y describe otra cosa: que no hay batería, o que la línea de medida está rota. El firmware introduce por ello un suelo de verosimilitud (`BATTERY_ABSENT_THRESHOLD_MV`, 2000 mV): por debajo se registra el código 51 y **no** se apaga.

La asimetría es deliberada y merece justificarse, porque a primera vista parece debilitar RR-4. Apagar el equipo no protege ninguna celda que no existe; y si la línea de medida se rompiera en campo, apagar significaría interrumpir una donación por una lectura que ya se sabe falsa. Lo que sí se exige es que el fallo no sea silencioso. La última línea de defensa de una celda real sigue siendo, en cualquier caso, su propio circuito de protección.

Qué le ocurre exactamente al código 51. Conviene describir su recorrido con precisión, porque decir simplemente que «queda registrado en la Caja Negra» prometería más de lo que el firmware hace. En el instante en que se detecta, el código se anota en el **anillo volátil** de incidencias, actualiza el contador de salud de su categoría y queda marcado como *último código persistente*. Nada de eso toca todavía la memoria flash, y no por descuido: escribir en NVS desde un hilo cualquiera bloquearía durante la recolección de basura del sistema de ficheros, de modo que la escritura se reserva a la acción de error, que ya está fuera del camino crítico.

De ahí se sigue que el código 51 puede conservarse u observarse por dos vías distintas, y sólo por ellas, pero conviene no confundirlas. **Sólo la primera lo lleva a la memoria flash:** que *después* ocurra un fallo que provoque un informe, porque al escribirlo el anillo volátil se copia íntegro en el campo de eventos previos y el 51 viaja con él como parte del contexto; si además no se había fijado ninguna causa fatal explícita, es el propio 51 —por ser el último código persistente— el que encabeza el informe. La segunda no persiste nada: permite consultarlo mientras el equipo sigue encendido mediante la consola USB, que vuelca el anillo vivo tal como está.

La consecuencia práctica hay que declararla sin adornos: **si el equipo funciona sin batería y no falla ninguna otra cosa, el código 51 no llega a la flash.** Permanece visible mientras el equipo esté encendido y desaparece al reiniciarlo. Es una decisión coherente con la naturaleza del aviso —la ausencia de celda es una condición del montaje, no una avería sobrevenida— pero significa que el diagnóstico de un banco de pruebas mal cableado exige mirar el anillo en caliente, no el informe de flash.

Una limitación del acondicionamiento analógico. El mismo ensayo dejó a la vista una restricción del diseño que conviene declarar. El canal está configurado con ganancia 1/6 y referencia interna de 0,6 V, lo que sitúa el fondo de escala en 3,6 V, mientras que una celda de litio cargada alcanza 4,2 V. Con conexión directa, por tanto, toda tensión por encima de 3,6 V se satura y la medida deja de ser representativa; el umbral de seguridad de 3200 mV queda dentro del rango y la protección funciona, pero la tensión *informada* no sería fiable en la zona alta de la carga. Un montaje correcto exige un divisor resistivo y ajustar en consecuencia `BATTERY_DIVIDER_NUM` y `BATTERY_DIVIDER_DEN`, que hoy valen 1/1 por corresponder a una medida directa. Se documenta como requisito del hardware definitivo, no como defecto del firmware.

Capa 3: Detección de errores del sensor

FDP-08: Error de lectura física en tiempo de ejecución

- **Fichero:** `sensor_task.c` | **Función:** `sensor_thread()`
- **Detección:** La rutina `sensor_read_frame()` retorna un código distinto de cero al componer una trama.
- **Severidad:** RECUPERABLE | **Código de Error:** 21 (`APP_ERR_SENSOR_I2C_RUNTIME`, persistencia RAM).
- **Descripción lógica del algoritmo:** El hilo de adquisición compone tramas de `SAMPLES_PER_FRAME` (20) muestras. No las lee de una en una a intervalos fijos, sino que sondea la FIFO del sensor cada `SENSOR_FIFO_POLL_MS` (25 ms) y se lleva en cada vuelta el lote completo de muestras disponibles —dos o tres, a la cadencia de 100 Hz—, según se detalla en la Sección 3.2.4. Si una lectura falla, el hilo registra el código 21 con el *errno* devuelto en el campo *aux*, **notifica el evento** `APP_EVT_SENSOR_ERROR` **a la red de Petri de forma inmediata** y espera `SENSOR_RETRY_DELAY_MS` (200 ms) antes de reintentar. Ese retardo cumple una función concreta: un bus I²C perturbado por una desconexión momentánea necesita tiempo para estabilizarse, y reintentar de inmediato solo produciría una ráfaga de fallos idénticos.
- **Ausencia deliberada de contador local:** El hilo del sensor **no acumula** fallos ni decide cuándo un error es definitivo; se limita a informar. La política de tolerancia reside íntegramente en la red de Petri (FDP-09). Esta separación es intencionada: concentrar la decisión en un único punto evita que dos contadores independientes puedan discrepar, y permite que la política quede sujeta a verificación formal junto con el resto del control.
- **Número de secuencia:** El contador de secuencia de las tramas solo avanza con lecturas válidas. De este modo, un salto en la numeración recibida por el cliente identifica sin ambigüedad una pérdida de datos, y no un reenvío o un artefacto del transporte.

Los metadatos de la trama existen para el receptor. Conviene explicitar por qué la trama transporta dos campos que el dispositivo no necesita para sí mismo. El número de secuencia y la marca de tiempo de la primera muestra no sirven al firmware: sirven a quien recibe.

El firmware **descarta tramas deliberadamente** cuando la radio se congestiona. La rutina de transmisión intenta adquirir el testigo de envío durante un plazo acotado y, si no lo consigue, abandona la trama en lugar de bloquearse. Es la decisión correcta —en una señal fisiológica la frescura del dato vale más que su integridad histórica— pero deja al receptor ante un problema: la serie que recibe tiene huecos y nada en las muestras lo delata, porque una muestra descartada sencillamente no llega.

El número de secuencia resuelve esa ambigüedad. Un salto en la numeración es prueba de pérdida, y su magnitud indica cuántas tramas faltan. La marca de tiempo cumple una función complementaria: permite reconstruir la base temporal a partir del reloj *del dispositivo* en lugar de suponerla desde el instante de llegada, que incluye la latencia variable del enlace y la del planificador del receptor.

Ambos campos son, por tanto, parte del contrato con el cliente y no un detalle interno. Un receptor que los ignore seguirá decodificando las muestras sin error aparente —la

codificación empleada descarta en silencio los campos que no conoce— pero perderá la capacidad de saber si los datos que muestra están completos. La forma en que los clientes consumen este contrato se detalla en el Capítulo 7.

FDP-09: Política de tolerancia a fallos del sensor en la Red de Petri

- **Fichero:** `app_pn.c` | **Función:** `action_handle_sensor_error()`
- **Detección:** Disparo de la transición 22 (`P_STREAMING + P_EVT_SENSOR_ERROR`).
- **Severidad:** `RECUPERABLE` → `FATAL` | **Código de Error:** 20 (`APP_ERR_SENSOR_READ_LIMIT`) al alcanzar el umbral.
- **La transmisión no se interrumpe:** La transición asociada a este evento es una de las cuatro de la red cuyo lugar de salida coincide con el de entrada —las otras tres son la del retorno del sensor y las dos que mantienen el reposo cuando no se dan las condiciones para emitir—: la marca permanece en `P_STREAMING`. Un fallo aislado de lectura —provocado casi siempre por el movimiento del paciente— no justifica cortar el servicio ni descender al estado de reposo. La acción se limita a incrementar el contador `sensor_error_count` y a registrar el suceso.
- **Umbral único:** Cuando el contador alcanza `SENSOR_ERROR_LIMIT` (3), la acción registra el código 20 con persistencia en Flash, fija la causa en `last_fatal_err_code` e inyecta `APP_EVT_FATAL_ERROR`. La red atiende ese evento con prioridad 1 y transita a `P_ERROR`, lo que dispara la escritura de la Caja Negra y el reinicio de recuperación descrito en FDP-01. El umbral efectivo del sistema es, por tanto, de **tres fallos consecutivos**: no existe ningún segundo contador que lo multiplique.
- **Condición de reinicio del contador:** El contador vuelve a cero por dos caminos. El primero es entrar en `P_STREAMING` desde otro estado, que descarta el historial de la sesión anterior. El segundo —y es el que hace cierta la palabra *consecutivos*— es la transición `in_str_sensor_ok`, declarada *antes* que la del error: la tarea del sensor emite `APP_EVT_SENSOR_OK` en cuanto una lectura vuelve a salir bien tras un fallo, y la acción asociada devuelve el contador a cero sin abandonar `P_STREAMING`.
Conviene detenerse en por qué ese evento es de *flanco* y no de nivel. Emitirlo en cada lectura correcta inundaría la cola de la red con cientos de mensajes por segundo que no cambiarían nada; la tarea del sensor mantiene por ello una bandera local y solo avisa cuando una racha existente queda rota. Es el mismo criterio que ya aplicaba la cadena de transmisión con `tx_error_streak`, y su ausencia en la rama del sensor era una asimetría real del diseño: sin esta transición el sistema contaba fallos *acumulados* dentro de la sesión, de modo que tres movimientos del paciente separados por minutos de captura correcta acababan declarando un fallo fatal que RR-1 no contempla.

Arquitectura de la cadena de tolerancia a fallos del sensor La gestión del error de sensor ilustra un principio de diseño que recorre todo el firmware: **detectar donde se produce el fallo, decidir donde reside el control**. La Tabla 3.5 resume el reparto de responsabilidades.

	Hilo del sensor (FDP-08)	Red de Petri (FDP-09)	Efecto
Responsabilidad	Detectar e informar	Contar y decidir	Separación de funciones
Código registrado	21 (RAM)	20 (Flash) al llegar al umbral	Traza sin desgaste de Flash
Estado resultante	Sin cambio	P_STREAMING en bucle hasta el tercer fallo	Continuidad del servicio
Espera aplicada	200 ms de estabilización del bus	Ninguna	El retardo va donde está el bus
Verificable formalmente	No	Sí	La política es parte del modelo

Tabla 3.5: Reparto de responsabilidades en la cadena de tolerancia a fallos del sensor.

El resultado observable es que el equipo absorbe hasta dos fallos consecutivos de lectura sin que el usuario perciba interrupción alguna —únicamente un hueco en la numeración de secuencia, visible para el cliente—, y que cualquier lectura correcta entre medias devuelve el margen íntegro. Solo al tercer fallo seguido concluye que la avería es permanente. Los códigos 21 registrados en el anillo volátil quedan disponibles para el diagnóstico posterior aunque el equipo nunca llegue a fallar, lo que permite detectar un deterioro progresivo del contacto físico antes de que produzca una parada.

Capa 4: Detección de errores de comunicaciones (BLE)

FDP-10: Fallo en la inicialización física del chip Bluetooth

- **Fichero:** `ble_task.c` | **Función:** `bt_ready()`
- **Detección:** Callback de inicialización asíncrona que retorna un código de error de la pila de radio.
- **Severidad:** FATAL | **Código de Error:** 40 (`APP_ERR_BLE_STACK_INIT`, con el *errno* en *aux*).
- **Descripción lógica del algoritmo:** Si el driver de bajo nivel de Bluetooth no consigue inicializarse o calibrar la frecuencia de la antena, el callback de Zephyr interrumpe el arranque normal, guarda el código de error y notifica a la Red de Petri para evitar que el dispositivo opere sin conectividad inalámbrica.

FDP-11: Fallo al activar el modo de publicidad (Advertising)

- **Fichero:** `ble_task.c` | **Función:** `bt_ready()`
- **Detección:** Comprobación de si la rutina del stack de radio `bt_le_adv_start()` reporta error.
- **Severidad:** FATAL | **Código de Error:** 41 (`APP_ERR_BLE_ADV_START`, con el *errno* en *aux*).
- **Descripción lógica del algoritmo:** Si el subsistema inalámbrico no puede iniciar la emisión de paquetes publicitarios (por saturación de canales o anomalías de ra-

diofrecuencia), se captura la excepción, se almacena el código del error y se desvía la Red de Petri hacia el estado de error fatal.

FDP-12: Fallo de codificación de la trama Protocol Buffers

- **Fichero:** `ble_task.c` | **Función:** `ble_send_frame()`
- **Detección:** `proto_encode_frame()` retorna `false`.
- **Severidad:** RECUPERABLE | **Código de Error:** 45 (`APP_ERR_BLE_ENCODE_FAIL`) | **Acción:** Descarte de la trama y liberación del semáforo de transmisión.
- **Descripción lógica del algoritmo:** Si se produce una anomalía al serializar las 20 muestras de la trama (`SAMPLES_PER_FRAME`), la función de codificación la considera inválida. Se registra el código 45 (`APP_ERR_BLE_ENCODE_FAIL`) y el paquete se descarta, evitando transmitir datos corruptos al cliente. El fallo se clasifica como recuperable y con persistencia en RAM: no interrumpe la sesión, pero queda anotado en el anillo de incidencias por si se repite.

FDP-13: Tamaño de carga útil mayor que la MTU negociada

- **Fichero:** `ble_task.c` | **Función:** `ble_notify_locked()`
- **Detección:** La longitud serializada supera la carga útil disponible, calculada como `mtu - 3`.
- **Severidad:** RECUPERABLE | **Código de Error:** 44 (`APP_ERR_BLE_MTU_TOO_SMALL`, con los bytes requeridos en `aux`).
- **Descripción lógica del algoritmo:** El protocolo binario del microcontrolador calcula la longitud de la carga útil del mensaje Protocol Buffers y verifica de forma previa si este tamaño cabe en el límite de la MTU de transmisión negociada con el smartphone (restando 3 bytes obligatorios del protocolo ATT). Si la trama supera el tamaño máximo permitido, la transmisión se cancela de forma inmediata para evitar un desbordamiento de búfer y una posible corrupción de memoria en la pila Bluetooth de Zephyr, reportando la anomalía.

FDP-14: Fallo de transmisión del stack Bluetooth

- **Fichero:** `ble_task.c` | **Funciones:** `ble_notify_locked()` y `ble_tx_failed()`
- **Detección:** `bt_gatt_notify_cb()` retorna un código de error.
- **Severidad:** RECUPERABLE | **Código de Error:** 42 (`APP_ERR_BLE_TX_STREAK`) al agotar la tolerancia.
- **Tolerancia por racha, no por fallo aislado:** Un envío fallido aislado es normal en un enlace de radio: basta con que el receptor se aleje un instante. Por ello el firmware no reacciona al primer fallo, sino que contabiliza una *racha* en `tx_error_streak`. Solo cuando esta alcanza `BLE_TX_ERROR_LIMIT` (5 envíos consecutivos fallidos) se registra el código 42 y se inyecta `APP_EVT_NOTIFY_ERROR`, que la red atiende con la transición 20 devolviendo el sistema a `P_CONNECTED_IDLE`. Cualquier envío correcto, así como una conexión o desconexión, pone la racha a cero: el criterio cuenta fallos *consecutivos*, no acumulados.
- **Liberación de recursos:** En todas las rutas de error se devuelve el semáforo

`notify_slot_sem` antes de retornar. Omitirlo en una sola rama bloquearía la transmisión de forma permanente, pues nadie volvería a liberarlo.

FDP-15: Pérdida silenciosa de conexión (Silent Disconnect)

- **Fichero:** `app_pn.c` | **Función:** `silent_disc_timer_cb()`
- **Detección:** Ausencia de *finalizaciones locales* de notificación durante más de 2 segundos: la marca `last_notify_done` deja de refrescarse porque el controlador de radio no completa ningún envío. No se trata de un acuse de recibo del cliente —las notificaciones GATT no lo llevan—, sino de la constatación de que la radio local ha dejado de despachar tramas.
- **Severidad:** RECUPERABLE | **Código de Error:** 43 (`APP_ERR_BLE_SILENT_DISCONNECT`), con los milisegundos transcurridos en el campo `aux`.
Acción: Retorno automático al modo de publicidad.
- **Descripción lógica del algoritmo:** Cuando el dispositivo está transmitiendo, el *callback* de finalización del Bluetooth actualiza una variable atómica (`last_notify_done`) con el instante en que el controlador de radio dio por despachado el último paquete. El temporizador de supervisión periódica evalúa cada segundo si la diferencia entre el reloj actual y esa marca supera los 2 segundos. Si es así, se concluye que **el enlace ha dejado de cursar tráfico**: la radio ya no consigue completar envíos, sea porque el cliente se ha alejado del alcance, porque el enlace se ha degradado o porque el controlador ha dejado de recibir confirmaciones de capa de enlace del otro extremo. El sistema detiene las lecturas del sensor y vuelve al modo de publicidad en espera de reconexión.
- **Alcance de la detección:** Es importante no atribuir a este mecanismo más de lo que puede observar. Lo que se supervisa es la finalización *local* del envío, no la recepción por parte de la aplicación. Un cliente que cerrase la aplicación manteniendo abierta la conexión de transporte seguiría aceptando los paquetes en la capa de enlace, y el mecanismo no lo distinguiría de un cliente activo. Detectar ese caso exigiría un *heartbeat* de nivel de aplicación, que se recoge como línea futura en el Capítulo 9.

Capa 5: Robustez de la capa de transmisión y adquisición

Esta capa agrupa los mecanismos de control de flujo y resiliencia que actúan durante la fase de transmisión activa (estado `P_STREAMING`), previniendo el desbordamiento de buffers del stack BLE y la pérdida de frescura de las bioseñales.

FDP-16: Renegociación dinámica del Supervision Timeout de BLE

- **Fichero:** `ble_task.c` | **Función:** `connected()`
- **Detección:** Evento de conexión exitosa con el smartphone receptor.
- **Severidad:** PREVENTIVA | **Acción ejecutada:** Actualización dinámica de los parámetros de enlace BLE.
- **Descripción lógica del algoritmo:** Inmediatamente tras establecer una conexión BLE, el firmware no acepta los parámetros de enlace por defecto del cliente. En su lugar, invoca `bt_conn_le_param_update()` para imponer de forma activa un *Supervision Timeout* de 4000 ms (400×10 ms) e intervalos de conexión de entre 30 y 50

ms. Si el microcontrolador no recibe ningún paquete de la radio durante 4 segundos seguidos, el stack Bluetooth de Zephyr genera de forma autónoma el evento de desconexión física, sin necesidad de que el temporizador de desconexión silenciosa de la Red de Petri lo detecte manualmente. Esto proporciona una segunda barrera de seguridad hardware ante pérdidas abruptas de enlace, complementaria al mecanismo software de FDP-15.

FDP-17: Control de flujo de notificaciones BLE mediante semáforo

- **Fichero:** `ble_task.c` | **Funciones:** `ble_send_frame()` y `notify_complete()`
- **Detección:** Ocupación de la ranura de transmisión del stack Bluetooth.
- **Severidad:** PREVENTIVA | **Acción ejecutada:** Bloqueo de nuevas transmisiones hasta confirmación del stack.
- **Descripción lógica del algoritmo:** El subsistema de transmisión BLE utiliza un semáforo binario (`notify_slot_sem`) para implementar un control de flujo estricto. Antes de componer la trama, `ble_send_frame()` debe adquirir el semáforo con una espera acotada de `NOTIFY_SLOT_WAIT_MS`; si no lo consigue en ese plazo, descarta la trama en lugar de bloquearse indefinidamente, decisión coherente con la política de priorizar la frescura de la señal sobre su integridad histórica. El semáforo solo se devuelve cuando la devolución de llamada `notify_complete()` confirma que la pila de radio ha despachado el paquete anterior, o bien desde cualquiera de las rutas de error. Este mecanismo garantiza que en ningún momento existan dos tramas de bioseñales en tránsito simultáneo, previniendo el desbordamiento de las colas de transmisión internas de Zephyr y la consecuente corrupción de la memoria RAM dinámica del sistema operativo.

FDP-18: Descarte selectivo de muestras ante desbordamiento del buffer de adquisición

- **Fichero:** `sensor_task.c` | **Función:** `sensor_thread()`
- **Detección:** Imposibilidad de inserción en la cola de muestras `sample_msgq` por capacidad agotada.
- **Severidad:** GESTIONADA | **Acción ejecutada:** Descarte del paquete más antiguo y reinserción de la muestra más reciente.
- **Descripción lógica del algoritmo:** Si el hilo de transmisión BLE experimenta una demora temporal y la cola de muestras PPG se llena, el intento de inserción de la nueva muestra con `os_msgq_send()` falla. En lugar de descartar la muestra recién adquirida o bloquear el hilo de adquisición indefinidamente, el firmware extrae mediante `os_msgq_receive()` únicamente el paquete más antiguo presente en la cola, liberando exactamente una ranura, y acto seguido reintenta insertar la muestra actual. Esta política de *oldest-drop* selectivo —frente a una purga total— minimiza la pérdida de datos garantizando que los registros que lleguen al médico sean siempre los más recientes, priorizando la frescura de la señal hemodinámica.

FDP-19: Captura de excepciones no tipificadas en tiempo de ejecución (Código 60)

- **Fichero:** `app_pn.c` | **Función:** `action_enter_error()`

- **Detección:** Captura de eventos de fallo fatal (`APP_EVT_FATAL_ERROR`) durante la fase operacional (`ctrl_state > 0`) sin un código de error explícito asignado.
- **Severidad:** FATAL | **Código de Error:** 60 (`APP_ERR_UNKNOWN_FATAL`)
- **Descripción lógica del algoritmo:** Si el sistema experimenta un colapso inesperado en tiempo de ejecución (p. ej., agotamiento de memoria dinámica o fallos no tipificados en el *pipeline* de datos), la rutina de gestión de errores solo recurre al código 60 tras agotar dos vías previas: primero consulta si alguna capa ha fijado explícitamente una causa fatal, y después si existe algún código persistente registrado en el anillo de incidencias. El genérico es, por tanto, el último recurso y no la opción por defecto. Al tratarse de un fallo en vuelo (`ctrl_state > 0`), el *firmware* guarda el registro en la memoria Flash NVS (*Caja Negra*) y ejecuta de forma inmediata un reinicio por *software* (`os_system_reset()`) para desfragmentar la memoria y restablecer la continuidad del servicio.

Decisión de diseño del HAL: Adaptabilidad de canales ópticos

DD-01: Inversión de canales ópticos en tiempo de compilación

- **Fichero:** `app_hw.h` (declaración) | `app_hw.c` (uso)
- **Mecanismo:** Constante de preprocesador `SWAP_RED_IR_CHANNELS`, evaluada mediante compilación condicional.
- **Naturaleza:** DECISIÓN DE DISEÑO | **Estado actual:** activada (valor 1).
- **Justificación de diseño:** El controlador `maxim,max30101` de Zephyr entrega las muestras en el orden de canal que impone el modo de operación configurado en el sensor, orden que no coincide necesariamente con la asignación rojo/infrarrojo que espera la cadena de procesado. Una discordancia de este tipo no impide medir, pero invierte el cociente R de la Ecuación 2.3 y produce valores de SpO_2 sistemáticamente erróneos, con el agravante de que la señal sigue pareciendo válida. Resolverlo con una constante de compilación permite corregir la asignación sin modificar el controlador de Zephyr ni rediseñar el soporte físico. En la configuración empleada en este Trabajo la constante está activa, es decir, los canales *se intercambian* antes de propagarse al resto del firmware.
- **Limitación asumida:** Al resolverse en tiempo de compilación, la corrección no puede ajustarse sobre un equipo ya programado. Se acepta esa rigidez porque la asignación de canales es una propiedad fija de la combinación sensor-controlador, no un parámetro de uso.

DD-02: Parámetros de configuración del sensor en `prj.conf`

- **Fichero:** `prj.conf` | **Mecanismo:** Directivas `Kconfig` del controlador `maxim,max30101`.
- **Naturaleza:** DECISIÓN DE DISEÑO | **Efecto:** Determinan la cadencia, la amplitud y la resolución de la señal PPG adquirida.
- **Justificación de diseño:** Los cuatro parámetros siguientes fijan el compromiso entre calidad de señal y consumo:
 - **Frecuencia de conversión** (`CONFIG_MAX30101_SR=2`): selecciona 200 Hz de tasa de conversión en el sensor.

- **Promediado en el sensor** (`CONFIG_MAX30101_SMP_AVE=1`): promedia **dos** muestras adyacentes por canal dentro del propio integrado. Conviene subrayar que el valor 1 no significa «sin promediado»: la ausencia de promediado corresponde al valor 0. La combinación de ambos parámetros produce la **cadencia efectiva de 100 Hz** exigida por RF-1, es decir, una muestra cada 10 ms, que es también el valor de `SAMPLE_INTERVAL_MS` en `app_common.h`. Promediar en el sensor en lugar de en el microcontrolador reduce a la mitad el tráfico por el bus I²C y atenúa el ruido de conversión sin coste de CPU.
- **Corriente de pulso de los LED** (`CONFIG_MAX30101_LED1_PA = CONFIG_MAX30101_LED2_PA = 0x2F`): la escala del controlador es de 0,2 mA por unidad, de modo que el valor 47 corresponde a 9,4 **mA por LED** y a 18,8 mA de consumo conjunto durante el pulso. Es un compromiso: una corriente mayor mejora la relación señal-ruido en tejidos poco perfundidos, pero eleva el consumo y la temperatura del encapsulado, que a su vez desplaza la respuesta espectral de los emisores.
- **Rango del ADC del sensor** (`CONFIG_MAX30101_ADC_RGE=2`): fija el fondo de escala del convertidor interno en 31,25 pA/LSB. El parámetro no controla la resolución en bits —fija en el integrado— sino la corriente de fotodiodo que corresponde al valor máximo, y por tanto el punto en que la señal satura.

DD-03: Extracción del diagnóstico por USB y no por radio

- **Fichero:** `app_console.c` (consola USB) | `ble_task.c` (característica de control).
- **Naturaleza:** DECISIÓN DE DISEÑO DEL PROTOCOLO | **Efecto:** Reduce la superficie de ataque del enlace inalámbrico.
- **Justificación de diseño:** La característica de control GATT admite **únicamente dos órdenes** de un byte: 0x00 para detener la transmisión y 0x01 para iniciarla. El acceso al informe de fallos no se expone por radio. La razón no es la falta de cifrado —el enlace va cifrado y exige emparejamiento, según la Sección 3.2.5— sino que un equipo emparejado lo está para *medir*, no para administrar; y a ello se suma que el dispositivo apaga la radio al declarar un fallo, es decir, justo cuando el informe resultaría interesante. Cada orden adicional expuesta inalámbricamente es una entrada que habría que validar, autenticar y proteger. Restringir la superficie de escritura al mínimo imprescindible es una medida de seguridad más sólida que validar cuidadosamente una superficie amplia.
- **Protección adicional en el estado de error:** La orden de *inicio* (0x01) se rechaza con el código `ATT_BT_ATT_ERR_VALUE_NOT_ALLOWED` cuando el dispositivo se encuentra en `P_ERROR`. Un equipo que ha declarado una avería no debe poder reanudar la medida por orden remota, porque los datos que produjera no serían fiables. La orden de *parada* (0x00), en cambio, se acepta siempre: detener nunca es una operación insegura.
- **Interfaz de diagnóstico local:** La extracción se realiza por el puerto USB CDC-ACM mediante órdenes de un carácter atendidas por `app_console_run()`, según recoge la Tabla 3.6. El protocolo es deliberadamente elemental —un carácter, una respuesta de texto— para que pueda emplearse desde cualquier terminal serie sin herramientas específicas.

Orden	Función	Respuesta
h	Ayuda y señal de presencia	Lista de órdenes disponibles
2	Leer el informe de la Caja Negra	Registro completo, o NVS_REPORT EMPTY si no hay ninguno
3	Borrar el informe de la Caja Negra	NVS_REPORT CLEARED, o NVS_REPORT CLEAR_FAILED
4	Volcar el anillo de incidencias	Hasta ocho entradas con código, repeticiones y estado
5	Volcar los contadores de salud	Recuento acumulado por categoría de incidencia
F	Reformatear la partición NVS	NVS_REPORT WIPE_ARMED en la primera pulsación; sólo una segunda dentro de diez segundos ejecuta el borrado

Tabla 3.6: Órdenes de la consola de diagnóstico por USB CDC-ACM relacionadas con la Caja Negra. La consola atiende además las órdenes de gestión de vínculos Bluetooth, que se recogen en el Capítulo 5.

Obsérvese que el borrado del informe (orden 3) requiere una acción explícita del operador. El firmware **nunca vacía el anillo por iniciativa propia**: vaciarlo exige siempre esa orden —que invalida las entradas en el índice de NVS— o la de mantenimiento F, que reformatea los sectores de la partición y por eso pide doble confirmación. Lo que sí ocurre sin intervención es la rotación, y conviene no confundirla con un borrado: cuando las cuatro ranuras están ocupadas, un quinto fallo desplaza al informe *más antiguo*, nunca al más reciente. El criterio se justifica en detalle en la subsección siguiente.

3.2.3. Persistencia de fallos y Caja Negra en Flash NVS

Para asegurar la trazabilidad clínica ante incidentes imprevistos, el dispositivo integra una “Caja Negra” persistente en memoria flash basada en el subsistema NVS de Zephyr.

El registro se define en `src/core/crash_recovery.h` y ocupa **exactamente 108 bytes** sin relleno interno, propiedad garantizada por el orden en que se declaran los campos y comprobada en tiempo de compilación. La Tabla 3.7 detalla su contenido.

Por qué un anillo de cuatro informes y no uno solo. La primera implementación guardaba un único registro, y el ensayo sobre la placa mostró que esa decisión incumplía el propio requisito que la Caja Negra debía satisfacer. RR-3 exige que el informe **no se elimine de forma automática en ninguna circunstancia**; con una sola ranura, el segundo fallo en marcha sobrescribía al primero sin preguntar. En la sesión de prueba ocurrió exactamente eso: una avería del sensor a los 11 s quedó registrada, y otra a los 27 s —provocada por la misma causa física— la borró. Lo que se perdió no fue un dato cualquiera, sino *la primera manifestación* de la cadena, que en un diagnóstico suele ser la que señala a la pieza correcta.

El almacenamiento pasó por ello a un **anillo de cuatro ranuras**. Cada informe lleva un contador de orden de escritura (`save_seq`) que permite reconstruir la secuencia sin

depender de la posición física, y al guardar se elige siempre la ranura libre o, si no queda ninguna, la del informe *más antiguo*: nunca la más reciente. El coste es de $4 \times 108 = 432$ bytes sobre una partición de 16 KiB, despreciable frente a la garantía que compra.

Sobrevive, eso sí, una protección contra el bucle de reinicios, pero su criterio tuvo que cambiar con el anillo. Un equipo que no puede arrancar reinicia una y otra vez con el mismo fallo: sin freno, en cuatro vueltas las cuatro ranuras contendrían copias idénticas y habrían expulsado a los fallos anteriores. La versión inicial suprimía cualquier fallo de arranque mientras existiera un informe *reciente*, y el ensayo sobre la placa mostró que ese criterio es el equivocado: tras registrar una avería del sensor en marcha (código 20), el equipo arrancó sin el sensor conectado, detectó el código 10 —un fallo distinto, fatal y marcado como persistente— y **no lo guardó**, pese a que quedaban tres ranuras libres.

El error de fondo era confundir dos preguntas. La frescura del informe anterior responde a «¿cuántas veces queda por avisar al operador?», que es información de presentación; lo que la decisión necesita saber es «¿se trata del *mismo* fallo?». La condición compara ahora el código de error y el estado de control, que son lo que identifica al suceso, y sólo suprime la repetición exacta de un fallo de arranque. Un fallo de arranque nuevo y distinto se guarda siempre; y un fallo en marcha no se suprime nunca, porque ocurre con el equipo en servicio y aporta contexto nuevo —otro instante, otra secuencia, otro rastro de incidencias— aunque el código coincida con el del informe anterior.

El cambio de un registro único a un anillo trajo consigo un efecto secundario que sólo apareció sobre la placa, y que ilustra bien el riesgo de alterar la semántica de una función sin revisar a quién sirve. La operación de guardado pasó de *sobrescribir el registro* a *ocupar una ranura nueva*, pero dos llamadas del sistema no crean un informe: lo corrigen. Una descuenta los avisos automáticos en cada arranque; la otra anota el motivo del apagado. Ambas seguían usando la operación de guardado, de modo que cada arranque añadía al anillo una copia del mismo fallo con un aviso menos. El informe aparecía duplicado —idéntico salvo en ese contador— y, en tres arranques, las cuatro ranuras habrían acabado ocupadas por el mismo suceso, expulsando a los anteriores: la consecuencia exactamente opuesta a la que el anillo perseguía.

La corrección separa las dos operaciones. `app_hw_crash_save()` añade un informe nuevo; `app_hw_crash_update()` reescribe uno existente en su propia ranura, localizándola por su número de orden de escritura, que no cambia. Una prueba unitaria fija la regla comprobando que anotar el motivo de apagado no incrementa el número de informes guardados, y falla si se vuelve a confundir una operación con la otra.

Campo	Tipo	Bytes	Función
<code>magic</code>	<code>uint32_t</code>	4	Firma 0xDEADBEEF que distingue un registro válido de memoria sin inicializar
<code>version</code>	<code>uint16_t</code>	2	Versión del formato (4); permite rechazar registros de firmwares anteriores
<code>aux</code>	<code>uint16_t</code>	2	Contexto numérico del fallo: <i>errno</i> , milivoltios o máscara de tareas
<code>seq_num</code>	<code>uint32_t</code>	4	Última trama transmitida antes del fallo
<code>uptime_ms</code>	<code>uint32_t</code>	4	Tiempo de funcionamiento acumulado en el instante del fallo
<code>error_code</code>	<code>int16_t</code>	2	Código de la tabla maestra que originó el fallo
<code>ctrl_state</code>	<code>uint8_t</code>	1	Marcado estable de la red en el momento del fallo
<code>boots_left</code>	<code>uint8_t</code>	1	Arranques en los que aún se mostrará el aviso automático
<code>severity</code>	<code>uint8_t</code>	1	Severidad del código, copiada de la tabla maestra
<code>shutdown_reason</code>	<code>uint8_t</code>	1	Motivo del apagado ordenado, si lo hubo
<code>event_count</code>	<code>uint8_t</code>	1	Entradas válidas en el vector de eventos
<code>reserved</code>	<code>uint8_t</code>	1	Alineación; siempre cero
<code>save_seq</code>	<code>uint32_t</code>	4	Orden de escritura dentro del anillo; identifica el informe y permite ordenarlos del más reciente al más antiguo
<code>health</code>	<code>struct</code>	12	Contadores acumulados por categoría de incidencia
<code>events</code>	<code>struct[8]</code>	64	Anillo de las ocho últimas incidencias previas al fallo
<code>crc</code>	<code>uint32_t</code>	4	CRC32 de todos los campos anteriores
Total		108	

Tabla 3.7: Estructura del registro de la Caja Negra (`retained_data_t`, versión 4).

Tres decisiones de diseño merecen justificación explícita.

El contexto pesa más que el código. Los primeros doce bytes identifican *qué* falló; los setenta y seis siguientes describen *cómo se llegó* a ese fallo. Esta proporción es deliberada. En un dispositivo empujado la causa raíz casi nunca coincide con el último suceso registrado: un fallo fatal del sensor suele ir precedido de una serie de lecturas erróneas aisladas que, tomadas por separado, el firmware absorbió sin incidencia. Por ello, al escribir el registro se vuelca el anillo volátil de las ocho últimas incidencias —cada una con su código, su repetición y el estado en que se produjo— junto con los contadores acumu-

lados de salud. Sin ese contexto, el técnico recibe un código de error; con él, recibe una secuencia.

Integridad verificable. La firma `magic` por sí sola no basta: una escritura interrumpida por una pérdida de alimentación puede dejar la firma correcta y el resto del registro a medias. El campo `version` permite además detectar registros escritos por una versión anterior del firmware, cuyo significado sería distinto. Un registro solo se acepta si la firma, la versión y el CRC32 concuerdan; en caso contrario se descarta en lugar de presentar información posiblemente falsa.

El informe no se borra solo. El campo `boots_left` se inicializa a `CRASH_REPORT_MAX_BOOTS` (3) y se decrementa en cada arranque en que el aviso se imprime automáticamente por consola. Al llegar a cero **el registro permanece intacto**: lo único que cesa es el aviso automático. La eliminación exige una orden explícita de la consola USB —la `3`, que invalida las entradas en el índice de NVS, o la `F`, que reformatea la partición—, al margen de la rotación que desplaza al informe más antiguo cuando las cuatro ranuras están ocupadas. El criterio es que un sistema de caja negra que descarta su propio contenido tras un arranque aparentemente correcto destruye precisamente la evidencia de los fallos intermitentes, que son los más difíciles de diagnosticar y los que más justifican la existencia del subsistema.

El ciclo de vida completo del registro es el siguiente:

1. **Escritura.** Al entrar en `P_ERROR`, la acción `action_enter_error()` compone el registro en RAM —determinando la causa por orden de preferencia: código fatal fijado explícitamente, último código persistente registrado o, en su defecto, el genérico 60—, copia el anillo de incidencias y los contadores de salud, y lo escribe mediante el subsistema NVS de Zephyr, que aplica nivelación de desgaste sobre la partición reservada.
2. **Lectura en el arranque.** Tras el reinicio, `main()` valida firma, versión y CRC. Si el registro es válido y `boots_left` no ha llegado a cero, imprime por consola el código, su nombre simbólico, el estado en que se produjo, el tiempo de funcionamiento, el motivo de apagado y el anillo de incidencias previas; después decrementa `boots_left` y reescribe el registro.
3. **Protección frente a bucles de reinicio.** Antes de ocupar una ranura nueva, `action_enter_error()` comprueba tres condiciones simultáneas: que el fallo se haya producido en `P_BOOT`, que exista un informe anterior legible, y que ese informe tenga *el mismo código de error y el mismo estado de control* que el que se acaba de producir. Si las tres se cumplen, el firmware deduce que el equipo está atrapado en un ciclo de reinicios y **no duplica el informe**. La tercera condición es la que hace correcta a la regla, y su formulación costó un ensayo sobre la placa: identifica al suceso por lo que realmente lo distingue. El contador `boots_left` no interviene aquí —es información de presentación, no de identidad—, y usarlo, como hacía la primera versión, impedía registrar un fallo de arranque *nuevo y distinto* cuando aún quedaba un informe anterior por mostrar.

3.2.4. Adquisición dirigida por la FIFO del sensor

La primera implementación de la adquisición leía una muestra cada 10 ms mediante una espera del sistema operativo. El razonamiento parecía sólido: si el sensor está configurado a 100 Hz, una muestra cada 10 ms respeta exactamente esa cadencia. La medida sobre el hardware demostró que no lo era, y el motivo merece exponerse porque afecta a cualquier adquisición periódica contra un periférico con reloj propio.

Por qué una espera fija no puede funcionar

El MAX30102 no deriva su cadencia del microcontrolador, sino de un oscilador interno con tolerancia de fabricación. No produce 100,000 Hz: produce 100 Hz con un error de algunas unidades porcentuales. El microcontrolador, en cambio, duerme exactamente lo que se le pide. Son dos relojes independientes que nadie sincroniza, y su desfase es acumulativo.

Las consecuencias no son simétricas. Si el firmware lee más deprisa de lo que el sensor produce, encuentra la FIFO vacía y repite la última muestra: la señal se deforma con mesetas artificiales. Si lee más despacio —el caso realmente peligroso—, la FIFO se va llenando hasta desbordar, y a partir de ese punto el sensor **descarta muestras sin informar a nadie**. La señal transmitida presenta entonces huecos que ningún mecanismo del sistema detecta, y que en una exploración clínica podrían interpretarse como ausencia de pulso.

Invertir quién marca el ritmo

La solución adoptada elimina el problema en lugar de calibrarlo: el firmware deja de imponer un ritmo y pasa a seguir el del sensor. En cada vuelta consulta los punteros de la FIFO y se lleva exactamente lo que haya, ni más ni menos. La secuencia se resuelve con cuatro registros contiguos del MAX30102, recogidos en la Tabla 3.8.

Tabla 3.8: Registros consultados en cada ciclo de sondeo. Los tres primeros se leen en una única transacción por ser contiguos.

Dirección	Registro	Uso en el algoritmo
0x04	FIFO_WR_PTR	Cuántas muestras ha escrito el sensor.
0x05	OVF_COUNTER	Cuántas ha tenido que descartar por desbordamiento. Se satura en 31 y se limpia al leer los datos.
0x06	FIFO_RD_PTR	Hasta dónde ha leído el firmware.
0x07	FIFO_DATA	Zona de datos; el puntero de lectura avanza solo dentro del chip, de modo que un lote entero se extrae con una única lectura en ráfaga.

El número de muestras pendientes se obtiene por resta modular, $n = (\text{WR} - \text{RD}) \bmod 32$. Con los punteros iguales la situación es ambigua —la FIFO puede estar vacía o completamente llena—, y es el contador de desbordamiento el que deshace la ambigüedad: si vale cero, está vacía.

Hacer visible la única pérdida inevitable

El registro 0x05 es la pieza que convierte este diseño en algo más que una optimización. El sensor informa del *número exacto* de muestras que ha tenido que descartar, y el firmware registra ese valor como código 23 (SENSOR_FIFO_OVERFLOW) con la cuenta en el campo *aux*. La pérdida en el origen es la única que el firmware no puede evitar por completo—depende de que el planificador le devuelva la CPU a tiempo—, y es precisamente por eso la que más importa dejar anotada. Un hueco en la señal deja de ser invisible y pasa a ser un dato con nombre, cantidad e instante.

El margen disponible es amplio: la FIFO admite 32 muestras y se sondea cada 25 ms, de modo que en cada vuelta se acumulan del orden de dos o tres. Desde que una vuelta termina hasta que el sensor empezaría a descartar transcurren unos 320 ms, más de doce veces el periodo de sondeo. Ese factor es el que absorbe los retrasos por prioridad de otros hilos.

Efectos colaterales del cambio

Sustituir la lectura por temporizador tuvo tres consecuencias que conviene explicitar:

- **El coste del bus deja de escalar con las muestras.** Antes se ejecutaban tres accesos I²C por muestra; ahora, cuatro transferencias por lote con independencia de su tamaño. El efecto medido sobre el hardware se cuantifica en la Sección 8.5.7.
- **El sello de tiempo debe corregirse hacia atrás.** Las muestras de un lote ya estaban en la FIFO cuando se consultó, de modo que el instante de lectura no es el de captura. El campo *t_ms* de la trama se retrasa en $(n - 1) \times 10$ ms para referirlo a la primera muestra del lote y no al momento en que se extrajo.
- **El periodo de trama deja de ser exacto y pasa a ser exacto en media.** Como una trama se cierra al acumular 20 muestras y los lotes son de tamaño entero, el número de sondeos necesarios oscila entre siete y ocho. El fenómeno, su magnitud y su carácter autocompensado se analizan con las trazas en la Sección 8.5.4.

3.2.5. Seguridad del enlace de radio

Las tramas que el dispositivo emite contienen señal fisiológica asociada a una donación identificada. Transmitirlas en claro dejaría cualquier equipo cercano en condiciones de leerlas, y también de ordenar el arranque o la parada de la adquisición: la característica de control admite escritura, de modo que un enlace abierto no solo expone el dato sino que cede el mando del instrumento.

Cifrado obligatorio en todas las características

El firmware exige enlace cifrado en cada característica del servicio. La política es un parámetro de compilación, APP_BLE_REQUIRE_ENCRYPTION, que conmuta los permisos GATT entre sus dos variantes:

```
#define APP_PERM_READ    BT_GATT_PERM_READ_ENCRYPT
#define APP_PERM_WRITE   BT_GATT_PERM_WRITE_ENCRYPT
```

Concentrar la decisión en dos macros evita el modo de fallo más probable en este terreno: proteger unas características y olvidar otra. La característica de notificación es el único

caso particular —sus datos no se leen, se envían— y su protección recae sobre el descriptor CCC, que sí lleva los permisos cifrados: sin poder escribir en él, un cliente no autorizado no puede suscribirse.

Clave fija: por qué, y qué protege realmente

El equipo no tiene pantalla ni teclado. No puede mostrar una clave aleatoria ni recibir la que el operador teclee, lo que reduce las alternativas de emparejamiento a dos. La primera, *Just Works*, cifra el enlace pero no autentica al otro extremo: como se expone en la Sección 2.4, un intermediario situado entre los dos equipos puede emparejarse con ambos sin que ninguno lo advierta [27]. La segunda consiste en declarar una **clave fija** conocida de antemano, impresa en la etiqueta del dispositivo.

Se ha adoptado la segunda. Con ella, Zephyr anuncia capacidades de solo visualización (`BT_SMP_IO_DISPLAY_ONLY`) y el emparejamiento se resuelve por introducción de clave, que sí autentica y alcanza el nivel de seguridad 3. La clave se declara en `ble_task.h` y se registra en el arranque mediante `bt_passkey_set()`.

Conviene una precisión adicional sobre el nivel alcanzado, porque el número depende del cliente y no solo del dispositivo. La configuración exige emparejamiento autenticado y cifrado, lo que garantiza el nivel 3 como mínimo; si el cliente admite *LE Secure Connections* —como ocurrió en todas las sesiones medidas— la negociación alcanza el nivel 4, que añade el intercambio de claves por curva elíptica. Lo que la configuración *no* hace es rechazar al cliente que no lo admita: forzarlo exigiría activar `CONFIG_BT_SMP_SC_ONLY`, a costa de excluir equipos antiguos. Es una decisión consciente de compatibilidad, y se recoge entre las líneas futuras.

Conviene ser preciso también sobre el alcance de la clave, porque una clave fija invita a exagerar lo que ofrece. **No es un secreto criptográfico**: quien tenga el dispositivo en la mano la lee de la etiqueta. Lo que impide es la conexión casual de un equipo cercano y el ataque de intermediario durante el emparejamiento, que es exactamente la superficie de riesgo de una sala de donación. Un despliegue con requisitos superiores exigiría claves por dispositivo provisionadas en fabricación.

Persistencia de las claves y reparto de la flash

Para que el vínculo sobreviva a un reinicio, las claves se guardan con el subsistema `settings` de Zephyr. Esa decisión, aparentemente menor, obligó a reorganizar el mapa de memoria no volátil.

El motivo es que `settings` está fijado *por código* a la partición `storage_partition` —así lo declara `subsys/settings/src/settings_nvs.c`— y no admite redirección sin parchear Zephyr. Esa es la misma partición que empleaba la Caja Negra descrita en la Sección 3.2.3, que monta su propia instancia NVS. Dos instancias NVS independientes escribiendo sobre el mismo espacio se corromperían mutuamente, y el fallo no sería inmediato ni evidente: aparecería como un informe de fallo ilegible mucho después.

La solución fue partir en dos los 32 KiB que el mapa estándar del nRF52840 reserva al final de la flash, dejando `settings` en la partición que tiene fijada y mudando la Caja Negra a una propia:

Dirección	Partición	Tamaño	Contenido
0x000f8000	storage	16 KiB	Claves de emparejamiento (settings)
0x000fc000	blackbox	16 KiB	Informe de fallo y anillo de incidencias

Tabla 3.9: Reparto de la partición de almacenamiento entre las claves Bluetooth y la Caja Negra. Cada mitad son cuatro sectores de flash.

Al cambiar el mapa, una reprogramación incremental dejaría la Caja Negra leyendo una dirección que ya no le corresponde: la actualización exige borrado completo de la flash, y los informes anteriores se pierden.

Política de vínculos: cuántos, y qué ocurre al llenarse

El dispositivo admite cuatro equipos emparejados a la vez. Alcanzado el límite, el comportamiento por omisión de Zephyr es rechazar todo emparejamiento nuevo: `bt_keys_get_addr()` devuelve nulo y la capa SMP responde con `-ENOMEM`. En una sala de donación eso dejaría el equipo inservible para cualquier móvil que no estuviera ya vinculado, con un mensaje que el operador no podría interpretar.

Se ha activado por ello el desalojo automático del vínculo más antiguo que no esté en uso, que protege explícitamente al equipo conectado en ese momento. La contrapartida es que el desalojo es **silencioso para el desalojado**: fallará en su siguiente conexión sin motivo aparente. Para que esa situación sea diagnosticable, el firmware anota el desalojo en la Caja Negra con el código 47 desde la devolución `bond_deleted`, del mismo modo que anota con el código 46 los emparejamientos que fracasan.

El control explícito recae en la consola USB, que ofrece tres órdenes descritas en el Capítulo 5: listar los vínculos con su índice y marcar el activo, borrar uno concreto, y borrar todos. Ninguna de las tres se expone por radio, por la razón que allí se detalla: un cliente emparejado lo está para medir, no para administrar.

4. Verificación formal del modelo de control

El Capítulo 3 ha descrito cómo se implementa el control del dispositivo mediante una red de Petri interpretada. Este capítulo aborda una pregunta distinta y complementaria: cómo se *demuestra* que esa lógica es correcta.

La respuesta no es ejecutar más pruebas. Un sistema reactivo y concurrente falla, cuando lo hace, en entrelazados de eventos que no se dieron durante las pruebas; y no darse no significa no ser posible. La técnica empleada, la comprobación de modelos introducida en la Sección 2.6, invierte el planteamiento: en lugar de recorrer las trazas que el planificador decide producir, explora *exhaustivamente* todas las que el sistema admite.

4.1. Del firmware al modelo en Promela

El modelo, contenido en `formal/model.pml`, reproduce las 25 transiciones de la tabla `transitions[]` de `app_pn.c` en el mismo orden, junto con las guardas, el contador de errores del sensor, el reinicio que sigue a un fallo en marcha y el apagado de la radio al entrar en el estado de error.

4.1.1. Alcance de la abstracción

Un modelo no es una copia del sistema: es una simplificación deliberada, y declarar qué se ha simplificado forma parte del resultado. La Tabla 4.1 recoge esa frontera.

El modelo sí representa	El modelo no representa
Los seis estados de control y los catorce eventos	Los tiempos absolutos: un vencimiento significa «el plazo expiró», no cuándo
Las 25 transiciones y su orden de prioridad	La duración concreta de cada temporizador
Las guardas <code>can_stream</code> y su negación	El contenido de las tramas ni la señal adquirida
El contador de errores del sensor y su umbral	La capa HAL y los periféricos físicos
El reinicio tras un fallo en marcha	La escritura efectiva en memoria no volátil
El apagado de la radio al declarar el error	La pila Bluetooth y su comportamiento interno

Tabla 4.1: Frontera de la abstracción: qué elementos del firmware se trasladan al modelo y cuáles se omiten deliberadamente.

La omisión del tiempo es la más significativa. El modelo razona sobre el *orden* de los sucesos, no sobre su instante. Esto basta para las propiedades que interesan —que no se transmita en estado de error, que el arranque termine, que un aviso de batería conduzca al apagado— y evita el coste de un modelo temporizado, que exigiría otra herramienta y otro formalismo.

4.1.2. Codificación de la prioridad de las transiciones

Existe una discrepancia semántica entre el motor real y la construcción natural de Promela que, de no corregirse, invalida el modelo. El motor `rx_pn.tick()` recorre la tabla en orden disparando todas las transiciones habilitadas, pero aplica el efecto de cada disparo antes de evaluar la siguiente. De ahí se sigue la regla de prioridad que el modelo debe reproducir: entre transiciones que compiten por una misma marca se dispara **la declarada antes**, que la consume y deja inhabilitadas a las demás.

Conviene enunciar esa regla sin excederse, porque no equivale a que sólo cambie de estado una transición por ciclo. Como el efecto se aplica de inmediato, la marca que una transición deposita en su estado de destino puede habilitar a otra declarada *después* que parta de ese destino. Si en un mismo tic quedan marcados los sucesos de inicio y de parada de transmisión —el drenaje vacía en cada ciclo la cola completa de eventos pendientes—, la transición 14 lleva de reposo a transmisión y la 18, examinada más adelante en esa misma pasada, devuelve a reposo. La cascada está acotada porque la pasada es única y sólo avanza: ninguna transición anterior a la que ya disparó vuelve a examinarse en ese ciclo. Lo que el modelo debe reproducir es, por tanto, el orden de examen, no una supuesta unicidad de disparo. La construcción `do : : . . . od` de Promela, en cambio, elige de forma *no determinista* entre todas las opciones habilitadas.

Un modelo escrito sin más admite entonces ejecuciones que el firmware nunca produce, como atender una orden de inicio de transmisión teniendo pendiente un aviso de batería baja. Verificar sobre ese modelo produce dos clases de resultado engañoso: propiedades

refutadas por contraejemplos imposibles, y un espacio de estados inflado con comportamientos irreales.

La solución adoptada consiste en añadir a cada rama una guarda que excluya la existencia de transiciones de mayor prioridad, mediante tres macros que reproducen los cinco niveles descritos en el Capítulo 3:

```
#define NO_FATAL      (!evt_fatal_error)
#define URGENT_CLEAR  (!evt_fatal_error && !evt_low_battery)
#define STR_FLOW      (URGENT_CLEAR && !evt_disconnected
                        && !evt_silent_disconnect)
```

Esta corrección no es un detalle de estilo. Como se detalla en la Sección 4.4, hay al menos una propiedad que solo se sostiene porque la prioridad está codificada.

4.1.3. El entorno y la cota de reinicios

El modelo consta de dos procesos concurrentes. El primero, `petri_net_engine`, reproduce el motor. El segundo, `environment`, genera los eventos externos: representa al usuario, al cliente Bluetooth, al sensor y a la batería, y produce en cada estado cualquier evento que sea físicamente posible en él. Es deliberadamente adversario: no supone ninguna secuencia razonable de uso, sino todas las secuencias posibles.

El entorno incorpora dos restricciones que merecen justificación explícita, por ser artefactos de la abstracción y no propiedades del firmware:

- **Cota de reinicios encadenados** (`MAX_REBOOTS = 2`). Un fallo en marcha provoca un reinicio, que devuelve el sistema al arranque, donde puede volver a fallar. Sin cota, el espacio de estados no sería finito y la verificación no terminaría. La cota no limita el firmware: limita la profundidad de la cadena de reinicios que el modelo explora.
- **Salida del estado de error al agotar la cota.** Alcanzado el límite, un fallo en marcha pasa a tratarse como fallo de arranque, es decir, con ventana de diagnóstico y apagado posterior. Sin esta previsión el estado de error se quedaba sin ninguna salida —no podía reiniciar por haber agotado la cota, ni recibir el vencimiento por no haberlo armado— y el modelo entraba en *deadlock*. Conviene subrayar que ese bloqueo era un defecto de la abstracción y no del firmware, distinción que solo puede establecerse examinando el contraejemplo.

El entorno termina cuando el sistema alcanza el sueño profundo. De no hacerlo, SPIN señalaría un estado final inválido que no corresponde a ningún fallo real.

4.2. Propiedades verificadas

Se han formulado **diecisiete propiedades** en lógica temporal lineal, agrupadas en cuatro familias. Las tres primeras corresponden a las categorías clásicas introducidas en la Sección 2.6; la cuarta recoge reglas propias de este sistema.

4.2.1. Seguridad

Afirman que algo indeseable no ocurre nunca, y tienen la forma $\Box \neg \varphi$. Se comprueban sin hipótesis de equidad: un contraejemplo es una traza finita que alcanza el estado prohibido.

Propiedad	Enunciado
p1	Nunca se dan simultáneamente el estado de error y la condición de transmisión habilitada
p2	El sueño profundo es terminal: alcanzado, el sistema no lo abandona
p3	En transmisión, el contador de errores del sensor nunca supera su umbral
p14	En el estado de error la radio está siempre apagada

Tabla 4.2: Propiedades de seguridad verificadas.

La propiedad p14 formaliza una corrección concreta del firmware. Un equipo averiado que continúe anunciándose puede recibir una conexión, y esa conexión cancelaba el plazo que debía llevarlo al sueño profundo, dejándolo indefinidamente en estado de error. Apagar la radio al declarar el fallo elimina la causa; p14 convierte esa decisión en una propiedad comprobable.

4.2.2. Vivacidad

Afirman que algo deseable acaba ocurriendo. Su contraejemplo no es un estado sino un ciclo infinito, por lo que se verifican bajo hipótesis de equidad débil (opción $\neg f$ de SPIN).

Propiedad	Enunciado
p4	El arranque siempre termina: el sistema no permanece en él indefinidamente
p5	El estado de error nunca es terminal: se sale por sueño profundo o por reinicio
p5b	Un fallo ocurrido en el arranque conduce siempre al sueño profundo
p6b	Un aviso de batería baja nunca se ignora indefinidamente
p7	El estado de anuncio no es un bloqueo: siempre se abandona

Tabla 4.3: Propiedades de vivacidad verificadas bajo equidad débil.

4.2.3. Alcanzabilidad

Comprueban que ciertos estados son efectivamente alcanzables, lo que descarta que una guarda mal escrita haya dejado una parte del sistema muerta. Su formulación exige una

precaución: la forma $\Diamond$ estado **no sirve**, pues afirma que *todos* los caminos llegan al estado, y su contraejemplo solo demuestra que existe uno que no llega, que es otra cosa. Se escriben por tanto *negadas*, como $\Box \neg$ estado, y es el contraejemplo —la traza concreta que sí alcanza el estado— el que constituye la prueba. En estas cuatro propiedades, y solo en ellas, `errors:1` es el resultado correcto.

Propiedad	Enunciado
p8	El estado de transmisión es alcanzable
p9	El estado de error es alcanzable
p10	El sueño profundo es alcanzable
p15	La recuperación del sensor es alcanzable: existe una ejecución en la que una racha de errores queda rota por una lectura correcta y la transmisión continúa

Tabla 4.4: Propiedades de alcanzabilidad, formuladas de forma negada.

La propiedad p15 merece un comentario, porque su presencia responde a un defecto real detectado al contrastar el modelo con el código. El requisito RR-1 exige tolerar tres fallos *consecutivos* del sensor, pero ni el firmware ni el modelo devolvían el contador a cero tras una lectura correcta: contaban fallos acumulados dentro de la sesión. La corrección introduce una transición de recuperación, y p15 comprueba que esa transición no es código muerto —que existe realmente una ejecución en la que el sensor falla, se recupera y el flujo continúa—. Lo que p15 *no* demuestra es la propiedad completa «tres fallos separados por lecturas correctas nunca provocan el apagado»: expresarla en LTL exigiría una variable de historia, y se comprueba en su lugar mediante pruebas unitarias dedicadas (Sección 8.2). Es un ejemplo del reparto de competencias entre ambas técnicas que se discute al final de este capítulo.

4.2.4. Lógica de negocio

Recogen reglas específicas del dispositivo que no encajan en las categorías anteriores.

Propiedad	Enunciado
p6	Cuando la red consume un aviso de batería baja, el resultado es siempre el apagado
p11	Alcanzado el umbral de errores del sensor, el sistema está en error o apagado
p12	Al volver al estado de anuncio el contador de errores está a cero
p13	Una desconexión silenciosa siempre saca al sistema de la transmisión

Tabla 4.5: Propiedades específicas del dispositivo.

La propiedad p12 garantiza que los errores de una sesión de transmisión no se arrastran a

la siguiente: sin ese reinicio del contador, tres fallos aislados repartidos a lo largo de horas de uso acabarían declarando una avería inexistente.

4.3. Resultados

La verificación se ejecuta mediante el guion `run_verification.sh`, que traduce el modelo con `spin -a`, compila el verificador generado y lo invoca una vez por propiedad con `./pan -a -f -N propiedad`. La Tabla 4.6 recoge la salida completa.

Propiedad	Familia	Estados	Prof.	Resultado
p1_no_stream_in_error	Seguridad	621 106	4024	Verificada
p2_deep_sleep_is_terminal	Seguridad	962 446	4024	Verificada
p3_sensor_count_bounded	Seguridad	621 106	4024	Verificada
p14_radio_off_in_error	Seguridad	621 106	4024	Verificada
p4_boot_terminates	Vivacidad	621 176	4024	Verificada
p5_error_never_terminal	Vivacidad	649 866	4024	Verificada
p5b_boot_error_sleeps	Vivacidad	621 126	4024	Verificada
p6b_low_battery_not_ignored	Vivacidad	766 258	4024	Verificada
p7_advertising_no_deadlock	Vivacidad	685 618	4024	Verificada
p8_streaming_reachable	Alcanzabilidad	180	95	Alcanzable
p9_error_reachable	Alcanzabilidad	5	8	Alcanzable
p10_sleep_reachable	Alcanzabilidad	6	10	Alcanzable
p15_recuperacion_alcanzable	Alcanzabilidad	22 984	1453	Alcanzable
p6_low_battery_shuts_down	Negocio	621 106	4024	Verificada
p11_three_errors_fatal	Negocio	621 106	4024	Verificada
p12_advertising_clears_counter	Negocio	621 106	4024	Verificada
p13_silent_disc_recovers	Negocio	688 690	4024	Verificada

Tabla 4.6: Resultados de la verificación con SPIN 6.5.2. Ninguna propiedad resultó violada.

En las propiedades de seguridad, vivacidad y lógica de negocio —las que exigen recorrer el espacio completo— la exploración se sitúa entre 621 000 y 962 000 estados según la propiedad, con un vector de estado de 52 bytes y una profundidad máxima de 4024. Las de alcanzabilidad quedan por debajo en varios órdenes de magnitud, entre 5 y 22 984 estados, porque les basta con hallar un camino. La verificación completa de las diecisiete propiedades se resuelve en menos de diez segundos, lo que la hace apta para integrarse en el ciclo de desarrollo y no solo como comprobación final.

El contraste entre las dos primeras familias y la de alcanzabilidad resulta ilustrativo: las propiedades de seguridad y vivacidad exigen recorrer el espacio completo, mientras que a las de alcanzabilidad les basta con encontrar *un* camino, y lo hacen en cinco u ocho estados. La asimetría es la esperada: demostrar que algo nunca ocurre requiere mirarlo todo; demostrar que algo puede ocurrir requiere un solo ejemplo. El caso de p15 matiza la regla sin contradecirla: necesita 22 984 estados porque lo que busca no es un estado suelto

sino una secuencia —fallar, recuperarse y seguir transmitiendo—, y aun así se resuelve con más de un orden de magnitud menos de exploración que cualquier propiedad de seguridad.

4.4. Lecciones del proceso de verificación

Los resultados de la tabla anterior muestran diecisiete propiedades satisfechas, pero ese estado final oculta lo más instructivo del proceso. Tres episodios merecen constar, por ilustrar los modos de fallo característicos de esta técnica.

Una propiedad verificada sobre un modelo incompleto no demuestra nada. Durante el desarrollo, la propiedad que exigía la salida del estado de error figuraba como *verificada* mientras el firmware real la incumplía: un cliente que se conectaba durante la ventana de diagnóstico cancelaba el plazo de sueño y el equipo quedaba en error indefinidamente. El modelo no lo detectaba porque no representaba ni los plazos ni el efecto de la conexión sobre ellos. El defecto no estaba en la propiedad ni en el verificador, sino en la frontera de la abstracción. La corrección consistió en introducir la variable que modela el estado de la radio y la propiedad `p14` que la gobierna. Este episodio delimita el alcance real de la técnica: **verificar un modelo demuestra propiedades del modelo, no del código.**

Una propiedad mal formulada puede pasar desapercibida. Dos de las diecisiete propiedades resultaron ser, en versiones anteriores, copias literales de otras: una duplicaba el enunciado sobre el sueño profundo sin mencionar la batería, y otra duplicaba la cota del contador sin comprobar ningún reinicio. Ambas figuraban como verificadas, lo que es coherente: una propiedad duplicada se cumple exactamente igual que su original. Una batería de propiedades que se satisface por completo no es, por sí sola, garantía de nada; hace falta revisar que cada enunciado diga lo que se pretendía decir.

La equidad de SPIN es de procesos, no de ramas. La propiedad `p6b` —que un aviso de batería baja no se ignore indefinidamente— resultaba refutada de forma persistente. El contraejemplo mostraba al motor eligiendo eternamente otras transiciones sin atender nunca la batería. La causa no era el firmware, sino la construcción no determinista descrita en la Sección 4.1.2: la hipótesis de equidad débil que activa la opción `-f` garantiza que todo *proceso* listo acabe avanzando, pero no que cada *rama* de un *do* acabe seleccionándose. Codificar la prioridad de las transiciones eliminó el contraejemplo y, de paso, redujo apreciablemente el espacio de estados al suprimir entrelazados que el sistema real nunca genera. La propiedad `p6b` solo se sostiene, por tanto, porque el modelo reproduce el determinismo del motor.

Conviene cerrar con la delimitación que se desprende de todo lo anterior. La verificación formal garantiza que el *modelo de control* está libre de bloqueos no deseados en los estados operativos —`DEEP_SLEEP` es absorbente por diseño, y la propiedad `p2` lo verifica— y de las violaciones enunciadas, y que los estados y recorridos enumerados en el grupo de alcanzabilidad son efectivamente alcanzables —lo que no equivale a haber demostrado que no exista ningún otro estado inalcanzable en el modelo—. No garantiza que la implementación en C sea correcta, ni que la capa de abstracción funcione, ni que los plazos estén bien dimensionados. Esas garantías proceden de los otros dos niveles de validación,

que se presentan en el Capítulo 8. Los tres son complementarios y ninguno sustituye a los demás.

5. Herramienta de extracción diagnóstica

Este capítulo describe `error_inspector.py`, la herramienta de escritorio que cierra el ciclo de diagnóstico iniciado por la Caja Negra descrita en la Sección 3.2.3. Su cometido cubre tres frentes: extraer y presentar de forma legible el registro de fallos almacenado en la memoria no volátil del dispositivo; permitir al personal técnico borrarlo de manera explícita y auditada; y, desde que el enlace exige emparejamiento (Sección 3.2.5), administrar los vínculos Bluetooth guardados cuando uno de ellos deja de responder o la lista se llena.

5.1. Decisiones de diseño

Dos decisiones condicionan toda la arquitectura de la herramienta y conviene justificarlas antes de describirla.

5.1.1. Conexión exclusivamente por USB

La herramienta se comunica con el dispositivo **únicamente por el puerto serie virtual USB CDC-ACM**. No contempla ningún canal inalámbrico, porque el firmware no expone el informe de fallos por radio: la característica de control GATT admite solo las órdenes de inicio y parada de la transmisión, según se justifica en la Sección 3.2.

La razón es doble, y la primera es determinante: **cuando el equipo declara un fallo, apaga la radio**. La acción `action_enter_error()` detiene el anuncio y corta la conexión activa, de modo que en el estado en que el informe resulta interesante no queda ningún canal inalámbrico por el que consultarlo. Un diagnóstico que dependiera de la radio dejaría de funcionar precisamente cuando hace falta.

La segunda es de control de acceso. Aunque el enlace va cifrado y exige emparejamiento, cualquier equipo emparejado lo está para *medir*, no para administrar: exponer la lectura o el borrado del registro por radio permitiría a un cliente cualquiera manipular el historial de incidencias de otro. Restringir esas órdenes al puerto físico convierte la posesión del dispositivo en el mecanismo de autorización, que es el criterio adecuado para una operación de mantenimiento. Como efecto colateral, la extracción no interfiere con la sesión de medida ni consume ancho de banda de radio.

5.1.2. Una única tabla de códigos, compartida con el firmware

Una herramienta de diagnóstico que mantenga su propia copia de la tabla de códigos está condenada a desincronizarse: basta con añadir un código al firmware y olvidar actualizarla para que el técnico lea una descripción equivocada, que es peor que no leer ninguna.

Por ello la herramienta **no contiene ninguna tabla de errores escrita a mano**. En su arranque localiza `app_errors.h` —el mismo fichero que compila el firmware— y extrae de él las entradas de `APP_ERROR_TABLE` mediante una expresión regular, obteniendo para cada código su nombre simbólico, su severidad, su persistencia y su descripción. La búsqueda recorre varias ubicaciones en orden de preferencia: la ruta relativa dentro del árbol del proyecto y, cuando la herramienta se distribuye empaquetada con PyInstaller, el directorio temporal de datos que este genera.

Si la cabecera no se encuentra, la herramienta no aborta: continúa con la tabla vacía. Esto es posible porque el firmware transmite, junto a cada código numérico, su nombre simbólico, de modo que el informe sigue siendo interpretable aunque se pierda la descripción larga. Se prefiere una degradación parcial a un fallo total.

5.2. Arquitectura de la aplicación

La interfaz gráfica se construye sobre `tkinter`, incluido en la biblioteca estándar de Python, y la comunicación serie sobre `pyserial`. Son las dos únicas dependencias, criterio deliberado: una herramienta de diagnóstico debe poder ejecutarse en el ordenador que haya disponible en el momento de la avería, sin preparar un entorno.

El bucle de eventos de `tkinter` es bloqueante, de modo que leer del puerto serie desde el hilo principal congelaría la ventana. La solución adoptada es un **hilo de escucha dedicado** (*daemon thread*) que lee líneas del puerto de forma continua mientras la conexión permanece abierta, y un segundo hilo periódico que envía la orden `h` como señal de presencia. Ambos entregan sus resultados al hilo de la interfaz, que es el único que manipula los objetos gráficos. No se emplea programación asíncrona: el problema —un único puerto serie con tráfico escaso— no la justifica, y los hilos mantienen el código más simple.

La conexión se establece a **115 200 baudios** sobre el puerto que el operador seleccione de la lista de puertos serie detectados en el sistema, obtenida mediante `serial.tools.list_ports`.

5.3. Protocolo de diagnóstico

El protocolo es deliberadamente elemental: el ordenador envía **un único carácter** y el dispositivo responde con líneas de texto. Esta simplicidad permite operar la Caja Negra desde cualquier terminal serie, sin la herramienta gráfica, lo que resulta valioso precisamente en las situaciones de avería en que uno no dispone de su equipo habitual. La Tabla 5.1 recoge las órdenes.

Orden	Botón en la interfaz	Efecto
h	— (automática)	Señal de presencia periódica; prolonga diez segundos la ventana de diagnóstico si el equipo está en ella
2	Informe (2)	Vuelca el registro persistente de la Caja Negra
3	Borrar (3)	Borrado lógico: invalida las entradas del registro en el índice de NVS, sin actuar sobre las celdas
4	Incidencias (4)	Vuelca el anillo volátil de las ocho últimas incidencias
5	Salud (5)	Vuelca los contadores acumulados por categoría
6	Desemparejar (6)	Borra todos los vínculos Bluetooth guardados; los clientes deberán emparejarse de nuevo
7	Vínculos (7)	Lista los equipos emparejados con su índice, marcando el que está conectado
a–d	Quitar uno	Borra un único vínculo por su índice (a=0 ... d=3)
F	— (no expuesta)	Borrado físico: reformatea los sectores de la partición. Exige doble pulsación en diez segundos y la herramienta no la envía

Tabla 5.1: Órdenes del protocolo de diagnóstico por USB. La herramienta no envía ninguna orden de control de la medida: su alcance se limita al diagnóstico.

Obsérvese que no existe ninguna orden para iniciar o detener la adquisición. La herramienta de diagnóstico y el control de la medida están separados a propósito: son funciones distintas, con operadores y momentos distintos.

El borrado (orden 3) no se deshace desde la propia interfaz, por lo que ésta interpone un diálogo modal de confirmación antes de enviarlo. Conviene precisar su alcance: invalida las entradas del registro en el índice de NVS, pero no actúa sobre las celdas de memoria; el borrado *físico* de la partición corresponde a la orden de mantenimiento F, que la herramienta no expone y que el firmware protege con doble confirmación. Ambos se corresponden con el criterio expuesto en la Sección 3.2.3: el firmware nunca vacía el anillo por iniciativa propia —sólo la rotación desplaza al informe más antiguo cuando las cuatro ranuras están llenas—, y vaciarlo exige siempre una acción humana explícita.

5.4. Formato de las tramas y decodificación

Las respuestas del firmware son líneas de texto con campos separados por el carácter |, en las que cada campo tiene la forma **CLAVE:valor**. La herramienta identifica la clase de línea por su prefijo, según recoge la Tabla 5.2, y la convierte en un diccionario de campos mediante una única función de análisis común a todos los formatos.

Prefijo	Contenido
NVS_REPORT	Cabecera del informe persistente: código, estado, tiempo de funcionamiento y motivo de apagado
NVS_EVENT	Una entrada del anillo de incidencias almacenado junto al informe
NVS_HEALTH	Contadores de salud guardados en el momento del fallo
RAM_EVENT	Una entrada del anillo volátil de la sesión en curso
RAM_HEALTH	Contadores de salud de la sesión en curso
BLE_BOND	Una entrada de la lista de vínculos Bluetooth (orden 7)
BLE BONDS	Resultado de una operación sobre los vínculos: recuento total, borrado completo o de uno solo

Tabla 5.2: Prefijos que emite el firmware y contenido asociado. La distinción NVS/RAM separa lo que sobrevivió a un reinicio de lo que solo describe la sesión actual.

La separación entre los prefijos NVS_ y RAM_ es la que permite al técnico distinguir dos preguntas distintas: qué ocurrió en el fallo que provocó el último reinicio, y qué está ocurriendo ahora. Un equipo puede no haber fallado nunca y presentar aun así un anillo volátil con incidencias recuperables, lo que revela un deterioro en curso antes de que produzca una parada.

La herramienta reconoce además las líneas del volcado automático que el firmware imprime durante el arranque cuando existe un informe pendiente. Estas líneas no llevan prefijo, por lo que se detectan mediante marcadores textuales; de este modo el operador ve el informe sin necesidad de solicitarlo si conecta el cable antes de alimentar el dispositivo.

Con la tabla cargada desde `app_errors.h`, cada código recibido se traduce a su nombre simbólico, su descripción y sus dos clasificaciones. La severidad se representa además mediante un código de color en la ventana de registro, de modo que un fallo fatal se distinga de una incidencia recuperable sin necesidad de leer el texto. Los estados de control se traducen con una tabla equivalente al enumerado `app_ctrl_state` del firmware. Los códigos que no figuren en la tabla se presentan con su valor numérico y una marca de desconocido, en lugar de ocultarse: un código no reconocido es en sí mismo una información de diagnóstico.

5.5. Interfaz de usuario

La ventana se organiza en tres zonas. En la superior se sitúan la selección del puerto serie y el botón de conexión. La central la ocupa el panel de registro, donde se acumulan cronológicamente los mensajes de sesión y los diagnósticos decodificados, con el resaltado por severidad ya mencionado. En la inferior se disponen los siete botones de acción: los cuatro de consulta —Informe, Incidencias, Salud y Vínculos— y los tres que modifican el estado del equipo —Quitar uno, Desemparejar y Borrar—, junto a una casilla que permite mostrar las tramas en crudo, útil cuando se sospecha de un problema en el propio formato de las respuestas.

Los botones de acción permanecen deshabilitados mientras no exista una conexión confirmada, lo que evita enviar órdenes al vacío e induce al operador a seguir la secuencia correcta. Los tres botones que modifican el estado del equipo se distinguen cromáticamente de los cuatro de consulta, y entre ellos el de borrado lleva el color de mayor alarma por ser el que destruye el historial de diagnóstico.

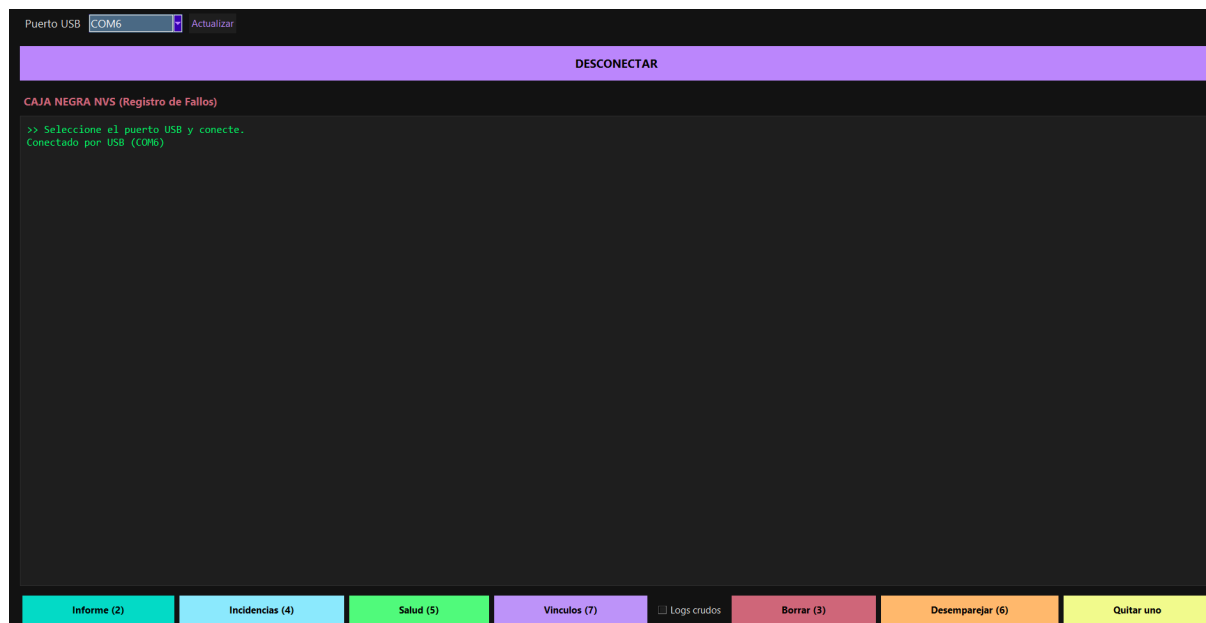


Figura 5.1: Interfaz de la herramienta tras establecer la conexión USB. El desplegable ofrece únicamente los puertos serie detectados en el sistema; no existe ningún selector de canal inalámbrico. Los siete botones de acción, deshabilitados mientras no hay enlace, quedan activos al confirmarse la conexión.

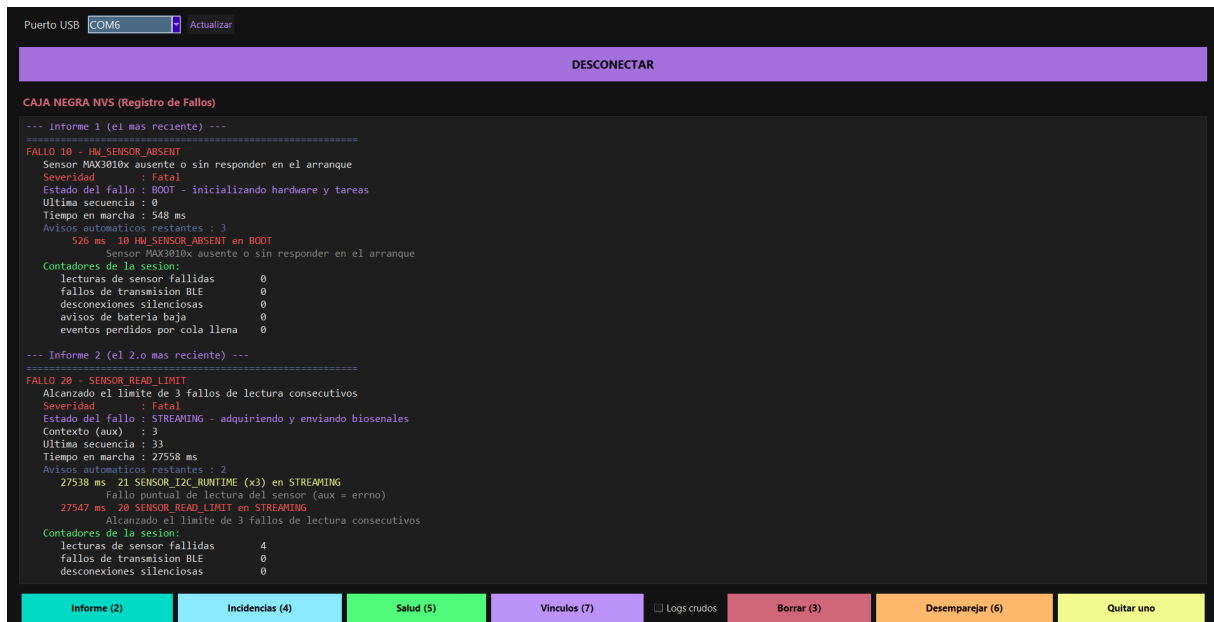


Figura 5.2: Los dos informes que la Caja Negra conservaba tras el ensayo descrito en la Sección 3.2.3, extraídos con la orden 2. El primero —el más reciente— es el código 10, `HW_SENSOR_ABSENT`, detectado en `BOOT` a los 548 ms al arrancar con el sensor desconectado. El segundo es el código 20, `SENSOR_READ_LIMIT`, ocurrido en `STREAMING` a los 27 558 ms de la sesión anterior. Lo que la figura documenta no es cada fallo por separado sino que **el más reciente no borró al anterior**: el código 10, posterior en el tiempo, aparece el primero en pantalla porque el volcado va del más nuevo al más antiguo, y bajo él sobrevive intacta la avería que lo precedió. Con una única ranura, el fallo de arranque habría sustituido a esa avería y con ella la secuencia que explica la causa. Dos detalles confirman que cada informe es autónomo: los contadores de sesión del código 10 están todos a cero —el reinicio los reinició— mientras que los del 20 registran cuatro lecturas fallidas; y ese mismo cuatro frente al límite de tres muestra el criterio de consecutividad en funcionamiento, pues sólo la racha de tres seguidos (x3 a los 27 538 ms) llegó a declararse fatal.

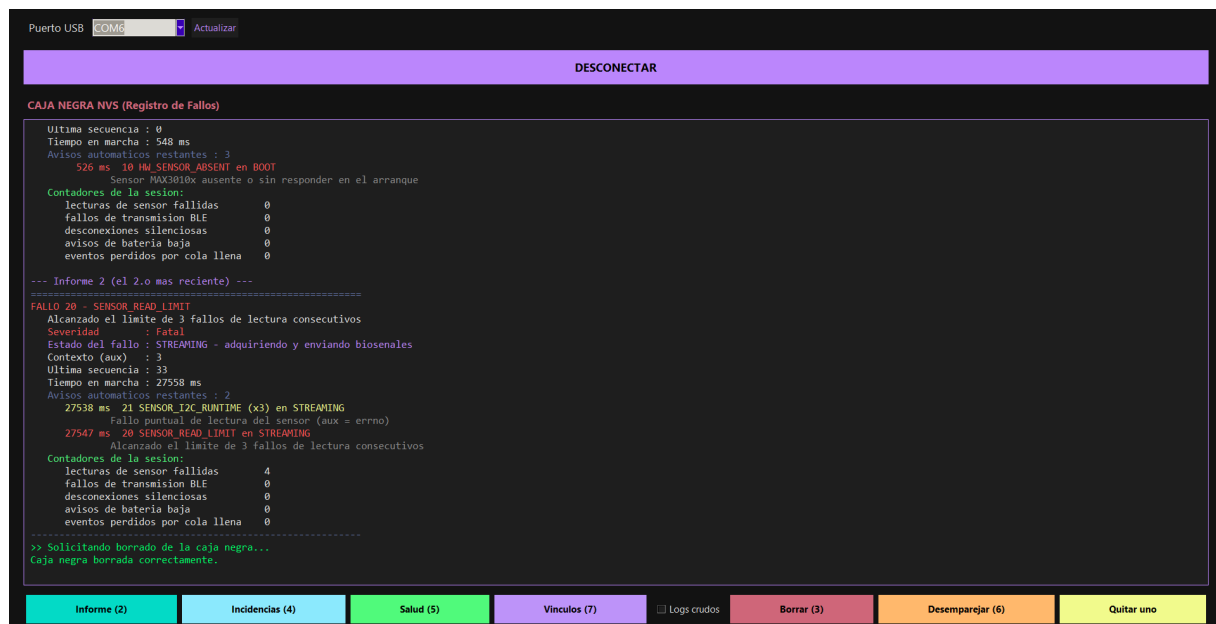


Figura 5.3: Continuación de la misma sesión: bajo la cola del segundo informe aparece la respuesta a la orden 3, con la confirmación «Caja negra borrada correctamente» en verde. La secuencia refleja el orden de trabajo que impone RR-3 —consultar antes de destruir—, y es la única vía por la que el registro puede desaparecer *desde esta herramienta*: el firmware no lo borra por iniciativa propia, y la orden sólo se envía tras aceptar el diálogo modal. Fuera de la interfaz quedan otras dos: la orden de mantenimiento F de la consola, que reformatea los sectores de la partición, y la rotación del anillo, que desplaza al informe más antiguo cuando las cuatro ranuras están llenas. Es también el estado en el que conviene dejar el equipo antes de una nueva sesión de ensayo, para que los informes que aparezcan después correspondan con certeza a ella.

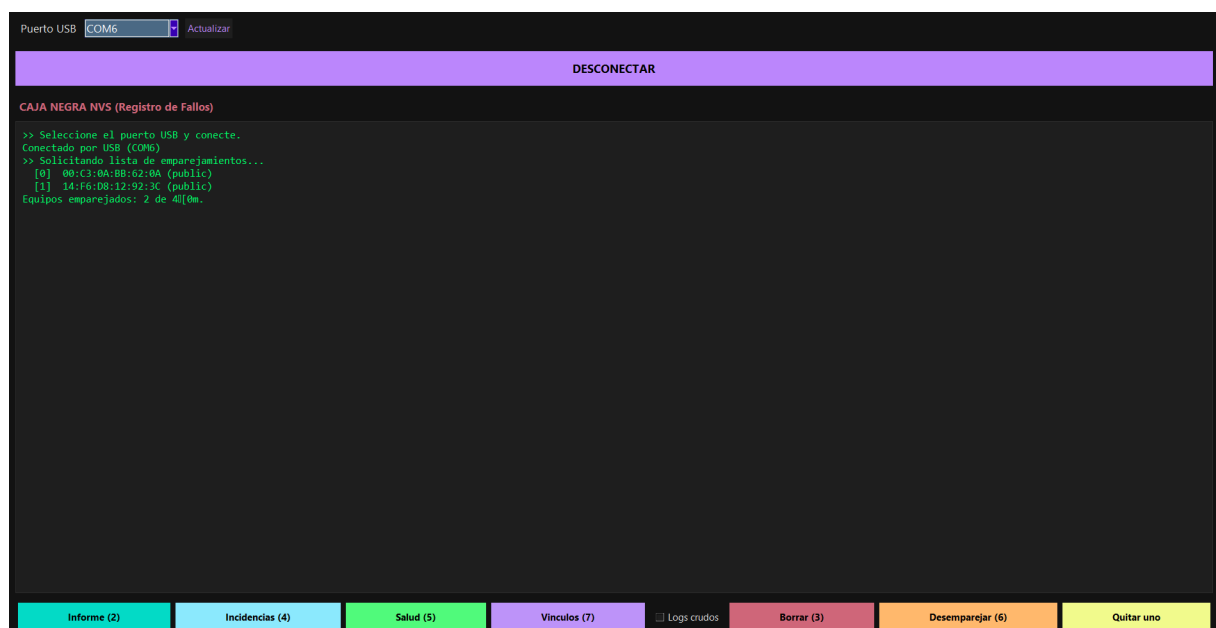


Figura 5.4: Respuesta a la orden 7: los dos equipos emparejados con su índice y su dirección, y el recuento frente al límite de cuatro. Los índices son los que aceptan las órdenes a–d para retirar un vínculo concreto, mecanismo descrito en la Sección 3.2.5.

6. Cliente Android de donación

El Capítulo 5 ha descrito la herramienta de extracción por cable, concebida para el diagnóstico del dispositivo sobre la mesa de trabajo. Este capítulo presenta el cliente destinado al uso real: una **aplicación Android** que dialoga con el equipo por radio y deposita lo medido en la plataforma de integración descrita en el Capítulo 7.

La aplicación no es un visor. Es el **segundo cliente** de un protocolo clínico cuyo primer cliente es la aplicación web. Ambos clientes son compatibles con el mismo contrato de radio y la misma interfaz REST, aunque no son completamente intercambiables debido a sus diferencias funcionales y de configuración. La diferencia está en el enlace, no en el protocolo. La aplicación web necesita un puente en Python porque el navegador no puede abrir un enlace BLE de forma fiable (Sección 7.2); la aplicación móvil habla con el dispositivo directamente y no necesita intermediario.

6.1. El flujo de trabajo determina la estructura

La aplicación organiza el trabajo en **cinco pasos numerados**. Esa secuencia no es una preferencia de diseño sino una imposición del servidor: como se detalla en la Sección 7.3, los pasos 2 a 5 responden **404** mientras no exista el registro creado en el paso 1.

Paso	Pantalla	Cometido
1	Registro	Lectura de los códigos ISBT 128 del donante y de la bolsa; el servidor devuelve el <code>register_id</code>
2	Basales	Fecha de nacimiento, peso, talla y sexo
3	Clínicos	Constantes vitales y hemoglobina de cribado
4	Medición	Captura de la señal PPG y de la temperatura
5	Laboratorio	Hemograma completo, cuando llega del laboratorio

Tabla 6.1: Correspondencia entre los pasos del protocolo y las pantallas de la aplicación.

Que el paso 5 pueda llegar horas más tarde —el hemograma sale del laboratorio, no de la sala— obliga a que una donación pueda quedar a medias y retomarse después. Es la razón de que el estado de cada paso se persista localmente y no se infiera de la sesión en curso.

6.1.1. Navegación e indicador de progreso

La barra inferior ofrece **cuatro pestañas** —panel en vivo, donación, registros y ajustes—, y no cinco o más: en un teléfono no caben. Los cinco pasos se recorren mediante un **indicador de progreso** que encabeza cada pantalla del flujo, con un botón numerado por paso que distingue tres condiciones: completado, actual y bloqueado.

El indicador *bloquea de verdad* la navegación en lugar de limitarse a colorear. Un botón deshabilitado evita un error de red que el operador no sabría interpretar: sin esa comprobación previa, tocar el paso 3 antes de haber registrado la donación produciría un 404 opaco.

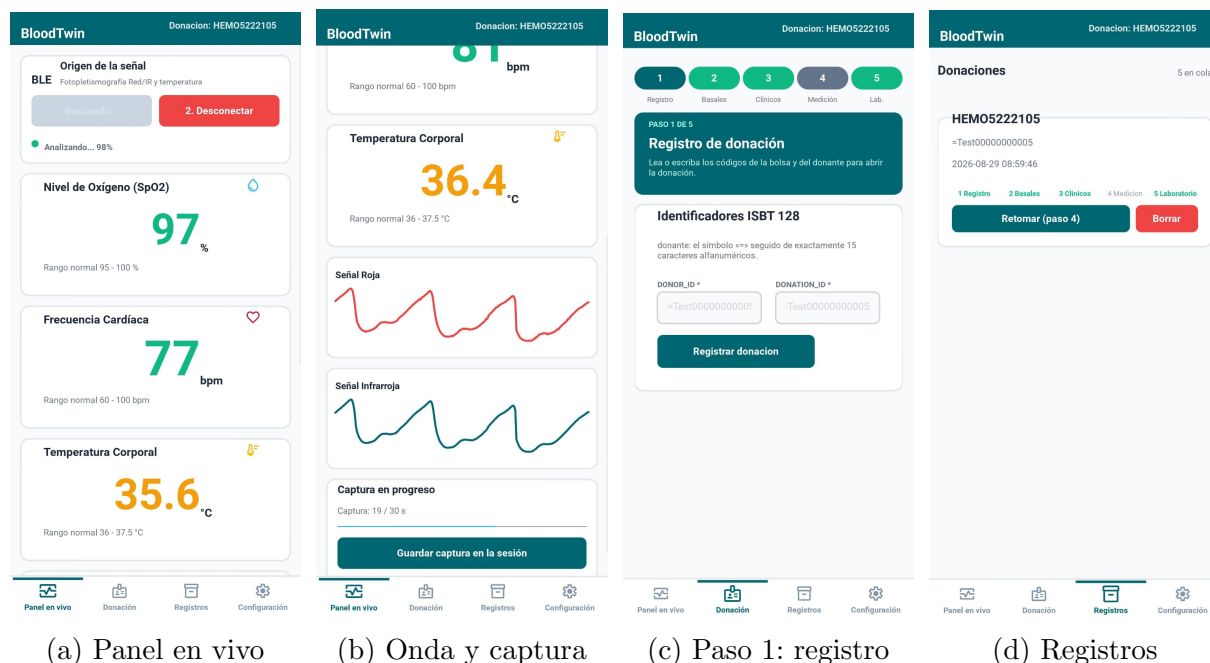


Figura 6.1: Pantallas del cliente Android durante una sesión de captura sobre hardware real. En (a) y (b), el panel de adquisición con el dedo colocado: los valores instantáneos, las dos ondas —roja e infrarroja— y la barra que contabiliza los treinta segundos de captura. En (c), el paso 1 con el indicador de progreso encabezando la pantalla: los pasos 2, 3 y 5 aparecen en verde por estar ya subidos y el 4 en gris. En (d), la lista de registros, donde la misma donación muestra qué pasos han llegado al servidor y ofrece retomarla por el que falta.

6.2. Arquitectura del software

La aplicación está escrita en **Python sobre Kivy**, que proporciona el árbol de widgets y el bucle de eventos, y se empaqueta como APK nativa mediante Buildozer (Sección 6.6). No hay navegador embebido ni capa web: la interfaz se dibuja directamente sobre el lienzo de Kivy.

El código se organiza en cuatro capas, con la regla de que las rutas se piden siempre a un único módulo y nunca se calculan con `__file__` desde donde haga falta.

Capa	Contenido
<code>app/paths.py</code>	Resolución única de rutas: material empaquetado y estado generado en ejecución
<code>app/core/</code>	<code>api</code> (los cinco endpoints, JWT y cola de envíos), <code>store</code> (SQLite), <code>validators</code> (espejo del validador del servidor), <code>forms</code> (descriptores de campo), <code>logger</code>
<code>app/device/</code>	<code>dsp</code> (procesado de la señal) y el esquema Protobuf del firmware
<code>app/ui/</code>	Widgets propios y una pantalla por paso

Tabla 6.2: Organización del código de la aplicación.

Esta división reproduce deliberadamente la de la aplicación web —cuyos módulos `core/api.js`, `core/validators.js`, `core/store.js` y `core/outbox.js` cumplen el mismo papel—, de modo que una corrección del protocolo pueda trasladarse de un cliente al otro sin traducir la arquitectura.

6.2.1. Enlace de radio y decodificación de tramas

El diálogo con el dispositivo emplea `bleak`, con recurso a `pyjnius` para el adaptador nativo cuando Android lo exige. La aplicación localiza el equipo por nombre, se suscribe a la característica de datos y escribe `0x01` en la característica de control para iniciar la transmisión; `0x00` la detiene.

Las tramas llegan serializadas en Protocol Buffers, con el esquema que el firmware define en `app/proto/simple.proto`:

```
message PPG { uint32 red = 1; uint32 ir = 2; }  
message DataFrame { repeated PPG ppgs = 1; uint32 seq = 2; uint32 t_  
ms = 3; }
```

Cada trama transporta **20 muestras**, que a la cadencia de 100 Hz del sensor equivalen a 200 ms de señal. Los dos campos escalares merecen atención porque no son adorno: el firmware *descarta tramas a propósito* cuando la radio se congestiona, y `seq` avanza con cada trama *completa generada*, en el instante en que el hilo de adquisición reúne las 20 muestras y antes de encolarla. Esa elección es deliberada: numerar en el momento de la generación —y no en el del envío— es lo que convierte un salto en la numeración recibida en evidencia de pérdida, con independencia de dónde se haya producido, ya sea porque el firmware descartó la trama al congestionarse la radio o porque no llegó a atravesar el enlace. La aplicación contabiliza esos saltos para calcular la fiabilidad extremo a extremo.

Conviene precisar el alcance de esa métrica. Mide la **fiabilidad de la radio**, no la calidad de la señal óptica, y por eso no se envía al campo homónimo de REDCap: una captura sin dedo puesto no pierde ni una trama y daría un 100 % teniendo una señal inservible.

6.3. Procesado digital de la señal

Las muestras de reflectancia roja e infrarroja se procesan sobre una ventana deslizante de **240 muestras**, que a 100 Hz corresponden a **2,4 segundos** de señal. Todo el cálculo se

apoya en NumPy.

6.3.1. Filtro eliminador de continua

La señal fotopletimográfica presenta una componente continua de gran amplitud, debida a la reflectividad estática de los tejidos, muy superior a la componente alterna pulsátil de interés. Se elimina con un filtro paso alto de primer orden:

$$y[n] = x[n] - x[n-1] + \alpha \cdot y[n-1] \quad (6.1)$$

con $\alpha = 0,95$. A la frecuencia de muestreo de 100 Hz, el polo en $z = \alpha$ sitúa la frecuencia de corte a -3 dB en torno a 0,82 Hz, esto es, en el borde inferior de la banda cardíaca (unos 49 lpm) y no muy por debajo de ella. Referida a la ganancia de banda de paso, la atenuación que introduce es de 5,6 dB a 0,5 Hz (30 lpm), 3,1 dB a 0,8 Hz (48 lpm), 2,2 dB a 1,0 Hz (60 lpm), 1,1 dB a 1,5 Hz (90 lpm), 0,7 dB a 2,0 Hz (120 lpm) y 0,3 dB a 3,0 Hz (180 lpm). El filtro suprime, por tanto, la componente continua, pero no es transparente en el extremo inferior del intervalo fisiológico: por debajo de unos 60 lpm la amplitud medida queda rebajada en más de 2 dB. Corregirlo exige acercar α a la unidad.

6.3.2. Detección de dedo y amplitud pulsátil

Antes de calcular nada se comprueba que haya tejido sobre el sensor: se promedia la señal en crudo y se compara con un umbral de reflectividad estática (`MIN_DC.THRESHOLD = 10 000` cuentas, el mismo valor que emplea el puente web para la misma comprobación). Por debajo de él se suspenden los cálculos y se avisa al operador.

Con el dedo presente, la amplitud pulsátil de cada canal se estima como el valor cuadrático medio de la señal filtrada sobre la ventana:

$$AC_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_{i=1}^N y[i]^2} \quad (6.2)$$

6.3.3. Saturación de oxígeno

La saturación se deriva del cociente R de las amplitudes pulsátiles normalizadas:

$$R = \frac{AC_{\text{Red,RMS}} / \bar{x}_{\text{DC,Red}}}{AC_{\text{IR,RMS}} / \bar{x}_{\text{DC,IR}}} \quad (6.3)$$

sobre el que se aplica la calibración empírica del sensor:

$$SpO_2 = 106,0 - 17,0 \cdot R \quad (6.4)$$

Los coeficientes difieren de los genéricos de la literatura ($A \approx 110$, $B \approx 25$) porque proceden de la documentación de referencia del fabricante para este sensor, no de una calibración realizada en este Trabajo: no se ha ejecutado ningún ensayo de desaturación controlada que permita ajustarlos al ejemplar concreto. El resultado se acota en el intervalo $[70\%, 100\%]$ —no en $[0\%, 100\%]$ — porque una saturación por debajo de 70 no es

un hallazgo clínico sino, en este montaje, un artefacto: el sensor es de reflectancia y no está validado en ese rango. Sobre la serie resultante se aplica además una mediana móvil de ocho estimaciones.

6.3.4. Frecuencia cardíaca

La frecuencia se deriva del canal infrarrojo, por ser el de mejor relación señal-ruido en reflectancia, mediante un detector de picos de cinco etapas:

1. **Eliminación de tendencia.** Se resta a la señal su ajuste **lineal** por mínimos cuadrados, que suprime la deriva de base residual sin deformar el pico sistólico. Un polinomio de orden superior seguiría la propia onda de pulso y la atenuaría.
2. **Suavizado.** Media móvil de cinco puntos, que reduce el ruido de alta frecuencia sin desplazar el máximo.
3. **Umbral adaptativo.** Recalculado en cada ventana como $\bar{y} + 0,3\sigma_y$, lo que lo hace robusto frente a las diferencias de perfusión periférica entre sujetos.
4. **Período refractario** de 350 ms tras cada pico, correspondiente a una frecuencia máxima de ≈ 171 lpm, que impide contar dos veces la misma onda.
5. **Estimación.** Se promedian los intervalos entre picos consecutivos —descartando los que caen fuera de la banda de 0,30 a 1,50 s— y el valor mostrado es la **mediana de las ocho últimas estimaciones**, lo que amortigua la arritmia sinusal respiratoria y los artefactos puntuales.

La cadencia empleada en el cálculo, **100 Hz**, no es una estimación: se deduce de la configuración del sensor descrita en el Capítulo 3, donde `CONFIG_MAX30101_SR` selecciona 200 Hz de conversión y `CONFIG_MAX30101_SMP_AVE` promedia dos muestras. Emplear un valor distinto sesgaría la frecuencia cardíaca en la misma proporción, sin afectar a la saturación: el cociente R es adimensional y no depende de la cadencia.

6.4. Almacenamiento local y trabajo sin cobertura

La unidad de trabajo persistida es la **donación**, no la sesión de medida. La tabla correspondiente guarda los dos códigos ISBT 128, el `register_id` que asignó el servidor y **cinco indicadores**, uno por paso, que registran qué se ha subido ya. Son esos indicadores los que alimentan tanto el indicador de progreso como la lista de registros: sin ellos la aplicación no podría saber qué le falta a una donación a medias.

Una sala de donación no garantiza cobertura, de modo que un envío que falle por un corte de red no debe darse por perdido. Cuando una subida falla de forma reintentable, la carga se guarda en disco dentro de un sobre que conserva además el paso al que pertenece y los códigos ISBT de la donación. Esto último es necesario porque sin red tampoco se puede obtener un `register_id`: al reintentar, la aplicación lo solicita primero y solo entonces sube el paso.

El tratamiento distingue por código de respuesta. Un 429 o cualquier `5xx` —incluido el 503 con que la plataforma señala que un servicio externo no responde— dejan el envío en la cola; un 400 o un 404 lo apartan a un directorio de rechazados, porque repetir lo mismo daría el mismo resultado y conviene que alguien lo revise. El reintento diferido solicita

siempre un token nuevo: una carga guardada anoche se reenvía hoy, y el token de anoche ya no vale.

6.5. Validación en el cliente

Todo lo que el servidor rechazaría con un 400 se detecta antes de enviarlo. En una sala de donación, esperar un viaje de red para descubrir que falta un carácter es tiempo perdido delante del donante.

Los códigos ISBT 128 se comprueban contra el mismo patrón que aplica el servidor — el carácter = seguido de exactamente quince caracteres alfanuméricos—, y los campos numéricos se validan en tipo y rango. Los descriptores de campo residen en un único módulo que define, para cada uno, la clave que viaja al servidor, la etiqueta, la unidad y el intervalo admisible; las pantallas se construyen a partir de esas listas y el validador las recorre igual, de modo que no puede haber discrepancia entre lo que se pinta y lo que se comprueba.

Dos decisiones merecen justificarse. Los nombres de clave **no son libres**: se han tomado uno a uno de los mapeos del servidor, porque inventarlos haría que REDCap los ignorase en silencio. Y el rango admitido en el hemograma es más ancho que el de referencia, porque un valor fuera de referencia es un hallazgo clínico legítimo y no una errata; el límite solo ataja errores de tecleo groseros.

6.6. Empaquetado como aplicación Android

El paso de Python a un ejecutable nativo se realiza con **Buildozer** y **python-for-android**, que evitan reescribir la aplicación en Java o Kotlin. Buildozer encapsula la instalación del SDK y el NDK de Android y ejecuta la compilación cruzada.

1. **Compilación de las bibliotecas nativas** —NumPy para el procesado y SQLite para el almacenamiento— para la arquitectura objetivo **arm64-v8a**, que cubre la práctica totalidad de los terminales en uso.
2. **Inclusión del intérprete de Python** junto al código de la aplicación, de modo que la lógica se ejecute de forma nativa sobre el núcleo Linux de Android.
3. **Declaración de permisos en el manifiesto**. **BLUETOOTH_SCAN** y **BLUETOOTH_CONNECT** para el enlace de radio; **ACCESS_FINE_LOCATION**, que Android sigue exigiendo para el rastreo BLE; **INTERNET** y **ACCESS_NETWORK_STATE** para el diálogo con la plataforma; y **CAMERA**, previsto para la lectura de los códigos ISBT 128 impresos en la bolsa.
4. **Generación del paquete instalable**, que se despliega directamente sobre el terminal.

Solo entran en el paquete el punto de entrada, el árbol **app/** y el material gráfico; el estado generado en ejecución, las utilidades de verificación y las copias de consulta quedan excluidos.

6.7. Simulación en escritorio

Depender de un dispositivo físico y de un servidor levantado para cada prueba de interfaz es un obstáculo real durante el desarrollo. La aplicación incorpora por ello un modo de simulación que arranca el código *real* sustituyendo únicamente sus dos extremos: el sensor, por un generador de señal sintética, y la plataforma, por un servidor mínimo que reproduce los cinco puntos de entrada y su autenticación.

La señal sintética está calibrada contra el propio módulo de procesado, de modo que la saturación y la frecuencia que muestra la aplicación coinciden con las solicitadas. Esto permite comprobar la cadena completa —incluidos los caminos de error, forzando respuestas 404 o 500— sin hardware. Conviene subrayar que nada de lo enviado en ese modo alcanza REDCap: para una prueba de extremo a extremo real hay que levantar el servidor y apuntar la aplicación a él.

7. Plataforma de integración clínica

Los capítulos anteriores han recorrido el dispositivo y sus dos clientes inmediatos. Falta el eslabón que convierte un conjunto de medidas en un dato clínico utilizable: la plataforma que las recibe, las valida y las deposita en la infraestructura de investigación donde otros las encontrarán meses después.

Ese eslabón se ha desarrollado bajo el nombre **BloodTwin**, y su cometido no es almacenar sino *integrar*. La distinción importa: guardar datos es un problema resuelto; hacer que un registro clínico y una señal de miles de muestras sigan estando asociados cuando cada uno vive en un sistema distinto no lo es.

7.1. Arquitectura del ecosistema

El sistema completo se organiza en cuatro etapas encadenadas, representadas en la Figura 7.1: el dispositivo embebido, un cliente que dialoga con él por radio, la plataforma de integración, y una última etapa de almacenamiento que reparte cada dato entre dos destinos.

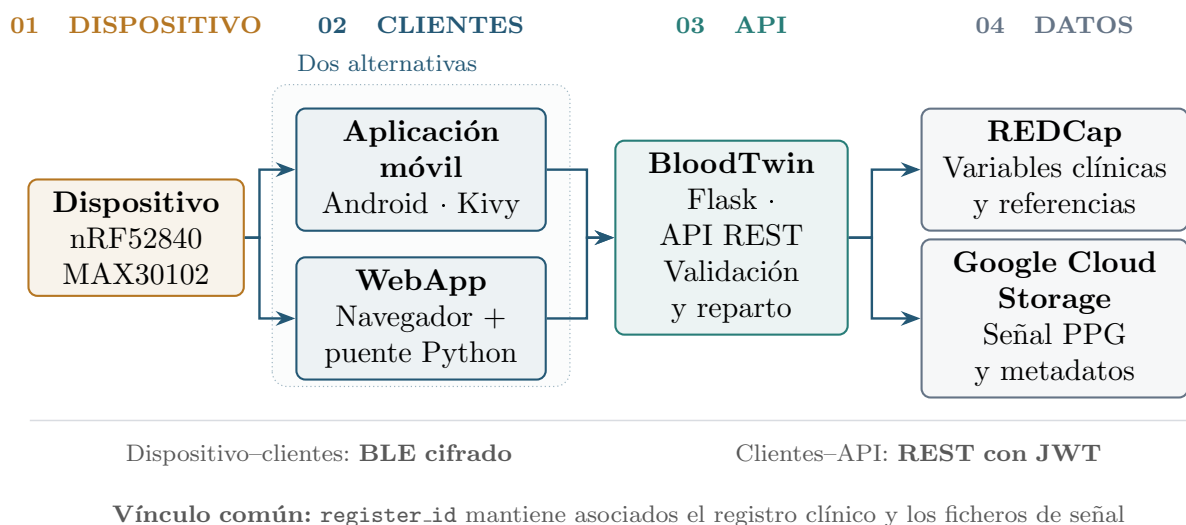


Figura 7.1: Arquitectura del sistema. El dispositivo entrega las muestras por radio — sobre un enlace cifrado y autenticado — a cualquiera de los dos clientes; ambos consumen la misma interfaz REST con autenticación por JWT, y la plataforma reparte cada dato al almacén que le corresponde. El tramo cliente–plataforma opera sobre HTTP en el entorno de desarrollo empleado en este Trabajo; su despliegue en producción exige interponer un *proxy* con TLS, requisito que se recoge entre las líneas futuras.

Conviene subrayar una propiedad de este diseño: **los dos clientes se apoyan en el mismo contrato**. Consumen las mismas características GATT, el mismo esquema de serialización y la misma interfaz REST. Ninguno es privilegiado, y añadir un tercero no exigiría tocar el dispositivo ni la plataforma.

Compartir el contrato no equivale, sin embargo, a ser intercambiables en el estado actual de la implementación, y conviene precisar dónde difieren. Ambos decodifican hoy la trama completa: el cliente Android mediante el módulo Protocol Buffers de Google, y el puente web mediante un decodificador escrito a mano que evita esa dependencia. Los dos leen los campos `seq` y `t_ms` además del vector de muestras, y los dos usan el número de secuencia para contabilizar las tramas que no llegan.

En una versión anterior el decodificador del puente extraía únicamente las muestras y descartaba los otros dos campos, de modo que la interfaz web no podía informar de la pérdida. Corregirlo resultó ser barato —son dos campos de longitud variable en una codificación que el propio puente ya recorría— y elimina la asimetría más molesta entre los dos clientes: la de que uno supiera la calidad del enlace y el otro no.

Lo que hasta hace poco seguía difiriendo era el umbral de la cadena de procesado que decide si hay dedo sobre el sensor: el cliente móvil exigía una componente continua de al menos 30 000 cuentas, frente a las 10 000 del puente web, de modo que ambos producían una estimación equivalente sobre una señal limpia pero se comportaban de forma distinta ante una marginal. Se ha unificado a 10 000 cuentas en los dos clientes, eliminando también esta asimetría. El valor sigue viviendo como una constante independiente en cada código base y no de una fuente única compartida; homogeneizarlo con el resto de parámetros del protocolo, de forma análoga a los rangos numéricos de validación recogidos entre las líneas futuras, evitaría que un cambio posterior en uno de los dos clientes reintroduzca la asimetría sin que el otro se entere.

7.2. El puente Bluetooth de la aplicación web

Una aplicación en el navegador plantea un obstáculo que la móvil no tiene: el acceso al Bluetooth. La opción evidente sería la API *Web Bluetooth*, y se descartó de forma deliberada por tres motivos concretos: solo está implementada en navegadores basados en Chromium, exige un contexto seguro con HTTPS incluso en red local, y obliga a un gesto explícito del usuario en cada conexión, lo que impide reanudar una sesión de forma automática tras una desconexión.

La solución adoptada mantiene el navegador al margen del enlace de radio. Un proceso local en Python —`serve.py`— dialoga con el dispositivo mediante `bleak`, la misma biblioteca que emplea la aplicación móvil, y publica los resultados hacia el navegador por Socket.IO:

Navegador $\longleftrightarrow$ Socket.IO $\longleftrightarrow$ `serve.py` (bleak) $\longleftrightarrow$ BLE $\longleftrightarrow$ Dispositivo

El navegador recibe eventos ya interpretados —estado del enlace, identificador del dispositivo, valores calculados de saturación y frecuencia, bloques de muestras para la representación de la onda— y nunca manipula tramas. La ventaja adicional es que el mismo puente sirve a cualquier navegador sin cambios y que la lógica de decodificación es común con el resto del ecosistema.

El decodificador de tramas merece una mención. En lugar de depender de la biblioteca

oficial de Protocol Buffers, el puente implementa un lector propio del formato binario. La decisión reduce las dependencias del proceso local a lo mínimo, y sobre todo permite que el decodificador **ignore con seguridad los campos que no conoce**: si una versión posterior del firmware añade un campo al esquema, el puente lo salta sin fallar en lugar de rechazar la trama completa. Esta tolerancia hacia adelante es una propiedad deseable en un sistema cuyos extremos se actualizan por separado.



Figura 7.2: Interfaz de adquisición durante una sesión de medida real. El indicador **Sensor conectado** confirma que el puente mantiene el enlace de radio; las dos ondas —roja e infrarroja— se representan en tiempo real sobre las últimas 1500 muestras, y las tarjetas superiores muestran los valores derivados de esa señal. La barra lateral sitúa la donación en curso, HEM05222106, con dos de los cinco pasos ya completados. Es la figura que demuestra que la cadena completa —sensor, radio, puente y navegador— funciona de extremo a extremo.

7.3. La interfaz de integración

La plataforma expone cinco puntos de entrada REST, todos por POST bajo el prefijo /api, que reciben una lista de objetos {"name": ..., "value": ...}. La Tabla 7.1 los resume.

Punto de entrada	Cometido
<code>register_donor</code>	Valida los códigos ISBT 128 del donante y de la donación, crea la fila en REDCap y devuelve el <code>register_id</code> que identificará todo lo demás
<code>upload_donor_basals</code>	Datos demográficos: fecha de nacimiento, peso, talla y sexo
<code>upload_donor_clinical</code>	Constantes vitales y hemoglobina de cribado
<code>save_device_measurements</code>	Señal del sensor: series de PPG rojo e infrarrojo y temperatura, con sus metadatos
<code>upload_donation_lab</code>	Hemograma completo procedente del laboratorio
...	

Tabla 7.1: Interfaz REST de la plataforma de integración.

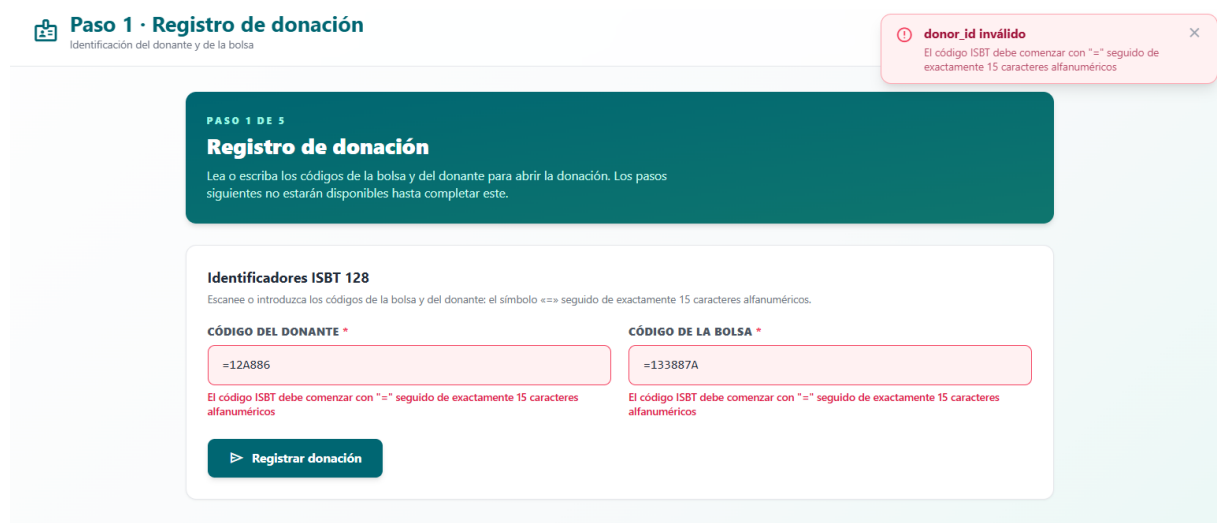
7.3.1. El orden no es opcional

El primero de los cinco puntos de entrada es condición previa de los demás: sin un `register_donor` que cree la fila en REDCap no existe el registro al que referirse. Esta rigidez es intencionada: reproduce el procedimiento clínico real, en el que la donación se identifica antes de tomar ninguna medida, y persigue evitar que queden en el sistema medidas huérfanas imposibles de atribuir a un donante.

Los cuatro puntos de datos aplican esa regla de manera uniforme: resuelven el identificador contra REDCap *antes* de escribir nada y responden **404** si no lo encuentran. El de mediciones del dispositivo es el único que escribe en dos almacenes y merece detalle, porque su orden es el resultado de haber corregido un defecto real. La secuencia es hoy: **validar** la donación en REDCap, **subir** los ficheros de señal al almacenamiento, **escribir** sus nombres en el registro clínico y, si esa última escritura falla, **deshacer** la subida borrando los ficheros recién creados.

La versión anterior invertía los dos primeros pasos, y esa inversión no era inocua: un identificador inexistente dejaba los ficheros en el *bucket* sin ninguna fila clínica a la que pertenecer —huérfanos, sin forma de atribuirlos— y la petición aún podía resolverse con un código de éxito. La comprobación existía, pero llegaba tarde. El caso ilustra algo que se repite en este trabajo: en un sistema que escribe en dos almacenes distintos, el orden de las operaciones *es* parte del diseño, y una compensación explícita ante el fallo del segundo es tan necesaria como la validación previa. Entre los pasos 2 a 5 el servidor no impone un orden relativo, decisión razonable porque el hemograma de laboratorio puede llegar horas después de la extracción.

La consecuencia práctica para el cliente es que **el identificador de registro lo asigna el servidor**, nunca el dispositivo ni la aplicación. Un identificador generado localmente no existiría en REDCap y todas las subidas posteriores se rechazarían.



Paso 1 · Registro de donación
Identificación del donante y de la bolsa

PASO 1 DE 5
Registro de donación
Lea o escriba los códigos de la bolsa y del donante para abrir la donación. Los pasos siguientes no estarán disponibles hasta completar este.

Identificadores ISBT 128
Escanee o introduzca los códigos de la bolsa y del donante: el símbolo «=» seguido de exactamente 15 caracteres alfanuméricos.

CÓDIGO DEL DONANTE *
=12A886
El código ISBT debe comenzar con "=" seguido de exactamente 15 caracteres alfanuméricos

CÓDIGO DE LA BOLSA *
=133887A
El código ISBT debe comenzar con "=" seguido de exactamente 15 caracteres alfanuméricos

Registrar donación

donor_id inválido
El código ISBT debe comenzar con "=" seguido de exactamente 15 caracteres alfanuméricos

Figura 7.3: Validación de formato en el cliente. Los dos códigos introducidos —=12A886 y =133887A— son más cortos que los quince caracteres que exige ISBT 128, y la interfaz los rechaza *antes* de contactar con el servidor: marca ambos campos en rojo, explica la regla incumplida bajo cada uno y emite un aviso. Lo que documenta la figura no es el aspecto del formulario sino que el viaje de red se ahorra por completo cuando el dato no puede ser válido. La cabecera sitúa además el paso dentro de la secuencia de cinco.

7.3.2. Autenticación y protección

Los cinco puntos de entrada exigen un token JWT [28] firmado con HS256, que se obtiene en un punto de entrada específico y se envía después en la cabecera **Authorization**. La plataforma incorpora además un límite de peticiones por origen, validadores propios para cada tipo de dato y un documento **OpenAPI 3.0** que el servidor publica en tiempo de ejecución, accesible como especificación descargable y como documentación navegable. Conviene ser preciso sobre su procedencia, porque determina cuánto puede uno fiarse de él: ese documento está escrito a mano en un módulo del propio servidor, no se deriva por introspección de las rutas. Se sirve, por tanto, desde el mismo despliegue que atiende las peticiones —lo que descarta que corresponda a otra versión— pero es una especificación mantenida en paralelo a la implementación y puede desincronizarse de ella. Contrastarla con las rutas en la integración continua se recoge entre las líneas futuras.

Los errores se devuelven con el código HTTP que corresponde a la causa —400 validación, 401 token ausente o caducado, 404 donación inexistente, 413 cuerpo mayor que el límite admitido, 429 exceso de peticiones, 500 fallo interno no previsto y 503 servicio externo no disponible— acompañados siempre de un mensaje explicativo. La distinción entre 500 y 503 no es cosmética: el primero indica un defecto no previsto del propio servidor y el segundo que REDCap o el almacén de objetos no responden. Ambos se consideran transitorios a efectos de reintento —un servidor puede recuperarse igual que un servicio externo—, y ambos alimentan por tanto la cola descrita en la Sección 7.5, junto con el 429 y los fallos de red. Lo que separa a esa familia de la otra no es la gravedad sino el pronóstico: 400, 401, 404 y 413 devolverían exactamente el mismo resultado por mucho que se repitan, y exigen intervención.

7.4. La decisión de almacenamiento

La decisión de diseño más característica de la plataforma es el reparto de cada dato entre dos almacenes de naturaleza distinta.

7.4.1. El problema

Un registro clínico de donación contiene unas decenas de variables: edad, peso, constantes vitales, resultados del hemograma. Encaja sin dificultad en un formulario. Una sesión de medida, en cambio, produce **series de miles de muestras** por canal. No cabe en un formulario, y forzarla dentro degradaría el sistema de captura hasta hacerlo inutilizable.

7.4.2. La solución

Las variables clínicas se registran en **REDCap**, el sistema de captura de datos de investigación clínica de la institución, donde la integración utiliza cuarenta y cuatro campos —los cinco pasos del protocolo más los dos identificadores de fila—, verificados uno a uno contra el diccionario de datos del proyecto activo mediante una herramienta de comprobación incluida en el repositorio. Las señales en bruto se depositan como ficheros CSV en **Google Cloud Storage**. Y el vínculo entre ambos mundos lo mantiene el propio registro clínico, que conserva el *nombre* de cada fichero de señal en tres campos dedicados.

De este modo, cuando la operación termina correctamente, el registro conserva la referencia a sus señales, sin que ninguno de los dos sistemas tenga que hacer aquello para lo que no está diseñado. En un despliegue completo con credenciales del almacén, un investigador que consulte REDCap encontraría la referencia al fichero y la señal permanecería accesible en un almacén pensado para objetos grandes. Es el comportamiento que la plataforma implementa; su comprobación efectiva es la que quedó pendiente por lo expuesto más adelante en este mismo capítulo.

Record ID	166
Register ID	HEM05222106 Register identifier
Donor ID * must provide value	=TEST00000000011 ISBT donor code
Donation ID * must provide value	=TEST00000000011 ISBT donation code
UTC Timestamp	2026-09-02 12:59 Record creation timestamp (UTC)
Date of Birth	1995-10-25 Donor date of birth
Biological Sex	☐ Male ☐ Female Registered biological sex
Weight (kg)	68 Donor weight in kilograms
Height (cm)	172 Donor height in centimeters
Basal Comments	test Additional notes regarding donor basal data Expand
Form Status	
Complete?	Incomplete
	-- Cancel --

Figura 7.4: Registro de una donación de prueba en el sistema de captura clínica, creado **a través de la interfaz REST** y no introducido a mano. Conviven en la misma fila el identificador que asignó la plataforma (HEM05222106), los dos códigos ISBT del donante y de la bolsa, y los datos basales del paso 2. La prueba de que el registro nació por la interfaz es el **sello de tiempo universal**: ese campo lo escribe la plataforma al crear la fila, y el formulario web de la herramienta no lo ofrece.

Record ID	166
Device Hardware ID	NRF52-A6233C39 Nordic MCU MAC address / Hardware ID
Red Channel File	 Filename or path of red PPG CSV in GCP Storage
IR Channel File	 Filename or path of IR PPG CSV in GCP Storage
Temperature File	 Filename or path of temperature CSV in GCP Storage
Signal Quality ID	 Signal quality assessment rating ID
Device Comments	Additional PPG acquisition notes Expand
Local Acquisition Datetime	2026-09-02 15:03:33 Local timestamp of sensor acquisition
Timezone Offset (Seconds)	7200 Timezone offset in seconds relative to UTC
Form Status	
Complete?	Incomplete -- Cancel --

Figura 7.5: Formulario de medidas del mismo registro HEM05222106. Los campos que la plataforma sí ha podido escribir —identificador físico del sensor, instante local de adquisición y desfase horario— llegaron por la interfaz REST, lo que confirma que el paso 4 alcanza el registro clínico. Los tres campos de nombre de fichero permanecen vacíos por la limitación que se explica a continuación. La captura es anterior a la corrección del orden de operaciones, de modo que refleja el recorrido previo del paso 4.

7.4.3. Una limitación del despliegue de pruebas

La Figura 7.5 muestra los tres campos `red_filename`, `ir_filename` y `temp_filename` sin valor, y conviene explicar por qué, porque son precisamente los que materializan el reparto descrito en esta sección.

El despliegue empleado en estas pruebas **no dispone de credenciales para el almacén de objetos**: los permisos de acceso al *bucket* del proyecto no llegaron a estar disponibles durante el periodo de pruebas, de modo que la limitación es del entorno de trabajo y no del diseño de la plataforma. La plataforma escribe esos tres nombres en REDCap únicamente después de confirmar la subida de los CSV correspondientes, de modo que sin acceso al almacén no hay fichero que referenciar y los campos quedan legítimamente vacíos. No se trata de un fallo del protocolo ni de la interfaz: el resto de la fila —identificador de registro, códigos ISBT, datos basales, constantes vitales y las diecinueve variables de laboratorio— se rellena con normalidad, como acredita la Figura 7.4.

La figura debe leerse, eso sí, con una precisión de fechas. Fue capturada antes de la corrección del orden de operaciones descrita en la Sección 7.3, cuando el fallo del almacén todavía permitía que los metadatos del dispositivo alcanzasen REDCap. Con el código actual esa petición se detiene antes: la ausencia del almacén produce un 503 y no se escribe nada en el registro clínico, precisamente para no dejar una fila que afirme tener señales inexistentes. La consecuencia sobre lo que la figura documenta es nula —los campos de

nombre de fichero siguen vacíos por la misma razón de fondo—, pero la fila del dispositivo que en ella aparece no se produciría hoy sin credenciales válidas.

La consecuencia para este Trabajo es acotada pero debe declararse: el mecanismo de vinculación entre el registro clínico y la señal en bruto **está implementado y descrito, pero no verificado de extremo a extremo**. Queda comprobado, por separado, que la plataforma acepta las series y las agrupa, y que la escritura de datos estructurados alcanza el registro clínico —esto último documentado sobre el orden de operaciones anterior a la corrección, no sobre el camino actual del paso de dispositivo—; queda por comprobar, sobre un despliegue con credenciales, que ese paso completa hoy su recorrido, que los nombres de fichero se escriben en REDCap y que los CSV son recuperables desde el almacén con esos nombres. Todo apunta a que se trata de una verificación de configuración y no de diseño —la ausencia de credenciales es una limitación del entorno—, aunque sólo esa prueba permitiría descartar además un defecto de integración. En cualquier caso, no afecta a las conclusiones sobre el resto de la cadena.

7.4.4. Por qué REDCap y no una base de datos propia

Podría haberse diseñado un esquema relacional a medida, y habría sido más sencillo de programar. Se descartó por una razón que no es técnica sino de contexto: REDCap [29, 30] es la herramienta que la institución ya emplea para sus estudios clínicos, con sus procedimientos de acceso, su trazabilidad de cambios y su soporte establecido. Integrarse en ella significa que los datos de este Trabajo son inmediatamente utilizables por el resto del proyecto, en lugar de quedar aislados en un sistema que habría que mantener y del que habría que exportarlos después.

Es la misma clase de decisión que la de emplear el controlador de Zephyr en lugar de escribir uno propio, discutida en el Capítulo 3: cuando existe una pieza establecida que cumple el cometido, el trabajo de valor está en integrarla correctamente, no en reemplazarla.

7.5. Continuidad del servicio sin conexión

Los dos clientes contemplan la pérdida de conectividad. Cuando una subida falla por una causa *transitoria* —no hay red, el servidor está saturado (429) o avería en el servidor (5xx)— la carga útil se conserva localmente y se reintenta más adelante, de modo que una sesión de medida no se pierde porque el punto de acceso estuviera saturado.

El criterio de qué se encola y qué no merece explicarse, porque encolarlo todo sería peor que no encolar nada. Un rechazo por validación (400), por credencial inválida (401) o por donación inexistente (404) no mejora con el tiempo: reintentarlo indefinidamente solo acumularía basura en la cola y ocultaría al operador un error que debe corregir en ese momento. Esos fallos se muestran y no se guardan.

Los dos clientes difieren, en cambio, en cómo tratan el primer paso, y conviene no presentarlo como una política común. El puente web **no lo encola**: sin el identificador que devuelve el alta, los cuatro pasos siguientes no tienen a qué referirse, de modo que diferirlo no permitiría continuar con la sesión y el operador debe resolver la conexión antes de empezar. El cliente Android sí lo difiere, porque guarda en el sobre los códigos ISBT de la donación además de la carga útil: al reintentar solicita primero el `register_id` y sólo entonces sube el paso (Sección 6.4). Es una diferencia de alcance, no de criterio: ambos

evitan encolar lo que no mejoraría con el tiempo.

Esta previsión responde al entorno real de uso. Una sala de donación no es un laboratorio con red garantizada, y el momento de la medida no puede supeditarse a la disponibilidad del enlace: el donante está delante, y la medida se toma entonces o no se toma.

Registros
Donaciones registradas y mediciones pendientes de enviar

HEM05222106

DONACIÓN ACTIVA
HEM05222106
Abierta el 2/9/2026, 15:00:00 - 4 de 5 pasos - donante = TEST00000000011

REGISTRO
register_id = HEM05222106

BASALES
nacimiento 1995-10-25 - 68 kg - 172 cm

CLÍNICOS
77 bpm - 130/80 mmHg - SpO₂ 99 % - Hb 15 g/dL

DISPOSITIVO
Pendiente

LABORATORIO
20 parámetros enviados

Donaciones de este equipo
Copia local para consulta. El registro oficial está en el servidor del centro.

Buscar por identificador...

Exportar todo Vaciar

Aún no hay donaciones archivadas
Complete el flujo de 5 pasos y pulse «Cerrar y archivar donación» para verla aquí.

Iniciar una donación

Figura 7.6: Vista de registros durante una donación en curso. El identificador HEM05222106 es el que asignó el servidor, no uno generado en el cliente. Las cinco tarjetas resumen el estado de cada paso y muestran, en verde, el dato efectivamente confirmado por la plataforma —fecha de nacimiento, peso y talla en basales; constantes vitales en clínicos; diecinueve parámetros en laboratorio—, mientras el paso de dispositivo aparece aún como pendiente. Es esa distinción entre confirmado y pendiente la que permite retomar una donación incompleta por el paso que le falta.

8. Validación experimental

El Capítulo 4 ha demostrado que el modelo de control está libre de bloqueos no deseados en los estados operativos y satisface las propiedades exigidas. Esa garantía, sin embargo, recae sobre el modelo y no sobre el código que se ejecuta en el microcontrolador. Este capítulo cubre los dos niveles restantes de validación: la comprobación, sobre los comportamientos que las pruebas ejercitan, de que la implementación en C se ajusta a lo que el modelo prescribe —lo que no equivale a haber demostrado su equivalencia completa—, y la medida del sistema real sobre el hardware.

Se presentan, en este orden, el entorno de pruebas unitarias sobre el ordenador anfitrión y su cobertura, la validación funcional de las transiciones de la red, los mecanismos de recuperación que solo pueden probarse físicamente, la caracterización del alcance de lo que puede afirmarse sobre el consumo —y de las medidas eléctricas que quedan pendientes—, y el análisis del comportamiento temporal mediante trazas del núcleo.

8.1. Pruebas unitarias sobre el ordenador anfitrión

8.1.1. Entorno y estrategia

Las pruebas se ejecutan en el ordenador de desarrollo, no sobre la placa. Esta decisión no es de conveniencia: los caminos que más interesa verificar —sensor ausente, bus caído, batería agotada, imposibilidad de crear un hilo— son precisamente los más difíciles de provocar sobre el hardware, y algunos exigirían dañar el equipo. Ejecutar sobre el anfitrión permite inyectar cada uno de ellos de forma determinista y repetible.

Lo que hace posible esa ejecución es la capa OSAL descrita en la Sección 2.7: el motor de la red y la lógica de control no invocan primitivas de Zephyr sino la interfaz abstracta, que en el anfitrión se resuelve contra sustitutos. El entorno se apoya en tres herramientas [24, 25, 26]: **Ceedling** orquesta la compilación y la ejecución, **Unity** aporta las aserciones y **CMock** genera automáticamente los dobles de prueba a partir de las cabeceras.

Conviene subrayar un punto que distingue esta batería de una simulación: las pruebas enlazan y ejercitan **el motor de red de Petri real**, el mismo código C que se compila para el microcontrolador. Lo sustituido es el sistema operativo y el hardware, no la lógica bajo prueba. Un resultado correcto significa, por tanto, que el código que se embarca se comporta como se espera, y no que lo haga un modelo suyo.

8.1.2. Cobertura alcanzada

La instrumentación con **gcov** y la generación de informes con **gcovr** producen las métricas de la Tabla 8.1.

Criterio	Cubierto	Total	Porcentaje
Líneas	344	356	96,6 %
Funciones	34	34	100,0 %
Ramas	134	160	83,8 %

Tabla 8.1: Cobertura de código del motor de control y del subsistema de errores, medida con `gcov` sobre la ejecución de las 68 pruebas.

La cobertura de funciones es completa y la de líneas alcanza el 96,6 %. Las diferencias merecen una explicación honesta, porque una cobertura no interpretada no significa nada:

- **Las doce líneas no cubiertas son código inalcanzable desde el anfitrión.** Corresponden a las tres devoluciones de llamada de temporizador y al bucle infinito del hilo de control, marcados en el código con directivas `GCOVR_EXCL`. Conviene precisar el alcance de esas directivas, porque excluir y no cubrir no son lo mismo: la Tabla 8.1 recoge el recuento en bruto de `gcov`, que no interpreta esos marcadores, de modo que las líneas afectadas siguen contando en el denominador y son las que figuran como no cubiertas; sólo el informe de `gcovr` las descuenta, y sobre él la cobertura de líneas del código efectivamente alcanzable es completa. No son código sin probar: son código *inalcanzable* desde el anfitrión, porque los temporizadores sustitutos no disparan y el bucle del hilo colgaría al ejecutor. La lógica que sí contienen —la decisión de batería y el drenaje de eventos descartados— se extrajo deliberadamente a `pn_handle_deferred_work()`, que sí se prueba.
- **La cobertura de ramas es menor por construcción.** Muchas de las ramas no tomadas corresponden a comprobaciones defensivas de puntero nulo y a códigos de retorno de error que el sustituto no produce. Alcanzar el 100 % de ramas exigiría añadir sustitutos capaces de fallar en cada punto, con un coste de mantenimiento desproporcionado frente al defecto que detectarían.

La interpretación correcta de estas cifras es la enunciada en la Sección 2.7: una cobertura alta no demuestra la ausencia de defectos, pero una cobertura baja sí demuestra que existe código cuyo comportamiento nadie ha comprobado. Con un 100 % de funciones ejercitadas, no queda ninguna función del motor de control en esa situación.

8.2. Validación funcional del motor de control

La batería consta de **68 pruebas**, agrupadas en doce bloques temáticos. La totalidad se ejecuta con éxito: 68 superadas, ninguna fallida y ninguna ignorada. La Tabla 8.2 recoge la distribución y el aspecto que verifica cada bloque; la relación exhaustiva, con el nombre de cada función y la aserción concreta que comprueba, figura en la Tabla C.1 del Anexo C.

Bloque	N.º	Qué verifica
Arranque	2	El marcado inicial y la salida nominal hacia el anuncio
Conexión y flujo de datos	9	Las transiciones entre anuncio, reposo y transmisión, con sus guardas
Errores del sensor	8	Que un fallo aislado no corta el flujo, que el tercero <i>consecutivo</i> sí es fatal y que una lectura correcta rompe la racha
Batería y apagado	4	Las tres vías que conducen al sueño profundo, más la vigilancia de batería durante la transmisión
Prioridad entre transiciones	3	Que ante conflicto se dispara la transición de mayor prioridad
Robustez de la cola de eventos	4	El descarte de eventos sin transición válida, el desbordamiento y que ni el fallo fatal ni el aviso de batería pueden ser expulsados por una ráfaga
Radio y rastro del watchdog	3	El apagado de la radio al entrar en error y la invalidación del rastro
Registro de incidencias	8	El anillo, la agrupación de repeticiones y los dos ejes de clasificación
Rutas de fallo del arranque	15	Cada modo de fallo de la secuencia de inicio por separado, incluidos los subsistemas de protección que no arrancan
Trabajo diferido	9	La decisión de batería —incluida la de no apagar el equipo cuando la alimentación es externa— y el drenaje de eventos descartados en interrupción
Tabla maestra de errores	2	La traducción de código a nombre y descripción, incluido el caso desconocido
Interfaz de temporizadores	1	Que las operaciones sobre el temporizador de apagado no fallan
Total	68	

Tabla 8.2: Distribución de las 68 pruebas unitarias por bloque temático.

Antes que enumerar las sesenta y ocho, resulta más informativo detenerse en las que verifican propiedades que no son evidentes o que corrigen errores cometidos durante el desarrollo.

La prioridad de las transiciones se comprueba explícitamente. Tres pruebas construyen deliberadamente situaciones de conflicto —un fallo fatal concurrente con el aviso de radio lista, una batería baja concurrente con una conexión, una conexión concurrente con un vencimiento— y comprueban que la red dispara siempre la transición de mayor prioridad. Son la contrapartida en el código de lo que el Capítulo 4 verifica sobre el

modelo: sin ellas, un cambio en el orden de la tabla `transitions[]` pasaría inadvertido.

El error de sensor no interrumpe la transmisión. Una prueba comprueba que un fallo aislado deja la marca en `P_STREAMING`, otra que el tercero consecutivo declara el fallo fatal, y una tercera que el contador se reinicia al volver a transmitir. Esta última protege contra un defecto sutil: sin el reinicio, tres fallos aislados repartidos a lo largo de horas acabarían declarando una avería inexistente.

El orden de las dos transiciones del sensor decide si la avería es falsa. La red no conserva el orden en que llegan los eventos: el capturador los convierte en banderas, y si el error de sensor y la recuperación caen en el mismo ciclo la red ve ambos lugares marcados sin saber cuál ocurrió antes. Como el motor dispara toda transición habilitada, las dos se ejecutan, y decide entonces el orden de declaración.

Con el error declarado primero, la secuencia real *contador a dos, lectura buena, lectura mala* se resolvía contando el tercer fallo antes de aplicar la recuperación: el umbral se alcanzaba y el equipo declaraba una avería. Son exactamente los tres fallos *no* consecutivos que RR-1 excluye, es decir, el mismo defecto que se acababa de corregir, sobreviviendo en un caso de borde. Declarar la recuperación primero lo elimina.

El orden inverso —error y después lectura buena en el mismo ciclo— debería dejar el contador a cero y lo deja a uno. Se documenta porque es una imprecisión conocida y no un descuido, y porque la asimetría es deliberada. Contar un fallo *de más* sólo puede adelantar la declaración de una avería: nunca la oculta. Contar uno *de menos* retrasaría la detección de una avería real y dejaría en servicio un equipo averiado, que es el error que un dispositivo médico no puede permitirse. Ante un caso de borde que exige que ambos sucesos quepan en diez milisegundos, se prefiere por tanto errar del lado que nunca esconde un fallo. Tres fallos realmente consecutivos no se ven afectados en ningún caso: entre ellos no hay lectura correcta alguna, luego no se emite ninguna recuperación que compita.

Una ráfaga no puede expulsar un fallo fatal ni un aviso de batería. La cola de eventos tiene dieciséis huecos y, al llenarse, sacrifica el más antiguo. Ese criterio es el correcto para una orden de flujo —que envejece: arrancar una transmisión que se pidió hace medio segundo ya no interesa— pero es exactamente el equivocado para los dos eventos que obligan a apagar el equipo. Un fallo fatal o un aviso de batería encolados podían quedar expulsados por una ráfaga de eventos ordinarios llegada después, y el sistema seguiría midiendo con una avería ya declarada o con la celda por debajo del umbral de seguridad.

Ambos se han sacado por ello de la cola: viajan en sendas banderas atómicas que ninguna presión de tráfico puede desalojar, y que el capturador lee *después* de vaciar la cola. Dos pruebas lo verifican encolando el evento crítico y sepultándolo bajo una ráfaga del triple de la capacidad; en ambos casos el ciclo siguiente alcanza el estado que corresponde. Merece señalarse la dirección del hallazgo, porque es la contraria a la habitual: el modelo formal representaba los eventos como variables booleanas —es decir, con la semántica de bandera— mientras que la implementación usaba una cola finita. La discrepancia no invalidaba las propiedades verificadas, pero sí significaba que el modelo describía un sistema más robusto que el construido. La corrección elimina esa distancia en lugar de documentarla.

Los eventos descartados en interrupción se registran después. El firmware no puede anotar una incidencia desde una rutina de interrupción, porque registrarla exige tomar un semáforo. La solución adoptada —un contador atómico que el hilo de control drena— se verifica llenando la cola de eventos, comprobando que el anillo sigue vacío inmediatamente después, y comprobando que la incidencia aparece tras ejecutar el trabajo diferido. Esta prueba existe porque el defecto contrario llegó a introducirse durante el desarrollo.

Un bus caído no se confunde con un sensor ausente. Una prueba registra previamente el código de bus inoperativo y comprueba que el arranque conserva esa causa en lugar de sustituirla por la de sensor ausente. Su valor no es lógico sino de mantenimiento: distingue dos averías que conducen a reparaciones distintas.

Un fallo del convertidor no se confunde con una batería llena. Cuando la lectura del ADC falla, la rutina retorna sin actualizar la tensión almacenada. La prueba correspondiente verifica que el valor previo permanece intacto, evitando que un periférico averiado se interprete como una medida válida.

Las pruebas del bloque de registro de incidencias merecen una mención aparte porque verifican el subsistema de diagnóstico y no el control: comprueban que el anillo agrupa repeticiones en lugar de desbordarse, que conserva el orden de incidencias distintas, que solo los códigos clasificados como persistentes se recuerdan como causa, que los dos ejes de clasificación son independientes, y que el informe guardado incluye la secuencia de sucesos que condujo al fallo y no únicamente el último.

8.3. Validación de los mecanismos de recuperación física

Tres mecanismos del firmware solo pueden validarse sobre el hardware, porque dependen de sucesos físicos que ningún sustituto reproduce: la pérdida de alimentación, la pulsación de un botón y la desconexión de un cable.

8.3.1. Ventana de diagnóstico y consola de emergencia

Cuando un fallo se declara durante el arranque —por ejemplo, sensor ausente o bus I²C inoperativo— el firmware no se apaga de inmediato. La acción `action_enter_error()` abre una **ventana de diagnóstico** de `ERROR_WINDOW_MS` (10 segundos) durante la cual el equipo permanece encendido y accesible por el puerto USB CDC-ACM. Transcurrida sin intervención, la red transita a sueño profundo para no agotar la batería.

Conviene precisar dos extremos que suelen malinterpretarse. En primer lugar, **la pila Bluetooth sí se inicializa** aunque el sensor esté ausente —porque `app_pn_start()` lanza la tarea sin condición—, pero **no permanece activa**: al declararse el fallo, `action_enter_error()` llama a `ble_task_stop()`, que detiene el anuncio y corta cualquier conexión abierta. Durante la ventana de diagnóstico el equipo está accesible *solo* por USB, y el informe de fallo no se expone en ningún momento por Bluetooth. En segundo lugar, si quien falla es el propio motor de control —de modo que `app_pn_start()` retorna error— no hay ventana temporizada en absoluto, porque la ventana es una acción de la red y la

red no ha llegado a existir; en ese caso `main()` cede el control a la consola, que permanece disponible de forma indefinida.

La validación consistió en desconectar físicamente el sensor del bus antes de alimentar el equipo y comprobar, por consola, la secuencia completa: declaración del fallo, escritura del informe, apertura de la ventana y apagado posterior. Las ocho órdenes de consulta y gestión —consulta del informe, borrado, volcado del anillo de incidencias, volcado de los contadores de salud, borrado de los emparejamientos Bluetooth, listado de vínculos, borrado de un vínculo concreto y señal de presencia— se comprobaron una a una según se describe en el Capítulo 5.

Queda fuera de esa comprobación la orden de mantenimiento 'F', que reformatea la partición de la caja negra. Se incorporó con posterioridad a esta campaña de medidas, y su verificación exige provocar físicamente un montaje fallido de NVS, de modo que no puede atribuírsele el mismo grado de validación experimental que a las ocho anteriores: su comportamiento se ha comprobado por inspección del código y queda pendiente de ensayo sobre el banco.

8.3.2. Despertar del sueño profundo

Al entrar en sueño profundo el firmware apaga los LED indicadores, duerme el sensor y detiene la radio antes de invocar el modo *System OFF* del SoC, en el que la CPU queda sin alimentación y solo permanece activo el detector de nivel de los pines.

El despertar se configura sobre el pin **P0.11** mediante `nrf_gpio_cfg_sense_input()` con resistencia de elevación interna y detección por nivel bajo, de modo que la pulsación del botón físico —que lleva el pin a masa— provoca la reactivación del SoC. El despertar desde *System OFF* no es una reanudación sino un **reinicio completo**: la RAM ordinaria no se retiene y el firmware arranca desde el principio. Es precisamente esa característica la que obliga a que el rastro del perro guardián se aloje en una región declarada `_noinit` y se proteja con firma y CRC, según se explicó en el Capítulo 3: sin esa comprobación, un arranque en frío leería basura y fabricaría un fallo inexistente.

La prueba consistió en forzar el apagado por batería baja, verificar el consumo residual, pulsar el botón y comprobar que el equipo reanuda el ciclo nominal identificando correctamente la causa del arranque.

8.3.3. Tolerancia a fallos del sensor en marcha

El comportamiento ante un fallo de lectura durante la transmisión es el descrito en el Capítulo 3: la marca **permanece en P_STREAMING** y la red se limita a contabilizar el suceso. No hay descenso al estado de reposo ni reinicio del sensor; la única espera es la de 200 ms que aplica el hilo de adquisición para permitir que el bus se estabilice.

La validación se realizó desconectando momentáneamente el cable del sensor durante una sesión de transmisión activa. Se comprobaron los dos comportamientos esperados: con una o dos interrupciones breves, la transmisión continúa y el único efecto observable para el cliente es un salto en la numeración de secuencia; con el cable desconectado de forma sostenida, el tercer fallo consecutivo declara el código de límite de lecturas, escribe el informe en memoria no volátil y reinicia el equipo. Los códigos recuperables registrados en el anillo permanecen consultables aunque el fallo fatal no llegue a producirse, lo que permite detectar un contacto deteriorado antes de que cause una parada.

8.4. Consumo: referencias de los fabricantes y caracterización pendiente

Conviene abrir esta sección delimitando con exactitud su alcance, porque es la única del capítulo que no se apoya en medidas propias. **No se ha realizado ninguna medición de corriente sobre el prototipo:** no se dispuso de un analizador de potencia ni de un amperímetro intercalado en la línea de alimentación durante el desarrollo. En consecuencia, este Trabajo no aporta ni consumo medio medido ni autonomía verificada del montaje completo, y lo que sigue se limita a las referencias publicadas por los fabricantes y a los parámetros de configuración que fija el propio *firmware*.

Los fabricantes publican valores de consumo para los componentes individuales bajo condiciones específicas de tensión, temperatura y modo de operación. Esos valores constituyen únicamente una referencia teórica y **no representan una medición del prototipo completo**, que integra además el regulador de la placa, los diodos indicadores y el resto de la circuitería: el nRF52840 publica sus corrientes de radio en transmisión y recepción y la de su modo *System OFF* en las especificaciones de Nordic Semiconductor [13], y el MAX30102 las de sus emisores y de su modo de reposo en la hoja de datos de Analog Devices [8]. Sumar esas cifras no produciría una autonomía válida del dispositivo, de modo que aquí no se hace.

Sí es un dato del diseño, y no una medida, la **corriente de pulso configurada para los emisores**: `CONFIG_MAX30101_LED1_PA` y `CONFIG_MAX30101_LED2_PA` valen `0x2F` en `prj.conf`, lo que con la escala de 0,2 mA por paso del controlador corresponde a 9,4 mA por LED durante el pulso (Capítulo 3). Es el parámetro que, junto con la actividad del transceptor de 2,4 GHz, cabe esperar que domine el consumo en *streaming* frente al resto de estados, aunque establecer esa jerarquía con cifras exige medirla.

Del mismo modo, la arquitectura de estados prevé que el reposo conectado resulte menos costoso que el anuncio, porque un enlace ya establecido —con el intervalo de conexión negociado de 30 a 50 ms— exige menos actividad de radio que la emisión periódica de paquetes de anuncio, y que el estado de error consuma menos desde que `ble_task_stop()` detiene la radio al declarar el fallo. Son consecuencias esperables del diseño, no resultados: ninguna de ellas se ha cuantificado.

Lo que sí queda verificado, y por una vía distinta de la eléctrica, es el **funcionamiento** del modo de bajo consumo: la entrada en *System OFF* y el despertar por el pulsador se comprobaron físicamente sobre la placa según se describe en la Sección 8.3. Esa comprobación acredita que la secuencia de apagado y arranque es correcta, no cuál es la corriente residual que queda.

La caracterización eléctrica del conjunto —consumo medio por estado, perfil de corriente durante una sesión completa y autonomía efectiva con una celda concreta— queda por tanto **pendiente de trabajo experimental**, para el que el instrumento natural es un analizador de potencia como el Nordic Power Profiler Kit II intercalado en la alimentación de la placa. Hasta disponer de esas medidas, el cumplimiento del requisito de autonomía no puede darse por acreditado, y así se refleja en las conclusiones.

8.5. Análisis de temporización en tiempo real mediante trazas Perfetto

Para ir más allá de la validación funcional por pruebas unitarias y corroborar el cumplimiento de los requisitos de tiempo real del firmware, se instrumentó el sistema con el *framework* de trazado de Zephyr RTOS, que genera trazas en formato compatible con **Perfetto Trace Viewer** [31]. Esta herramienta permite visualizar la ejecución real de cada hilo del sistema operativo con resolución de microsegundos, ofreciendo evidencia empírica del comportamiento temporal del firmware sobre el hardware físico.

La captura que se presenta en esta sección se realizó **tras activar el cifrado del enlace** descrito en la Sección 3.2.5: el cliente se emparejó con la clave fija antes de iniciar la transmisión. Esto no es incidental. Permite comprobar, sobre la misma traza que verifica la cadencia de muestreo, que el coste del emparejamiento –el intercambio de claves por curva elíptica– no perturba la periodicidad de la adquisición.

8.5.1. Configuración e instrumentación del sistema de trazado

El flujo de captura combina **dos fuentes** de eventos, fusionadas por un script Python propio en un único fichero JSON compatible con Perfetto: el *backend* CTF [32] del núcleo Zephyr, que registra los cambios de contexto entre hilos, y el canal RTT de `app_trace.c`, que anota cuándo la aplicación entra y sale de sus estados de alto nivel.

Fase 1: Activación del *backend* CTF en RAM (Zephyr Kconfig)

La elección del *backend* RAM en lugar del UART responde a una limitación fundamental: el *stack* de Bluetooth del SoC nRF52840 genera miles de interrupciones por segundo. Con el *backend* UART, el puerto serie no puede transferir los eventos al anfitrión con suficiente velocidad, llenando el búfer de trazas en décimas de segundo y perdiendo la mayoría de los eventos. El *backend* RAM escribe directamente en un búfer interno sin interrumpir el flujo de ejecución.

La Tabla 8.3 resume la configuración de `prj.conf` para el canal RTT de estados y el *backend* CTF en RAM.

Tabla 8.3: Opciones de Kconfig que activan el canal RTT de estados y el *backend* CTF en RAM.

Opción	Valor	Motivo
<i>Canal RTT (<code>app_trace.c</code>): anota los cambios de estado de alto nivel</i>		
CONFIG_USE_SEGGER_RTT	y	Habilita el canal. Es barato: solo se emiten los cambios (<code>SENSOR_IDLE/STREAMING</code> , <code>BLE_TX_CONNECTED/DISCONNECTED</code>), no cada muestra.
CONFIG_SEGGER_RTT_BUFFER_SIZE_UP	4096	Tamaño del búfer de subida del canal.
<i>Trazado CTF del núcleo con backend en RAM</i>		
CONFIG_TRACING	y	Activa el subsistema de trazado.
CONFIG_TRACING_CTF	y	Selecciona el formato CTF.
CONFIG_TRACING_SYNC	y	Sincroniza la marca de tiempo del trazado con el reloj del sistema.
CONFIG_TRACING_BACKEND_RAM	y	Es el único <i>backend</i> que no altera lo que mide: escribe cada evento con un <code>mempcpy</code> , sin hilo de trazas ni transporte.
CONFIG_RAM_TRACING_BUFFER_SIZE	153 600	150 KB. Debe coincidir exactamente con <code>ram_size</code> en <code>script.gdb</code> : si el volcado pide menos, sale truncado sin ningún aviso.
CONFIG_TRACING_BUFFER_SIZE	4096	Tamaño del búfer de paso intermedio del subsistema de trazado.
CONFIG_TRACING_ISR	n	El <i>stack</i> BLE genera miles de interrupciones por segundo y llenaría el búfer en décimas de segundo.
CONFIG_TRACING_SYSCALL	n	Emite eventos que <code>ctf2perfetto.py</code> no sabe descodificar y corrompe el resto de la captura al resincronizar.
<i>Nombres de hilo legibles en la traza</i>		
CONFIG_THREAD_NAME	y	Asocia un nombre legible a cada hilo.
CONFIG_THREAD_MONITOR	y	Se declaran bajo «Registro» en <code>prj.conf</code> porque el propio subsistema de <i>logging</i> las usa también.

El tamaño del búfer (153 600 bytes, 150 KB) es el mayor que admite el mapa de memoria del proyecto sin comprometer las pilas de los hilos ni los búferes del *stack* Bluetooth, sobre los 256 KB de RAM del SoC. Con él se capturan algo más de tres segundos de ejecución en transmisión activa —la sesión analizada en este capítulo dura 3 s 088 ms—; la supresión del trazado de interrupciones y de llamadas al sistema es lo que hace viable esa duración, pues con ambos activos la actividad del controlador de radio agotaría el

búfer en fracciones de segundo. Ampliar la ventana mucho más allá no sería solo cuestión de memoria: la marca de tiempo del formato CTF ocupa 32 bits en nanosegundos, de modo que vuelve a cero cada 4,295s y el conversor debe reconstruir el tiempo absoluto a partir de esos ciclos.

Fase 2: Extracción por J-Link/SWD y fusión con el canal RTT

Una vez finalizada la sesión de trazado, el firmware sigue ejecutándose y el contenido del búfer RAM permanece intacto en la memoria del SoC. El guion de automatización `tools/tracing/run_trace.sh` orquesta los siguientes pasos:

1. **Localización dinámica del símbolo en el ELF:** el guion no codifica ninguna dirección. Busca el `zephyr.elf` bajo el árbol de compilación y extrae con `nm`, filtrado por `grep`, la dirección y el tamaño del símbolo `ram_tracing`, de modo que un cambio del tamaño del búfer no obliga a tocar el guion.
2. **Registro RTT en paralelo (opcional):** si `JLinkRTTLogger` está disponible, se lanza en segundo plano sobre el canal 0 (`-RTTChannel 0`) antes del `reset`, volcando la salida a `rtt_output.txt`, para capturar las anotaciones de `app_trace.c` durante toda la ventana.
3. **Reset y ejecución limpia:** el guion escribe un guion temporal de J-Link Commander con las órdenes de `reset`, arranque y salida, y lo ejecuta para que la sesión de trazado –y el canal RTT– comiencen desde el estado inicial del firmware, con el búfer y sus índices a cero.
4. **Volcado del búfer RAM:** tras la ventana de captura se repite el procedimiento, esta vez con la orden `savebin` de J-Link Commander, a la que se pasan la dirección y el tamaño leídos del ELF en el primer paso. El resultado es un fichero binario de 150 KB con la secuencia de eventos del núcleo en formato CTF. El propio guion comprueba que el tamaño volcado coincide con el del símbolo y avisa si no es así.

El script Python `ctf2perfetto.py` funde ambas fuentes. Del binario CTF lee eventos de 5 bytes de cabecera (marca de tiempo de 32 bits más identificador de evento) seguidos de un *payload* de tamaño fijo definido por el esquema de Zephyr; los eventos de conmutación de hilo se convierten en pares **B/E** (*begin/end*) del formato *Chrome Trace Event*, y el resto –semáforos, temporizadores– en eventos instantáneos **i**. Del registro RTT lee líneas JSON que `app_trace.c` emite en cada cambio de estado, con la forma `{"name": ..., "ph": ..., "ts": ..., "pid": 0, "tid": ..., "args": {"state": ...}}` —las claves son ya las del formato *Chrome Trace Event*, de modo que el conversor no tiene que traducirlas—, y las traduce a dos pistas semánticas fijas: **Sensor (App)**, que alterna entre **IDLE** y **STREAMING** según `sensor_task.c`, y **BLE-TX (App)**, que alterna entre **DISCONNECTED** y **CONNECTED** según `ble_task.c`. Estas dos pistas no son trazas de planificación: son la interpretación semántica que el propio firmware hace de su estado, y por eso complementan –no duplican– las pistas **Sensor_Task** y **BLE-TX_Task** de más abajo, que sí son conmutaciones de hilo puras.

La invocación final es `python3 ctf2perfetto.py -c trace.bin -r rtt_output.txt -o trace.json`, y el fichero `trace.json` resultante se abre directamente en <https://ui.perfetto.dev> para su inspección interactiva.

8.5.2. Vista general del sistema

La Figura 8.1 muestra la traza completa de una sesión de 3 s 087 ms 799 μ s —la duración que impone el tamaño del búfer—, desde el arranque hasta la transmisión activa con el enlace ya cifrado.

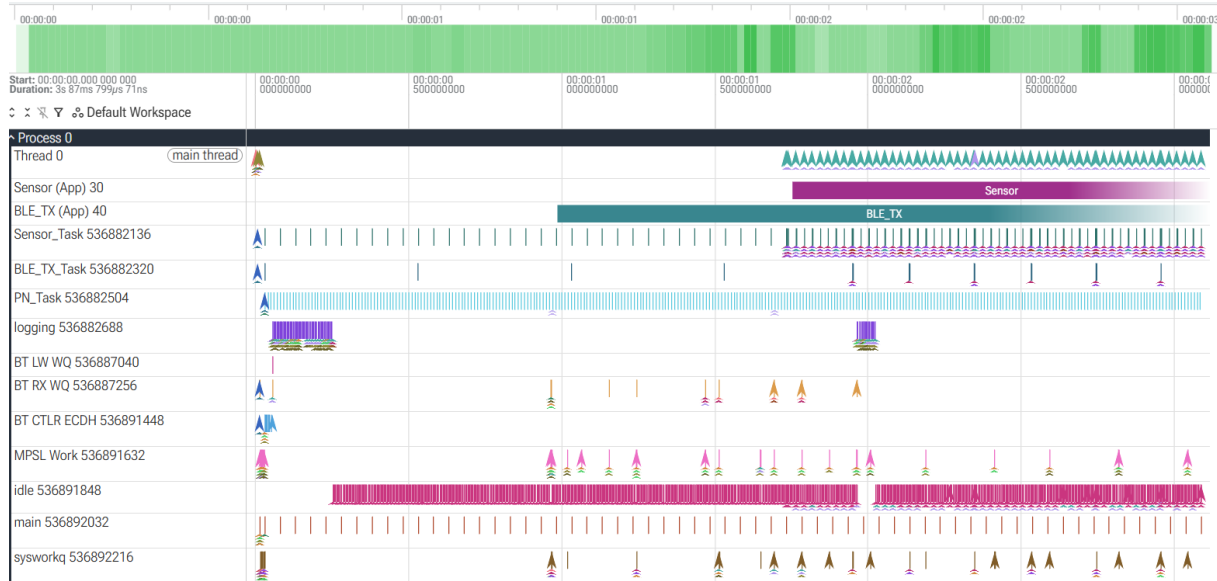


Figura 8.1: Vista global de la traza durante 3 s 088 ms. Las dos primeras pistas, **Sensor (App)** y **BLE_TX (App)**, son las anotaciones semánticas del canal RTT y delimitan las tres fases de la sesión: anuncio, conexión establecida y *streaming* activo. Debajo, las pistas de planificación del núcleo muestran los hilos reales: **Sensor_Task** y **BLE_TX_Task** (aplicación), **PN_Task** (motor de la Red de Petri), **logging**, los hilos del *stack* Bluetooth de Zephyr (**BT LW WQ**, **BT RX WQ**, **BT CTRLR ECDH**), **MPSL Work** (radio de Nordic), **sysworkq**, **main** e **idle**.

La Tabla 8.4 identifica la función de cada pista, para que la línea de tiempo se lea sin ambigüedad.

Tabla 8.4: Leyenda de las pistas visibles en la traza Perfetto.

Pista	Origen	Función
<i>Anotaciones semánticas de la aplicación (canal RTT)</i>		
Sensor (App)	app_trace.c	Estado de la adquisición: alterna entre IDLE y STREAMING.
BLE_TX (App)	app_trace.c	Estado del enlace: alterna entre DISCONNECTED y CONNECTED.
<i>Hilos de la aplicación (conmutación de contexto, CTF)</i>		
Sensor_Task	Aplicación	Sondea la FIFO del MAX30102 por I ² C y compone tramas de 20 muestras en <code>sample_msgq</code> .
BLE_TX_Task	Aplicación	Desencola las tramas y las envía por notificaciones GATT.
PN_Task	Aplicación	Motor de la Red de Petri; evalúa guardas y dispara transiciones cada 10 ms.
<i>Hilos del sistema operativo y del stack de radio</i>		
main	Zephyr	Inicializa el sistema y lanza los demás hilos.
logging	Zephyr	Vuelca las trazas de depuración al canal RTT.
sysworkq	Zephyr	Cola de trabajo del sistema; ejecuta el trabajo diferido de <code>settings</code> .
idle	Zephyr	Prioridad mínima; pone el SoC en modo WFI cuando nadie más requiere CPU.
BT LW WQ	Zephyr BT	Cola de baja prioridad; procesa <i>callbacks</i> de conexión y GATT.
BT RX WQ	Zephyr BT	Gestiona los eventos HCI y ACL entrantes del controlador de radio.
BT CTLR ECDH	Zephyr BT	Curva elíptica del emparejamiento LE Secure Connections (Sec. 3.2.5).
MPSL Work	Nordic	Capa multiprotocolo; planifica los <i>timeslots</i> de radio del nRF52840.

Las dos pistas semánticas dividen la sesión en tres fases, y esa división es la clave para leer correctamente todas las medidas que siguen. Confundirlas es el error de interpretación más fácil de cometer sobre esta traza, porque el mismo hilo exhibe periodos distintos en cada una:

1. **Anuncio (0 — 985 ms).** No hay cliente. `Sensor_Task` despierta cada 50 ms —la constante `SENSOR_IDLE_DELAY_MS`— únicamente para realimentar el perro guardián, y `BLE_TX_Task` cada 500 ms, que es el plazo con el que espera en `sample_msgq` cuando no llegan tramas (`BLE_TX_POLL_MS`).
2. **Conectado sin flujo (985 ms — 1,753 s).** La pista `BLE_TX (App)` pasa a `CONNECTED`, pero el cliente todavía no ha activado las notificaciones: la Red de Petri permanece en `P_CONNECTED_IDLE` y las cadencias no cambian.

3. **Streaming activo (desde 1,753 s).** **Sensor (App)** pasa a **STREAMING** y **Sensor_Task** adopta el patrón de ráfagas densas que caracteriza la adquisición. Solo esta fase es representativa del régimen de trabajo del dispositivo, y es sobre ella —no sobre la traza completa— donde se toman las medidas de las secciones siguientes.

De la vista global se extraen además dos observaciones sobre el diseño:

- **El emparejamiento no compite con la adquisición.** El hilo **BT_CTRLR_ECDH** —el cálculo de curva elíptica sobre el que se apoya *LE Secure Connections*— aparece en los primeros milisegundos de la traza, porque la pila precalcula el par de claves al inicializarse, no en el instante del emparejamiento. El emparejamiento propiamente dicho no puede ocurrir durante el anuncio: exige una conexión ya establecida, de modo que se resuelve después del instante marcado como **CONNECTED**, a los 985 ms. Conviene ser exacto sobre lo que ese instante señala: el firmware emite esa marca en la devolución de llamada de conexión, es decir, cuando el enlace *físico* queda establecido, no cuando termina la negociación de seguridad. Esta última se confirma más tarde, en la devolución **security_changed**, y la traza no la instrumenta con un suceso propio.

La conclusión que la traza sí sostiene es indirecta pero suficiente: el acceso a las características **GATT** está protegido por permisos que exigen cifrado, de modo que el hecho de que el flujo arranque a los 1,753 s demuestra que la negociación había concluido antes de ese instante. Lo relevante para la medida es que tanto la pre-computación como el intercambio de claves quedan resueltos antes de que la pista **Sensor (App)** pase a **STREAMING**: activar el cifrado no introduce ningún hilo nuevo en la fase en que la periodicidad importa. Marcar el fin del emparejamiento con un suceso explícito permitiría acotar su coste temporal, y es una mejora sencilla de la instrumentación.

- **El hilo idle nunca desaparece.** Sigue ejecutándose de forma continuada incluso en la tercera fase, lo que anticipa el resultado que se cuantifica más adelante: el sistema no está saturado en ningún momento de la captura.

8.5.3. El ciclo de sondeo de la FIFO

La capa de adquisición no lee una muestra cada 10 ms, sino que consulta los punteros de la FIFO del sensor cada 25 ms y se lleva de una vez cuantas muestras haya disponibles. La razón de ese diseño es que el oscilador interno del MAX30102 tiene tolerancia de fabricación y no produce exactamente 100 Hz: una espera fija acaba desincronizándose del sensor, y basta con quedarse ligeramente por detrás para que la FIFO se desborde y descarte muestras. La traza permite comprobar directamente ese ciclo, y conviene hacerlo en dos pasos: primero su periodo, después su contenido.

Periodo del sondeo

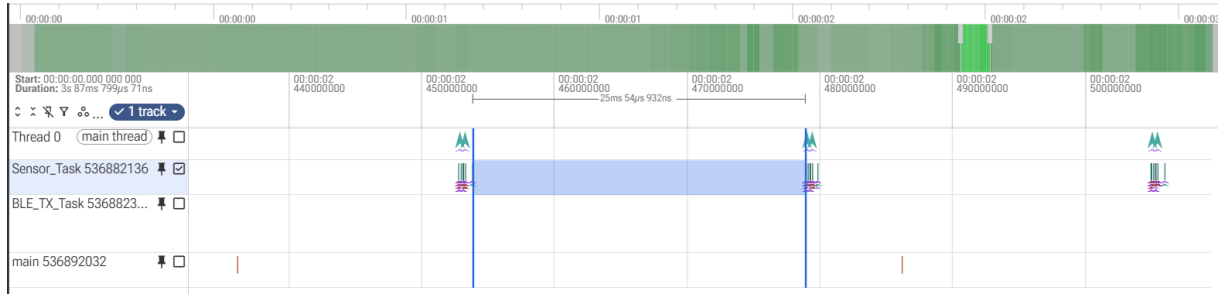


Figura 8.2: Separación entre dos ráfagas consecutivas de **Sensor_Task** durante el *streaming*: 25 ms 054 μ s 932 ns. La medida se toma entre el final de una ráfaga y el comienzo de la siguiente, es decir, sobre el intervalo de espera propiamente dicho.

El valor medido, 25,055 ms, reproduce la constante `SENSOR_FIFO_POLL_MS` con un error de 55 μ s, esto es, un 0,22 %. La desviación no es ruido de medida sino la latencia de planificación: entre que el temporizador vence y el planificador devuelve la CPU a **Sensor_Task** transcurre un tiempo pequeño pero no nulo.

Merece la pena detenerse en una distinción que la propia figura obliga a hacer. La espera se implementa con `os_delay_ms()`, que es un retardo **relativo**: se cuenta a partir del instante en que el hilo lo invoca, no desde un origen absoluto. En consecuencia, el periodo real de repetición no es de 25 ms sino de 25 ms más lo que dure el trabajo de la vuelta:

$$T_{\text{repetición}} = T_{\text{espera}} + T_{\text{trabajo}} = 25,055 \text{ ms} + 1,04 \text{ ms} \approx 26,1 \text{ ms} \quad (8.1)$$

Ambas cifras son correctas y miden cosas distintas: la figura mide la espera, y el análisis del fichero de traza —que recorre los instantes de comienzo de cada ráfaga— mide la repetición. Esta diferencia, de aspecto menor, es la que explica por completo el fenómeno que se analiza en la Sección 8.5.4, de modo que conviene tenerla presente.

Anatomía de un sondeo

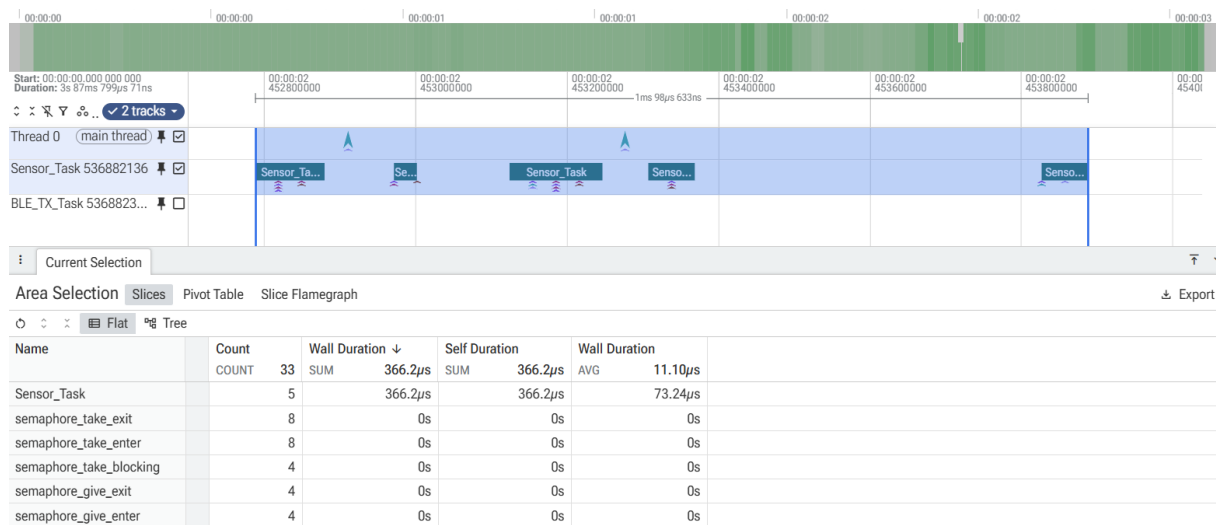


Figura 8.3: Ampliación sobre un único ciclo de sondeo (ventana de 1 ms 098 μ s). Perfetto contabiliza 33 eventos: 5 *slices* de **Sensor_Task** que suman 366,2 μ s de CPU, y los semáforos del controlador I²C que los separan.

Tabla 8.5: Desglose de un ciclo de sondeo completo, leído sin redondear del panel *Current Selection* de Perfetto.

Elemento	Recuento	Suma	Media
Sensor_Task	5	366,2 μ s	73,24 μ s
semaphore_take_enter	8	≈ 0	—
semaphore_take_exit	8	≈ 0	—
semaphore_take_blocking	4	≈ 0	—
semaphore_give_enter	4	≈ 0	—
semaphore_give_exit	4	≈ 0	—
Total	33	366,2 μs	11,10 μs

La estructura de cinco *slices* no significa que el hilo despierte cinco veces: despierta **una**, y esa única activación queda partida en cinco tramos porque cuatro veces cede la CPU mientras espera al bus. El contador `semaphore_take_blocking` = 4 lo confirma, y esas cuatro esperas se corresponden una a una con las cuatro transacciones que el ciclo necesita:

1. Escritura del registro 0x04 para posicionar el puntero de lectura.
2. Lectura de los tres bytes de punteros: `FIFO_WR_PTR`, `OVF_COUNTER` y `FIFO_RD_PTR`.
3. Escritura del registro 0x07 para apuntar a la zona de datos.
4. Lectura en ráfaga de todas las muestras disponibles, en una sola transacción.

Cada acceso `i2c_burst_read_dt()` se descompone en el controlador TWIM del nRF52840 en dos transferencias —escritura de la dirección de registro y lectura de datos—, y cada una espera su semáforo de terminación. De ahí que dos llamadas de la capa HAL produzcan cuatro bloqueos.

Este es el punto donde la traza documenta el efecto de la reforma de la adquisición. En la implementación anterior el firmware ejecutaba tres accesos I²C **por cada muestra**; en la actual ejecuta cuatro transferencias **por lote**, con independencia de cuántas muestras contenga el lote. El coste del bus ha dejado de escalar con el número de muestras.

8.5.4. El periodo de trama y su cuantificación

Cada 20 muestras el firmware cierra una trama, la encola y despierta a BLE_TX_Task. A 100 Hz, el periodo nominal de ese ciclo es de 200 ms. La Figura 8.4 mide el intervalo real entre dos transmisiones consecutivas.

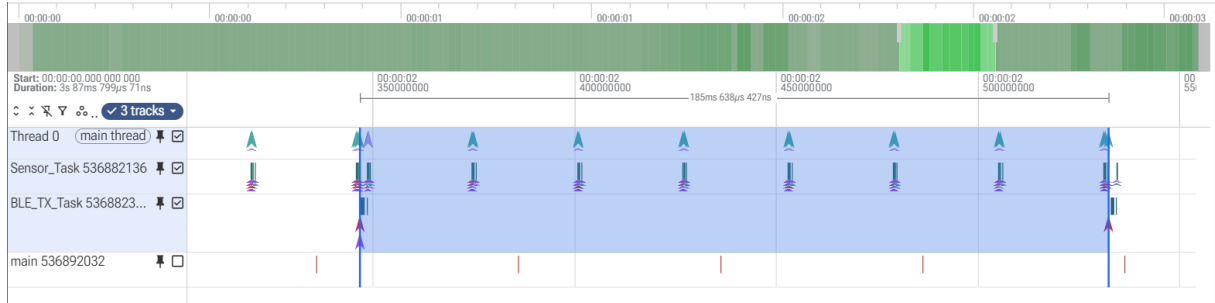


Figura 8.4: Intervalo entre dos activaciones consecutivas de BLE_TX_Task: 185ms 638µs 427ns. Entre ambos extremos se cuentan las ráfagas de Sensor_Task que han ido acumulando las 20 muestras de la trama.

El intervalo medido es de 185,638 ms, no de 200 ms. La discrepancia es del 7,2 % y es compatible con el efecto de **cuantificación** que se describe a continuación, sin que se aprecie, en la sesión analizada, ningún defecto del firmware: se trata de un efecto que conviene explicar con detalle, porque afecta a cómo debe interpretarse el periodo de trama en todo el trabajo.

Origen del fenómeno

Una trama se cierra cuando se han acumulado 20 muestras, y las muestras no llegan de una en una sino en lotes, uno por cada sondeo. Con un sondeo cada 26,1 ms (Ecuación 8.1) y una producción de 100 Hz, el lote medio contiene

$$\bar{n} = 100 \text{ Hz} \times 26,1 \text{ ms} = 2,61 \text{ muestras por sondeo} \quad (8.2)$$

Pero un lote es necesariamente un número *entero* de muestras: el sensor entrega 2 o 3, nunca 2,61. La trama se cierra, por tanto, en el primer sondeo cuyo acumulado alcanza o supera 20, y el número de sondeos necesarios oscila entre 7 y 8 según cómo se hayan repartido los enteros:

$$7 \times 26,1 \text{ ms} = 182,7 \text{ ms} \quad \text{frente a} \quad 8 \times 26,1 \text{ ms} = 208,8 \text{ ms} \quad (8.3)$$

La comprobación decisiva es la diferencia entre el intervalo más corto y el más largo observados en la traza:

$$212,067 \text{ ms} - 185,638 \text{ ms} = 26,429 \text{ ms} \approx T_{\text{repetición}} \quad (8.4)$$

La diferencia **coincide con un periodo de sondeo** dentro del margen de la propia medida: 26,429 ms frente a los 26,1 ms que dura un ciclo completo, un 1,3 % de desviación. No es un valor arbitrario ni proporcional al error de un oscilador, y es la explicación que mejor se ajusta a lo observado: la cuantificación del instante de envío por el periodo de sondeo. No constituye, en rigor, una demostración de exclusividad —una sola sesión no descarta formalmente otras causas—, pero sí una concordancia difícil de atribuir al azar.

Por qué el efecto no se acumula

Un desfase que se repitiera siempre en el mismo sentido produciría deriva, y la deriva sí sería un defecto. No es el caso, porque el mecanismo se compensa a sí mismo. La capa HAL lee como máximo las muestras que faltan para completar la trama en curso (`SAMPLES_PER_FRAME - filled`); las que sobran permanecen en la FIFO del sensor y encabezan la trama siguiente. Una trama que se cierra pronto deja menos remanente a su sucesora, que necesitará un sondeo más. La Tabla 8.6 recoge los intervalos consecutivos medidos.

Tabla 8.6: Intervalos entre activaciones consecutivas de `BLE_TX_Task` durante el *streaming*, extraídos del fichero de traza.

Trama	Intervalo	Desviación	Sondeos
1	185,638 ms	−15,6 ms	7
2	212,067 ms	+10,9 ms	8
3	185,638 ms	−15,6 ms	7
4	211,426 ms	+10,2 ms	8
5	211,304 ms	+10,1 ms	8
Media	201,215 ms	+1,2 ms	—

El patrón alterna intervalos cortos y largos, y la media sobre cinco tramas consecutivas es de 201,215 ms: un 0,61 % por encima del valor nominal de 200 ms. Ese residuo es del mismo orden que la tolerancia del oscilador interno del MAX30102, que es precisamente la razón por la que la adquisición se rediseñó para seguir al sensor en lugar de a un temporizador propio. Dicho de otro modo: el sistema no persigue un periodo fijo, sino que se adapta al ritmo real del sensor, de manera que la fluctuación del intervalo entre tramas no implica por sí misma pérdida de datos. Conviene formular el resultado con la precisión que admite: **en la sesión analizada no se registró ningún desbordamiento de la FIFO ni ningún descarte de trama** —los contadores de los códigos 23 y 24 permanecieron a cero durante toda la captura—. Eso no equivale a una garantía absoluta: el firmware conserva tres vías de pérdida posibles. Las dos primeras son el desbordamiento en el propio sensor, si el planificador retrasase el sondeo más allá del margen de 320 ms, y el descarte de una trama si la cola de muestras se saturase por congestión del enlace. La tercera ocurre más tarde, ya fuera de la cola: una trama extraída correctamente puede perderse al enviarla por radio, sea porque no hay crédito disponible en el semáforo de notificaciones o porque la propia pila la rechaza.

Esa tercera vía se instrumenta de forma distinta a las otras dos, y conviene decirlo. El fallo *persistente* de transmisión sí se registra: el código 42 tras cinco envíos consecutivos fallidos, y el 45 cuando la trama no llega siquiera a serializarse. Un rechazo aislado —la pila que devuelve error, o el crédito de notificaciones que no llega a tiempo— alimenta esa racha pero no genera código propio, y por tanto no deja constancia del número de secuencia concreto que se perdió. Sí lo detecta el receptor, que lleva la cuenta de los huecos en `seq`: la información existe, pero está en el cliente y no en la caja negra del dispositivo. Completar esa simetría —anotar en el equipo la secuencia de cada trama no enviada— es una mejora pendiente de la instrumentación, no un defecto del transporte. La aportación del diseño no es que las tres vías desaparezcan, sino que ninguna es silenciosa: las dos primeras se registran en el propio dispositivo con causa y cantidad exacta, y la tercera es observable extremo a extremo desde el cliente.

Conviene por ello enunciar el resultado con precisión: el periodo de trama es de **200 ms en promedio**, con una desviación típica de 12,7 ms coherente con la cuantificación del sondeo, que es la explicación que mejor se ajusta a lo observado. El valor instantáneo de una trama concreta no es una magnitud que el diseño garantice, ni necesita serlo: lo que la aplicación cliente requiere es que la base de tiempos de las muestras sea correcta, y de eso se ocupa el campo `t_ms`, que la capa de adquisición retrasa en $(n - 1) \times 10$ ms para referirlo a la primera muestra del lote y no al instante de la lectura.

Métricas del ciclo completo

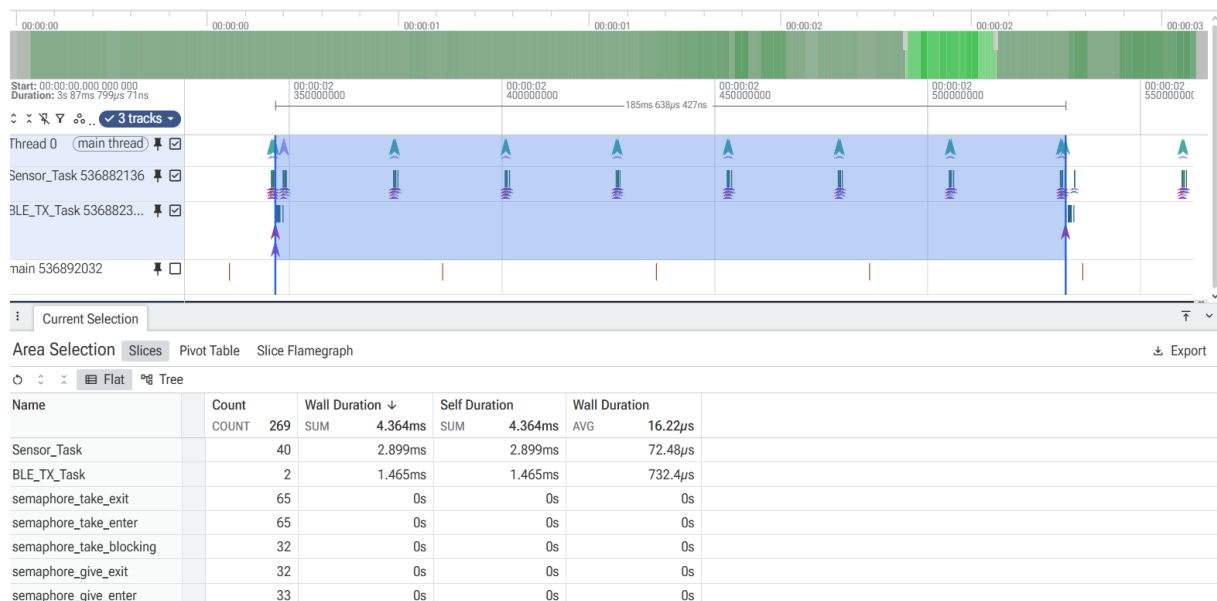


Figura 8.5: La misma ventana de 185,638 ms con el panel *Current Selection* desplegado. Los 269 eventos se reparten entre las activaciones de los dos hilos de datos y los semáforos del bus.

Tabla 8.7: Desglose exacto de la ventana de 185,638 ms (269 eventos), correspondiente a una trama completa de 20 muestras.

Elemento	Recuento	Suma	Media
Sensor_Task	40	2,899 ms	72,48 μ s
BLE_TX_Task	2	1,465 ms	732,4 μ s
semaphore_take_enter	65	≈ 0	—
semaphore_take_exit	65	≈ 0	—
semaphore_take_blocking	32	≈ 0	—
semaphore_give_enter	33	≈ 0	—
semaphore_give_exit	32	≈ 0	—
Total	269	4,364 ms	16,22 μs

Los recuentos de la Tabla 8.7 son mutuamente consistentes, y esa consistencia es lo que permite darlos por buenos:

- **Sondeos ejecutados.** Las 40 activaciones de **Sensor_Task** divididas entre los 5 *slices* de cada sondeo dan $40 \div 5 = 8$ sondeos. El mismo resultado se obtiene por el otro camino: 32 esperas bloqueantes entre las 4 de cada sondeo dan igualmente 8. Dos contadores independientes arrojan el mismo número.
- **Muestras acumuladas.** Ocho sondeos a 2,61 muestras de media suman 20,9, coherente con las 20 muestras que cierran la trama más el pequeño remanente que pasa a la siguiente.
- **Una única transmisión.** Los 2 *slices* de **BLE_TX_Task** no son dos envíos: son un envío partido en dos por la expropiación de **PN_Task**, que tiene mayor prioridad. Lo confirma la suma, 1,465 ms, coherente con el envío único de una notificación de 212 bytes.

De estas cifras se derivan las cargas de CPU de los dos hilos de datos, calculadas sobre la suma de tiempo activo —no sobre la media— y referidas al ciclo completo de trama:

$$U_{\text{sensor}} = \frac{2,899 \text{ ms}}{185,638 \text{ ms}} = 1,56 \% \quad U_{\text{BLE}} = \frac{1,465 \text{ ms}}{185,638 \text{ ms}} = 0,79 \% \quad (8.5)$$

8.5.5. Periodicidad del motor de la Red de Petri

Las vistas anteriores caracterizan las tareas que mueven el dato. Queda la tercera pieza: el motor de control que las gobierna.

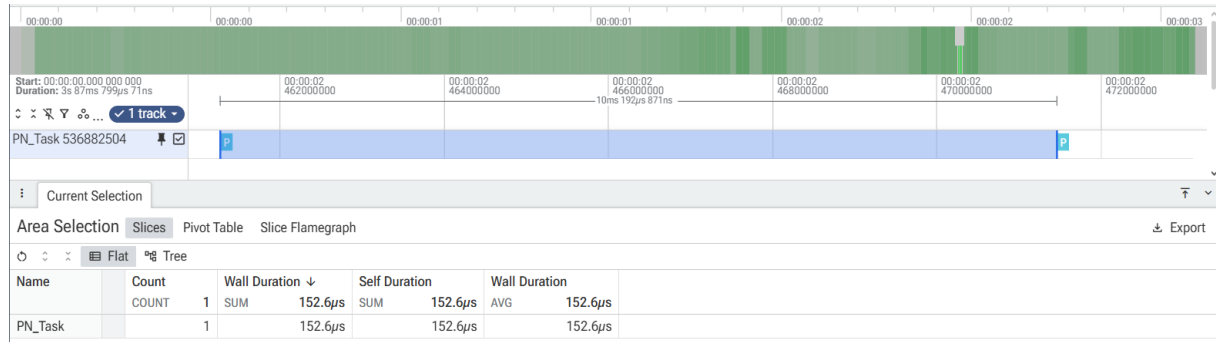


Figura 8.6: Ventana de 10 ms 192 μ s 871 ns con PN.Task como única pista seleccionada. Perfetto contabiliza **una sola activación** de 152,6 μ s, delimitada por los dos extremos de la selección.

Esta medida verifica dos propiedades que hasta ahora solo se habían afirmado por diseño:

- **El periodo de tictac coincide con el declarado.** El firmware fija PN_TICK_PERIOD_MS a 10 ms en `app_common.h`, y la separación medida entre dos activaciones es de 10,193 ms. La diferencia de 193 μ s se descompone en los 152,6 μ s que dura la propia evaluación —por el mismo carácter relativo de la espera que se discutió en la Ecuación 8.1— más unos 40 μ s de latencia de planificación. El presupuesto temporal cierra sin residuo inexplicado.
- **El coste del control es marginal.** Los 152,6 μ s que consume una evaluación completa de la red —recorrer las 25 transiciones, comprobar sus guardas y disparar todas las que resultan habilitadas según la semántica secuencial voraz— representan un **1,53 %** del periodo nominal de 10 ms —un 1,50 % del periodo medido de 10,193 ms—. El determinismo que aporta el formalismo no se paga, por tanto, con carga de CPU apreciable.

De una medida a una distribución

Una ventana que contiene una sola activación es una comprobación, no una demostración de periodicidad: podría ser una coincidencia afortunada. Para descartarlo se han extraído del fichero de traza los instantes de comienzo de *todas* las activaciones de PN.Task durante la sesión y se han medido los intervalos consecutivos. La Tabla 8.8 resume el resultado.

Tabla 8.8: Distribución de los intervalos entre activaciones consecutivas de PN_Task sobre la sesión completa.

Magnitud	Valor
Activaciones registradas	301
Intervalos analizados	299
Media	10,216 ms
Mediana	10,223 ms
Mínimo	10,162 ms
Máximo	10,773 ms
Desviación típica	0,052 ms
<i>Jitter</i> pico a pico	0,610 ms
Dentro de $\pm 0,5$ ms del nominal	296 de 299 (99,0 %)

La desviación típica de $52 \mu\text{s}$ sobre trescientos ciclos, con un *jitter* pico a pico de 0,610 ms —un 6 % del periodo— es la evidencia cuantitativa de que la periodicidad del control no depende de la ventana elegida.

Se ha excluido del cómputo un único intervalo de 0,153 ms observado en $t = 1,583$ s. No corresponde a dos ciclos consecutivos, sino a una activación que el planificador partió en dos tramos por expropiación: la traza registra dos comienzos de *slice* para una sola evaluación de la red. Documentarlo, en lugar de promediarlo en silencio, es parte de la medida.

Conviene subrayar que PN_Task es la tarea de mayor prioridad del sistema (PN_TASK_PRIORITY = 4, frente al 5 de las tareas de sensor y radio). Que aun siendo la más prioritaria consuma el 1,50 % del tiempo hace muy improbable que bloquee a las demás durante un intervalo apreciable: termina y cede el procesador mucho antes de que llegue el siguiente lote de muestras. Conviene no ir más lejos con este dato: una ocupación baja acota la duración de los bloqueos, pero no demuestra por sí sola la ausencia de inversión de prioridad, que depende de los recursos compartidos entre tareas y no sólo del tiempo de CPU. La expropiación que parte en dos la activación de BLE_TX_Task, observada en la Sección 8.5.4, es exactamente esta prioridad funcionando según lo previsto.

8.5.6. Cuatro relojes independientes

Las secciones anteriores han medido, por separado, cuatro periodos que provienen de temporizadores distintos y no comparten ningún mecanismo de sincronización entre sí. La Tabla 8.9 los reúne frente a la constante que los declara en el código fuente.

Tabla 8.9: Periodos medidos sobre el hardware frente a la constante que los fija en el código fuente. Los cuatro primeros son temporizadores independientes; el quinto no es un temporizador, sino el resultado emergente de acumular 20 muestras.

Magnitud medida	Medido	Error	Constante del código
Ciclo de la Red de Petri	10,193 ms	+1,9 %	PN_TICK_PERIOD_MS = 10
Sondeo de la FIFO	25,055 ms	+0,2 %	SENSOR_FIFO_POLL_MS = 25
Reposo del sensor	50,110 ms	+0,2 %	SENSOR_IDLE_DELAY_MS = 50
Sondeo del transmisor	500,092 ms	+0,02 %	BLE_TX_POLL_MS = 500
Periodo de trama (media)	201,215 ms	+0,6 %	20 muestras ÷ 100 Hz

El resultado admite una lectura directa: cuatro temporizadores que no se comunican entre sí reproducen sus constantes con errores comprendidos entre el 0,02 % y el 1,9 %, y el error residual es compatible en todos los casos con la latencia de planificación y el carácter relativo de la espera; dentro de la ventana medida no se observa deriva acumulativa. Es la evidencia más directa de determinismo temporal que ofrece este trabajo, porque no depende de una única medida afortunada sino de la coincidencia simultánea de cinco magnitudes obtenidas por caminos distintos.

8.5.7. Efecto de la reforma de la adquisición

La captura analizada en este capítulo corresponde al firmware con la adquisición dirigida por los punteros de la FIFO. Disponer de la traza equivalente de la implementación anterior —la que leía una muestra cada 10 ms— permite cuantificar el efecto de la reforma sobre la misma unidad de trabajo: una trama completa de 20 muestras.

Tabla 8.10: Coste de adquirir una trama de 20 muestras, antes y después de sustituir la lectura por temporizador por la lectura dirigida por los punteros de la FIFO.

Magnitud (por trama de 20 muestras)	Antes	Después	Variación
Activaciones de <code>Sensor_Task</code>	60	40	−33,3 %
CPU consumida por la adquisición	4,364 ms	2,899 ms	−33,6 %
Carga de CPU atribuible al sensor	2,12 %	1,56 %	−26,4 %
Transacciones I ² C	3 por muestra	4 por lote	—
Pérdida de muestras detectable	No instrumentada	0x05, código 23	—
Muestras perdidas en la sesión	—	0	—

La reducción de un tercio en activaciones y en CPU es consecuencia directa de haber desacoplado el coste del bus del número de muestras: cuatro transferencias por lote en lugar de tres por muestra. Con todo, el resultado que justifica la reforma no es el ahorro de CPU —que era holgada de partida— sino la última fila de la tabla. La implementación anterior no disponía de ningún mecanismo para saber si el sensor había descartado muestras; la actual lee el contador de desbordamiento en cada sondeo y registra la pérdida

exacta como código 23 en la caja negra. En la sesión completa analizada, ese contador se mantuvo a cero.

8.5.8. Tabla de conformidad con los requisitos de tiempo real

La Tabla 8.11 consolida los parámetros de tiempo real medidos sobre esta traza, con el enlace cifrado y emparejado, frente a los requisitos formales del sistema.

Tabla 8.11: Conformidad de los requisitos de tiempo real, verificados mediante trazas Perfetto sobre hardware físico con el enlace cifrado y emparejado.

Parámetro	Valor esperado	Valor medido	Estado
Periodo de sondeo de la FIFO	25 ms	25,055 ms	PASS
Periodo de PN_Task	10 ms	10,193 ms	PASS
Periodo de trama (RF-1)	200 ms (medio)	201,215 ms	PASS
Muestras por trama (RF-3)	20 muestras	20 muestras	PASS
Detección de pérdida de muestras	Pérdidas detectables por contadores y seq	0 observadas en la sesión (contador 0x05); instrumentación disponible	PASS
Latencia de PN_Task	< 1 ms	152,6 μ s	PASS
Duración de envío BLE	< 5 ms	1,465 ms	PASS
Margen antes de desbordar la FIFO	> 2 $\times$ el sondeo	12,3 $\times$	PASS
Carga CPU de Sensor_Task	< 20 %	1,56 %	PASS
Carga CPU de PN_Task	< 10 %	1,50 %	PASS
Carga CPU de BLE_TX_Task	< 5 %	0,79 %	PASS
Emparejamiento fuera de la ruta crítica	No solapar con <i>streaming</i>	BT CTLR ECDH previo	PASS
Presencia del hilo idle	Maximizar	Activo en todo el <i>streaming</i>	PASS

Sobre la fila de pérdida de muestras conviene ser explícito: RF-2 define la transmisión como *best-effort* y no promete ausencia de pérdidas, de modo que lo que la tabla declara conforme no es un cero garantizado sino que **la pérdida es detectable** —por el contador de desbordamiento del sensor, por el de descartes en cola y por el número de secuencia en el cliente—. El cero corresponde a la sesión analizada y no puede extrapolarse como garantía de diseño.

Conviene no sumar aquí las cifras citadas más arriba: el 1,56 % de adquisición y el 0,79 % de

transmisión están medidos sobre la ventana de un ciclo de trama (185,638 ms), mientras que el 1,50 % del motor de control es la proporción de una única evaluación de la red frente al periodo nominal de su temporizador (10 ms), no una ocupación medida sobre esa misma ventana; sumar cifras con denominadores distintos no produce una ocupación real. La cifra homogénea es la de la Tabla 8.12, medida sobre la ventana completa de *streaming* (1,335 s): **Sensor_Task** 1,53 %, **PN_Task** 1,53 % y **BLE_TX_Task** 0,66 %, que sitúan a los tres hilos de la aplicación en el **3,72 %** de esa ventana. Conviene, además, no atribuir todo el resto al hilo ocioso: el reparto completo de la fase de *streaming*, calculado sobre el fichero de traza, es el de la Tabla 8.12.

Tabla 8.12: Ocupación de CPU por hilo durante la fase de *streaming* (1,335 s). La suma no alcanza el 100 % porque los tramos abiertos en los bordes de la ventana quedan fuera del cómputo.

Hilo	CPU	Naturaleza
idle	90,25 %	Modo WFI: el SoC duerme.
logging	4,33 %	Instrumentación: vuelca las trazas por RTT. No existe en la compilación de producción.
Sensor_Task	1,53 %	Adquisición.
PN_Task	1,53 %	Motor de control.
BLE_TX_Task	0,66 %	Transmisión.
MPSL Work	0,24 %	Planificación de <i>timeslots</i> de radio.
sysworkq	0,16 %	Trabajo diferido del sistema.
main	0,06 %	Residual tras la inicialización.

La cifra que merece comentario es el 4,33 % del hilo **logging**: es el coste del propio instrumento de medida, no del sistema medido. Ese hilo existe únicamente porque la compilación instrumentada vuelca las trazas por el canal RTT, y desaparece en la configuración de producción. Descontándolo, los hilos de aplicación y de sistema ocupan el 4,18 % de la CPU y el hilo **idle** el 90,25 %; el 1,24 % restante corresponde a los tramos que los bordes de la ventana dejan fuera del cómputo.

Ese margen no es un dato ornamental: es el observado en esta sesión, y sugiere que el dispositivo admitiría trabajo adicional —un filtrado en el propio nodo, una segunda característica GATT— sin comprometer la periodicidad de la adquisición, que es la propiedad de la que depende la validez de la señal. Confirmarlo exigiría medir con esa carga añadida: lo que aquí se acredita es la holgura disponible, no el resultado de haberla consumido.

Conviene delimitar el alcance de esta verificación. Se ha medido *una* sesión de poco más de tres segundos, la ventana que impone el búfer de trazas, y las cifras de carga proceden de selecciones concretas dentro de ella. No constituyen, por tanto, una caracterización estadística del peor caso, sino la comprobación de que el comportamiento nominal coincide con el diseñado y de que los márgenes son amplios: los tres hilos de aplicación al 3,72 % de CPU en la ventana común de *streaming* —y el conjunto de hilos no ociosos al 4,18 %— no están cerca de ningún límite. Lo que la medida acredita es, por tanto, un margen amplio en las condiciones nominales observadas; establecer que ese margen se mantiene en el peor caso exigiría una campaña de capturas que este Trabajo no ha realizado.

Hay además una limitación del instrumento que conviene documentar, porque condiciona cómo debe planificarse una captura. El *backend* de trazas en RAM no es circular: al llenarse el búfer deja de registrar en lugar de sobrescribir los eventos más antiguos. Con los 150 KB configurados, eso equivale a los primeros 3,1 s posteriores al *reset*, con independencia de la duración que se solicite al guion de captura. La consecuencia práctica es que el evento que se desea observar debe provocarse dentro de esa ventana inicial: en la sesión analizada, la conexión del cliente y el arranque del *streaming* ocurrieron a los 985 ms y 1,753 s respectivamente, dejando 1,3 s de *streaming* efectivo dentro de la captura.

Con todo, la aportación más relevante de esta traza es doble. Por un lado incorpora —por primera vez frente a las capturas previas del proyecto— el coste del emparejamiento Bluetooth activado en la Sección 3.2.5, y muestra que cifrar el enlace no compromete la cadencia: la precomputación criptográfica aparece durante la fase de *advertising*, mientras que el emparejamiento y el intercambio efectivo de claves se completan después de establecer la conexión y antes del inicio del *streaming*. La traza no permite fechar con precisión el final de la negociación —el evento **security_changed** no está representado con una marca propia—, de modo que lo que se acredita es ese orden, no un instante exacto. Por otro, es la que permitió detectar, explicar y verificar la cuantificación del periodo de trama analizada en la Sección 8.5.4, un fenómeno que ninguna prueba unitaria podía revelar porque solo se manifiesta cuando el firmware dialoga con el reloj real del sensor. La arquitectura en capas con Red de Petri reactiva, OSAL y protocolo binario ligero cumple los criterios de temporización de un sistema de captura biomédica en tiempo real, ahora con el enlace protegido y con la adquisición instrumentada frente a la pérdida de muestras.

9. Conclusiones y líneas futuras

9.1. Conclusiones

Este Trabajo de Fin de Máster ha cumplido los objetivos de diseño e implementación y ha validado los mecanismos funcionales, formales y temporales descritos, culminando en **BloodTwin**: una arquitectura de *firmware* robusta y verificable para un dispositivo de pulsioximetría, junto con la cadena —clientes Android y web, plataforma de integración clínica y herramienta de diagnóstico— que lleva la señal del donante hasta el registro de la donación conservando el vínculo entre ambos. Quedan pendientes, y así se recogen entre las limitaciones, la caracterización eléctrica del prototipo y la validación de extremo a extremo de la ruta actual con Google Cloud Storage.

Las principales conclusiones derivadas del desarrollo de este proyecto se resumen en los siguientes puntos:

1. **Un formalismo que hace posible la demostración:** El control se ha expresado como una red de Petri interpretada de 6 lugares de estado y 25 transiciones, ejecutada por un motor dirigido por tablas. La ventaja decisiva de esa elección no es la elegancia de la notación, sino que el motor mantiene una correspondencia directa entre el modelo y el código ejecutado, y esa correspondencia es lo que permite someter el control a análisis automático. El orden de declaración de las transiciones establece su prioridad, lo que convierte la resolución de conflictos —por ejemplo, un fallo fatal concurrente con un aviso de batería— en una propiedad del diseño y no en un efecto colateral de la implementación.
2. **Verificación formal del modelo de control:** Se han formulado y comprobado **17 propiedades** en lógica temporal lineal mediante el comprobador de modelos SPIN, cubriendo seguridad, vivacidad, alcanzabilidad y reglas propias del dispositivo. Ninguna resultó violada. La exploración exhaustiva del espacio de estados —entre 621 000 y 962 000 estados en las propiedades que exigen recorrerlo entero— ofrece una garantía que ninguna batería de pruebas por ejecución puede proporcionar, porque cubre también los entrelazados que no llegan a producirse. El proceso dejó además una lección metodológica que se documenta en el Capítulo 4: una propiedad verificada sobre un modelo incompleto no demuestra nada, y de hecho una de ellas figuraba como satisfecha mientras el firmware la incumplía.
3. **Aislamiento y portabilidad con OSAL:** La implementación de una capa de abstracción del sistema operativo (OSAL) ha permitido desacoplar las tareas y la lógica de negocio de las primitivas de Zephyr RTOS. Esto ha permitido ejecutar **68 pruebas unitarias** en el ordenador de desarrollo, alcanzando el 100 % de cobertura de funciones y el 96,6 % de líneas sobre el motor de control y el subsistema de errores. El valor de esa capacidad no está en el número de pruebas sino en cuáles

son posibles: los caminos de error —sensor ausente, bus caído, batería agotada, imposibilidad de crear un hilo— son los más difíciles de provocar sobre el hardware y los más importantes de verificar.

4. **Tolerancia a fallos persistente (Caja Negra):** El diseño del subsistema de "Caja Negra" apoyado en el sistema de almacenamiento NVS ha dotado al dispositivo de trazabilidad clínica post-mortem. El almacenamiento no volátil de la secuencia de incidencias y códigos de error permite que un fallo que llegase a generar informe se diagnostique *a posteriori*, extrayéndolo por USB en un arranque posterior con la herramienta del Capítulo 5. El alcance tiene dos límites que conviene declarar: la extracción es local —no remota—, y las incidencias que no llegan a provocar un informe permanecen sólo en el anillo volátil. Completa el mecanismo una política activa contra el bucle de reinicios (*bootloops*), que suprime la repetición del mismo fallo de arranque sin impedir el registro de uno distinto.
5. **Comunicaciones eficientes y con contrato único:** La serialización binaria mediante Protocol Buffers, con la implementación *nanopb* en el lado embebido, permite acomodar una trama de 20 muestras en un solo paquete dentro de la unidad máxima de transmisión negociada de 247 bytes, evitando la fragmentación. El beneficio adicional, y no menor, es que el mismo esquema genera el código de decodificación de los terminales: el formato deja de ser un acuerdo implícito entre programas escritos en lenguajes distintos y pasa a ser un contrato versionado. Medido sobre la ventana completa y común de *streaming*, el coste de CPU de la transmisión es del 0,66 %.
6. **Una cadena completa, no un prototipo aislado:** El sistema no termina en el dispositivo. Las medidas atraviesan un enlace inalámbrico con contrato explícito, uno de dos clientes compatibles con ese contrato, y una plataforma de integración que las deposita en la infraestructura de captura de datos clínicos de la institución. Que los dos clientes —móvil y web— consuman las mismas características GATT, el mismo esquema de serialización y la misma interfaz REST no es casual: es la propiedad que permite añadir un tercero sin tocar el dispositivo. La decisión de repartir cada dato entre el sistema de captura clínica y un almacén de objetos, conservando en el primero la referencia a los ficheros del segundo, está dirigida a resolver la tensión entre lo que un formulario clínico admite y lo que una señal fisiológica ocupa; su comportamiento en un despliegue completo con credenciales de Cloud Storage queda pendiente de comprobación.
7. **Mecanismos de bajo consumo implementados, consumo sin caracterizar:** El *firmware* emplea de forma sistemática los mecanismos de ahorro que ofrece la plataforma —modo *System OFF* con despertar por pulsador, radio detenida al declarar un fallo, sensor en reposo fuera de la adquisición y espera pasiva en el hilo ocioso—, y su entrada y salida del sueño profundo se comprobaron físicamente sobre la placa. Ahora bien, **no se ha medido la corriente del prototipo:** este Trabajo no dispuso de analizador de potencia, de modo que no aporta consumo medio ni autonomía verificada, y el cumplimiento del requisito de autonomía queda *pendiente de validación experimental* (Sección 8.4). El trazado del núcleo sí confirma que el sistema opera lejos de la saturación: medido sobre la ventana completa de *streaming* (1,335 s), la adquisición consume el 1,53 % de CPU, el motor de control el 1,53 % y la transmisión el 0,66 %, con el hilo de bajo consumo ocupando el 90,25 % del tiempo.

9.1.1. Limitaciones del Sistema

A pesar de los resultados satisfactorios obtenidos, es importante reconocer ciertas limitaciones inherentes al diseño actual del prototipo:

- **Sensibilidad a artefactos de movimiento:** Aunque se han implementado algoritmos de filtrado temporal para amortiguar el ruido, la tecnología fotoplethysmográfica (PPG) reflexiva sigue siendo susceptible a los movimientos bruscos y las contracciones musculares intensas, lo que en situaciones dinámicas extremas puede reducir temporalmente la fiabilidad de las mediciones.
- **Dependencia del tono de piel y perfusión:** La precisión óptica del sensor MAX30102 puede verse afectada en pacientes con fototipos de piel oscura (debido a la mayor absorción lumínica por la melanina) o en condiciones de baja perfusión periférica (hipotermia, hipotensión o vasoconstricción local).
- **La clave de emparejamiento es fija y compartida:** El equipo no tiene pantalla donde mostrar una clave aleatoria, de modo que el emparejamiento se apoya en una constante impresa en el dispositivo. Basta para impedir la conexión casual y para autenticar el enlace frente a un intermediario, pero no es un secreto criptográfico fuerte: quien tenga acceso físico al equipo la lee. Un despliegue con requisitos más estrictos exigiría claves por dispositivo, aprovisionadas en fabricación.
- **La verificación formal cubre el modelo, no la implementación:** Las 17 propiedades demuestran que el modelo satisface las propiedades formuladas; no demuestran que su traducción a C lo sea, ni que los plazos estén bien dimensionados. Esas garantías provienen de las pruebas unitarias y del trazado, que son necesariamente parciales.
- **El vínculo con el almacén de objetos no está verificado de extremo a extremo:** La integración con Google Cloud Storage está implementada, pero no pudo validarse de extremo a extremo porque las credenciales y los permisos de acceso al *bucket* del proyecto no estuvieron disponibles durante el periodo de pruebas. Las etapas se comprobaron por separado; queda pendiente verificar, con credenciales válidas, que la ruta actual sube los CSV, escribe sus nombres en REDCap y permite recuperarlos posteriormente (Sección 7.4.3). Se trata de una limitación del entorno de pruebas y no de un defecto acreditado del diseño, pero mientras no se complete esa comprobación la cadena no puede declararse validada en toda su longitud.
- **Ausencia de calibración clínica:** Las constantes de la curva de saturación proceden de la referencia del fabricante y no de un estudio de desaturación controlada. El dispositivo es, por tanto, una herramienta de monitorización de tendencia y no un instrumento de medida validado clínicamente.
- **El modelo inferencial queda fuera de este Trabajo:** La infraestructura entrega el par (*señal óptica, referencia de laboratorio*) de forma trazable, pero no la función que traduce el primero en una estimación del segundo. Esa pieza exige una cohorte que solo puede reunirse una vez el sistema está desplegado, y pertenece al dominio del tratamiento de señal y del aprendizaje estadístico más que al de los sistemas empotrados. Lo aquí construido es su condición de posibilidad, no su sustituto.

En definitiva, este Trabajo sostiene una tesis concreta: que las técnicas de ingeniería de software con garantías —modelado formal del control, verificación por comprobación de modelos, pruebas sobre el ordenador anfitrión y diagnóstico persistente— son aplicables

a un sistema empotrado construido sobre mecanismos de bajo consumo sin comprometer su cadencia, y que el resultado se integra mediante las interfaces desarrolladas en una infraestructura de investigación clínica existente, aunque la validación completa con el almacén de objetos quede pendiente. Las cifras de cobertura, de verificación formal y de temporización recogidas en los capítulos anteriores son la evidencia de esa afirmación; las limitaciones enumeradas más arriba —entre ellas la ausencia de caracterización eléctrica— delimitan hasta dónde alcanza.

9.2. Líneas futuras

A partir del trabajo desarrollado en esta memoria, se abren interesantes líneas de investigación y desarrollo futuro para evolucionar el dispositivo de monitorización hacia un producto médico comercial certificado:

- **Caracterización eléctrica y validación de la autonomía:** Es la continuación más inmediata, porque cierra una carencia declarada en el Capítulo 8: este Trabajo no dispuso de instrumentación de medida, de modo que el consumo del prototipo y su autonomía real no se han cuantificado. Procede intercalar un analizador de potencia —el Nordic Power Profiler Kit II es el instrumento natural— en la alimentación de la placa y levantar el perfil de corriente de cada estado de la red, el de una sesión completa de donación y el residual en *System OFF*. Sólo con esas medidas puede darse por acreditado el requisito de autonomía y contrastarse el ahorro que se atribuye a las políticas de bajo consumo implementadas.
- **Validación de extremo a extremo con el almacén de objetos:** La integración con Google Cloud Storage está implementada, pero no pudo comprobarse en su recorrido completo porque las credenciales y los permisos del *bucket* no estuvieron disponibles durante el periodo de pruebas (Sección 7.4.3). Con un despliegue acreditado procede verificar que la ruta actual del paso de dispositivo sube los tres CSV, escribe sus nombres en REDCap y permite recuperarlos después, y convertir esa comprobación en una prueba de integración automatizada que impida futuras regresiones silenciosas.
- **Calibración clínica y modelo inferencial:** Las constantes de la curva de saturación proceden de la referencia del fabricante y no de un estudio de desaturación controlada, de modo que el dispositivo es hoy una herramienta de monitorización de tendencia y no un pulsioxímetro calibrado. Cerrar ese frente exige un estudio de calibración frente a un equipo patrón y, a partir del registro continuo que esta arquitectura hace posible, abordar el **modelo inferencial** que quedó fuera de este Trabajo y que la Sección 1.1 define como objetivo de partida: la estimación de hemoglobina a partir de la señal óptica, alimentada por los pares (*señal, referencia de laboratorio*) que esta arquitectura produce. Sobre ese mismo registro cabe además una línea de investigación distinta y complementaria: caracterizar la caída de perfusión periférica que precede a la reacción vasovagal y evaluar si permite anticiparla con antelación clínicamente útil. Es la línea que da sentido último a la señal capturada, y la que exigiría un protocolo con donantes reales y aprobación del comité de ética.
- **Robustez de la señal frente a artefactos y tono de piel:** Las dos limitaciones de naturaleza óptica declaradas en las conclusiones —la sensibilidad a los artefactos de movimiento y la dependencia del fototipo cutáneo y de la perfusión periférica—

no se resuelven en el firmware actual. Abordarlas exige, en el plano algorítmico, incorporar detección y rechazo de artefactos e índices de calidad de señal por ventana, y en el plano del hardware, revisar la corriente de los emisores y la geometría del acoplamiento óptico. Es un requisito previo a cualquier uso clínico en población diversa.

- **Claves de emparejamiento por dispositivo:** El emparejamiento se apoya hoy en una clave fija compartida, suficiente para impedir la conexión casual pero legible por quien tenga acceso físico al equipo. Un despliegue con requisitos más estrictos exigiría claves individuales aprovisionadas en fabricación y almacenadas en la región protegida del SoC, junto con la activación de `CONFIG_BT_SMP_SC_ONLY` mencionada más abajo.
- **Cifrado en reposo y extremo a extremo:** El enlace de radio ya está cifrado y su emparejamiento autenticado mediante clave fija (Sección 3.2.5), de modo que la confidencialidad *en tránsito* sobre BLE está razonablemente cubierta. Queda ahí un frente menor: la configuración exige cifrado y autenticación, pero no *fuera LE Secure Connections* mediante `CONFIG_BT_SMP_SC_ONLY`, de modo que un cliente que solo admitiera emparejamiento heredado podría negociarlo. Los enlaces observados en las pruebas alcanzaron el nivel 4, que implica LE Secure Connections, pero convertir esa observación en garantía exige activar esa opción y comprobar después la compatibilidad de los clientes. Quedan además dos frentes mayores. El primero es el cifrado *en reposo*: los informes de fallo persistidos en la memoria flash NVS del dispositivo —que no contienen señal en bruto, sino códigos de error y contexto diagnóstico—, la base de datos local de la aplicación Android y el almacenamiento del navegador conservan hoy su contenido en claro, y el módulo CryptoCell del nRF52840 permitiría proteger con aceleración por hardware la parte que reside en el dispositivo —la base de datos de Android y el almacenamiento del navegador exigen mecanismos propios de cada plataforma—, si bien su coste efectivo sobre la CPU y el consumo deberá medirse. El segundo es el tramo que va del cliente a la plataforma de integración. El cifrado del enlace de radio protege la señal hasta el cliente, pero ese segundo tramo no está hoy protegido: el entorno de desarrollo empleado en este Trabajo opera sobre HTTP, según se detalla más adelante en este mismo apartado. Resolverlo con terminación TLS es la condición mínima; aun así, la terminación en el servidor dejaría el contenido legible para un intermediario con acceso a él, y evitarlo es precisamente lo que exigiría cifrado *extremo a extremo*. Ambos son medidas que el RGPD contempla en su artículo 32 en función del riesgo, y que HIPAA clasifica como especificación *addressable*: un despliegue real tendría que justificarlas mediante un análisis de riesgos, no adoptarlas por defecto.
- **Heartbeat de nivel de aplicación:** El mecanismo de desconexión silenciosa (RR-2) solo observa que la radio local siga despachando tramas, no que la aplicación cliente las reciba; un cliente que cerrase la aplicación manteniendo abierto el enlace de transporte no sería detectado. Cerrar esa brecha exige que el propio protocolo de aplicación lleve una confirmación periódica —un *heartbeat*— que el firmware pueda verificar de forma independiente de la capa de enlace.
- **Endurecimiento de la plataforma de integración:** La revisión cruzada entre la implementación y esta memoria dejó identificados varios frentes en el lado del servidor y de los clientes. Los más graves ya están corregidos y se documentan en el Capítulo 7: el punto de mediciones valida el identificador antes de subir los ficheros y los borra si la escritura en REDCap falla; el alta es idempotente respecto

al par de códigos ISBT; los validadores numéricos se aplican efectivamente en las cuatro rutas de datos; y la interfaz web distingue visualmente un dato encolado de uno confirmado. Quedan abiertos cuatro frentes. El primero es la **asignación atómica del identificador de registro**: hoy se lee el último libre y se crea la fila en dos operaciones separadas, y aunque una comprobación posterior detecta la colisión y la reasigna, lo correcto sería derivarlo del identificador único que ya asigna REDCap. El segundo es una **fuentes única de rangos admisibles** compartida por el servidor y los dos clientes: hoy el servidor tiene la suya centralizada, pero Android y la interfaz web mantienen las propias, y nada garantiza que coincidan. El tercero es la **renovación del testigo de autenticación** en la cola en disco: por decisión deliberada el token no se persiste junto a los datos clínicos, de modo que un reenvío posterior al cierre de la sesión exige autenticarse de nuevo; resolverlo sin volver a escribir una credencial en disco exige un mecanismo de refresco que no se ha abordado. El cuarto es **derivar el documento OpenAPI del propio código** en lugar de mantenerlo escrito a mano en paralelo (Anexo D): al menos, contrastarlo con las rutas reales en la integración continua, para que una divergencia entre la especificación publicada y el comportamiento del servidor se detecte antes del despliegue y no la descubra un cliente en producción.

- **Terminación TLS en producción:** Interponer un *proxy* inverso con certificado válido entre los clientes y la plataforma, y rechazar el tráfico en claro. El entorno de desarrollo empleado en este Trabajo opera sobre HTTP, decisión aceptable en una red de laboratorio pero incompatible con el tratamiento de datos clínicos reales.
- **Diseño de una PCB personalizada (Hardware dedicado):** Desarrollar una placa de circuito impreso (PCB) personalizada integrando el System on Chip nRF52840, el sensor MAX30102, el circuito de carga de batería LiPo por USB y una antena de chip compacta para reducir el tamaño del prototipo a un formato de dispositivo de monitorización ergonómico y compacto.
- **Certificación médica:** Documentar los procesos del desarrollo de firmware según el Reglamento (UE) 2017/745 sobre los productos sanitarios (MDR) y los estándares internacionales de calidad de software embebido médico, específicamente la norma **IEC 62304** (Software para Dispositivos Médicos - Procesos del Ciclo de Vida del Software) e **ISO 13485** (Sistemas de Gestión de la Calidad para Dispositivos Médicos).

Bibliografía

- [1] Michael Grieves. Digital twin: Manufacturing excellence through virtual factory replication. White Paper, 2014. Accedido: septiembre 2026.
- [2] John Allen. Photoplethysmography and its application in clinical physiological measurement. *Physiological Measurement*, 28(3):R1–R39, 2007.
- [3] Jianming Zhu, Ruiyang Sun, Huiling Liu, Tianjiao Wang, Lijuan Cai, Zhencheng Chen, and Baoli Heng. A non-invasive hemoglobin detection device based on multispectral photoplethysmography. *Biosensors*, 14(1):22, 2024.
- [4] Paul Ashford and Matthew Delgado. ISBT 128 standard for coding medical products of human origin. *Transfusion Medicine and Hemotherapy*, 44(6):386–390, 2017.
- [5] Christos G. Cassandras and Stéphane Lafortune. *Introduction to Discrete Event Systems*. Springer, 2nd edition, 2008.
- [6] John G. Webster. *Medical Instrumentation: Application and Design*. Wiley, 4th edition, 2009.
- [7] Edwin D. Chan, Michael M. Chan, and Mallory M. Chan. Pulse oximetry: Understanding its basic principles facilitates appreciation of its limitations. *Respiratory Medicine*, 107(6):789–799, 2013.
- [8] Maxim Integrated. MAX30102 high-sensitivity pulse oximeter and heart-rate sensor for wearable health. Datasheet Rev. 1, 2018. Accedido: junio 2026.
- [9] Andrés Moreno Miguel. Diseño e implementación de un sistema de monitorización ambulatoria y no intrusiva de hemoglobina. Trabajo fin de grado, Escuela Técnica Superior de Ingenieros de Telecomunicación, Universidad Politécnica de Madrid, 2021. Grado en Ingeniería de Tecnologías y Servicios de Telecomunicación. No publicado.
- [10] Esther Moreno Moreno. Diseño e implementación de un banco de pruebas de un sistema inalámbrico de monitorización ambulatoria. Trabajo fin de grado, Escuela Técnica Superior de Ingenieros de Telecomunicación, Universidad Politécnica de Madrid, 2022. Grado en Ingeniería de Tecnologías y Servicios de Telecomunicación. No publicado.
- [11] María Esther Ortega Sánchez. Implementación y puesta en marcha de un sistema de monitorización inalámbrico para el cribado de pacientes con anemia en el sistema de hemodonación. Trabajo fin de grado, Escuela Técnica Superior de Ingenieros de Telecomunicación, Universidad Politécnica de Madrid, 2025. Grado en Ingeniería de Tecnologías y Servicios de Telecomunicación. No publicado.
- [12] Michael Barr and Anthony Massa. *Programming Embedded Systems: With C and GNU Development Tools*. O'Reilly Media, 2nd edition, 2006.
- [13] Nordic Semiconductor. nRF52840 product specification v1.7, 2021. Accedido: junio 2026.

- [14] The Zephyr Project. Zephyr project documentation, 2024. Accedido: junio 2026.
- [15] The Zephyr Project. Non-volatile storage (NVS), 2024. Accedido: junio 2026.
- [16] Bluetooth SIG. Bluetooth core specification v5.3, 2021. Accedido: junio 2026.
- [17] Google LLC. Protocol buffers: Developer guide. <https://protobuf.dev/>. Consultado en 2025.
- [18] Petteri Jokinen. Nanopb – protocol buffers for embedded systems, 2023. Accedido: junio 2026.
- [19] Carl Adam Petri. Kommunikation mit automaten. *Schriften des Rheinisch-Westfälischen Institutes für instrumentelle Mathematik*, 2, 1962. Tesis doctoral, Universidad de Bonn.
- [20] José M. Moya. **rxnet**: Reactive petri net and FSM runtime for embedded systems, 2026. Licencia MIT. Universidad Politécnica de Madrid.
- [21] Edmund M. Clarke, Orna Grumberg, and Doron A. Peled. *Model Checking*. MIT Press, 1999.
- [22] Amir Pnueli. The temporal logic of programs. In *18th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 46–57, 1977.
- [23] Gerard J. Holzmann. *The SPIN Model Checker: Primer and Reference Manual*. Addison-Wesley, 2004.
- [24] Throw The Switch. Ceedling: C project build and test system, 2024. Accedido: junio 2026.
- [25] Throw The Switch. Unity: Simple unit testing for C, 2024. Accedido: junio 2026.
- [26] ThrowTheSwitch. CMock: Automatic mock generation for C. <https://github.com/ThrowTheSwitch/CMock>. Consultado en 2025.
- [27] Matthias Cäsar, Tobias Pawelke, Jan Steffan, and Gabriel Terhorst. A survey on Bluetooth Low Energy security and privacy. *Computer Networks*, 205:108712, 2022.
- [28] Michael Jones, John Bradley, and Nat Sakimura. JSON web token (JWT). RFC 7519, Internet Engineering Task Force (IETF), 2015. Accedido: septiembre 2026.
- [29] Paul A. Harris, Robert Taylor, Robert Thielke, Jonathon Payne, Nathaniel Gonzalez, and Jose G. Conde. Research electronic data capture (REDCap) – a metadata-driven methodology and workflow process for providing translational research informatics support. *Journal of Biomedical Informatics*, 42(2):377–381, 2009.
- [30] Paul A. Harris, Robert Taylor, Brenda L. Minor, Veida Elliott, Michelle Fernandez, Lindsay O’Neal, Laura McLeod, Giovanni Delacqua, Francesco Delacqua, Jacqueline Kirby, and Stephany N. Duda. The REDCap consortium: Building an international community of software partners. *Journal of Biomedical Informatics*, 95:103208, 2019.
- [31] Google LLC. Perfetto – system profiling, app tracing and trace analysis, 2024. Accedido: septiembre 2026.
- [32] Mathieu Desnoyers and EfficiOS Inc. Common trace format (CTF) specification, 2024. Accedido: septiembre 2026.

-
- [33] World Health Organization. Global diffusion of ehealth: making universal health coverage achievable, 2016. Accedido: junio 2026.
- [34] OpenAPI Initiative. OpenAPI specification v3.0.3, 2021. Accedido: septiembre 2026.

Anexo A: Aspectos éticos, económicos, sociales y ambientales

El apartado “Requisitos de las acreditaciones internacionales EUR-ACE y ABET” de la Normativa de TFT de la ETSIT-UPM establece que *“La memoria del TFT del GITST, GIB y MUIT, y en general la de aquellas titulaciones que hayan obtenido o para las que se desee solicitar una acreditación internacional EUR-ACE o ABET, debe mostrar conciencia de la responsabilidad de la aplicación práctica de la ingeniería, el impacto social y ambiental, el compromiso con la ética profesional, la responsabilidad y las normas de la aplicación práctica de la ingeniería, así como sobre las prácticas de gestión de proyectos, gestión, y control de riesgos, entendiendo sus limitaciones”*.

Este anexo obligatorio del TFT tiene un carácter sintético con los siguientes apartados:

A.1. Introducción

El desarrollo de BloodTwin —un dispositivo de pulsioximetría y la cadena que enlaza su señal con el registro clínico de una donación de sangre— no es una actividad aislada en el vacío tecnológico, sino que tiene importantes ramificaciones éticas, sociales, legales, económicas y ambientales. El contexto de aplicación acentúa esas ramificaciones: el donante es una persona sana que acude de forma altruista, de modo que cualquier instrumentación añadida al proceso debe justificar su presencia por el beneficio que aporta a su seguridad, y los datos que genera son información clínica sobre un individuo identificable mediante un código de trazabilidad internacional. El proyecto se concibe bajo el prisma de la salud digital, cuyo propósito último es mejorar la calidad y la seguridad de la atención mediante el uso eficiente de las tecnologías de la información y la comunicación (TIC) [33].

En este sentido, la responsabilidad del ingeniero abarca desde la selección de materiales sostenibles hasta la protección estricta del derecho fundamental a la privacidad de los datos médicos de los pacientes, pasando por un riguroso análisis de viabilidad financiera y del impacto positivo en la sostenibilidad de los sistemas públicos de salud.

A.2. Descripción de impactos relevantes relacionados con el proyecto

Tras un análisis sistemático del ciclo de vida y despliegue del sistema diseñado, se han identificado tres dimensiones de impacto clave:

1. **Impacto Social y Sostenibilidad Sanitaria:** El alcance del sistema desarrollado es el puesto de donación de sangre, no el domicilio del paciente crónico. Dentro

de ese alcance, el dispositivo permite registrar de forma continua la saturación de oxígeno (SpO_2) y la frecuencia cardíaca del donante durante la extracción, un intervalo que hoy queda sin instrumentar. El beneficio esperado es doble: dotar al personal de una señal que precede a la reacción vasovagal —la complicación más frecuente en donación— y construir un registro trazable que asocie esa señal a la donación concreta, lo que abre la puerta a estudios retrospectivos sobre la seguridad del donante. Conviene no sobrestimar ese impacto: es una herramienta de instrumentación y trazabilidad, y su valor clínico deberá acreditarse con un estudio con donantes reales. La extensión a la monitorización remota de pacientes crónicos —enfermedad pulmonar obstructiva crónica, insuficiencia cardíaca— es una aplicación plausible de la misma arquitectura, pero queda fuera de lo desarrollado y validado en este Trabajo.

2. **Impacto Ambiental y Ecodiseño:** La fabricación y el desecho de dispositivos electrónicos portátiles plantean importantes desafíos ambientales debido a la generación de residuos electrónicos (e-waste). La selección de componentes para este prototipo ha priorizado el cumplimiento de la directiva europea RoHS (Restricción de Sustancias Peligrosas) para evitar el uso de metales pesados. Asimismo, el uso sistemático de los mecanismos de bajo consumo de la plataforma —en particular el modo *System OFF* del SoC en el estado `P_DEEP_SLEEP`— podría reducir la frecuencia de recarga y prolongar la vida útil de la celda de polímero de litio, retrasando su degradación y su posterior necesidad de reciclado. Se trata de una contribución potencial y no de un resultado acreditado: el consumo del prototipo no se ha medido en este Trabajo, de modo que el alcance real de ese ahorro queda pendiente de la caracterización experimental descrita en el Capítulo 8.
3. **Aspectos Éticos y Protección de Datos Médicos:** Los datos capturados y transmitidos por el dispositivo representan información clínica confidencial de alta sensibilidad. El uso de protocolos de transmisión inalámbrica BLE sin cifrado o con perfiles vulnerables constituye un riesgo ético importante ante posibles ataques de interceptación de datos (*eavesdropping*). El diseño implementado mitiga este impacto estableciendo mecanismos de control rígidos en la característica de datos y sentando las bases criptográficas para futuras integraciones.

A.3. Análisis detallado de alguno de los principales impactos

Se analiza en profundidad el **Impacto Ético y Legal sobre la Confidencialidad de los Datos Médicos**, el cual está sujeto a marcos regulatorios muy estrictos como el Reglamento General de Protección de Datos (RGPD) en la Unión Europea.

En el diseño e implementación del firmware se ha aplicado el principio de *Privacy by Design* (Privacidad desde el Diseño) mediante dos decisiones técnicas clave. Conviene, no obstante, acotar el alcance de esa afirmación antes de enunciarlas: el principio se ha aplicado *en el dispositivo*, no todavía en toda la cadena. Los datos en reposo de los nodos terminales —la base de datos local de la aplicación móvil y el almacenamiento del navegador— conservan hoy su contenido sin cifrar, de modo que un despliegue conforme al RGPD exigiría completar ese frente, tal como se recoge en las líneas futuras del Capítulo 9.

Las dos decisiones aplicadas en el firmware son:

- **Pseudonimización de tramas:** El esquema Protocol Buffers y el perfil GATT desarrollados no contienen ningún dato de carácter identificativo personal del donante (como nombres, identificadores únicos o datos demográficos). Las notificaciones transmiten exclusivamente valores brutos de reflectancia roja e infrarroja asociados a un número de secuencia interno; un atacante con alcance de radio no puede identificar al donante a partir del contenido de las tramas —lo que no impide, en cambio, seguir la pista del propio dispositivo por sus identificadores de radio—. La asociación entre el identificador físico del hardware y el registro de la donación se realiza fuera del enlace, en el servidor de la plataforma de integración, mediante el `register_id` que la propia plataforma asigna. Esa asociación no está hoy cifrada en reposo —como ya se ha señalado más arriba—, y completar ese cifrado es uno de los frentes recogidos entre las líneas futuras.
- **Mecanismos de borrado seguro:** El subsistema de almacenamiento en flash NVS (Caja Negra) registra la secuencia de fallos, y el borrado solicitado por el operador está deliberadamente restringido a un único canal: la consola de mantenimiento sobre USB CDC-ACM. La orden '3' realiza un borrado lógico de los informes almacenados —invalida su entrada en el índice de NVS, sin actuar sobre las celdas—, mientras que la orden 'F' ejecuta un borrado físico de los sectores de la partición mediante la API de flash del SoC. Esta segunda opera tanto sobre una partición corrupta como sobre una sana, de modo que cubre los dos escenarios en que el borrado es legítimo: la recuperación de un equipo averiado y el vaciado previo a su retirada. Precisamente porque el historial no se recupera, la orden exige **doble confirmación**: la primera pulsación solo advierte, y únicamente una segunda dentro de los diez segundos siguientes ejecuta el borrado; cualquier otra tecla intercalada lo cancela. La operación queda además registrada con el código 36, cuyo campo `aux` distingue si se trató de una recuperación o de un vaciado deliberado. Conviene, no obstante, ser preciso sobre el alcance de esta medida. El borrado elimina el contenido accesible por los medios normales de lectura y es adecuado para la retirada o el desecho del equipo en el marco previsto, pero **este trabajo no ha realizado ninguna verificación forense** que permita afirmar la irrecuperabilidad frente a técnicas de análisis de carga residual sobre el silicio. Una declaración de borrado seguro en sentido estricto exigiría un procedimiento de sobrescritura múltiple y una validación específica que quedan fuera del alcance definido. **Ninguna de las dos operaciones es accesible por Bluetooth**: el servicio GATT solo admite las órdenes de arranque y parada del flujo de datos. Esa restricción es una decisión de diseño con doble motivación —un atacante con alcance de radio no puede destruir la evidencia de un fallo, y el propio firmware corta la radio al declarar una avería— y se corresponde con el invariante p14 verificado formalmente en el Capítulo 4.

A.4. Conclusiones

El análisis ético, social y ambiental desarrollado sugiere que la introducción de criterios de sostenibilidad y robustez metodológica supone una contribución potencial relevante para el proyecto. Diseñar un software tolerante a fallos y determinista mediante redes de Petri no es solo un objetivo técnico, sino una responsabilidad ética ineludible al tratar con

software médico de cuya fiabilidad depende la seguridad del paciente; conviene precisar, eso sí, que este Trabajo acredita propiedades del modelo de control y del *firmware*, no un beneficio clínico demostrado sobre pacientes. Asimismo, el ecodiseño aplicado a la gestión energética del SoC nRF52840 podría reducir la huella ambiental del dispositivo — extremo que exigiría medir el consumo real para cuantificarlo—, en línea con las directivas internacionales de desarrollo sostenible y economía circular en el sector de las tecnologías de la salud.

Anexo B: Presupuesto económico

El apartado “Requisitos de las acreditaciones internacionales EUR-ACE y ABET” de la Normativa de TFT de la ETSIT-UPM establece que *“La memoria del TFT del GITST, GIB y MUIT, y en general la de aquellas titulaciones que hayan obtenido o para las que se desee solicitar una acreditación internacional EUR-ACE o ABET, ... debe incluir un presupuesto económico”*.

El presupuesto económico del proyecto se divide en costes directos (mano de obra y amortización de recursos materiales) y costes indirectos (gastos generales y beneficio industrial), aplicando finalmente el material fungible y los impuestos vigentes.

B.1. Desglose de costes directos

B.1.1. Costes de mano de obra

El desarrollo del firmware, los dos clientes —la aplicación Android y la aplicación web con su puente de radio—, la plataforma de integración clínica y la herramienta de diagnóstico han sido llevados a cabo por un ingeniero electrónico júnior durante un período de 6 meses de duración (24 semanas), con una dedicación media estimada de 25 horas semanales, lo que supone un total de 600 horas de desarrollo. Aplicando una tarifa media de mercado de 30 €/hora para este perfil profesional, se desglosa el coste de mano de obra en la Tabla B.1.

Concepto / Tarea	Horas	Precio / Hora	Total
Diseño de arquitectura y requisitos	80	30 €	2,400 €
Implementación del firmware (Zephyr)	180	30 €	5,400 €
Aplicación Android	90	30 €	2,700 €
Aplicación web y puente de radio	60	30 €	1,800 €
Plataforma de integración clínica (RED-Cap, Cloud Storage)	60	30 €	1,800 €
Herramienta de diagnóstico por USB	20	30 €	600 €
Verificación formal, pruebas y trazado	110	30 €	3,300 €
Total Mano de Obra	600		18.000 €

Tabla B.1: Costes de recursos humanos y desarrollo.

B.1.2. Amortización de recursos materiales

Los recursos de hardware y software utilizados para la realización del proyecto se amortizan en función de su vida útil y de los meses en los que han sido utilizados de forma

exclusiva para el desarrollo del TFM (6 meses), de acuerdo con la siguiente fórmula de amortización lineal:

$$\text{Amortización} = \text{Precio Compra} \cdot \left(\frac{\text{Uso en meses}}{\text{Vida útil en meses}} \right) \quad (9.1)$$

Se amortizan un ordenador personal, el kit de desarrollo de Nordic Semiconductor y el sensor MAX30102. No se incluye ninguna sonda de depuración externa: tanto la programación del dispositivo como la extracción por SWD de las trazas del núcleo analizadas en el Capítulo 8 se realizaron con la interfaz **SEGGER J-Link On-Board** integrada en la propia placa nRF52840 DK, cuyo coste ya está comprendido en el del kit; contabilizarla aparte duplicaría la partida. Tampoco se incluye instrumentación de laboratorio —fuente de alimentación, osciloscopio o multímetro—: el desarrollo no requirió medidas eléctricas, coherentemente con la caracterización de consumo que se declara pendiente en ese mismo capítulo. El desglose detallado se muestra en la Tabla B.2.

Equipamiento / Recurso	Precio Compra	Uso (meses)	Vida útil (años)	Amortización
Ordenador portátil de desarrollo	1,500,00 €	6	5 (60 meses)	150,00 €
Kit de desarrollo nRF52840 DK	50,00 €	6	2 (24 meses)	12,50 €
Placa sensor MAX30102	10,00 €	6	2 (24 meses)	2,50 €
Total Recursos Materiales				165,00 €

Tabla B.2: Amortización de recursos materiales.

B.2. Presupuesto total acumulado

El presupuesto global se calcula aplicando los gastos generales indirectos (asumidos en un 15 % sobre los costes directos de desarrollo y amortización) y el beneficio industrial típico del sector (6 % sobre la suma de costes directos e indirectos). Asimismo, se incluye el material fungible (encuadernaciones, impresión y componentes pasivos y cables destruidos) y el Impuesto sobre el Valor Añadido (IVA) vigente del 21 %.

La Tabla B.3 resume las partidas económicas globales del proyecto.

Partida / Concepto	Coste
Coste de mano de obra (directo)	18,000,00 €
Amortización de recursos materiales (directo)	165,00 €
COSTE DIRECTO TOTAL (CD)	18.165,00 €
Gastos generales (15 % sobre CD)	2,724,75 €
Beneficio industrial (6 % sobre CD + Gastos)	1,253,39 €
Material fungible (Consumibles, componentes y encuadernación)	200,00 €
SUBTOTAL PRESUPUESTO	22.343,14 €
IVA aplicable (21 %)	4,692,06 €
TOTAL PRESUPUESTO ACUMULADO	27.035,20 €

Tabla B.3: Presupuesto final acumulado.

Anexo C: Batería completa de pruebas unitarias

Este anexo recoge la relación exhaustiva de las 68 pruebas unitarias que componen la batería de validación del firmware, agrupadas en los doce bloques temáticos descritos en la Sección 8.2. Los nombres se transcriben literalmente del fichero `tests/test/test_app_pn.c`, y la aserción indicada es la comprobación efectiva que realiza cada prueba, no una paráfrasis de su intención.

La ejecución completa se resuelve en el ordenador de desarrollo mediante `ceedling gcov:all`, con el resultado global de **68 superadas, 0 fallidas y 0 ignoradas**.

Tabla C.1: Relación exhaustiva de las 68 pruebas unitarias del firmware, con la aserción que verifica cada una y su resultado. El prefijo `test_` se omite en la columna de nombres por brevedad.

Función de prueba	Aserción verificada	Res.
Bloque 1 — Arranque		
<code>el_arranque_deja_un_testigo_en_boot</code>	Tras la inicialización, la marca está en <code>P_BOOT</code> y ningún otro lugar de estado la tiene	PASS
<code>boot_pasa_a_advertising_con_bt_ready</code>	El evento de radio lista traslada la marca a <code>P_ADVERTISING</code> y actualiza <code>ctrl_state</code>	PASS
Bloque 2 — Conexión y flujo de datos		
<code>advertising_pasa_a_streaming_si_se_cumple_la_guarda</code>	Con la guarda satisfecha, la conexión lleva directamente a <code>P_STREAMING</code>	PASS
<code>advertising_pasa_a_reposo_si_no_se_cumple_la_guarda</code>	Sin guarda satisfecha, la misma conexión lleva a <code>P_CONNECTED_IDLE</code>	PASS
<code>reposo_pasa_a_streaming_al_activar_notificaciones</code>	La suscripción del cliente activa la transmisión desde reposo	PASS
<code>reposo_se_mantiene_si_falta_la_suscripcion</code>	Sin suscripción, la marca permanece en reposo y no alcanza <code>P_STREAMING</code>	PASS
<code>streaming_vuelve_a_reposo_al_parar_el_flujo</code>	La orden de parada devuelve la marca a <code>P_CONNECTED_IDLE</code>	PASS

continúa en la página siguiente

Función de prueba	Aserción verificada	Res.
streaming.vuelve_a_reposo_si_falla_la_notificacion	El error de notificación degrada a reposo sin declarar avería	PASS
streaming.vuelve_a_anunciar_al_desconectar	La desconexión del cliente devuelve el equipo a P_ADVERTISING	PASS
desconexion_silenciosa.vuelve_a_anunciar	La pérdida silenciosa del enlace produce el mismo retorno al anuncio	PASS
volver_a_anunciar_desactiva_el_flujo	Al reanudar el anuncio, la bandera streaming_enabled queda a cero	PASS

Bloque 3 — Errores del sensor

un_error_de_sensor_no_corta_el_flujo	Un fallo aislado deja la marca en P_STREAMING: la transmisión continúa	PASS
tres_errores_de_sensor_provocan_fallo_fatal	Al tercer fallo consecutivo se fija el código 20 y la marca pasa a P_ERROR	PASS
el_contador_de_errores_se_reinicia_al_volver_a_streaming	El contador vuelve a cero al reanudar, sin arrastrar fallos de la sesión anterior	PASS
una_lectura_correcta_rompe_la_racha_de_errores	La secuencia error–acierto–error–acierto–error contiene tres fallos pero ninguna racha de tres, y no declara avería	PASS
la_racha_rota_no_impide_detectar_tres_errores_seguidos	Romper la racha no desactiva la protección: tres fallos seguidos posteriores siguen llevando a P_ERROR	PASS
una_lectura_correcta_sin_errores_previos_no_hace_nada	El evento de recuperación sin racha pendiente es inocuo: ni cambia de estado ni registra incidencia	PASS
error_y_recuperacion_en_el_mismo_ciclo_no_declaran_averia	Con ambos eventos en el mismo ciclo se disparan las dos transiciones en orden de declaración —primero la recuperación, después el error— y no se declara avería alguna	PASS
recuperacion_seguida_de_error_en_el_mismo_ciclo_no_es_fatal	El caso inverso: la lectura buena rompe la racha antes de contar el error, y no se declara avería	PASS

Bloque 4 — Batería y apagado

bateria_baja_lleva_a_apagado_profundo	El aviso de batería conduce a P_DEEP_SLEEP y actualiza el estado publicado	PASS
---------------------------------------	--	------

continúa en la página siguiente

Función de prueba	Aserción verificada	Res.
<code>timeout_anunciando_lleva_a_apagado_profundo</code>	El vencimiento del plazo de anuncio conduce al sueño profundo	PASS
<code>la_ventana_de_error_termina_en_apagado</code>	Agotada la ventana de diagnóstico, el equipo pasa a sueño profundo	PASS
<code>streaming_mantiene_armada_la_vigilancia_de_bateria</code>	Al entrar en <code>P_STREAMING</code> el temporizador de batería queda armado en modo periódico y con el valor de <code>BATTERY_CHECK_PERIOD_MS</code>	PASS
Bloque 5 — Prioridad entre transiciones en conflicto		
<code>el_fallo_fatal_tiene_prioridad_sobre_bt_ready</code>	Con ambos eventos pendientes se dispara la transición al error, no la de anuncio	PASS
<code>la_bateria_baja_tiene_prioridad_sobre_la_conexion</code>	Con ambos pendientes se atiende la batería y el resultado es el apagado	PASS
<code>la_conexion_tiene_prioridad_sobre_el_timeout</code>	Con ambos pendientes prevalece la conexión frente al vencimiento	PASS
Bloque 6 — Robustez de la cola de eventos		
<code>un_evento_sin_transicion_valida_se_descarta</code>	Un evento sin transición aplicable se consume sin alterar el marcado	PASS
<code>la_cola_llena_descarta_el_evento_mas_antiguo</code>	Al desbordar la cola, la inserción sigue teniendo éxito descartando el más antiguo	PASS
<code>la_cola_llena_no_puede_expulsar_un_fallo_fatal</code>	Un fallo fatal sepultado bajo una ráfaga del triple de la capacidad sigue llevando la red a <code>P_ERROR</code>	PASS
<code>la_cola_llena_no_puede_expulsar_un_avisode_bateria</code>	Lo mismo para el aviso de batería, que alcanza <code>P_DEEP_SLEEP</code> pese a la ráfaga	PASS
Bloque 7 — Radio y rastro del perro guardián		
<code>entrar_en_error_apaga_la_radio</code>	Al declarar el fallo se invoca <code>ble_task_stop()</code> exactamente una vez	PASS
<code>el_fallo_en_marcha_invalida_el_rastro_del_watchdog</code>	Un fallo en operación borra el rastro antes de reiniciar	PASS
<code>el_fallo_en_arranque_conserva_el_rastro</code>	Un fallo de arranque <i>no</i> borra el rastro: la causa podría ser el watchdog	PASS

continúa en la página siguiente

Función de prueba	Aserción verificada	Res.
Bloque 8 — Registro de incidencias		
el_anillo_agrupa_las_incidencias_repetidas	Tres incidencias idénticas ocupan una entrada con contador de repetición igual a 3	PASS
el_anillo_conserva_el_order_de_incidencias_distintas	Dos incidencias distintas ocupan dos entradas en orden cronológico	PASS
solo_los_codigos_de_flash_se_re cuer dan_como_persistentes	Un código de RAM no altera la causa persistente; uno de Flash sí	PASS
los_dos_ejes_de_clasificacion_son_independientes	Severidad y persistencia se combinan libremente, según la tabla maestra	PASS
los_contadores_de_salud_agrupan_por_categoria	Cada incidencia incrementa únicamente el contador de su categoría	PASS
el_informe_guarda_la_secuencia_que_llevo_al_fallo	El registro persistido incluye el anillo previo y los contadores de salud	PASS
el_informe_hereda_la_causa_del_ultimo_codigo_persistente	Sin causa fijada, se adopta el último código de Flash con su contexto y severidad	PASS
anotar_el_motivo_de_apagado_no_crea_un_informe_nuevo	Corregir un informe existente lo reescribe en su ranura en vez de ocupar otra, de modo que el anillo no se llena de copias del mismo fallo	PASS
Bloque 9 — Rutas de fallo del arranque		
el_arranque_falla_si_no_se_pueden_crear_las_colas	Retorna -1 y registra el código 33	PASS
sin_sensor_el_arranque_sigue_en_modo_seguro	Con sensor ausente el arranque concluye, registra el código 10 y queda en P_BOOT	PASS
un_bus_i2c_caído_no_se_confunde_con_un_sensor_ausente	Si la capa HAL anotó el código 13, esa causa prevalece sobre la de sensor ausente	PASS
un_watchdog_que_no_arranca_deja_el_equipo_fuera_de_servicio	Un fallo del watchdog fija el código 15 como causa de arranque: RR-2 deja de poder cumplirse	PASS
un_adc_que_no_arranca_deja_el_equipo_fuera_de_servicio	Un fallo del ADC fija el código 14: sin él nada impide sobredescargar la celda	PASS

continúa en la página siguiente

Función de prueba	Aserción verificada	Res.
una_funcion_ausente_por_compilacion_no_impide_arrancar	-ENOTSUP no es avería sino decisión de construcción: el arranque continúa sin causa fijada	PASS
un_sensor_ausente_manda_sobre_un_watchdog_averiado	Con varios fallos simultáneos se registra el primero de la secuencia, que es el que explica al resto	PASS
un_fallo_de_arranque_distinto_se_guarda_junto_al_anterior	Tras una avería en marcha, un fallo de arranque <i>distinto</i> se guarda como segundo informe en vez de suprimirse	PASS
el_mismo_fallo_de_arranque_no_se_duplica_en_cada_reinicio	La repetición exacta de un fallo de arranque no ocupa una ranura nueva: el bucle de reinicios no vacía el anillo	PASS
dos_fallos_de_arranque_distintos_se_guardan_por_separado	Perder el sensor y perder el bus son averías distintas y generan informes distintos	PASS
sin_informe_previo_el_fallo_de_arranque_se_guarda	Con la caja negra recién borrada no hay con qué comparar: el informe se guarda	PASS
un_fallo_en_marcha_nunca_se_suprime_por_ser_repetido	La supresión es exclusiva del arranque; en marcha cada fallo aporta contexto nuevo	PASS
el_fallo_de_la_tarea_del_sensor_se_registra	El fallo al crear el hilo de adquisición registra el código 11 sin abortar el arranque	PASS
el_fallo_de_la_tarea_bluetooth_se_registra	El fallo al arrancar la tarea de radio registra el código 12	PASS
el_arranque_falla_si_no_hay_hilo_de_control	Sin hilo de control el arranque retorna -1 y registra el código 34	PASS

Bloque 10 — Trabajo diferido

una_bateria_sana_no_dispara_nada	La lectura válida actualiza la tensión y consume la petición pendiente	PASS
una_bateria_por_debajo_del_umbral_pide_el_apagado	Bajo umbral se incrementa el contador de salud y la red alcanza el sueño profundo	PASS
el_umbral_de_bateria_no_se_dispara_en_el_limite	Exactamente en el umbral no hay aviso: la comparación es estricta	PASS
bateria_baja_no_dispara_apagado_si_hay_alimentacion_usb	Con tensión USB presente, una lectura por debajo del umbral no apaga el equipo ni incrementa el contador de salud	PASS

continúa en la página siguiente

Función de prueba	Aserción verificada	Res.
<code>un_fallo_del_adc_no_se_confunde_con_bateria_llena</code>	Si la conversión falla, la tensión almacenada permanece intacta	PASS
<code>una_lectura_imposible_no_se_confunde_con_bateria_agotada</code>	Una lectura de 400 mV, imposible para una celda con protección, se registra como código 51 y no apaga el equipo	PASS
<code>el_umbral_de_ausencia_no_tapa_una_bateria_realmente_baja</code>	Justo por encima del suelo de verosimilitud la medida vuelve a ser creíble y el apagado se produce	PASS
<code>sin_peticion_pendiente_no_se_lee_la_bateria</code>	Sin petición activa no se invoca el convertidor	PASS
<code>los_eventos_descartados_en_interrupcion_se_anotan_despues</code>	El anillo permanece vacío hasta que el hilo de control drena el contador atómico	PASS
Bloque 11 — Tabla maestra de errores		
<code>cada_codigo_devuelve_su_nombre_y_su_descripcion</code>	La traducción de código a nombre simbólico y descripción es correcta y no nula	PASS
<code>un_codigo_fuera_de_la_tabla_no_inventa_nada</code>	Un código desconocido degrada a valores reconocibles sin indexar fuera de la tabla	PASS
Bloque 12 — Interfaz de temporizadores		
<code>las_funciones_del_temporizador_de_apagado_no_fallan</code>	Las operaciones de arranque y parada del temporizador retornan sin error	PASS

C.1. Observaciones sobre la batería

Tres características de este conjunto merecen destacarse, porque no se deducen de la tabla anterior.

En primer lugar, las pruebas **ejercitan el motor de red de Petri real**, el mismo código C que se compila para el microcontrolador. Lo sustituido es el sistema operativo y el hardware, no la lógica bajo prueba; por eso el fichero incluye directamente la unidad `app_pn.c`, cuyas estructuras internas son estáticas y no serían accesibles de otro modo.

En segundo lugar, el bloque de *rutras de fallo del arranque* concentra quince pruebas sobre un camino que en operación normal no se recorre nunca. Esa desproporción es intencionada: son los caminos que el hardware no permite provocar con facilidad y los que, cuando fallan, dejan el equipo inservible.

Por último, varias pruebas existen porque el defecto contrario llegó a introducirse duran-

te el desarrollo. Es el caso de `los_eventos_descartados_en_interrupcion_se_anotan_despues`, que fija el comportamiento correcto tras haberse detectado una llamada al registro de incidencias desde contexto de interrupción, donde tomar un semáforo está prohibido. Y es también el caso de `streaming_mantiene_armada_la_vigilancia_de_bateria`, incorporada después de descubrirse que el firmware detenía el temporizador de batería precisamente al entrar en el modo de mayor consumo: la transición `in_str_lowbat` seguía declarada en la tabla, pero nadie generaba ya el evento capaz de dispararla, de modo que se había convertido en código inalcanzable. El caso ilustra con precisión el límite de la verificación formal: el modelo de Promela contemplaba esa transición y la propiedad `p6b` la comprobaba, pero SPIN razona sobre eventos, no sobre los temporizadores que los originan, y por eso no podía detectar que la fuente del evento estuviera desconectada. Una prueba de regresión de este tipo vale más que varias escritas a priori: documenta un error real y garantiza que no vuelva.

Anexo D: Especificación de la interfaz de integración

Este anexo recoge la especificación de la interfaz REST que expone la plataforma de integración descrita en el Capítulo 7. El contenido se corresponde con el documento **OpenAPI 3.0** que la aplicación publica en tiempo de ejecución. Conviene precisar su procedencia: ese documento está escrito a mano en el módulo `utils/openapi.py` y se sirve desde el propio servidor, no se deriva por introspección del código. Es, por tanto, una especificación mantenida en paralelo a la implementación, y puede desincronizarse de ella si una y otra evolucionan por separado. Derivar el esquema del código —o, al menos, contrastarlo con él en la integración continua— se recoge entre las líneas futuras.

D.1. Convenios generales

- Los cinco puntos de entrada de datos, junto con la obtención del testigo, cuelgan del prefijo `/api` y emplean el método `POST`. Los recursos de consulta descritos al final del anexo responden a `GET`.
- El cuerpo de los **cinco puntos de datos** es siempre una **lista** de objetos con la forma `{"name": campo, "value": valor}`. Este formato uniforme permite que un cliente construya la carga útil sin conocer el esquema completo de cada punto de entrada.
- Las respuestas son objetos JSON. Las de error incluyen siempre `"error"` con un mensaje explicativo. Las de éxito incluyen `"success": true` en los cinco puntos de entrada, `register_donor` incluido. Este último acompaña además el campo `"granted": 1`, heredado de la primera versión de la interfaz y conservado a propósito: los clientes ya desplegados lo interrogan, y retirarlo los rompería sin aportar nada. La respuesta de un alta es por tanto `{"success": true, "granted": 1, "register_id": id}`, y añade `"already_registered": true` cuando la donación ya existía.
- Los cinco puntos de entrada de datos exigen autenticación.

D.2. Autenticación

El acceso se controla mediante tokens JWT firmados con el algoritmo **HS256**. El token se obtiene en un punto de entrada específico y tiene una validez configurable, con un valor por omisión de **12 horas**.

```
POST /api/auth/token
{"usuario": ..., "clave": ...}
```

La respuesta devuelve el token y su instante de caducidad en formato ISO. En cada petición posterior debe enviarse en la cabecera:

```
Authorization: Bearer token
```

La plataforma aplica además un **límite de peticiones** por dirección de origen dentro de una ventana temporal deslizante, ambos parámetros configurables en el arranque. Superarlo produce una respuesta 429 que indica los segundos que restan para poder reintentar.

D.3. Secuencia de invocación

El orden de las llamadas no es libre: reproduce el procedimiento clínico. Sin un registro previo de la donación no existe la fila en el sistema de captura, y los cuatro puntos de entrada restantes responden 404.

1. `register_donor` — crea la donación y devuelve el `register_id`
2. `upload_donor_basals` — datos demográficos
3. `upload_donor_clinical` — constantes vitales
4. `save_device_measurements` — señal del sensor
5. `upload_donation_lab_measurements` — hemograma

El identificador de registro lo asigna **siempre el servidor**. Un cliente que lo genere localmente verá rechazadas todas las subidas posteriores.

D.4. Puntos de entrada

D.4.1. Registro de la donación

Campo	Tipo	Validación
<code>donor_id</code>	cadena	Formato ISBT 128: carácter = seguido de exactamente 15 caracteres alfanuméricos
<code>donation_id</code>	cadena	Mismo formato ISBT 128

Tabla D.1: Entrada de `register_donor`.

Devuelve `{"success": true, "granted": 1, "register_id": ...}`. La plataforma calcula el siguiente identificador libre, crea la fila y sella el instante en tiempo universal coordinado. El alta es **idempotente** respecto al par de códigos ISBT: si la donación ya existe —porque la respuesta de un intento anterior se perdió y el cliente reintentó— no se crea una segunda fila, sino que se devuelve el identificador existente acompañado de `"already_registered": true`. Sin esa comprobación, una respuesta perdida en la red repartiría las medidas de una misma donación entre dos filas.

D.4.2. Datos demográficos y clínicos

Punto de entrada	Campo	Contenido
upload_donor_ basals	register_id	Identificador devuelto en el paso 1
	birth	Fecha de nacimiento
	weight_kg	Peso en kilogramos
	height_cm	Talla en centímetros
	sex	Sexo
	comments	Observaciones libres
upload_donor_ clinical	register_id	Identificador de la donación
	hr_bpm	Frecuencia cardíaca
	sys_mmHg	Presión sistólica
	dia_mmHg	Presión diastólica
	spo2_percent	Saturación de oxígeno
	hgb_g_dL	Hemoglobina de cribado
	comments	Observaciones libres
local_datetime		Marca de tiempo del cliente. Obligatorio en ambos pasos
offset_seconds		Desfase horario del cliente. Obligatorio en ambos pasos

Tabla D.2: Entrada de los pasos 2 y 3. Los campos numéricos se validan en tipo y, cuando procede, en rango. Las dos últimas filas se exigen también en el paso 5.

D.4.3. Medidas del dispositivo

Es el punto de entrada que consume la salida del firmware descrito en esta memoria.

Campo	Tipo	Contenido
<code>register_id</code>	cadena	Identificador de la donación. Obligatorio
<code>device_id</code>	cadena	Identificador del dispositivo que tomó la medida
<code>ppg_red</code>	lista	Serie completa del canal rojo
<code>ppg_ir</code>	lista	Serie completa del canal infrarrojo
<code>temp_dice</code>	lista	Serie de temperatura del encapsulado
<code>local_datetime</code>	entero	Marca de tiempo del cliente, en milisegundos
<code>offset_seconds</code>	entero	Desfase horario del cliente en segundos. Obligatorio: sin él no puede reconstruirse la hora local de la captura
<code>quality</code>	—	Indicador de calidad de la sesión. Opcional
<code>comments</code>	cadena	Observaciones libres. Opcional

Tabla D.3: Entrada de `save_device_measurements`.

Reparto del contenido. Este punto de entrada es el único que escribe en los dos almacenes. Las tres series se depositan como ficheros CSV en el almacén de objetos, con dos columnas —marca de tiempo y valor— y una nomenclatura que las vincula a la donación:

```
device_measurements/{register_id}_ppg_red-{timestamp}.csv
device_measurements/{register_id}_ppg_ir-{timestamp}.csv
device_measurements/{register_id}_temp_dice-{timestamp}.csv
```

Junto a los tres CSV se deposita un cuarto objeto, `metadata/{register_id}-{timestamp}-metadata.json`, que recoge los parámetros de la sesión —dispositivo, frecuencias de muestreo, número de muestras por serie y marca de tiempo— y permite interpretar los CSV sin depender de ninguna otra fuente. No se referencia desde REDCap: su nombre se deriva de los mismos `register_id` y marca de tiempo que los CSV, de modo que localizarlo no exige guardarlo.

En el sistema de captura clínica se anotan únicamente los metadatos de la sesión y los *nombres* de los tres ficheros de señal, en los campos `red_filename`, `ir_filename` y `temp_filename`. De este modo, cuando la operación finaliza correctamente, el registro clínico conserva los nombres que permiten referenciar las señales sin tener que contenerlas, según se razona en la Sección 7.4.

D.4.4. Resultados de laboratorio

Recibe el `register_id` junto con las diecinueve variables del hemograma completo, un campo de observaciones libres y el par `local_datetime` / `offset_seconds`, obligatorio igual que en los pasos 2 y 3. Es el último paso de la secuencia y cierra el registro de la donación.

Conviene justificar por qué el instante de la toma se exige en los cuatro pasos de datos y no se deduce en el servidor. Un hemograma puede llegar horas después de la extracción,

y sellarlo con la hora de recepción falsearía la cronología clínica precisamente en el paso donde más separadas están ambas. El desfase se pide aparte de la marca porque el cambio de hora repite una hora entera al año: sin él, dos donaciones de esa madrugada serían indistinguibles en el orden.

D.5. Códigos de respuesta

Código	Significado
200	Operación completada
400	Fallo de validación: campo obligatorio ausente, formato ISBT incorrecto, valor no numérico o fuera de rango, o cuerpo que no es una lista
401	Token ausente, mal formado o caducado
404	La donación referida por <code>register_id</code> no existe
413	El cuerpo de la petición supera el tamaño máximo admitido
429	Se ha superado el límite de peticiones de la ventana actual
500	Fallo interno no previsto del servidor
503	Un servicio externo —REDCap o el almacén de objetos— no responde, o la autenticación no está configurada en el servidor

Tabla D.4: Códigos de estado devueltos por la plataforma.

D.6. Documentación en línea

Además de los puntos de entrada de datos, la plataforma expone cuatro recursos de consulta que no requieren autenticación: una página raíz que enumera la interfaz, un indicador de estado del servicio, el documento OpenAPI 3.0 [34] en formato JSON y una versión navegable de esa misma especificación. Ninguno de los cuatro devuelve dato clínico alguno.

Las figuras que siguen reproducen esa documentación tal como la sirve la plataforma, recorriéndola en el mismo orden en que el protocolo obliga a invocarla. Cada punto de entrada aparece con su descripción, un ejemplo completo de la carga útil y la lista de códigos de respuesta que contempla. Conviene subrayar el alcance exacto de esa garantía, para no reclamar más de lo que hay: las páginas se *sirven* desde el mismo despliegue que atiende las peticiones, de modo que corresponden con certeza a la versión en ejecución y no a otra anterior; pero su contenido procede del documento escrito a mano descrito al principio de este anexo, no de una introspección de las rutas. Corresponden a lo que el servidor *declara* aceptar, y contrastar esa declaración con lo que realmente acepta es la comprobación que queda pendiente.

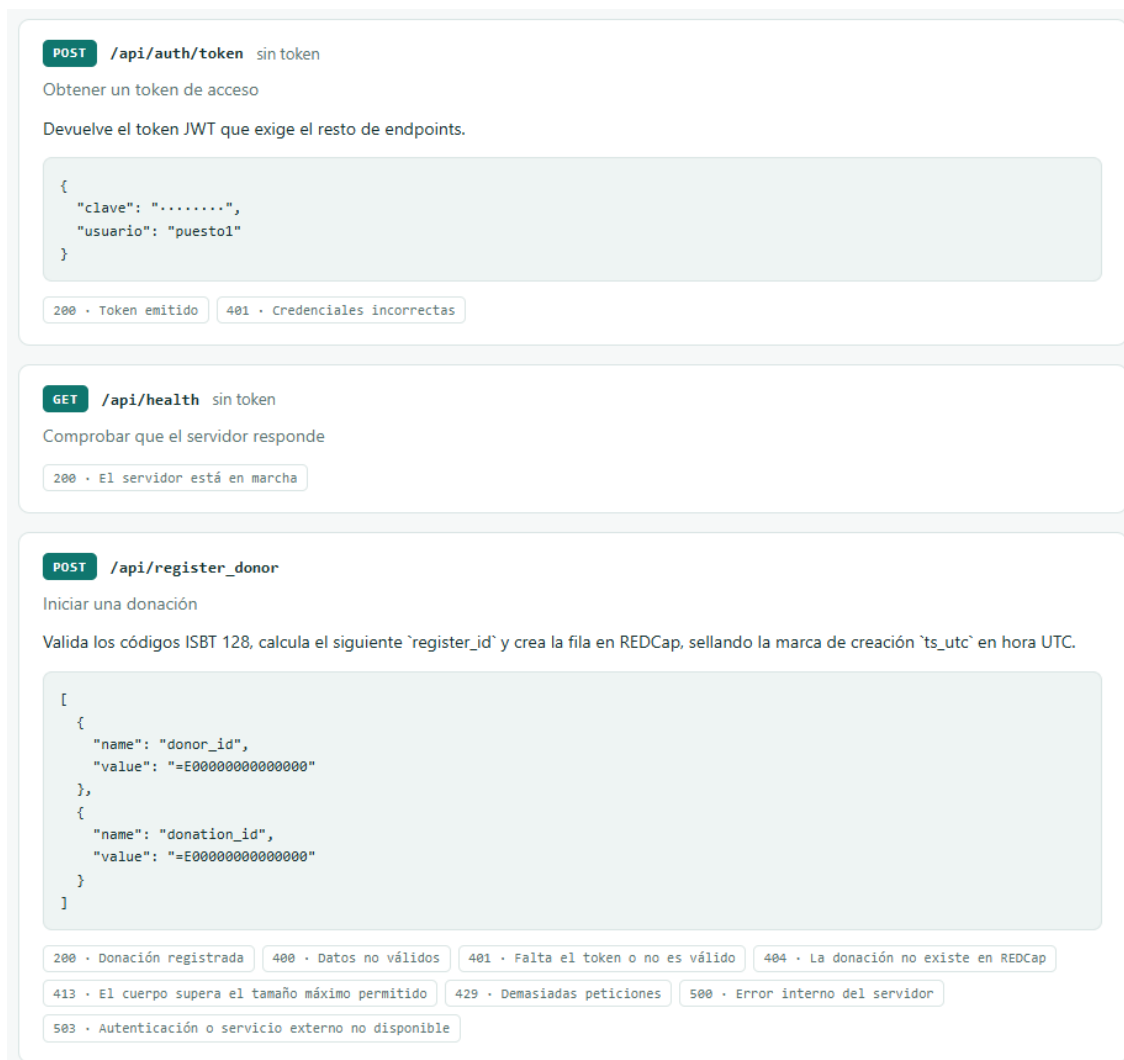


Figura D.1: Los tres primeros recursos. La obtención del token y la comprobación de estado llevan la marca **sin token**: son los dos únicos de esta figura accesibles sin autenticar —los otros recursos de consulta, descritos más abajo, tampoco la exigen—, y su ausencia en el resto es lo que indica que los puntos de entrada de datos sí la requieren. El tercero, **register_donor**, abre la donación: valida los códigos ISBT 128, calcula el **register_id** y sella la marca de creación en hora UTC.



Figura D.2: Paso 2, datos demográficos del donante. La carga útil es la lista de pares {name, value} común a toda la interfaz, encabezada siempre por el register_id que devolvió el paso anterior.

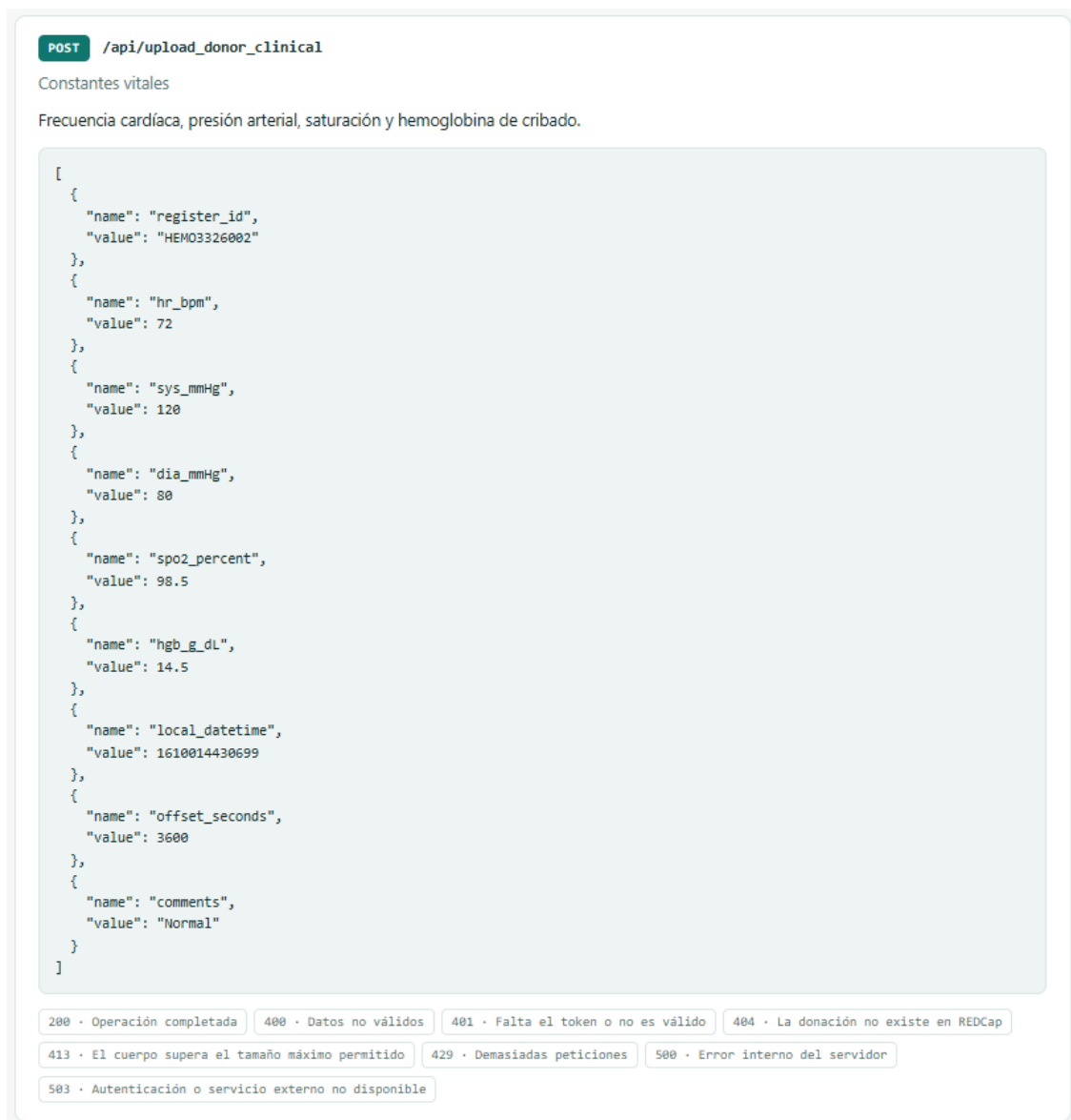


Figura D.3: Paso 3, constantes vitales y hemoglobina de cribado. Los cinco parámetros llevan el nombre y la unidad con que se registran, sin abreviaturas propias del almacén de destino.



Figura D.4: Paso 4, el único que no deposita su contenido en el registro clínico. Las tres series —roja, infrarroja y de temperatura— viajan como vectores dentro de la misma estructura de pares, acompañadas de los metadatos de trama; la plataforma las convierte en tres ficheros CSV y anota sus nombres en el registro.

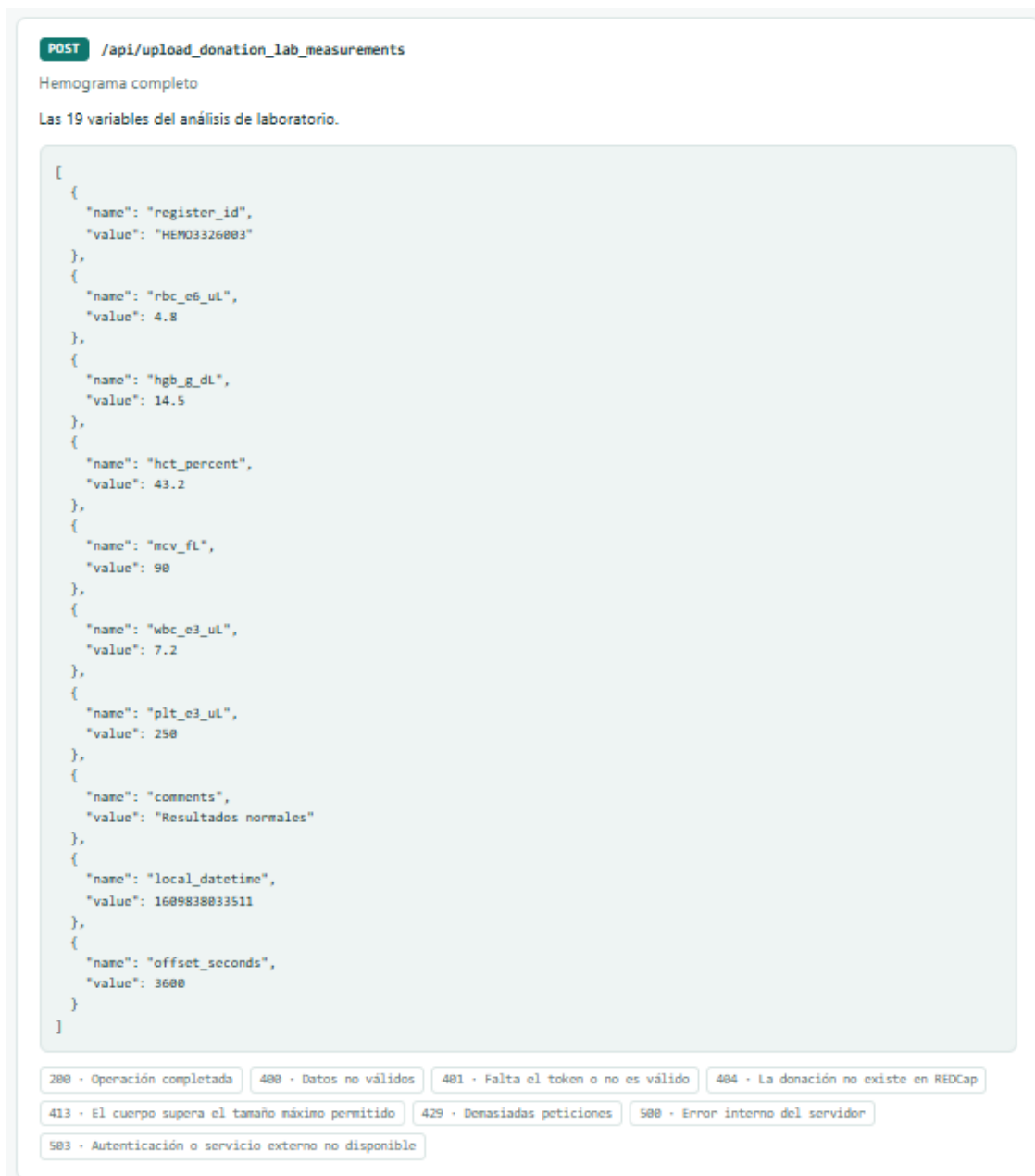


Figura D.5: Paso 5, las diecinueve variables del análisis de laboratorio. Es el paso que puede llegar horas después de los anteriores, y la interfaz no impone ninguna restricción temporal: basta con que el `register_id` exista.

El repertorio de códigos es común a los cinco puntos de entrada de datos, lo que no es casual: la plataforma responde de forma uniforme para que un cliente pueda tratar los errores con un único mecanismo, sin conocer qué paso ha fallado. La única salvedad es el 404, que sólo puede darse en los cuatro puntos que reciben un `register_id` ya existente; el alta lo crea, de modo que no tiene ninguna donación previa que no encontrar. La distinción decisiva es la que separa los que **no** mejoran con el tiempo —400, 401, 404 y 413, que exigen intervención porque repetir la petición daría el mismo resultado— de los que sí admiten reintento: 429, 500 y 503. Solo estos tres alimentan la cola descrita en la Sección 7.5. El 503 es el caso más nítido de la segunda familia: significa que REDCap o el almacén de objetos no responden en este instante, y esa es justamente la condición que un reintento posterior puede encontrar resuelta.